

IC Compiler™ II Implementation User Guide

中文学习译本

原版版本：V-2023.12, December 2023

Synopsys, Inc.

原文版权归 Synopsys, Inc. 所有（Version V-2023.12, December 2023）。
本文件为非官方个人学习译本，仅供学习参考，不得用于商业传播。
命令名、选项名、文件名等技术标识符保持英文原文不译。

基于官方 PDF 的结构化翻译工程

当前进度：前言 + 第 1–6 章完整；第 7–12 章目录占位

目录

关于本用户指南	x
本版本新增内容	x
相关产品、文档与商标	x
排版约定	x
客户支持	x
包容性与多样性声明	xi
第一章 使用 IC Compiler II 工具	1
1.1 布局布线设计流程概览	2
1.2 IC Compiler II 相关概念	3
1.2.1 功耗意图概念	3
1.2.2 多重图案化概念	5
1.3 使用 3DIC Compiler 用户界面	6
1.3.1 启动 CLI	7
1.3.2 退出工具	7
1.4 输入 icc2_shell 命令	7
1.4.1 中断或终止命令处理	8
1.4.2 获取命令信息	8
1.5 使用应用选项	8
1.6 使用变量	9
1.7 查看 Man 页	9
1.8 使用 Tcl 脚本	10
1.9 通过检查点向脚本添加变更	10
1.9.1 定义检查点	11
1.9.2 配置检查点	12
1.9.3 查询检查点与行为	14
1.9.4 查看检查点历史	14
1.10 使用 Setup 文件	15
1.11 使用命令日志文件	15
1.12 启用多核处理	15
1.12.1 配置多线程	15
1.12.2 配置分布式处理	16
1.12.3 报告与移除多核配置	16

1.12.4	并行运行任务	16
1.12.5	在本地主机并行运行命令	17
第二章	准备设计	18
2.1	定义搜索路径	19
2.2	设置库	19
2.2.1	处理设计库	19
2.2.2	设置参考库	20
2.2.3	库配置	21
2.2.4	限制库单元使用	22
2.2.5	限制所用目标库	23
2.3	使用 Pin Access Checker 实用程序	24
2.4	处理设计	25
2.5	导入 Floorplan 信息	26
2.5.1	读取 DEF 文件	26
2.6	设置多电压设计	31
2.6.1	应用多电压功耗意图	32
2.6.2	准备电源网络	34
2.6.3	定义电压区域	36
2.6.4	插入多电压单元	43
2.6.5	控制多电压单元的布局	44
2.6.6	启用多电压 Net 的改进缓冲	45
2.6.7	分析多电压信息	45
2.7	指定时序约束与设置	45
2.8	指定逻辑设计规则约束	46
2.9	控制时钟门控延迟	47
2.9.1	集成时钟门控延迟估计	47
2.9.2	用户指定的时钟门控延迟	48
2.10	为布局与合法化指定物理约束	50
2.10.1	定义 Keepout Margins	50
2.10.2	定义基于区域的布局障碍	52
2.10.3	定义布局 Bounds	56
2.10.4	查询布局 Bound	59
2.10.5	移除布局 Bound	59
2.10.6	定义布局 Attraction	59
2.10.7	定义合法化单元间距约束	60
2.10.8	报告单元间距约束	61
2.10.9	移除单元间距约束	62
2.11	指定布局设置	62
2.11.1	在早期阶段使用不精确约束进行布局	62
2.11.2	为时钟门控单元自动生成 Group Bound	63
2.11.3	控制布局密度	63
2.11.4	控制布局期间的拥塞驱动重构	64

2.11.5	降低拥塞	64
2.11.6	布局时考虑单元密度	65
2.11.7	布局时考虑单元 Pin 的影响	65
2.11.8	布局时考虑时钟 Net 非默认布线规则带来的拥塞影响	65
2.11.9	布局时考虑时序阵列中时钟门控单元的影响	66
2.11.10	布局时考虑合法化影响	66
2.11.11	布局时考虑 DFT 连接	66
2.11.12	布局时考虑动态功耗 QoR	66
2.11.13	执行 IR-Drop 感知布局	66
2.11.14	布局时分散 Repeater 单元	68
2.12	指定合法化设置	69
2.12.1	合法化期间最小化大位移	69
2.12.2	合法化期间优化 Pin Access	69
2.12.3	启用高级 PG Net 检查	69
2.12.4	启用高级合法化算法	70
2.12.5	设置 Variant 感知合法化	70
2.12.6	检查库单元是否可合法放置	72
2.13	控制单元、Net、Pin 与 Port 的优化	73
2.13.1	优化期间保留单元与 Net	73
2.13.2	将优化限制为仅做单元尺寸调整	74
2.13.3	优化期间保留网络	75
2.13.4	标记时钟网络	75
2.13.5	禁用设计规则检查 (DRC)	75
2.13.6	尺寸调整期间保留 Pin 名称	76
2.13.7	保留已有层次的 Port	77
2.13.8	隔离输入与输出 Port	77
2.13.9	修复多 Port Net	78
2.13.10	控制向 Module、层次单元与 Voltage Area 添加新单元	78
2.13.11	为优化指定单元名前缀	79
2.14	指定布线前优化设置	79
2.14.1	控制 place_opt 的 DFT 优化	80
2.14.2	指定布线前优化的寄生参数估计设置	81
2.14.3	指定布线前优化的自动 Via Ladder 插入设置	81
2.14.4	在高利用率区域启用面积回收	83
2.14.5	启用高级逻辑重构	83
2.15	设置功耗相关功能	84
2.15.1	标注开关活动	84
2.15.2	为 place_opt 与 clock_opt 启用功耗优化	86
2.15.3	通过限制 LVT 单元百分比改善良率	88
2.15.4	更新 Activity 以改善功耗优化	88
2.15.5	启用电源完整性功能	89
2.16	指定布线资源	91
2.16.1	指定全局层约束	91

2.16.2	指定 Net 专用层约束	93
2.16.3	指定时钟树层约束	94
2.16.4	启用基于 Level 的时钟非默认布线规则	95
2.16.5	为层设置优选布线方向	95
2.17	使用 Early Data Check Manager 处理设计数据	96
2.18	应用 Mega-Switch 命令设置	97
2.18.1	应用先进工艺节点所需设置	97
2.18.2	应用高性能核所需设置	98
2.18.3	应用改善特定 QoR 指标所需设置	99
第三章	布局与优化	100
3.1	执行布局与优化	100
3.1.1	执行独立布局与合法化	101
3.1.2	使用 place_opt 命令执行布局与优化	101
3.1.3	使用 Design Compiler 工具的物理引导	106
3.1.4	执行多比特寄存器优化	107
3.1.5	执行 Magnet Placement	111
3.1.6	细化布局	113
3.1.7	对多电压 Block 执行布局与优化	113
3.1.8	重建缓冲树	114
3.1.9	添加与移除 Tie Cell	114
3.2	识别优化期间无法修复的问题	114
3.2.1	分析 Net 的可缓冲性	115
3.2.2	分析无法修复的违例	115
3.3	分析布局	116
3.3.1	报告利用率	116
3.3.2	报告布局 QoR	117
3.3.3	查询与更改布局状态	118
3.3.4	在 GUI 中分析布局	118
3.4	分析时序	119
3.5	分析功耗	119
3.5.1	为报告创建功耗组	120
3.5.2	报告基于引脚的时钟网络功耗	120
3.6	对比 QoR 数据	121
3.6.1	设置基线运行	122
3.6.2	更改 QoR 显示样式	123
3.6.3	排序与过滤数据	123
3.6.4	探索详细对比数据	125
第四章	时钟树综合	128
4.1	CTS 前提条件	128
4.2	定义时钟树	129
4.2.1	推导时钟树	129

4.2.2	定义时钟树例外	132
4.2.3	限制时钟网络上的优化	134
4.2.4	跨 Mode 复制时钟树例外	135
4.2.5	由理想时钟延迟推导时钟树例外	135
4.2.6	处理存在平衡冲突的端点	136
4.3	验证时钟树	137
4.4	设置时钟树设计规则约束	138
4.5	指定 CTS 设置	138
4.5.1	指定时钟树 References	139
4.5.2	设置 Skew 与 Latency 目标	140
4.5.3	启用局部 Skew 优化	140
4.5.4	指定 CTS 主 Corner	141
4.5.5	阻止综合特定时钟	141
4.5.6	保留既有时钟树	141
4.5.7	启用时钟树降功耗技术	141
4.5.8	启用线长感知的功耗驱动搬移	141
4.5.9	降低电迁移	142
4.5.10	处理不准确约束	142
4.5.11	定义时钟单元间距规则	142
4.5.12	创建 Skew Groups	143
4.5.13	定义时钟单元名前缀	143
4.5.14	初始 CTS 使用全局路由器	143
4.5.15	为时钟网络指定约束	143
4.5.16	为特定端点构建更快时钟树	144
4.5.17	改进延迟驱动的时钟门分裂	144
4.5.18	降低时钟网络上的信号完整性影响	145
4.5.19	指定时钟延迟调整设置	145
4.5.20	报告时钟树设置	145
4.6	实现时钟树并执行 CTS 后优化	145
4.6.1	独立时钟树综合	146
4.6.2	用 clock_opt 综合、优化并布线时钟树	146
4.6.3	控制并发时钟与数据优化	148
4.6.4	分裂时钟单元	152
4.6.5	平衡不同时钟树之间的 Skew	152
4.6.6	使用机器学习数据的基于全局布线优化	153
4.6.7	布线时钟树	154
4.6.8	在 CTS、优化与时钟布线中插入 Via Ladder	154
4.6.9	CTS 后将时钟标记为 Propagated	154
4.6.10	执行布线后时钟树优化	155
4.6.11	执行电压优化	155
4.6.12	将时钟树标记为已综合	155
4.6.13	移除时钟树	155
4.7	实现多源时钟树	157

4.7.1	多源时钟树结构简介	157
4.7.2	实现常规多源时钟树	158
4.7.3	使用集成 Tap Assignment 的常规多源时钟树	158
4.7.4	仅含 H-Tree 全局结构的常规多源时钟树	159
4.7.5	实现 MSCTS 多重 H-Tree	159
4.7.6	实现功耗感知柔性 H-Tree	159
4.7.7	实现结构型多源时钟树	159
4.7.8	使用集成子树综合的 SMSCTS	160
4.7.9	改善 SMSCTS 与 CCD 的相关性	160
4.7.10	插入时钟驱动器	160
4.7.11	综合全局时钟树	161
4.7.12	创建时钟 Strap	161
4.7.13	布线到时钟 Strap	162
4.7.14	分析 Clock Mesh	163
4.7.15	自动 Tap 插入与 H-Tree 综合	163
4.7.16	指定 Tap Assignment 选项与设置	164
4.7.17	构建本地时钟子树结构	164
4.8	分析时钟树结果	164
4.8.1	生成时钟树 QoR 报告	164
4.8.2	报告时钟树功耗	165
4.8.3	创建时钟网络引脚集合	165
4.8.4	分析时钟时序	165
4.8.5	在 GUI 中分析时钟树	166
第五章	布线与布线后优化	167
5.1	Zroute 简介	168
5.2	基本 Zroute 流程	168
5.3	布线前提条件	169
5.4	定义 Via	169
5.4.1	从 LEF 读取 Via 定义	170
5.4.2	创建 Via 定义	170
5.5	插入 Via Ladder	171
5.5.1	定义 Via Ladder 规则	172
5.5.2	为电迁移 Via Ladder 生成规则	172
5.5.3	为性能 Via Ladder 生成规则	173
5.5.4	基于约束的 Via Ladder 插入	173
5.6	检查可布线性	175
5.7	布线约束	176
5.7.1	定义 Routing Blockage	176
5.7.2	定义 Routing Guide	178
5.7.3	推导 Routing Guide	179
5.7.4	控制 Block 边界周围布线	180
5.7.5	在特定区域内布线 (Routing Corridor)	181

5.7.6	使用非默认布线规则 (NDR)	181
5.7.7	控制 Off-Grid 布线	184
5.7.8	Must-Join Pin 布线	184
5.7.9	控制 Pin 连接与 Tapering	185
5.7.10	设置重布线模式	186
5.7.11	定义与使用 Routing Corridor (详述)	186
5.7.12	布线层资源与冻结层 (章内相关)	186
5.7.13	天线规则与冗余 Via (章内要点)	187
5.8	布线应用选项	187
5.9	时钟网络布线	187
5.10	关键网络布线	188
5.11	次级电源与地 Pin 布线	188
5.11.1	验证属性	188
5.11.2	设置约束并布线	188
5.12	信号网络布线	189
5.12.1	全局布线	189
5.12.2	轨道分配	190
5.12.3	详细布线	190
5.12.4	自动布线	191
5.13	网络屏蔽	191
5.13.1	定义并分配屏蔽规则	192
5.13.2	布线前屏蔽	192
5.13.3	信号布线中的软屏蔽规则	192
5.13.4	布线后屏蔽与增量屏蔽	192
5.14	执行布线后优化	193
5.14.1	布线后逻辑优化	193
5.14.2	使用 hyper_route_opt	194
5.14.3	修复 Pin Access 引起的 DRC	194
5.15	分析并修复信号电迁移违例	194
5.16	执行 ECO 布线	195
5.17	在 GUI 中布线	195
5.18	清理已布线网络	196
5.19	分析布线结果	196
5.19.1	拥塞报告与拥塞图	196
5.19.2	用 Zroute 做设计规则检查	197
5.19.3	签核 DRC、外部工具与 LVS	197
5.19.4	报告布线结果与线长	197
5.19.5	DRC 查询命令	198
5.20	保存布线信息	198
5.21	推导掩模颜色	198
5.22	插入与移除 Cut Metal 形状	198

第六章 芯片收尾与可制造性设计	200
6.1 插入 Tap Cell	200
6.1.1 使用 <code>create_tap_cells</code> 命令	201
6.1.2 使用 <code>create_cell_array</code> 命令	203
6.1.3 插入 Exterior Tap Wall	204
6.1.4 插入 Interior Tap Wall	204
6.1.5 插入 Tap Mesh	205
6.1.6 插入 Dense Tap Array	206
6.2 执行 Boundary Cell 插入	206
6.2.1 指定 Boundary Cell 插入要求	206
6.2.2 Boundary Cell 插入期间创建 Routing Guide	209
6.2.3 Boundary Cell 插入期间创建 Placement Blockage	209
6.2.4 报告 Boundary Cell 插入要求	209
6.2.5 移除 Boundary Cell 插入要求	210
6.2.6 插入 Boundary Cell	210
6.2.7 验证 Boundary Cell 布局	210
6.3 查找并修复天线违例	211
6.3.1 定义天线规则	211
6.3.2 设置天线模式	216
6.3.3 指定天线属性	217
6.3.4 分析并修复天线违例	218
6.3.5 详细布线期间插入二极管	219
6.4 插入冗余 Via	220
6.4.1 在时钟网上插入冗余 Via	220
6.4.2 在信号网上插入冗余 Via	221
6.4.3 报告冗余 Via 率	224
6.5 优化线长与 Via 数量	224
6.6 减小临界面积	225
6.6.1 执行 Wire Spreading	225
6.6.2 执行 Wire Widening	226
6.7 插入金属-绝缘体-金属电容	226
6.8 插入 Filler Cell	227
6.8.1 标准 Filler Cell 插入	227
6.8.2 控制标准 Filler Cell 插入	228
6.8.3 检查 Filler Cell DRC 违例	228
6.8.4 修复剩余 Mask 设计规则违例	229
6.8.5 随机 Filler Cell 替换	232
6.8.6 用特定 Filler 邻接标准单元	233
6.8.7 控制基于单元的 Filler 插入	233
6.8.8 基于阈值电压的 Filler 插入	233
6.8.9 控制基于阈值电压的 Filler 插入	234
6.8.10 移除阈值电压 Filler 信息	234
6.9 插入 Metal Fill	234

第七章	IC Validator In-Design	235
第八章	使用 Custom Router 布线	236
第九章	相对布局物理数据通路	237
第十章	层次化实现	238
第十一章	RedHawk 与 RedHawk-SC Fusion	239
第十二章	ECO 流程	240

关于本用户指南

About This User Guide

本版本新增内容

关于新功能、增强与变更、已知限制，以及已解决的 Synopsys Technical Action Requests (STAR) 信息，请参阅 SolvNetPlus 站点上的 *IC Compiler II Release Notes*。

相关产品、文档与商标

有关 IC Compiler II 工具的更多信息，请参阅 Synopsys SolvNetPlus 支持站点上的文档：<https://solvnetplus.synopsys.com>

您可能还希望参阅以下相关 Synopsys 产品文档：

- Design Compiler®
- IC Validator
- PrimeTime Suite®
- StarRC™

排版约定

Synopsys 文档使用下列约定。

客户支持

客户支持通过 SolvNetPlus 提供。

访问 SolvNetPlus

SolvNetPlus 站点包含技术文章知识库以及对 Synopsys 工具常见问题的解答，并可访问软件下载、文档与技术支持等在线服务。

访问地址：<https://solvnetplus.synopsys.com>

如提示，请输入用户名与密码。若尚无 Synopsys 账号，请按页面说明注册。若使用 SolvNetPlus 时需要帮助，请点击右上角菜单栏中的 **REGISTRATION HELP**。

表 1: 文档排版约定

约定	说明
Courier	表示语法，例如 <code>write_file</code>
Courier italic	表示语法中的用户自定义值，例如 <code>write_file design_list</code>
Courier bold	表示示例中用户原样输入的内容，例如 <code>prompt> write_file top</code>
紫色高亮	- 在示例中表示特别关注的信息； - 在命令语法中表示默认值，例如 <code>include_enclosing = true false</code>
<code>[]</code>	表示语法中的可选参数，例如 <code>write_file [-format fmt]</code>
<code>...</code>	表示参数可按需重复，例如 <code>pin1 pin2 ... pinN</code>
<code> </code>	表示在多个备选中选择其一，例如 <code>low medium high</code>
<code>\</code>	表示命令行续行
<code>/</code>	表示目录层次
粗体	表示带有关联操作的图形用户界面（GUI）元素
Edit > Copy	表示菜单命令路径，例如打开 Edit 菜单并选择 Copy
Ctrl+C	表示组合键，例如按住 Ctrl 再按 C

联系客户支持

请访问 <https://solvnetplus.synopsys.com>。

包容性与多样性声明

Synopsys 致力于营造包容的环境，使每位员工、客户与合作伙伴都感到受欢迎。公司正在审查并从产品及面向客户的材料中移除排他性用语；内部亦推进从工程与工作环境中去除带有偏见的语言，包括嵌入软件与 IP 中的术语。同时，Synopsys 努力确保 Web 内容与软件应用可供不同能力的用户使用。由于部分 IP 实现仍遵循尚在审查中的业界标准规范，软件或文档中可能仍可见非包容性用语。

Chapter 1

使用 IC Compiler II 工具

Working With the IC Compiler II Tool

IC Compiler II 工具支持以下功能：

- 提取（extraction）与时序分析（timing analysis）
- 布局与优化（含相对布局 relative placement）
- 时钟树综合（clock tree synthesis）
- 布线（routing）
- 芯片收尾（chip finishing）
- 层次化设计的顶层收敛（top-level closure）
- 工程变更指令（ECO）
- 报告（reporting）
- ASCII 输出接口

工具以 Verilog 网表、时序约束、逻辑与物理库以及晶圆厂工艺数据为输入；输出可为 OASIS 或 GDSII 格式的版图，或可供第三方路由器使用的已布局网表 DEF（Design Exchange Format）文件。也可随时将设计输出为二进制设计数据库，或以 ASCII 文件形式输出（Verilog、DEF 与时序约束）。

本章包括以下主题：

- 布局布线设计流程概览
- IC Compiler II 相关概念
- 使用 3DIC Compiler 用户界面
- 输入 icc2_shell 命令
- 使用应用选项（application options）
- 使用变量
- 查看 man 页

- 使用 Tcl 脚本
- 通过检查点（checkpoints）向脚本添加变更
- 使用 setup 文件
- 使用命令日志文件
- 启用多核处理

参见

有关在 IC Compiler II 中处理设计数据，请参阅 *IC Compiler II Data Model User Guide*。

1.1 布局布线设计流程概览

Place and Route Design Flow Overview

图 1.1 给出使用 IC Compiler II 的基本布局布线设计流程。

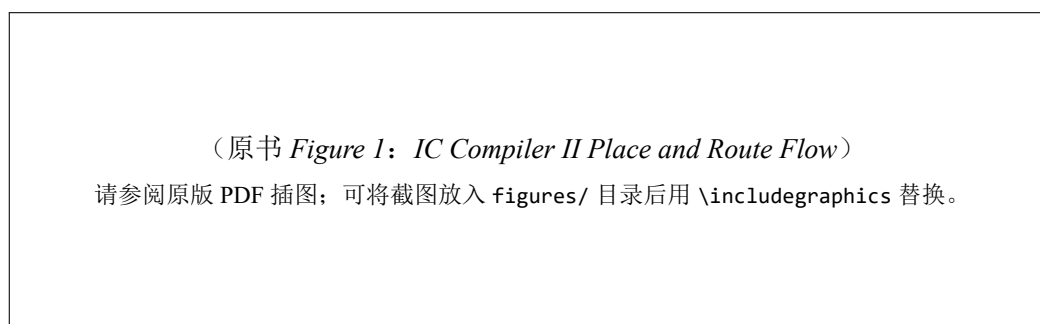


图 1.1: IC Compiler II 布局布线流程

运行 IC Compiler II 布局布线流程的步骤如下：

1. 设置库并准备设计数据，见第二章「准备设计」。

2. 执行设计规划与电源规划。

此时创建 floorplan，以确定设计尺寸、创建边界与 core 区域、为标准单元布局创建 site rows、设置 I/O pad，并创建电源计划。

更多信息见 *IC Compiler II Design Planning User Guide*。

3. 执行布局与优化。

使用 `place_opt` 命令。该命令面向并解决设计的时序收敛：通过增强的布局与综合技术，迭代生成标准单元的合法化布局与优化后的设计。还可补充功耗优化、布局面积回收、降低拥塞，以及最小化时序与设计规则违例。详见第三章。

4. 执行时钟树综合与优化。

使用 `clock_opt` 命令。ICC2 的 CTS 与内嵌优化可处理复杂时钟树问题（如 blockage 规避、布线前与布线后数据相关性等）。时钟树优化通过缓冲器/门的尺寸调整与搬移、电平调整、重构、插入延时、插入 dummy load，以及平衡时钟间延时等方式，改善 skew 与 insertion delay。详见第四章。

5. 执行布线与布线后优化，见第 五 章。

工具使用 Zroute 进行全局布线、track assignment、详细布线、拓扑优化与 ECO 布线。布线后优化使用 route_opt。对多数设计，默认设置即可；必要时可优化布线模式、降低串扰，或按特殊需求定制。

6. 执行芯片收尾与可制造性设计（DFM），见第 六 章。

可在流程各阶段应用芯片收尾、DFM 与可制造性（DFY）能力，以应对制造中的工艺设计问题。

7. 保存设计。

1.2 IC Compiler II 相关概念

IC Compiler II Concepts

本节介绍工具中的以下概念：

- 功耗意图（Power Intent）概念
- 多重图案化（Multiple-Patterning）概念

1.2.1 功耗意图概念

Power Intent Concepts

IC Compiler II 使用统一功耗格式（Unified Power Format, UPF）描述多电压设计的功耗意图。本节概述 UPF 概念及所支持的 UPF 流程。

UPF 概念

UPF Concepts

UPF 语言定义了一组命令，用于规定电子系统的低功耗设计意图。通过 UPF，可描述供电网络、开关、隔离、保持（retention）以及与芯片功耗管理相关的其他方面。同一套低功耗规范应贯穿设计、分析、验证与实现流程。Synopsys 工具遵循正式 UPF 标准。

UPF 规定设计的功耗需求，但不显式规定如何实现这些需求：它说明如何为各设计元素创建供电网络、供电网络之间的行为关系，以及如何扩展逻辑功能以支持对设计元素的动态电源开关；不包含布局或布线信息。

在 UPF 中，**power domain**（电源域）是逻辑层次中共享一组供电需求的元素集合。默认情况下，域内所有逻辑元素使用相同的主供电（**primary supply**）与主地（**primary ground**）；也可为域定义其他电源。电源域在物理版图上通常实现为连续的 **voltage area**，但这并非语言强制要求。

每个电源域有 **scope**（作用域）与 **extent**（范围）：**scope** 是电源域所在的逻辑层次；**extent** 是属于该域、共享相同供电需求的逻辑元素集合。换言之，**scope** 是电源域存在的层次，**extent** 是域内包含的内容。

每一 **scope**（层次）都有 **supply net**（供电网络）与 **supply port**（供电端口）。**supply net** 在给定电源域中承载供电电压或地；跨越多个电源域的 **supply net** 称为在多个域中「复用」。**supply port** 是相邻层次（父/子模块）之间的供电连接点；跨层次的 **supply net** 必须经过 **supply port**。

Power switch（电源开关）用于接通/关断某条 **supply net** 的供电，具有输入供电网、可开关的输出供电网，以及至少一个控制开关的输入信号；也可有多个控制输入与一个或多个应答输出。**Power state table** 列出设计中各电源域允许的电压值与开关状态组合。

当逻辑信号从一个电源域进入另一个电压明显不同的电源域时，必须插入 **level-shifter**（电平移位单元）以转换摆幅。当信号离开某域进入另一域时，必须插入 **isolation cell**（隔离单元），以便在关机期间产生确定逻辑值。若两域电压差异显著，接口单元需在上电时做电平移位、在关机时做隔离；能同时完成两者的单元称为 **enable level shifter**。

在具有电源开关的域中，关机期间需保持数据的寄存器必须实现为 **retention register**（保持寄存器），其具有独立的 **always-on** 供电（有时称 **backup supply**），在域主供电关闭时保持数据稳定。

UPF 流程

UPF Flows

IC Compiler II 同时支持传统 UPF 流程与 golden UPF 流程。Golden UPF 是维护多电压功耗意图的可选方法：在综合、物理实现与验证全过程中使用原始「golden」UPF 文件，并配合 Design Compiler 与 IC Compiler II 生成的 supplemental UPF 文件。

（原书 Figure 2: UPF-Prime / Traditional 与 Golden UPF 流程对比）

图 1.2: 传统 UPF 与 Golden UPF 流程

Golden UPF 流程在整条链路中保持并使用同一份原始 golden UPF；工具将功耗意图变更写入单独的 supplemental UPF。下游与验证工具使用 golden + supplemental 的组合，而不是单一的 UPF' 或 UPF'' 文件。

其优势包括：

- golden UPF 在流程中保持不变，保留原始结构、注释与通配命名；
- 可使用工具相关条件语句在不同工具中执行不同任务（传统 UPF-prime 流程中会丢失）；
- 功耗意图变更易于在 supplemental 文件中追踪；
- 可选地用 Verilog 网表存储全部 PG 连接信息，从而不必在 UPF 中使用 **connect_supply_net**，显著简化并缩小 UPF。

在加载 UPF 之前，用下列应用选项启用 golden UPF 流程：

```
icc2_shell> set_app_options -as_user_default \
-list {mv.upf.enable_golden_upf true}
```

加载 supplemental UPF 时，对 **load_upf** 使用 **-supplemental** 选项。更多信息见 SolvNet 文章 1412864, *Golden UPF Flow Application Note*。

1.2.2 多重图案化概念

Multiple-Patterning Concepts

在 20 nm 及以下工艺节点，现有光刻工具难以印制所需几何图形。为此采用 多重图案化：将版图掩模划分为两张或更多掩模，以提高制造间距、分辨率与可印制性。图 1.3 给出双图案化示例 (MASK A / MASK B)。

(原书 Figure 3: Double-Patterning Example)

图 1.3: 双图案化示例

使用多重图案化时，必须能将版图分解为满足多重图案化间距要求的两张或多张掩模。若某区域中相邻图形对数为奇数，且每对间距均小于多重图案化最小间距，则出现违例，称为 **odd cycle** (奇环)，见图 1.4。

(原书 Figure 4: Odd-Cycle Violation)

图 1.4: 奇环违例

若环中任一对间距大于多重图案化最小间距，则无违例，版图可分解（例如图 1.5 中 B 与 C 间距 x 足够大）。

(原书 Figure 5: No Odd-Cycle Violation)

图 1.5: 无奇环违例

IC Compiler II 在布局与布线中考虑多重图案化间距要求并防止奇环，使版图利于双图案化。一般仅在最底层金属 (multiple-patterning layers) 上执行双图案化；这些层上的金属形状（无论布线还是标准单元/宏内金属）须满足更严格间距；其他层不必。

多重图案化影响布局布线全流程。根据标准单元库，可走 **uncolored** 或 **precolored** 流程：

- **Uncolored** 流程：单元到边界有足够间距，布局时不会产生多重图案化违例。此类库称 **correct-by-construction library**（多数多重图案化库属此类）。工具为 pin 与 net shape 决定合适的 mask 设置。
- **Precolored** 流程：单元内部金属形状已分配 mask（常称 color），此类库称 **precolored library**。工具必须考虑这些分配，避免布局违例，并据此决定 pin/net 的 mask。Mask 分配在工具中表现为

mask constraints，须在开始布局布线前正确设置。

掩模约束

Mask Constraints

Mask constraints 表示采用多重图案化技术的 block 中物理 pin 与 net 金属形状的掩模要求，驱动布局与布线以保证版图合规。

注意

Mask constraints 仅用于 precolored 流程；uncolored 流程不需要。

可对时序关键 net（net shapes、routing rules、routing blockages）以及 via 设置 mask constraints。对 net，约束定义在 mask_constraint 属性；对 via，定义在 lower_mask、upper_mask、cut_mask。支持取值见表 1.1。

表 1.1: Mask Constraint 取值

属性值	说明
same_mask	掩模颜色尚未确定。带此属性的形状与任意已着色金属形状之间至少须满足多重图案化最小间距。
mask_one	形状为 mask1 色；与其他 mask1 形状之间至少须满足多重图案化最小间距。
mask_two	形状为 mask2 色；与其他 mask2 形状之间至少须满足多重图案化最小间距。
mask_three	形状为 mask3 色；与其他 mask3 形状之间至少须满足多重图案化最小间距。
any_mask	形状未着色；仅需满足标准最小间距，多重图案化最小间距规则不适用。

1.3 使用 3DIC Compiler 用户界面

Working With the 3DIC Compiler User Interfaces

IC Compiler II 在 Linux 上的 X Window 环境中运行，同时提供 shell 命令行界面（CLI）与 GUI。会话期间 CLI 始终可用；可在 shell 或 GUI 中启动/退出会话，也可在会话中打开或关闭 GUI。

工具使用 Tcl（EDA 业界广泛采用）。可用 Tcl 通过可复用过程与脚本扩展 icc2_shell 命令语言（见 *Using Tcl With Synopsys Tools*）。

本节说明如何用命令行启动与退出工具：

- 启动 CLI
- 退出 IC Compiler II

GUI 用法见 *IC Compiler II Graphical User Interface User Guide*。另见：启动 GUI、关闭 GUI。

1.3.1 启动 CLI

Starting the CLI

CLI 是纯文本环境，在提示符下输入命令，常用于脚本、批处理与一键流程。启动前请确保 `bin` 目录已在 `$PATH` 中。

启动方式：在 Linux shell 中输入

```
% icc2_shell
```

启动时可附加选项，例如：

- `-file script_file_name`: 执行脚本
- `-x command`: 执行一条命令
- `-output_log_file file_name`: 创建会话日志
- `-help`: 列出可用选项（不启动 shell）

启动时工具依次：

1. 创建命令日志文件；
2. 读取并执行 `setup` 文件；
3. 执行命令行 `-file/-x` 指定的脚本或命令；
4. 显示程序头与 `icc2_shell>` 提示符。

参见

使用命令日志文件；使用 `setup` 文件；使用 Tcl 脚本。

1.3.2 退出工具

Exiting the IC Compiler II Tool

可用 `exit` 或 `quit` 随时结束会话。

注意

从命令行退出时，工具不会保存已打开的 block。

1.4 输入 `icc2_shell` 命令

Entering `icc2_shell` Commands

与工具交互使用基于 Tcl 并带有 ICC2 扩展的 `icc2_shell` 命令。命令语言提供类似 Linux shell 的变量、条件执行与流程控制。你可以：

- 在 `icc2_shell>` 提示符下交互输入；
- 在 GUI 控制台命令行交互输入；

- 运行包含 `icc2_shell` 命令的 Tcl 脚本。

输入命令、选项或文件名时，可在字符足以唯一确定名称后按 **Tab** 自动补全；若有多个候选，工具会列出并由方向键与 **Enter** 选择。

若需复用命令行输出中的内容，可选中后移到命令行并用鼠标中键粘贴。

运行命令时，输出（含处理信息、警告与错误）会回显到 `icc2_shell`；若 GUI 打开，也会出现在控制台日志视图。默认非分页模式，输出可能滚动。要启用分页，将 `sh_enable_page_mode` 设为 `true`。

1.4.1 中断或终止命令处理

Interrupting or Terminating Command Processing

若选项或命令有误，可中断处理并留在 `icc2_shell` 中：按 **Ctrl+C**。

某些命令/进程不可中断，须在系统级终止 `icc2_shell`；终止进程或 shell 时不保存任何数据。

使用 **Ctrl+C** 时注意：

- 若正在处理脚本且中断其中一条命令，脚本处理被中断，后续脚本命令不再执行；
- 若在命令响应中断前连续按三次 **Ctrl+C**，`icc2_shell` 会被中断并以如下信息退出：

Information: Process terminated by interrupt.

1.4.2 获取命令信息

Getting Information About Commands

在线资源包括：

- 命令帮助（`command help`）
- Man pages

显示命令帮助

Displaying Command Help

命令帮助可以是命令的简要说明，或其所支持选项与参数的列表。

- 简要说明：`help` 后跟命令名，例如

```
icc2_shell> help report_timing
```

- 选项列表：命令名加 `-help`，例如

```
icc2_shell> report_timing -help
```

1.5 使用应用选项

Using Application Options

应用选项（application options）控制工具行为，命名约定为：

`category[.subcategory].option_name`

其中 **category** 为受影响引擎名；部分类别还有子类别。

作用域分为：

- **Block** 作用域：仅作用于设置所在的 **block**，保存在设计库中，跨会话持久。
- **Global** 作用域：作用于所有 **block**，但仅在当前会话有效，不写入设计库；每个 `icc2_shell` 会话都需重新指定。可将全局设置加入 `.synopsys_icc2.setup`。

列出可用应用选项：`get_app_options`（默认可列出全部；可提供 **pattern** 限制）。

```
icc2_shell> get_app_options
icc2_shell> get_app_options timer.*
```

生成报告：`report_app_options`。详细说明见 *IC Compiler II Data Model User Guide* 中的 **Application Options**。

参见

使用 `setup` 文件。

1.6 使用变量

Using Variables

一般而言，ICC2 通过应用选项而非应用变量修改默认行为；但仍支持用户定义的 Tcl 变量，以及少量应用变量（如 `search_path`）。

列出变量及其值：`printvar`。

```
icc2_shell> printvar *
icc2_shell> printvar search_path
```

参见

定义搜索路径（Defining the Search Path）。

1.7 查看 Man 页

Viewing Man Pages

显示命令或应用选项的 **man** 页：`man` 后跟名称。例如：

```
icc2_shell> man report_timing
```

也可查看应用选项摘要页：

- 类别摘要：`man category_options`，例如

```
icc2_shell> man timer_options
```

- 子类别摘要: `man category.subcategory_options`, 例如

```
icc2_shell> man route.common_options
```

- 命令相关摘要: `man command_options`, 例如

```
icc2_shell> man report_timing_options
```

在 `icc2_shell` 中输入 `man` 时, `man` 页显示在 `shell` 中 (GUI 打开时亦在控制台日志); 在 GUI 控制台输入时, 显示在 GUI `man` 页查看器中。

1.8 使用 Tcl 脚本

Using Tcl Scripts

可用 Tcl 脚本完成例行、重复或复杂任务: 将 ICC2 命令序列放入文本文件即可; 任何 ICC2 命令都可在脚本中执行。

Tcl 中以 `#` 开头的行为注释, 例如:

```
# This is a comment
```

编写脚本详见 *Using Tcl With Synopsys Tools*。运行脚本的方式:

- 启动时使用 `icc2_shell -file`;
- 在命令行使用 `source`;
- GUI 中选择 File > Execute Script。

命令出错时工具会抛出 `TCL_ERROR` 并立即停止脚本。若要容忍错误并继续执行, 可:

- 对可能出错的命令用 `Tcl catch` 检查 `TCL_ERROR`;
- 或在脚本中将 `sh_continue_on_error` 设为 `true`。

参见

启动 CLI; 通过检查点向脚本添加变更。

1.9 通过检查点向脚本添加变更

Adding Changes to a Script With Checkpoints

在 `block` 上做实验时, 可能以两种方式修改 `golden flow` 脚本:

- 流程变更: 设置应用选项、修改约束或其他改进 QoR 的改动;
- 生成报告: 增加报告或在流程新位置报告, 便于调试。

多数流程通过少量静态可改文件施加变更。典型 `place_opt` 流程示例: `source pre_place_opt_settings.tcl` 做流程改动, 末尾 `source generate_reports.tcl` 统一报告:

```
open_block ./design:init_design
source ./scripts/pre_place_opt_settings.tcl
remove_buffer_trees -all
place_opt -from initial_place -to initial_drc
create_placement -incremental -timing_driven -congestion
place_opt -from initial_drc -to initial_opto
place_opt -from final_place -to final_opto
source ./scripts/generate_reports.tcl
```

复杂改动有时需要直接改 `golden` 脚本，这在许多设计环境中不被鼓励或很难做，尤其当需在多阶段、多 `block` 或多组实验中实施时。

检查点（**checkpoint**）系统可简化该过程，使你无需修改 `golden flow` 脚本即可实验：在 `golden flow` 的重要步骤处定义 `checkpoint`，包裹相关命令。默认仅运行所包裹代码——单纯插入 `checkpoint` 不会改变流程。但可将 `checkpoint` 与「在包裹代码之前 / 之后 / 替代」执行的流程变更或报告关联；这些关联定义在可移植配置文件中，从而把 `golden` 脚本与某次运行的变更隔离。

同一配置文件可复制到各设计的 `run` 目录，从而在不修改各设计 `golden` 脚本的情况下统一施加变更。多用户项目中，每人可有独立的 `checkpoint.config.tcl`。

检查点系统还支持：

- 比非 `checkpoint` 脚本更精确地插入流程变更与报告；
- 以单一真相来源查看对默认 `golden flow` 的全部变更；
- 报告已运行 `checkpoint` 的历史，含当时执行的变更/报告；
- 报告各 `checkpoint` 的 `runtime` 与 `memory`；
- 生成唯一且描述性的报告名。

一般步骤：

1. 在脚本中插入 `checkpoints`；
2. 配置：定义要关联的流程变更/报告，并与 `checkpoints` 关联；
3. 运行脚本。

1.9.1 定义检查点

Defining Checkpoints

使用 `eval_checkpoint`，指定唯一名称并包裹代码块。

注意：

- 若新定义名称与已有 `checkpoint` 重名，运行遇到时工具会自动加唯一后缀（如 `placement` → `placement2`）；
- 不要嵌套 `checkpoint`：工具虽不阻止定义，但运行时忽略嵌套 `checkpoint`，只执行最外层（嵌套内的 Tcl 仍会执行，只是仿佛没有 `checkpoint`）。

示例：为 `golden flow` 各主要步骤包裹 `checkpoint`：

```
open_block ./design.nlib:init_design
```

```

source ./scripts/pre_place_opt_settings.tcl

eval_checkpoint remove_buffers {
    remove_buffer_trees -all
}

eval_checkpoint place_opt_to_initial_drc {
    place_opt -from initial_place -to initial_drc
}

eval_checkpoint incr_placement {
    create_placement -incremental -timing_driven -congestion
}

eval_checkpoint place_opt_to_initial_opto {
    place_opt -from initial_drc -to initial_opto
}

eval_checkpoint place_opt_to_final_opto {
    place_opt -from final_place -to final_opto
}

```

默认仅运行包裹代码；关联行为见下一小节。

1.9.2 配置检查点

Configuring Checkpoints

要使 checkpoint 影响流程，须将其与「之前 / 之后 / 替代」执行的流程变更或报告关联。

定义检查点行为

Defining Checkpoint Behaviors

可定义两类行为：**reports** 与 **actions**。Report 用于生成工具支持的报告或自定义报告；Action 用于在 checkpoint 包裹代码之前、之后或替代执行任意流程变更（例如在某 checkpoint 前设置应用选项）。

这些行为定义在检查点系统自动 source 的 checkpoint.config.tcl 中。

定义 Checkpoint Reports 使用 create_checkpoint_report，指定唯一名称与生成报告的 Tcl:

```

create_checkpoint_report timing {
    set name [get_current_checkpoint -name]
    set pos [get_current_checkpoint -position]

    report_qor -nosplit > ./checkpoint/$name.$pos.qor.rpt
    report_qor -summary -nosplit >> ./checkpoint/$name.$pos.qor.rpt

    report_timing -nosplit -max_paths 10 \
        > ./checkpoint/$name.$pos.path.rpt
}

```

```
}
```

注意用 `get_current_checkpoint -name/-position` 为报告命名：触发的 checkpoint 名以及 before/after 位置会体现在文件名中（例如 `checkpoint_A.after.qor.rpt`）。

再如写出全部非默认应用选项：

```
create_checkpoint_report app_options {
  set name [get_current_checkpoint -name]
  set pos [get_current_checkpoint -position]
  report_app_options -non_default \
    > ./checkpoint/$name.$pos.app_options.rpt
}
```

定义 Checkpoint Actions 使用 `create_checkpoint_action`。例如启用基于全局布线的 buffering：

```
create_checkpoint_action gropto {
  set_app_options -name place_opt.initial_drc.global_route_based \
    -value true
}
```

高努力拥塞缓解示例（用 `get_current_checkpoint -script` 取回原包裹命令并追加选项）：

```
create_checkpoint_action placer_high_effort_cong {
  set placer_command [get_current_checkpoint -script]
  set cong_option "-congestion_effort high"
  eval $placer_command $cong_option
}
```

关联检查点与行为

Associating Checkpoints and Checkpoint Behaviors

定义 checkpoint 与 behavior 后，须用关联命令使运行遇到 checkpoint 时执行行为。

关联 Reports 使用 `associate_checkpoint_report`：

- `-enable`：报告名；
- `-before / -after`：在指定 checkpoint 之前/之后运行；
- 多个 checkpoint 可列名或用 * 通配。

示例：

```
associate_checkpoint_report -enable timing \
  -after { place_opt_to_initial_opto place_opt_to_final_opto }

associate_checkpoint_report -enable app_options -before *
```

关联 **Actions** 使用 `associate_checkpoint_action`:

- `-enable`: action 名;
- `-before` / `-after` / `-replace`: 之前、之后或替代包裹代码。

示例:

```
associate_checkpoint_action -enable gropto \
    -before place_opt_to_initial_drc

associate_checkpoint_action -enable placer_high_effort_cong \
    -replace incr_placement
```

同一 **Checkpoint** 关联多个行为时的执行顺序

1. 配置为 `before` 的 reports;
2. 配置为 `before` 的 actions;
3. checkpoint 包裹代码, 或配置为 `replace` 的 actions;
4. 配置为 `after` 的 actions;
5. 配置为 `after` 的 reports。

同一位置有多个 report/action 时, 按 `associate_checkpoint_*` 关联的先后顺序执行。

1.9.3 查询检查点与行为

Querying Checkpoints and Checkpoint Behaviors

使用 `get_checkpoint_data`:

```
icc2_shell> get_checkpoint_data -list_names
icc2_shell> get_checkpoint_data -name place_opt_to_initial_drc
icc2_shell> get_checkpoint_data -list_reports
icc2_shell> get_checkpoint_data -list_actions
icc2_shell> get_checkpoint_data -report app_options
```

1.9.4 查看检查点历史

Viewing Your Checkpoint History

进入 checkpoint 目录: 其中 `checkpoint_history.rpt` 记录每次运行遇到的 checkpoint 及 run-time/memory; 该目录也存放写出的 checkpoint reports。

清除历史:

- `reset_checkpoints`: 清空整个 checkpoint 目录 (含报告); 清空前会在 `run` 目录保存带时间戳的副本;
- `reset_checkpoints -from <name>`: 从指定 checkpoint 起 (含) 从历史中移除后续条目, 但不删除已写出的报告文件。

1.10 使用 Setup 文件

Using Setup Files

启动时工具自动执行 `.synopsys_icc2.setup` 中的命令。会在家目录与项目目录（启动时的当前工作目录）中查找，读取顺序为：

1. 家目录中的 `.synopsys_icc2.setup`
2. 项目目录中的 `.synopsys_icc2.setup`

Setup 文件可包含初始化应用选项、设置 GUI 选项等基本任务。例如：

- 在家目录 `setup` 中设置定义工作环境的应用选项；
- 在设计目录 `setup` 中设置影响某 block 处理的项目/block 专用选项。

1.11 使用命令日志文件

Using the Command Log File

命令日志记录工具处理的命令，包括 `setup` 文件命令与应用选项设置。默认写入调用 `icc2_shell` 的目录下的 `icc2_command.log`。

可在 `.synopsys_icc2.setup` 中设置 `sh_command_log_file` 修改变量名；须在启动工具之前修改。若 `setup` 未含该变量，工具自动创建 `icc2_command.log`。

每次会话会覆盖命令日志；要保存请移动或重命名。可用于：生成某实现策略的脚本、记录物理实现过程、记录遇到的问题。

1.12 启用多核处理

Enabling Multicore Processing

若干功能支持多核处理：多线程、分布式处理或并行命令执行。并行可显著缩短周转时间。

注意

单台机器有一个或多个 CPU，每个 CPU 有一个或多个核；可用总核数 = CPU 数 × 每 CPU 核数。

使用多核时，每 8 个并行任务需要 1 个 IC Compiler II license（例如 16 个并行任务需 2 个 license）。

多数情况下用 `set_host_options` 配置。相关主题：配置多线程、配置分布式处理、报告配置、移除配置；层次设计上对多 block 并行跑同一任务；在本地主机并行运行检查/报告命令。

1.12.1 配置多线程

Configuring Multithreading

多线程在同一机器的多核上并行，共享单一进程内存映像；每个并行任务称为 `thread`。为最佳性能，线程数应不超过可用核数。

支持由 `set_host_options` 配置多线程的命令包括: `place_opt`、`clock_opt`、启用 `advanced legalizer` 时的 `check_legality`、`insert_via_ladders`、`route_auto`/`route_global`/`route_track`/`route_detail`/`route_eco`、`route_opt`、`signoff_check_drc`、`signoff_fix_drc`、`signoff_create_metal_fill`、`signoff_fix_isolated_via`、`write_def` 等。

注意

单线程布线结果确定；多线程时因任务划分不同，布线结果不确定。全局布线支持确定性多核模式：将 `route.global.deterministic` 设为 `on`。

默认所有命令单线程。对支持多线程的命令，将 `set_host_options -max_cores` 设为大于 1 且不超过可用核数的值；该值作用于所有支持多线程的命令。

启用后，即使超过可用核数，命令仍会创建并使用指定线程数——过度订阅会降低性能。最佳实践：每可用核不超过一个线程。例如双 CPU、每 CPU 三核：

```
icc2_shell> set_host_options -max_cores 6
```

1.12.2 配置分布式处理

Configuring Distributed Processing

分布式处理使用多台机器；每个进程有独立内存映像。每个并行任务称为 `process`。可用进程数不应超过各主机核数之和。

支持分布式的命令包括: `analyze_rail`、`create_placement -floorplan`、`signoff_check_drc`、`signoff_fix_drc`、`signoff_create_metal_fill`、`signoff_fix_isolated_via` 等。

可指定: 作业提交命令 (`-submit_command`, 默认 `rsh`)、主机列表、最大进程数 (`-num_processes`)。可用 `-name` 命名配置以便选用或删除。例如用 `qsub`:

```
icc2_shell> set_host_options -name dp_config \
-submit_command [list qsub -P bnormal -cwd]
```

1.12.3 报告与移除多核配置

Reporting / Removing Multicore Configurations

报告: `report_host_options`。移除: `remove_host_options (-all 或 -name)`。

1.12.4 并行运行任务

Running Tasks in Parallel

要对设计中多个 `block` 高效执行同一任务，可用 `run_block_script`: 接受 Tcl 脚本并应用到指定 `block`。用 `-run_order top_down` 或 `bottom-up` 控制处理顺序（默认 `bottom-up`）。

1.12.5 在本地主机并行运行命令

Running Commands in Parallel on Your Local Host

可通过后台（`redirect -bg`）或前台并行（`parallel_execute`）运行检查与报告命令，以缩短 runtime。这些技术不要求超出父进程所需的额外 license，也不要求配置分布式处理。

建议：

- 并行前先跑 `update_timing`，否则各需更新时序的命令会各自独立跑；
- 子进程变量传回父进程时，须在子进程写入文件，再由父进程读取。

后台运行命令

Running Commands in the Background

使用 `redirect -bg:icc2_shell` 立即返回执行下一条命令。父进程 `exit` 时会等待所有 `redirect -bg` 完成。

用 `list_commands -bg` 查看支持的命令。可跑单条命令或仅含支持命令的 Tcl；脚本内若再含 `redirect -bg`，其 `-bg` 会被忽略。最多同时两个后台任务，超出则排队。

用 `redirect -max_cores` 指定后台可用核数；父进程可用核数（由 `set_host_options -max_cores` 指定）会相应减少。

```
icc2_shell> set_host_options -max_cores 8
icc2_shell> redirect -bg -max_cores 3 -file bg_log.out \
    {source bg_script.tcl}
```

报告后台任务：`report_background_jobs`；`-reset` 可省略已完成任务。

并行运行命令

Running Commands in Parallel

使用 `parallel_execute`，在并行列表中最长命令完成后返回父进程，适合批处理脚本：

```
icc2_shell> update_timing
icc2_shell> parallel_execute {
    report_cmd1 log_file1
    report_cmd2 log_file2
    report_cmd3 log_file3
}
```

用 `-list_allowed_commands` 查看支持的命令。

参见

本章后续章节将展开设计准备、布局优化、CTS、布线等实现细节。命令与应用选项以原版 `man` 页及 `SolvNetPlus` 为准。

Chapter 2

准备设计

Preparing the Design

IC Compiler II 工具使用设计库（design library）来存储设计及其相关库信息。本节说明如何创建设计库，以及如何准备并保存设计。

这些步骤在以下主题中说明：

- 定义搜索路径（Defining the Search Path）
- 设置库（Setting Up Libraries）
- 使用 Pin Access Checker 实用程序
- 处理设计（Working With Designs）
- 导入 floorplan 信息
- 设置多电压设计（Setting Up Multivoltage Designs）
- 指定时序约束与设置
- 指定逻辑设计规则约束
- 控制时钟门控延迟（Clock-Gate Latencies）
- 为布局与合法化指定物理约束
- 指定布局设置
- 指定合法化设置
- 控制单元、net、pin 与 port 的优化
- 指定布线前优化设置
- 为功耗相关功能做准备
- 指定布线资源
- 使用 Early Data Check Manager 处理设计数据
- 应用 Mega-Switch 命令设置

2.1 定义搜索路径

Defining the Search Path

IC Compiler II 工具使用搜索路径（search path）查找以相对路径指定、或未指定路径的文件。

要用 `set_app_var` 命令将 `search_path` 应用变量设为按顺序查找文件的目录列表。工具查找文件时，从 `search_path` 中最左侧的目录开始，使用找到的第一个匹配文件。

也可用 Tcl 的 `lappend` 命令把目录追加到默认搜索路径（即启动工具时所在目录）。例如：

```
icc2_shell> lappend search_path ./mylibdir
```

2.2 设置库

Setting Up Libraries

block（块）是物理与功能设计数据的容器。**design library**（设计库）是一组相关 **block** 的集合，并附带适用于该 **block** 集合的工艺数据。芯片设计由一个或多个 **block** 组成，这些 **block** 常存储在不同设计库中。设计库使用在较低层库中定义的 **block** 实例，这些较低层库称为 **reference libraries**（参考库）。一个设计库也可作为另一设计库的参考库。

关于设置库，见以下主题：

- 处理设计库（Working With Design Libraries）
- 设置参考库（Setting Up Reference Libraries）
- 库配置（Library Configuration）
- 限制库单元使用（Restricting Library Cell Usage）
- 限制所用目标库（Restricting the Target Libraries Used）

2.2.1 处理设计库

Working With Design Libraries

可用绝对路径、相对路径或不指定路径，通过下列命令创建、打开、查询、保存或关闭设计库：

- **create_lib**

该命令在内存中创建库，并将其设为当前库。创建新设计库时必须指定库名。库名中不允许使用斜杠（/）与冒号（:）。下列命令用相对路径创建 `my_libA` 库：

```
icc2_shell> create_lib ../my_lib_dir/my_libA
{my_libA}
```

- **open_lib**

打开指定库，将其设为当前库，并打开其全部关联参考库。打开库意味着将其载入内存并使其 **block** 可访问。下例打开磁盘上已保存的 `my_libA`：

```
icc2_shell> open_lib my_libA
Information: Loading library file '/usr/lib/StdCells.ndm' (FILE-007)
```

```
Information: Loading library file '/usr/lib/RAMs.ndm' (FILE-007)
Information: Loading library file
'/usr/lib/PhysicalOnly.ndm' (FILE-007)
{my_libA}
```

- **current_lib**

默认情况下，最近打开的库为当前库。可用 **current_lib** 显式将任一已打开库设为当前库。例如：

```
icc2_shell> current_lib my_libA
{my_libA}
```

- **save_lib**

创建或修改库时，变更仅保存在内存中。要保存到磁盘，使用本命令。例如：

```
icc2_shell> save_lib lib_A
Saving library 'lib_A'
1
```

- **close_lib**

不再需要访问某库中的数据时，可用 **close_lib** 关闭。关闭前请务必保存库中的更改。例如：

```
icc2_shell> close_lib
Closing library 'lib_A'
1
```

此外，可用 **current_lib**、**get_libs** 与 **report_lib** 命令查询设计库。

参见

更多信息见 *IC Compiler II Data Model User Guide* 中的 Design Libraries 主题。

2.2.2 设置参考库

Setting Up Reference Libraries

创建设计库时，可用 **create_lib** 的 **-ref_libs** 选项为其指定参考库列表；也可随时用 **set_ref_libs** 更改参考库列表。

用下列命令指定、重新绑定（rebind）并报告参考库：

- **create_lib -ref_libs**

可用 **create_lib** 指定新设计库与其较低层参考库之间的关系。例如：

```
icc2_shell> create_lib lib_B \
-ref_libs {../LIBS/lib_c ../STND/stdhvt.ndm} ...
{lib_B}
```

- **set_ref_libs -ref_libs**

对已有设计库，先打开库，再用 **set_ref_libs** 指定参考库。例如：

```
icc2_shell> current_lib
{lib_B}
icc2_shell> set_ref_libs \
    -ref_libs {../LIBS/lib_C ../STND/stdhvt.ndm}
../LIBS/lib_C ../STND/stdhvt.ndm
```

- **report_ref_libs**

用 **report_ref_libs** 报告设计库的参考库。例如：

```
icc2_shell> create_lib lib_A -ref_libs \
    {../libs/SCLL.ndm ../libs/SCHH.ndm ../BLOCKS/MACROS}
{lib_A}
icc2_shell> report_ref_libs...
Name Path Location
-----
*+ SCLL ../libs/SCLL.ndm /remote/project/libs/SCLL.ndm
*+ SCHH ../libs/SCHH.ndm /remote/project/libs/SCHH.ndm
* MACROS ../BLOCKS/MACROS /remote/project/BLOCKS/MACROS
"*" = Library currently open
"+" = Library has technology information
```

- **set_ref_libs -rebind**

当参考库列表失效时（例如把参考库移到新位置），需要重新绑定参考库。使用 **-rebind** 选项，会把 **search_path** 变量所指定的各参考库路径重新绑定到当前已载入内存的库。例如：

```
icc2_shell> current_lib
{lib_A}
icc2_shell> set_app_var search_path { . ../REFLIBS ../CLIBS}
. ../LIBS ../BLOCKS
icc2_shell> set_ref_libs -rebind
../REFLIBS/lib_C ../REFLIBS/lib_D ../CLIBS/stdhvt.ndm}
```

重新绑定库不会影响设计库中已有 **block** 的绑定。要用更新后的参考库列表重新绑定这些 **block**，对 **link_block** 使用 **-rebind** 选项。

2.2.3 库配置

Library Configuration

库配置允许你指定将哪些厂商库用作当前设计的参考库。通过 **search_path** 与 **link_library** 变量指定技术文件、物理库与逻辑库，再用 **create_lib** 或 **set_ref_libs** 组装单元库。

在库配置过程中：

- IC Compiler II 工具自动调用 Library Manager，无需用户干预即可生成单元库，如下图所示：
- 工具将生成的单元库保存到磁盘，并加入设计库的参考库列表。
- 这些单元库与在 Library Manager 中进行库准备时所创建的单元库相同。

(原书示意: *.db / frame / ASCII technology file → Cell libraries → Synthesis*)

库配置流程示意; 请参阅原版 PDF。

图 2.1: 库配置中生成单元库

参见

更多信息见 *IC Compiler II Data Model User Guide* 中的 *Configuring Cell Libraries* 主题。

2.2.4 限制库单元使用

Restricting Library Cell Usage

默认情况下, 对 block 执行优化或时钟树综合时, IC Compiler II 可使用单元库中全部可用库单元。要限制单元使用, 在 block 已载入内存后使用 `set_lib_cell_purpose`。该命令具有 block 作用域, 并在工具会话间保持; 重新打开 block 时会恢复这些设置。

要用 `-include` 与 `-exclude` 选项指定工具可以使用或不可以使用库单元的方式。这两个选项均可接受下列一个或多个取值: `all`、`cts`、`hold`、`optimization`、`none`。

- `-include` 在指定库单元上设置 `included_purposes` 属性。
- `-exclude` 在指定库单元上设置 `excluded_purposes` 属性。

注意

若库单元带有 `dont_use` 属性, 则被排除在所有用途之外, 效果等同于对该单元执行 `set_lib_cell_purpose -include none`。

执行优化 (含 `setup`、`hold` 与逻辑 DRC 修复) 时, 工具可使用 `included purpose` 为 `optimization`、`hold` 或二者兼有的库单元。执行时钟树综合时, 可使用 `included purpose` 为 `cts` 的库单元。

例如, 禁止某组库单元用于所有用途:

```
icc2_shell> set_lib_cell_purpose -include none lib_cells
```

仅允许某组库单元用于时钟树综合:

```
icc2_shell> set_lib_cell_purpose -include none lib_cells
icc2_shell> set_lib_cell_purpose -include cts lib_cells
```

允许某组库单元用于除时钟树综合外的所有用途:

```
icc2_shell> set_lib_cell_purpose -exclude cts lib_cells
```

2.2.5 限制所用目标库

Restricting the Target Libraries Used

默认情况下，优化期间工具可从目标库中选择任意库单元。某些情况下，你可能希望限制用于时钟树综合与优化的库单元。例如，希望在某些设计 block 的优化中排除特定双高度（double-height）单元。

要限制用于时钟树综合与优化的库单元：

1. 用 `set_target_library_subset` 指定库子集。

默认情况下，库子集限制适用于：

- 顶层 block 及其全部子 block。要对特定子 block 设置，使用 `-objects` 选项。
- 时钟路径与数据路径二者。要仅对时钟或数据路径设置，分别使用 `-clock` 或 `-data`。

还可进一步限制目标库子集设置：

- 用 `-dont_use` 指定库中不应使用的单元列表。
- 用 `-only_here` 指定这些库除指定对象外，不得用于任何其他对象。

2. 将 `opt.common.enable_target_library_subset_opt` 应用选项设为 1，以启用目标库子集。

设置目标库子集时请注意：

- 子集限制适用于层次单元，但不适用于叶子单元（leaf cells）。
- 命令对指定 block 及其层次中的子设计强制子集限制，但已设置不同子集限制的子设计除外。
- 较低层指定的子集会覆盖较高层指定的子集。

（原书 *Figure 6: Logic Hierarchy of Design*）

图 2.2: 设计的逻辑层次

例如，假设设计逻辑层次如图 2.2，希望在优化与时钟树综合中实现如下库限制：

- 对 Sub1 block 及其子 block SubA、SubB，仅使用名为 `LVT_lib` 的库中的单元。
- 设计其余部分不得使用该库中的单元。

为此使用下列设置：

```
icc2_shell> set_target_library_subset -objects {top/Sub1} \
    -only_here [get_lib_cells LVT_lib/*] [get_libs LVT_lib]
icc2_shell> set_app_options \
    -name opt.common.enable_target_library_subset_opt -value 1
```

除上述设置外，假设还指定：

```
icc2_shell> set_lib_cell_purpose -include cts \
    {HVT_lib/buf1 HVT_lib/buf2 LVT_lib/buf1 LVT_lib/buf2}
```

则在时钟树综合期间于时钟网络上添加缓冲器时，工具会：

- 对名为 Sub1 的 block 及其子 block，使用 LVT_lib 中的 buf1、buf2；
- 对设计其余部分，使用 HVT_lib 中的 buf1、buf2。

报告目标库子集

Reporting Target Library Subsets

要查明已为顶层 block 或层次单元定义了哪些目标库子集，使用 `report_target_library_subset`。

由 `report_cells`、`report_timing` 等报告命令生成的报告中，对由 `-dont_use` 或 `-only_here` 指定的单元会附带 `td` 属性。

移除目标库子集

Removing Target Library Subsets

要从顶层 block 或层次单元移除目标库子集限制，使用 `remove_target_library_subset`。

2.3 使用 Pin Access Checker 实用程序

Using Pin Access Checker Utility

Pin access checker (PAC) 实用程序帮助你在库开发阶段识别 pin 可访问性问题。这有助于改进单元版图——库最终确定并发布给布局布线团队后，将无法再修改单元版图。

布线 DRC 会在布线后识别 pin 访问问题。当单元邻接 (abutted) 或放置过近时，两个或多个相邻 pin 会争夺公共布线资源，从而引起布线 DRC。

PAC 实用程序的好处包括：

- 生成带有多种邻接单元布局排列的布局布线数据库。
- 生成布局布线数据库，并根据报告评估所建库的可布局性与 pin 可访问性。

PAC 执行下列检查：

- 单元布局
- 每个库单元 pin 的可访问 track 数量
- 布局后的合法化检查
- 布线后的设计规则检查
- 上述全部检查 (含 PG 布线)

可用 `create_pin_check_lib` 根据输入的技术文件与参考库创建或打开新库。PAC 在该库中创建并存储 block。必须通过下列应用选项与 `create_pin_check_lib` 提供至少三项输入：

- 技术文件：用 `-technology` 提供技术文件信息。

- 参考库：用 `-ref_libs` 提供参考库信息。
- 金属层布线方向：在 Tcl 脚本中用 `set_attribute [get_layers Mx] routing_direction horizontal/vertical` 定义布线方向。必须用 `pin_check.place.preplace_option_file` 应用选项指定该脚本名。

要启用 `pin access check` 实用程序，可使用 `check_libcell_pin_access`。可用该命令与 `pin_check.*` 应用选项启用下列 `pin` 访问检查：

- Legalization
- Router
- 自动推导单元间距规则（Automatic derivation of cell spacing rule）

对 `check_libcell_pin_access` 使用 `-mode`，通过指定不同 `mode` 执行多种检查。

```
create_pin_check_lib myTest.nlib -ref_libs ../lib/refLib.ndm \
-technology ../tf/refTech.tf

set_app_options -as_user_default -name \
pin_check.place.preplace_option_file -value

check_libcell_pin_access -mode analyze_lib_cell

check_libcell_pin_access -mode analyze_lib_pin

check_libcell_pin_access -mode design_under_test
```

若此前使用基于 Tcl 的旧版本，可用 `translate_pin_check_ui` 将旧脚本转换为新的 IC Compiler II 命令：

```
translate_pin_check_ui -from old_script.tcl -to new_script.tcl
```

2.4 处理设计

Working With Designs

在多数情况下，在 IC Compiler II 中处理设计时，使用设计库中某一版本（**block**）的 **design view**。本手册中，**block** 指设计库中的特定设计版本（不论顶层设计还是子设计）；**design** 泛指正在处理的设计。除非另有说明，二者均指 **design view**。

IC Compiler II 以 Verilog 格式读取设计。Verilog 网表文件是单个文件中的结构级或门级设计。要将设计读入 IC Compiler II：

1. 创建或打开与该设计关联的设计库。
 - 若设计库尚不存在，用 `create_lib` 创建。
 - 若设计库已存在，用 `open_lib` 打开。
2. 用 `read_verilog` 读取设计的 Verilog 网表文件。

默认情况下，工具读取 Verilog 网表时会在当前设计库中创建一个 **block**，并增加其打开计数。工具通过识别在指定 Verilog 文件中未被其他模块例化的模块来确定 **block** 的顶层模块，并以该顶

层模块名作为 block 名。

工具还会为 block 创建默认电源域（power domain）。对多电压设计，在指定功耗意图（power intent）时该默认电源域会被替换，见「加载并应用 UPF 信息」。

用下列命令处理 block：

- `create_block`：创建 block
- `open_block`：打开已有 block
- `current_block`：设置或报告当前 block
- `save_block`：保存 block
- `close_blocks`：关闭 block

参见

参见：Blocks; Working With Design Libraries。

2.5 导入 Floorplan 信息

Importing the Floorplan Information

Floorplan 包含物理约束，例如 core 区域与形状、port 位置、宏单元位置与方向等，这些是执行物理综合与优化所必需的。

若 block 已有 floorplan，按「读取 DEF 文件」所述将其作为 DEF 文件读入。

若 block 尚无 floorplan，可按 *IC Compiler II Design Planning User Guide* 所述执行设计规划并生成 floorplan。

2.5.1 读取 DEF 文件

Reading DEF Files

要从 DEF 文件读取 floorplan 信息，使用 `read_def`：

```
icc2_shell> read_def block.def
```

注意

尽可能使用 DEF v5.8 或更高版本，因为该版本比早期版本支持更多类型的物理对象与障碍物（obstructions）。

默认情况下，`read_def` 命令会：

- 将 floorplan 信息标注到当前 block。
 - 要标注到其他 block，用 `-design` 指定 block 名。
 - 保留已有 floorplan 信息。
- 在增量（incremental）模式下：

- 布局区域根据当前 core 区域以及 DEF 中的 site rows 导入；

- 只能有一个取值的物理约束会被最新 DEF 中的值覆盖，例如 port 位置与宏单元位置会被覆盖；
- 可累积取值的物理约束会重新计算，即 core 区域可根据已有值与最新 DEF 中的 site row 定义重新计算。不同 DEF 文件中的 placement keepout 会累积，最终 keepout 几何在综合期间由内部计算。

要在标注 DEF 中的 floorplan 信息之前先清除已有 floorplan，使用 `-no_incremental`。该模式下，布局区域根据 DEF 中的 site rows 导入。

- 对宏单元与 port 使用基于规则的名称匹配（rule-based name matching）。
基于规则的名称匹配利用工具的智能名称匹配能力自动解决名称差异。启用时，默认下列字符视为等价：

- 层次分隔符 { / _ . }
 - 总线记号 { [] __ () }
- 例如，名为 `a.b_c/d_e` 的单元会自动与 DEF 中的字符串 `a/b_c.d/e` 匹配。
- 例如，名为 `a [4] [5]` 的单元会自动与 DEF 中的 `a_4__5_` 匹配。

要禁用基于规则的名称匹配并要求 DEF 与 block 之间精确名称匹配，将 `file.def.rule_based_name_matching` 设为 `false`。

参见

更多信息见 *IC Compiler II Data Model User Guide* 中的“Rule-Based Name Matching”。

- 忽略 DEF 中存在但 block 中不存在的任何对象（PG 对象除外）。
要允许从 DEF 创建新的非 PG 对象，用 `-add_def_only_objects` 指定要创建的对象类型。指定下列一个或多个关键字：
 - **cells**: 工具创建仅存在于 DEF 中的单元，并按 DEF 定义连接其电源与地 pin；即使 DEF 中定义了信号、时钟或 tie pin 连接，也不会连接这些 pin。工具也不会创建新的层次单元；DEF 中指定的任何层次必须已在 block 中存在。
 - **nets**: 工具创建仅存在于 DEF 中的信号、时钟与 tie net，并将它们连接到 DEF PINS 段中指定的 port；即使 DEF 中定义了到网表中其他 port 或 pin 的连接，也不会连接。工具不创建新的层次 net；DEF 中指定的任何层次必须已在 block 中存在。
 - **ports**: 工具创建仅存在于 DEF 中的信号、时钟与 tie port，并将它们连接到 DEF PINS 段中指定的 net。
 - **all**: 工具创建仅存在于 DEF 中的非 PG 单元、net 与 port，效果等同于同时指定 **cells**、**nets** 与 **ports**。

修复 Site 名称不匹配

Fixing Site Name Mismatches

若 DEF 中使用的 site 名与技术文件中定义的 site 名不匹配，用 `-convert_sites` 指定 site 名映射。例如，若 DEF 使用名为 CORE 的 site，而技术文件仅定义名为 unit 的 site，可用下列命令在读取 DEF 时转换 site 名：

```
icc2_shell> read_def -convert_sites { {CORE unit} } block.def
```

验证 DEF 文件

Validating DEF Files

要在将 floorplan 信息标注到 block 之前分析输入 DEF, 用 `-syntax_only` 启用仅检查 (check-only) 模式。该模式提供关于 DEF 正确性与完整性的诊断信息, 不会向 block 标注任何 floorplan 信息。

```
icc2_shell> read_def -syntax_only block.def
```

```
icc2_shell> check_duplicates -remove
```

`check_duplicates -remove` 可确保因执行 `read_def` 而不会在 block 中留下重复的 shape 与 via。

从 DEF 文件提取的物理约束

Physical Constraints Extracted From the DEF File

`read_def` 从 DEF 提取物理约束信息并标注到 block。但仅提取并标注下列物理约束:

- Placement Area (布局区域)
- Port Locations (Port 位置)
- Cell Locations (单元位置)
- Placement Blockages (布局障碍)
- Site Rows
- Routing Tracks (布线 track)
- Placement Bounds (布局 bound)
- Routing Blockages (布线障碍)
- Preroutes (预布线)

要在 GUI 中目视检查所提取的物理约束, 使用 `layout view`。从 DEF 提取的全部物理约束会自动加入 `layout view`。

布局区域

布局区域根据 site array 信息计算。

Port 位置

对 DEF 中指定了位置的每个 port, 工具在 block 中对应 port 上设置该位置。

注意

若 DEF 不含 port 位置信息, 工具会从 pad 单元的位置继承 port 位置。

(原书 *Example 1: DEF Port Location Information*)

图 2.3: DEF Port 位置信息 (示例占位)

```
PINS 2 ;
  -Out1 + NET Out1 + DIRECTION OUTPUT + USE SIGNAL +
    LAYER M3 ( 0 0 ) ( 4200 200 ) + PLACED ( 80875 0 ) N ;
  -Sel0 + NET Sel0 + DIRECTION INPUT + USE SIGNAL +
    LAYER M4 ( 0 0 ) ( 200 200 ) + PLACED ( 135920 42475 ) N ;
END PINS
```

支持名称已更改以及多层的 port。

```
# Example 2: DEF Port Locations With Changed Names and Multiple Layers
PINS 2 ;
  - sys_addr\[23\].extra2 + NET sys_addr[23] + DIRECTION INPUT +USE
  SIGNAL
  + LAYER METAL4 ( 0 0 ) ( 820 5820 ) + FIXED ( 1587825 2744180 ) N ;
  - sys_addr[23] + NET sys_addr[23] + DIRECTION INPUT + USE SIGNAL +
  LAYER
  METAL3 ( 0 0 ) ( 820 5820 ) + FIXED ( 1587825 2744180 ) N ;
END PINS
```

```
# Example 3: DEF Port Orientation Information
PINS 1;
  - OUT + NET OUT + DIRECTION INPUT + USE SIGNAL
  + LAYER m4 ( -120 0 ) ( 120 240 )
  + FIXED ( 4557120 1726080 ) S ;
END PINS
```

单元位置

对 DEF 中指定了位置且带有 **FIXED** 属性的每个单元，工具在 **block** 中对应单元上设置该位置。Example 4 给出 DEF 宏单元位置与方向信息，其中字母 **E**、**W** 分别表示东向与西向旋转。

```
# Example 4: DEF Cell Location Information
COMPONENTS 2 ;
  - macro_cell_abx2 + FIXED ( 4350720 8160 ) E ;
  - macro_cell_cdy1 + FIXED ( 4800 8160 ) W ;
END COMPONENTS
```

布局障碍 (Placement Blockages)

read_def 导入 DEF 中定义的 **hard**、**soft** 与 **partial** 布局障碍。

注意

DEF 5.7 之前的版本不支持 partial blockage。此外，若 floorplanning 工具生成的是 DEF 5.6，需手动添加 #SNPS_SOFT_BLOCKAGE pragma 以指定 soft blockage，见 Example 7。

```
# Example 5: DEF Hard Placement Blockage Information
BLOCKAGES 50 ;
...
- PLACEMENT RECT ( 970460 7500 ) ( 3247660 129940 )
...
END BLOCKAGES
```

```
# Example 6: DEF Version 5.7 Soft Placement Blockage Information
BLOCKAGES 50 ;
...
- PLACEMENT + SOFT RECT ( 970460 7500 ) ( 3247660 129940 ) ;
...
END BLOCKAGES
```

```
# Example 7: DEF Version 5.6 Soft Placement Blockage Information
BLOCKAGES 50 ;
...
- PLACEMENT RECT ( 970460 7500 ) ( 3247660 129940 ) ; #SNPS_SOFT_BLOCKAGE
...
END BLOCKAGES
```

```
# Example 8: DEF Partial Placement Blockage Information
BLOCKAGES 50 ;
...
- PLACEMENT + PARTIAL 80 RECT ( 970460 7500 ) ( 3247660 129940 ) ;
...
END BLOCKAGES
```

Site Rows

DEF 中的 site row 信息定义布局区域。

```
# Example 9: DEF Site Row Information
ROW ROW_0 core 0 0 N DO 838 BY 1 STEP 560 0;
```

布线 Tracks

DEF 中的 track 信息为基于标准单元的设计定义布线网格。该信息可用于布线；track 支持还可改善拥塞评估与报告，使其更接近布线结果。

```
# Example 10: DEF Routing Track Information
TRACKS X 330 DO 457 STEP 660 LAYER METAL1 ;
TRACKS Y 280 DO 540 STEP 560 LAYER METAL1 ;
```

布局 **Bounds**

若 DEF 中存在定义 bound 的 REGIONS，read_def 会导入这些布局 bound。此外，若相关 GROUP 中有单元附着到该 region，会在这些单元与 block 中的单元之间进行模糊单元匹配；匹配到的单元按下列方式附着到 bound：

- 若 block 中已有与 DEF 同名的 region，则相关 group 中的单元在增量模式下通过 add_to_bound 附着到该 region。
- 若 block 中尚无该 region，则通过 create_bound 以 DEF 中的同名创建；相关 group 中匹配到的单元也会附着。

```
# Example 11: DEF Placement Bound Information
REGIONS 1 ;
- c20_group ( 201970 40040 ) ( 237914 75984 ) + TYPE FENCE ;
END REGIONS
GROUPS 1 ;
- c20_group
  cell_abc1
  cell_sm1
  cell_sm2
  + SOFT
  + REGION c20_group ;
END GROUPS
```

布线障碍（**Routing Blockages**）

布线障碍的提取方式与布局障碍类似。

预布线（**Preroutes**）

工具提取 DEF 中定义的 preroute。

```
# Example 12: DEF Preroute Information
SPECIALNETS 2 ;
- vdd
  + ROUTED METAL3 10000 + SHAPE STRIPE ( 10000 150000 ) ( 50000 * )
  + USE POWER ;
...
END SPECIALNETS
```

2.6 设置多电压设计

Setting Up Multivoltage Designs

设置多电压设计时需完成下列任务：

- 应用多电压功耗意图（Applying the Multivoltage Power Intent）
- 准备电源网络（Preparing the Power Network）
- 定义电压区域（Defining Voltage Areas）

- 插入多电压单元（Inserting Multivoltage Cells）
- 控制多电压单元的布局
- 启用多电压 net 的改进缓冲
- 分析多电压信息

2.6.1 应用多电压功耗意图

Applying the Multivoltage Power Intent

关于应用多电压功耗意图，见：

- 加载并应用 UPF 信息
- 为仅物理单元指定 UPF 约束
- 保存 UPF 信息

加载并应用 UPF 信息

Loading and Applying UPF Information

要加载功耗意图并将其应用到多电压设计：

1. 用 `load_upf` 读取 UPF 文件。

```
icc2_shell> load_upf block.upf
```

2. 若使用 golden UPF 流程且有来自 Design Compiler 的名称映射文件，用 `read_name_map` 读取该文件。

```
icc2_shell> read_name_map block.nmf
```

- 3.（可选）验证 UPF 一致性，并识别网表、floorplan 与功耗意图之间的 PG 冲突。

用 `resolve_pg_nets -check_only` 验证 UPF 一致性并识别 PG 冲突。该命令会识别问题，并报告在提交（commit）功耗意图时为解决问题所做的更改。若希望以不同方式解决问题，可在运行 `commit_upf` 之前用手动编辑命令解决。

4. 用 `commit_upf` 提交功耗意图。

```
icc2_shell> commit_upf
```

`commit_upf` 对 UPF 一致性执行全局检查；解决网表、floorplan 与 UPF 规范之间的 PG 冲突；并将功耗策略（power strategies）与已有多电压单元关联。关于将功耗策略与已有多电压单元关联，见「将功耗策略与已有多电压单元关联」。

5. 用 `report_mv_path` 报告为多电压单元所做的关联。

若工具未能关联某些多电压单元，该命令会报告失败原因。必须成功提交功耗意图后，才能继续设计流程。

注意

成功运行 `commit_upf` 之后,若再尝试使用额外的 UPF 命令(下列除外:`set_related_supply_net`、`connect_supply_net`、`set_design_attributes`、`set_port_attributes`、`find_objects`、`set_scope`),工具会发出错误。要在运行 `commit_upf` 后修改功耗意图,必须先用 `reset_upf` 移除现有 UPF 规范,再重新加载功耗意图。

为仅物理单元指定 **UPF** 约束

Specifying UPF Constraints for Physical-Only Cells

仅物理单元(physical-only cells)的 UPF 约束,在读入不支持仅物理单元的其他工具(如验证工具)时可能引发问题。因此,请用下例所示语法将仅物理单元的 UPF 约束包起来:

```
if {[info exists snps_handle_physical_only] && \
    $snps_handle_physical_only} {

    /* Supply net connections for filler cells */
    connect_supply_net VDD5 -ports [get_pins FILLER*/VDD]

}
```

默认情况下,IC Compiler II 中 `snps_handle_physical_only` 变量为 `true`。因此加载并提交 UPF 时,这些约束会应用到仅物理单元。

用 `save_upf` 保存 UPF 时,工具对用户指定的与工具推导的仅物理单元 UPF 约束使用相同语法。因此,未将 `snps_handle_physical_only` 设为 `true` 的工具会忽略这些 UPF 约束。

要创建仅包含特定类型单元命令的 UPF 文件,对 `save_upf` 使用 `-include` 或 `-exclude`。这些选项接受如 `diode_cells`、`pad_cells`、`physical_only_cells` 等参数。完整参数列表见 `save_upf` 的 `man` 页。

对指定单元类型的命令过滤适用于 `connect_supply_net`、`set_related_supply_net`、`create_power_domain` 与 `set_isolation`。可对 block 级与全芯片 UPF 文件应用命令过滤。

保存 UPF 信息

Saving UPF Information

物理实现期间,工具会更新设计的功耗意图。要用 `save_upf` 生成更新后的 UPF 文件,供设计流程后续步骤使用。

`save_upf` 根据输入 UPF 文件分离 UPF 命令,并添加注释标明输入 UPF 文件及其加载时间。

假设按下列脚本加载、提交并保存 UPF:

```
load_upf top_1.upf
load_upf top_2.upf
load_upf top_3.upf
commit_upf
save_upf top.upf
```

生成的 `top.upf` 包含如下信息:

```
## Start - Load_upf top_1.upf on Wed Oct 26 17:03:18 2016
...
...
## End - Load_upf
## Start - Load_upf top_2.upf on Wed Oct 26 17:03:49 2016
...
...
## End - Load_upf
## Start - Load_upf top_3.upf on Wed Oct 26 17:04:22 2016
...
...
## End - Load_upf
```

2.6.2 准备电源网络

Preparing the Power Network

关于为物理实现准备电源网络，见：

- 创建逻辑电源与地连接
- 创建浮空逻辑供电网（Floating Logical Supply Nets）

创建逻辑电源与地连接

Creating Logical Power and Ground Connections

读入设计后，必须确保电源与地 net 与设计各单元的电源、地及 tie-off pin 之间存在逻辑连接。若设计尚无这些连接，用 `connect_pg_net` 创建。该命令可为单电压与多电压设计中的叶子单元、层次单元与仅物理单元创建逻辑电源与地连接。

创建逻辑电源与地连接之前，必须解决网表、floorplan 与 UPF 规范之间的任何 PG 冲突。

- 对多电压设计，冲突在提交功耗意图时解决，见「加载并应用 UPF 信息」。
- 对单电压设计，必须运行 `resolve_pg_nets` 解决冲突。注意：单电压设计的 UPF 规范即 `read_verilog` 生成的默认电源域。

若设计含有未映射（unmapped）实例，工具会发出信息消息，表明仅连接已映射实例。

`connect_pg_net` 有两种工作模式：

- 自动（**Automatic**）
自动模式下，命令从 UPF 规范推导全部电源与地 net、电源与地 pin、tie-off pin 及连接。若供电网尚不存在，工具会创建。
要在自动模式下创建逻辑电源与地连接，对 `connect_pg_net` 使用 `-automatic`。
- 手动（**Manual**）
手动模式下，命令按你的指定建立连接。若指定 pin 或 port 已有连接，工具先移除已有连接再创建指定连接。工具会验证连接是否与功耗意图匹配；若发现不匹配，发出警告但仍创建连接。
要在手动模式下创建逻辑电源与地连接，用 `-net` 指定电源或地 net，并将要连接到该 net 的 pin

与 `port` 作为命令参数。例如，将 `VDD` 电源 `net` 连接到所有名为 `vdd` 的 `pin`，将 `VSS` 地 `net` 连接到所有名为 `vss` 的 `pin`：

```
icc2_shell> connect_pg_net -net VDD [get_pins */vdd]
icc2_shell> connect_pg_net -net VSS [get_pins */vss]
```

要仅将 `PG net` 连接到电源与地 `pin`，使用 `-pg`；要仅连接到 `tie-off pin`，使用 `-tie`。默认若两者均未指定，工具连接到全部电源、地与 `tie-off pin`。这些选项不能与对象列表或 `-create_nets_only`、`-net` 一起使用。

无论使用何种命令选项，工具总会创建构建完整 `PG` 网表结构所需的连接，例如顶层 `PG port` 与中间层次 `PG pin`。

创建浮空逻辑供电网

Creating Floating Logical Supply Nets

浮空逻辑供电网是未连接到 `block` 中任何逻辑 `pin` 或 `port` 的电源或地 `net`。例如，电源规划期间创建的电源 `feedthrough net` 即为浮空供电网。

要用 `connect_pg_net` 创建浮空供电网，按下列步骤：

1. 在 `UPF` 文件中用 `supply_net_pg_type` 属性指定供电网类型，如下例：

```
...
create_supply_net VDD
...
...
set_design_attributes -elements VDD \
    -attribute supply_net_pg_type power
...
```

2. 在用 `connect_pg_net` 创建供电网之前，将 `mv.pg.create_floating_pg` 设为 `true` 以启用浮空供电网，如下例脚本：

```
...
...
# Apply the UPF
load_upf $upf_input_file_name
commit_upf
...
...
# create logical supply nets
set_app_options -as_user_default \
    -name mv.pg.create_floating_pg -value true
connect_pg_net -automatic
...
check_mv_design
```

对域无关（`domain-independent`）供电网，工具在当前 `UPF scope` 的最顶层层次中创建逻辑供电网；对域相关（`domain-dependent`）供电网，在该域的最顶层层次中创建。

下列情况下工具不创建逻辑供电网：

- 连接与用 `supply_net_pg_type` 指定的供电网类型冲突；
- 供电网已连接到某实例的 PG pin；
- `connect_pg_net` 未在当前或父物理层次中指定。

2.6.3 定义电压区域

Defining Voltage Areas

电压区域 (voltage area) 是与电源域关联的单元的物理布局区域。对多电压设计，电源域在 UPF 规范中定义。对单电压设计，工具在读取 Verilog 网表时创建默认电源域，并将其与默认电压区域关联，该默认电压区域由 block 的 core 区域构成。

布局器将电压区域视为排他性 move bound (exclusive move bound)：必须将电压区域内的单元放在指定区域内，且必须将所有其他单元放在电压区域之外。电压区域可以是矩形或直角多边形 (rectilinear)；此外可以不相交、嵌套或重叠。对重叠电压区域，各电压区域的有效形状由电压区域 shape 的堆叠顺序 (stacking order) 决定。

要用 `create_voltage_area` 定义电压区域。定义时至少须指定与该电压区域关联的电源域，使用 `-power_domains`。可指定一个或多个电源域；但所有指定电源域必须具有相同的主供电网 (primary supply net)。若指定单个电源域，电压区域名称由电源域名推导；若指定多个电源域，必须用 `-name` 为电压区域指定名称。

电压区域由一个或多个矩形或直角多边形 shape 组成，这些 shape 可以邻接、不相交或重叠。要用 `-region` 定义这些 shape 的边界：创建新电压区域时用 `create_voltage_area`；向已有电压区域添加 shape 时用 `create_voltage_area_shape`。注意：`create_voltage_area` 可一次指定一个或多个 shape，而每次 `create_voltage_area_shape` 只能指定单个 shape。

- 指定矩形边界时，用下列格式给出左下角与右上角：

```
{{l1x l1y} {urx ury}}
```

- 指定直角多边形边界时，用下列格式给出各顶点坐标：

```
{{x1 y1} {x2 y2} {x3 y3} {x4 y4} ...}
```

若电压区域由多个邻接或重叠的 shape 组成，可根据 shape 的堆叠顺序将其合并为最少的一组不相交 shape。关于如何合并电压区域 shape，见「合并电压区域 Shape」。

工具也用堆叠顺序解决来自不同电压区域的重叠 shape。关于解决重叠电压区域，见「解决重叠电压区域」。

为确保电压区域边界不发生短路，可为电压区域定义 guard band，作为环绕电压区域的硬 keepout 边距。若为电压区域 shape 定义了 guard band，guard band 计入该 shape 的有效边界，但不计入电压区域的有效布局面积。关于定义 guard band，见「定义 Guard Band」。

要用 `set_voltage_area` 修改已有电压区域，见「修改电压区域」。

多电压设计通常存在运行期间关断与上电的电源域，同时其他电源域保持上电。处理关断域时，关断部分中某些单元可能需要持续保持活动，例如实现保持寄存器、隔离单元、保持控制路径与隔离使能路径。这些单元称为 always-on 单元。要在电压区域内为 always-on 单元定义特殊布局区域 (always-on well)，在电压区域边界内定义排他性 move bound。关于定义排他性 move bound，见「定义 Move Bounds」。

创建电压区域后，运行 `check_mv_design` 以验证设计无多电压违例。

合并电压区域 Shape

Merging Voltage Area Shapes

要将电压区域 shape 合并为最少的一组不相交 shape, 对 `create_voltage_area`、`create_voltage_area_shape` 或 `set_voltage_area` 使用 `-merge_regions`。

- 使用 `create_voltage_area -merge_regions` 时, 工具合并用 `-region` 指定的 shape。
- 使用 `create_voltage_area_shape -merge_regions` 时, 工具合并用 `-region` 指定的 shape 与指定电压区域的已有 shape。
- 使用 `set_voltage_area -merge_regions` 时, 工具合并指定电压区域的全部已有 shape。

工具根据堆叠顺序合并电压区域 shape。默认堆叠顺序为定义 shape 的顺序, 最后定义的 shape 在最上层。合并后的 shape 替换一组邻接或重叠 shape 中的顶层 shape; 该组中的其他 shape 被移除, 不再与该电压区域关联。

例如, 假设用下列命令创建由三个矩形 shape 组成的电压区域, 如图 2.4 左侧所示:

```
icc2_shell> create_voltage_area -power_domains {PD1} \
  -region { {{0 0} {10 10}} {{10 0} {30 10}} {{15 5} {35 15}} }
{PD1}
icc2_shell> get_voltage_area_shapes -of_objects PD1
{VOLTAGE_AREA_SHAPE_1 VOLTAGE_AREA_SHAPE_2 VOLTAGE_AREA_SHAPE_3}
```

使用 `-merge_regions` 合并这些 shape 后, 电压区域变为单个直角多边形, 如图 2.4 右侧所示。合并后的电压区域 shape 名为 `VOLTAGE_AREA_SHAPE_3`, 即创建电压区域时最后定义的 shape。

```
icc2_shell> set_voltage_area PD1 -merge_regions
Information: Merging abutted and overlapping shapes in voltage_area
'PD1'. (NDMUI-154)
1
icc2_shell> get_voltage_area_shapes -of_objects PD1
{VOLTAGE_AREA_SHAPE_3}
```

(原书 Figure 7: Merging Voltage Area Shapes)

图 2.4: 合并电压区域 Shape

要用 `report_voltage_areas -verbose` 报告电压区域 shape 的堆叠顺序。

要用 `set_voltage_area_shape` 修改电压区域 shape 的堆叠顺序, 见「修改堆叠顺序」。

解决重叠电压区域

Resolving Overlapping Voltage Areas

若来自两个或多个电压区域的 shape 完全或部分重叠, 工具用 shape 的堆叠顺序确定各电压区域的有效形状。默认堆叠顺序为定义 shape 的顺序, 最后定义的在最上层。工具将重叠区域分配给与顶

层 shape 关联的电压区域。与合并同一电压区域内的 shape 不同，解决不同电压区域之间的重叠时，工具不会移除任何 shape；仅解释方式发生变化。

例如，假设要创建如图 2.5 所示的嵌套电压区域。

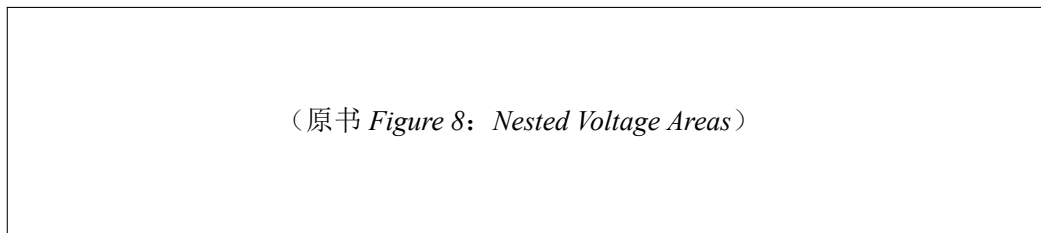


图 2.5: 嵌套电压区域

要生成这些有效电压区域，必须先指定外层电压区域，再指定内层电压区域，使内层电压区域的 shape 位于上层：

```
icc2_shell> create_voltage_area -power_domains PD1 \
    -region { {0 0} {30 30} }
{PD1}
icc2_shell> create_voltage_area -power_domains PD2 \
    -region { {10 10} {20 20} }
{PD2}
icc2_shell> get_attribute -objects [get_voltage_areas PD2] \
    -name effective_shapes
{{10.0000 10.0000} {20.0000 20.0000}}
icc2_shell> get_attribute -objects [get_voltage_areas PD1] \
    -name effective_shapes
{{0.0000 0.0000} {10.0000 0.0000} {10.0000 20.0000} {20.0000 20.0000}
 {20.0000 10.0000} {10.0000 10.0000} {10.0000 0.0000} {30.0000 0.0000}
 {30.0000 30.0000} {0.0000 30.0000}}
```

若先指定内层电压区域，则外层电压区域的 shape 在上层并遮蔽内层电压区域，因而被工具忽略，如下例：

```
icc2_shell> create_voltage_area -power_domains PD2 \
    -region { {10 10} {20 20} }
{PD2}
icc2_shell> create_voltage_area -power_domains PD1 \
    -region { {0 0} {30 30} }
{PD1}
icc2_shell> get_attribute -objects [get_voltage_areas PD2] \
    -name effective_shapes
icc2_shell> get_attribute -objects [get_voltage_areas PD1] \
    -name effective_shapes
{{0.0000 0.0000} {30.0000 30.0000}}
```

要用 `report_voltage_areas -verbose` 报告堆叠顺序；用 `set_voltage_area_shape` 修改堆叠顺序，见「修改堆叠顺序」。

修改堆叠顺序

Modifying the Stacking Order

可用 `set_voltage_area_shape` 修改电压区域 `shape` 的堆叠顺序：

- `-raise`：将 `shape` 上移一位
- `-lower`：将 `shape` 下移一位
- `-top`：将 `shape` 移到最顶层
- `-bottom`：将 `shape` 移到最底层
- `-above`：将 `shape` 移到另一 `shape` 正上方
- `-below`：将 `shape` 移到另一 `shape` 正下方

定义 Guard Band

Defining Guard Bands

Guard band 定义环绕电压区域的硬 keepout 边距，其中不能放置任何单元（包括电平移位单元与隔离单元）。Guard band 保证不同电压区域中的单元彼此分离，使电源规划不会引入短路。

默认电压区域没有 guard band。要用 `create_voltage_area` 或 `create_voltage_area_shape` 的 `-guard_band`，为 `-region` 中每个 `shape` 指定水平与垂直 guard band 宽度。水平 guard band 宽度适用于电压区域的全部垂直边；垂直 guard band 宽度适用于全部水平边。

注意

若同时使用 `-merge_regions`，必须为合并后的每个不相交 `shape` 指定 guard band 宽度。通常仅在待合并的全部 `shape` 邻接或重叠因而合并为单个 `shape` 时使用该选项。

电压区域 `shape` 的有效边界包含其 guard band；但其有效可布局面积不包含 guard band。

例如，图 2.6 显示由下列命令定义的 PD1 电压区域周围的 guard band：

```
icc2_shell> create_voltage_area -power_domains PD1 \
-region {{0 0} {30 0} {30 10} {40 10} {40 30} {20 30} {20 25} {0 25}} \
-guard_band { {3 1} }
```

（原书 *Figure 9: Voltage Area Guard Band*）

图 2.6: 电压区域 Guard Band

对同一电压区域关联的邻接或重叠 `shape`，工具按「合并电压区域 Shape」所述合并 `shape`，再将顶层 `shape` 定义的 guard band 应用到合并后的 `shape`。

例如，假设对图 2.4 右侧所示的电压区域 `shape` 用下列命令定义 guard band：

```
icc2_shell> create_voltage_area -power_domains {PD1} \
  -region { {0 0} {10 10}} { {10 0} {30 10}} { {15 5} {35 15}} } \
  -guard_band { {1 1} {3 3} {3 1} }
{PD1}
```

此时有效 guard band 与图 2.6 所示相同，即合并后 shape 所定义的 guard band。

对不同电压区域关联的重叠 shape，工具用有效边界按「解决重叠电压区域」所述解决 shape。顶层 shape 保留其布局面积与 guard band；较低层 shape 的有效布局面积与 guard band 不包含重叠区域。注意：若邻接 shape 带有 guard band，由于有效边界包含 guard band，它们不再邻接而是重叠。

例如，假设对图 2.5 所示电压区域用下列命令定义 guard band：

```
icc2_shell> create_voltage_area -power_domains PD1 \
  -region { {0 0} {30 30}} -guard_band { {2 2} }
{PD1}
icc2_shell> create_voltage_area -power_domains PD2 \
  -region { {10 10} {20 20}} -guard_band { {2 2} }
{PD2}
```

此时 PD1 的有效布局面积会因 PD2 的有效边界（含其 guard band）而减小，如图 2.7。

（原书 Figure 10: *Effective Boundaries of Overlapping Voltage Areas*）

图 2.7: 重叠电压区域的有效边界

定义 Gas Station

Defining Gas Stations

Gas station 是为尽量减少在物理 feedthrough 路径上使用双轨（dual-rail）缓冲器而创建的小型电压区域。Gas station 中仅允许单轨中继器，且仅优化步骤可向 gas station 添加单元。

在设计规划阶段用 create_voltage_area 创建 gas station。工具会自动识别 gas station，并在绕线迂回代价与使用双轨单元代价之间权衡以使用它们。

用 create_voltage_area 的 -nwell 与 -pwell 为电压区域指定 n-well 与 p-well 供电网。为 gas station 电压区域定义 well 供电可为不同设计风格使用 gas station 提供灵活性。若未对这些选项赋值，则假定 well bias 与域供电值相同。

report_voltage_areas 会列出 n-well 与 p-well 供电网，无论是否显式设置。

查询电压区域

Querying Voltage Areas

可查询关于电压区域的下列信息：

- 当前 block 中的电压区域: 用 `get_voltage_areas` 创建当前 block 中电压区域的集合 (含默认电压区域)。
- 电压区域的详细信息: 用 `report_voltage_areas` 显示详细信息 (含默认电压区域)。要用 `-verbose` 包含构成各电压区域的 `shape` 信息。
- 电压区域的有效布局面积: 查询电压区域的 `effective_shapes` 属性。
- 电压区域的有效 guard band: 查询电压区域的 `effective_guard_band_boundaries` 属性。也可对单个电压区域 `shape` 查询该属性。

修改电压区域

Modifying Voltage Areas

创建电压区域后, 可做下列修改:

- 更改与电压区域关联的电源域: 对 `set_voltage_area` 使用 `-add_power_domains` 与 `-remove_power_domains`。
- 更改电压区域名称: 对 `set_voltage_area` 使用 `-name`。
- 更改电压区域 region: 用 `create_voltage_area_shape` 添加 `shape`; 用 `remove_voltage_area_shapes` 移除 `shape`。

控制电压区域中的物理 Feedthrough Net

Controlling Physical-Feedthrough Nets in Voltage Areas

若 net 的一个或多个段位于某电压区域的逻辑层次中, 则该 net 对该电压区域为 native。若 net 必须在物理上布线经过非 native 电压区域, 则它是该电压区域的物理 feedthrough net, 如下图所示。

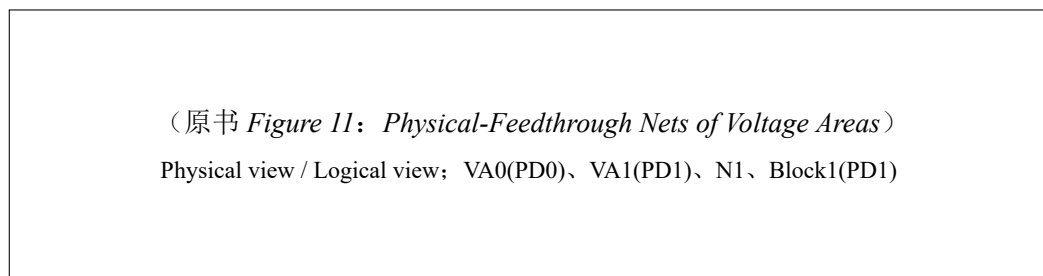


图 2.8: 电压区域的物理 Feedthrough Net

默认允许电压区域中存在物理 feedthrough net。要禁止某电压区域中的物理 feedthrough net, 用 `create_voltage_area_rule -allow_pass_through false` 定义电压区域规则, 例如:

```
icc2_shell> create_voltage_area_rule -allow_pass_through false \
-name VA1_rule -voltage_areas VA1
```

有此规则时, N1 net 会绕开 VA1 电压区域, 如图 2.9。

默认情况下, 优化不会在电压区域的物理 feedthrough net 上插入缓冲器。在物理 feedthrough net 上插入缓冲器可能导致缓冲器的逻辑视图与物理视图不匹配。工具通过支持下列优化类型来解决此类不匹配:

(原书 *Figure 12: Physical-Feedthrough Nets Disabled for a Voltage Area*)

图 2.9: 对电压区域禁用物理 Feedthrough Net

- 物理 **feedthrough** 缓冲 (**Physical-feedthrough buffering**)

期间工具会:

1. 在其驱动或负载的逻辑层次中插入缓冲器, 并用 power-domain-on-instance (PDOI) 约束将非 native 电压区域的电源域应用到该缓冲器;
2. 将缓冲器放在非 native 电压区域内。

要允许物理 feedthrough 缓冲, 使用 `create_voltage_area_rule -allow_physical_feedthrough true`, 例如:

```
icc2_shell> create_voltage_area_rule \
  -allow_physical_feedthrough true \
  -name VA1_rule -voltage_areas VA1
```

- 逻辑 **feedthrough** 缓冲 (**Logical-feedthrough buffering**)

期间工具会:

1. 在与非 native 电压区域对应的逻辑层次中添加缓冲器;
2. 将缓冲器放在非 native 电压区域内。

要允许逻辑 feedthrough 缓冲, 使用 `create_voltage_area_rule -allow_logical_feedthrough true`。要用 `-include_logical_feedthrough_hierarchy` 或 `-exclude_logical_feedthrough_hierarchy` 分别指定允许或不允许 feedthrough 缓冲器的逻辑层次。

下例为 VA1 启用逻辑 feedthrough 缓冲, 并将 feedthrough 缓冲器限制为仅 U1 与 U2 层次单元:

```
icc2_shell> create_voltage_area_rule \
  -allow_logical_feedthrough true \
  -include_logical_feedthrough_hierarchy {U1 U2}\
  -name VA1_rule -voltage_areas VA1
```

下例为 VA2 启用逻辑 feedthrough 缓冲, 并排除 U3 层次单元:

```
icc2_shell> create_voltage_area_rule \
  -allow_logical_feedthrough true \
  -exclude_logical_feedthrough_hierarchy {U3}\
  -name VA2_rule -voltage_areas VA2
```

注意

要向下层 block 添加逻辑 feedthrough 缓冲器，工具必须为该 block 添加新的边界 port。因此，用 `set_freeze_ports` 冻结 block 边界会阻止工具向该 block 添加逻辑 feedthrough 缓冲器。

(原书 Figure 13: *Difference in the Logical View After Physical- and Logical-Feedthrough Buffering*)

图 2.10: 物理与逻辑 Feedthrough 缓冲后逻辑视图的差异

要用 `create_voltage_area_rule` 配合 `-default_rule` 创建适用于所有尚无特定规则的电压区域的默认规则，例如：

```
icc2_shell> create_voltage_area_rule -default_rule \
    -allow_physical_feedthrough true
```

若某电压区域既无特定规则也无默认规则，则该电压区域的 feedthrough 缓冲由 `opt.common.allow_physical_feedthrough` 应用选项控制。

要用 `report_voltage_area_rules` 报告电压区域规则；用 `remove_voltage_area_rules` 移除规则。

移除电压区域

Removing Voltage Areas

要用 `remove_voltage_areas` 从 block 移除电压区域。要用 `-all` 移除全部电压区域；要移除特定电压区域，指定电压区域名称。

2.6.4 插入多电压单元

Inserting Multivoltage Cells

多电压设计在电源域接口处需要特殊的多电压单元，例如电平移位单元与隔离单元。电平移位单元用于工作在不同电压电平的电源域之间；隔离单元用于处于不同状态（关断相对 `always-on` 或上电）的电源域之间。通常多电压单元在逻辑综合期间插入；也可在 IC Compiler II 中插入。

插入电平移位单元

Inserting Level Shifters

电平移位单元作为工作在不同电压电平的电源域之间的接口，确保驱动的输出跳变能使接收单元切换，即使接收端工作在不同电压电平。

注意

若 block 使用 Design Compiler 综合，自动电平移位插入作为 `compile_ultra` 命令的一部分完成。

要在当前 block 中插入电平移位单元，使用 `create_mv_cells`。工具按 UPF 规范中定义的单元映射与策略插入电平移位单元。插入时，工具在电平移位单元上设置 `size_only` 属性，并在 port 到电平移位单元的 net 上设置 `dont_touch` 属性。

默认情况下，命令仅从 generic library 插入单元。若对 `create_mv_cells` 使用 `-mapped`，则仅从用户提供的逻辑库插入单元；若无合适单元则报错。

若工具未插入电平移位单元，可用 `analyze_mv_design -level_shifter` 获取更多信息。若用 `-through` 指定 net 或 pin，报告会列出该 net 或 pin 驱动端与负载端的电源域及相关供电，以及说明为何未插入电平移位单元的错误消息。报告包含针对特定插入点的错误，以及适用于经 `-through` 指定的 net 或 pin 整条路径的错误。

插入隔离单元

Inserting Isolation Cells

隔离单元用于有选择地关断电源域电压接口的输入侧；它们不进行电平移位。隔离单元应在 RTL 级例化，以避免形式验证错误。也可使用 IC Compiler II 插入。

要在当前 block 中插入隔离单元，使用 `create_mv_cells`。工具按 UPF 规范中定义的单元映射与策略插入隔离单元。插入时，工具在隔离单元上设置 `size_only` 属性，并在 port 到隔离单元的 net 上设置 `dont_touch` 属性。

默认情况下，命令仅从 generic library 插入单元。若对 `create_mv_cells` 使用 `-mapped`，则仅从用户提供的逻辑库插入单元；若无合适单元则发出错误消息。

将功耗策略与已有多电压单元关联

Associating Power Strategies With Existing Multivoltage Cells

运行 `associate_mv_cells` 或 `commit_upf` 时，IC Compiler II 会自动将功耗策略与已有多电压单元关联。若自动关联不正确，可用 `set_power_strategy_attribute` 手动修改关联。要用 `get_power_strategies` 确定某电源域的功耗策略。

2.6.5 控制多电压单元的布局

Controlling the Placement of Multivoltage Cells

连接多电压单元（如电平移位单元或隔离单元）到另一电压区域中至少一个单元的 net，称为多电压 net。要缩短多电压 net 长度，将 `place.coarse.enable_advanced_mv_cell_placement` 设为 `true`，以启用高级多电压单元布局。

启用后，工具通过将多电压单元放得更靠近电压区域边界来缩短多电压 net。为优化多电压 net，工具使用双轨缓冲器。缩短多电压 net 可减少优化期间使用的双轨缓冲器数量，并防止因缺乏双轨缓冲器而导致多电压 net 未优化。

2.6.6 启用多电压 Net 的改进缓冲

Enabling Improved Buffering for Multivoltage Nets

多电压 net 是在逻辑或物理上跨越多个电压区域的 net。可将 `opt.buffering.enable_hierarchy_mapping` 设为 `true`，以在布线前优化期间对这类 net 修复逻辑 DRC 违例时启用改进的缓冲技术。

该应用选项影响 `place_opt` 与 `clock_opt`。启用后可减少用于修复逻辑 DRC 违例的缓冲器数量，但可能略微增加总负松弛（total negative slack）或 hold 违例数。

2.6.7 分析多电压信息

Analyzing Multivoltage Information

要用 `check_mv_design` 确保设计无多电压设计违例。

可用 `check_mv_design` 限制要检查的规则类型。例如，`-isolation` 指定检查隔离策略与隔离单元规则；`-pg_pin` 指定检查与 PG pin 相关的规则。

要用表 2.1 所列命令报告设计的多电压信息。

表 2.1: 报告多电压信息的命令

目的	使用的命令
报告多电压信息摘要	<code>report_mv_design</code>
报告多电压单元信息	<code>report_mv_cells</code>
报告多电压库单元信息	<code>report_mv_lib_cells</code>
报告带多电压约束的路径及相关多电压单元	<code>report_mv_path</code>
报告电源域	<code>report_power_domains</code>
检查指定电源域是否等价	<code>check_equivalent_power_domains</code>
显示等价电源域列表	<code>get_equivalent_power_domains</code>
报告电压区域	<code>report_voltage_areas</code>

2.7 指定时序约束与设置

Specifying Timing Constraints and Settings

时序约束描述设计的时序要求。时序约束的关键部分是准确指定的时钟及其效应（如 `latency` 与 `uncertainty`）。指定时钟后，工具自动约束由这些时钟驱动的寄存器之间的路径。但可通过指定时序例外（`timing exceptions`）改变这些时序路径的默认行为，例如不应分析的路径（`false paths`）、需要多个时钟周期的路径（`multicycle paths`）等。此外，可通过为输入与输出 port 指定 `input/output delay` 来约束边界时序路径。

IC Compiler II 使用片上偏差（on-chip variation, OCV）模式进行时序分析，以建模芯片上工作条件变化的影响。该模式执行保守的时序分析，允许最小与最大延迟同时作用于不同路径。对 `setup` 检查，发射时钟路径与数据路径使用最大延迟，捕获时钟路径使用最小延迟；对 `hold` 检查，发射时钟路径与数据路径使用最小延迟，捕获时钟路径使用最大延迟。

一个 **block** 可能在多种条件（如不同温度与电压）下工作，并可能处于多种功能模式。时序分析中，每组条件由一个 **corner** 表示，每个功能模式由一个 **mode** 表示。**scenario** 是用于时序分析与优化的 **corner** 与 **mode** 的组合。开始处理 **block** 之前，必须定义用于该 **block** 的 **mode**、**corner** 与 **scenario**，以及延迟计算模型与所用布线层。所指定的布线层信息用于时序分析期间的 RC 估计。

详细信息见：

- 时钟与时钟效应： *IC Compiler II Timing Analysis User Guide* 中的 “Defining Clocks”；
- 时序路径例外与边界路径约束： 同上指南中的 “Constraining Timing Paths”；
- Mode、corner 与 scenario： 同上指南中的 “Defining Modes, Corners, and Scenarios”；
- 工作条件与 OCV 相关设置： 同上指南中的 “Specifying Operating Conditions”；
- RC 估计与提取的寄生信息： 同上指南中的 “Performing Parasitic Extraction”。

2.8 指定逻辑设计规则约束

Specifying Logical Design Rule Constraints

最小电容、最大电容与最大 **transition** 是设计必须满足才能按预期工作的逻辑设计规则约束。它们是在逻辑库中指定的工艺相关限制。但可用表 2.2 中的约束命令指定更严格的设计规则约束。

优化期间，IC Compiler II 会设法满足设计规则约束，即使这意味着违反时序、功耗与面积等优化约束——这些设计规则约束具有更高优先级。优化后，可用表 2.2 中的报告命令识别 **block** 中的设计规则约束违例。

表 2.2: 设计规则相关命令

目的	使用的命令
为输入 port、库单元 pin、叶子单元 pin、时钟或 block 指定允许的最小电容	set_min_capacitance
为输入 port、库单元 pin、叶子单元 pin、时钟或 block 指定允许的最大电容	set_max_capacitance
为输入 port、库单元 pin、叶子单元 pin、时钟或 block 指定允许的最大信号 transition 时间	set_max_transition
移除用户指定的最小电容约束	remove_min_capacitance
移除用户指定的最大电容约束	remove_max_capacitance
移除用户指定的最大 transition 约束	remove_max_transition
报告最小电容约束违例	report_constraints -min_capacitance
报告最大电容约束违例	report_constraints -max_capacitance
报告最大 transition 约束违例	report_constraints -max_transition

注意

IC Compiler II 不遵从最大 fanout 设计规则约束。但可用 `opt.common.max_fanout` 应用选项为 **block** 中的数据路径指定最大 fanout。这是软优化约束。优化期间工具会设法确保数据路径单元

驱动的扇出不得超过指定的最大 fanout。

2.9 控制时钟门控延迟

Controlling Clock-Gate Latencies

综合期间，IC Compiler II 假定时钟为理想时钟。理想时钟在时钟网络上不产生延迟。作出该假定是因为真实时钟网络延迟要到时钟树综合之后才可知。实际上时钟并非理想，时钟网络上存在非零延迟。对带时钟门控的设计，寄存器处的时钟网络延迟与时钟门控单元处的时钟网络延迟不同。寄存器与时钟门控单元之间的时钟网络延迟差异，会使时钟门控单元使能输入处的 setup 条件约束更紧。

工具可通过下列方式获得时钟门控延迟：

- 集成延迟估计（**Integrated latency estimation**）

默认情况下，工具在整个流程中估计并更新时钟门控延迟。估计的时钟延迟比用户指定的时钟延迟更准确。

- 用户指定延迟（**User-specified latency**）

可对特定实例禁用集成延迟估计，并手动指定延迟值。

2.9.1 集成时钟门控延迟估计

Integrated Clock-Gate Latency Estimation

默认情况下，工具在整个 place_opt 命令流程中估计并更新时钟门控延迟。这确保在时钟树综合之前、时钟仍为理想时，工具在数据路径优化与 useful-skew 优化中使用准确的时钟门控延迟。

若计划对设计执行结构性多源时钟树综合（structural multisource clock tree synthesis），请将 opt.clock_latency_estimation.estimate_smscts_subtrees 设为 true，以确保工具估计适合结构性多源时钟子树的时钟门控延迟。

为提高估计时钟门控延迟的准确性，在运行 place_opt 之前指定时钟树综合设置，包括时钟树缓冲器库单元列表、时钟树反相器库单元列表以及时钟非默认布线规则（NDR）。

估计的时钟延迟比用户指定的更准确。此外，工具在布局与多比特 banking 等步骤之后会更新估计的时钟延迟，而用户指定的延迟是静态值。

要阻止工具估计特定时钟门控的延迟，用 set_attribute 在其时钟 pin 上设置 dont_estimate_clock_latency 属性，例如：

```
icc2_shell> set_attribute \
[get_pins {ICG21/CK}] dont_estimate_clock_latency true
```

为获得最佳结果，请勿将下列功能与集成时钟门控延迟估计一起使用：

- Trial clock tree synthesis: 将 place_opt.flow.trial_clock_tree 设为 false 以禁用。
- Clock-gate optimization: 将 place_opt.flow.optimize_icgs 设为 false 以禁用。
- 用户指定延迟: 用户指定延迟由 set_clock_latency、set_clock_gate_latency、set_clock_gating_check -setup 或 set_path_margin -setup 设置。

工具不会修改你在时钟门控单元上指定的任何 `set_clock_latency` 约束。工具估计的时钟门控延迟值存储为相对用户指定值的偏移 (offset)。可用 `write_script -format icc2` 查看偏移值，并在生成输出中查找 `set_clock_latency -offset` 命令。若未设置任何延迟值，报告的偏移等于估计延迟值。

由 `write_sdc` 生成的 SDC 文件不会捕获估计的延迟偏移值。

若时钟门控及其被门控寄存器的相对位置在时钟树综合期间不变，延迟估计最准确。要保持集成时钟门控的布局，在时钟树综合之前将 `cts.compile.enable_cell_relocation` 设为 `auto`，以启用时钟网络单元的自动重定位。

2.9.2 用户指定的时钟门控延迟

User-Specified Clock-Gate Latency

不能禁用通用的集成时钟门控延迟估计；但可通过在时钟门控单元的时钟 pin 上设置 `dont_estimate_clock_latency` 属性，对特定时钟门控禁用该功能。本节说明如何对这些时钟门控手动指定延迟值。

用 `set_clock_latency` 或 `set_clock_gate_latency` 指定时钟网络延迟。`set_clock_gate_latency` 可用于门级与 RTL 设计。

用 `set_clock_gate_latency` 指定的时钟延迟，在 `compile_fusion` 插入时钟门控单元时标注到寄存器。但若在编译后修改时钟门控上的延迟值，必须用 `apply_clock_gate_latency` 手动将延迟值应用到已有时钟门控单元。

注意

用 `set_clock_gate_latency` 修改时钟门控延迟后，若再用 `compile_fusion` 编译设计，则不必再用 `apply_clock_gate_latency` 应用延迟值；工具会在编译期间标注指定值。

要用 `reset_clock_gate_latency` 移除先前由 `set_clock_gate_latency` 或 `apply_clock_gate_latency` 在时钟门控单元上设置的时钟延迟信息。该命令移除指定时钟上的时钟延迟值；若未指定时钟，则移除全部时钟门控单元上的时钟延迟值。

`set_clock_latency` 命令

The `set_clock_latency` Command

用 `set_clock_latency` 为特定时钟门控单元指定时钟网络延迟。

在图 2.11 中，`lat_cgtoreg` 是从时钟门控单元时钟 pin 到被门控寄存器时钟 pin 的估计延迟；`lat_reg` 是到无时钟门控的寄存器时钟 pin 的估计时钟网络延迟。

(原书 Figure 14: Clock Latency With Clock-Gating Design)

`lat_reg` / `lat_cgtoreg`; Clock-Gating Cell 与 Register

图 2.11: 带时钟门控设计的时钟延迟

对设计中由特定时钟驱动的全部寄存器（有门控或无门控）的时钟 pin，对 `set_clock_latency` 使用 `lat_reg` 值。对全部时钟门控单元的时钟 pin，使用 `lat_reg` 与 `lat_cgtoreg` 之差。由于设置延迟值的目的是计入寄存器与时钟门控单元之间不同的时钟网络延迟，获得合理准确的差值（`lat_cgtoreg`）很重要。除非要用这些值处理与时钟门控无关的时钟网络延迟问题，否则绝对值相对次要。

set_clock_gate_latency 命令

The set_clock_gate_latency Command

使用 `compile_fusion` 时，时钟门控在编译过程中插入。要在工具插入时钟门控单元之前指定时钟网络延迟，使用 `set_clock_gate_latency`。该命令允许你将时钟门控单元的时钟网络延迟指定为时钟域、时钟门控级数（stage）以及时钟门控单元扇出的函数。你指定的延迟在 `compile_fusion` 插入时钟门控单元时标注到这些单元。可用 `apply_clock_gate_latency` 手动将延迟值标注到设计中已有的时钟门控单元。

`set_clock_gate_latency` 接受下列参数：

- `-clock clock_list`
- `-stage clock_gate_stage`
- `-fanout_latency fanout_list`

`fanout_list` 是指定扇出范围与延迟减量的元组列表。下例创建三个扇出范围：1 到 5 延迟为 0.5；6–15 为 0.8；16 到无穷大为 0.6。

```
set_clock_gate_latency -stage 1 \
    -fanout_latency {{1-5 0.5} {6-15 0.8} {16-inf 0.6}}
```

要为被时钟门控的寄存器指定时钟延迟值，对 `-stage` 使用值 0。由于是为被门控寄存器指定延迟，`-fanout_latency` 应为 `1-inf`（1 到无穷大），且延迟为寄存器的绝对时钟延迟值，例如：

```
set_clock_gate_latency -clock CLK -stage 0 \
    -fanout_latency {1-inf 1.0}
```

若未指定 `-clock`，设置适用于设计中全部时钟。用 `set_clock_latency` 设置的时钟延迟优先级更高，不会被覆盖。

图 2.12 给出时钟门控级数与扇出示例。

（原书 *Figure 15: Clock-Gating Stages and Latency Calculations*）

图 2.12: 时钟门控级数与延迟计算

下列命令为图 2.12 设置延迟值：

```
set_clock_gate_latency -stage 0 -fanout_latency {{1-inf 1.0}}
set_clock_gate_latency -stage 1 -fanout_latency {{1-inf 0.3}}
set_clock_gate_latency -stage 2 -fanout_latency {{1-inf 0.2}}
set_clock_gate_latency -stage 3 -fanout_latency {{1-inf 0.1}}
```

图 2.13 给出另一扇出与延迟计算示例。

(原书 Figure 16: Latency Calculations With Varying Fanout)

图 2.13: 变化扇出下的延迟计算

下列命令为图 2.13 指定延迟与扇出：

```
set_clock_gate_latency -stage 0 \
    -fanout_latency {{1-inf 1.6}}
set_clock_gate_latency -stage 1 \
    -fanout_latency {{1-20 0.6} {21-inf 0.8}}
set_clock_gate_latency -stage 2 \
    -fanout_latency {{1-20 0.2} {21-inf 0.3}}
set_clock_gate_latency -stage 3 \
    -fanout_latency {{1-30 0.1} {31-65 0.8} {66-inf 0.18}}
set_clock_latency "0.4" {uicg_a/CK}
```

注意：对时钟门控 uicg_a，用 set_clock_latency 赋值为 0.4 的延迟不会被覆盖。

2.10 为布局与合法化指定物理约束

Specifying Physical Constraints for Placement and Legalization

布局与合法化期间，floorplan 信息决定单元放置位置。下列主题说明如何指定影响布局与合法化的额外物理约束：

- 定义 Keepout Margins
- 定义基于区域的布局障碍（Area-Based Placement Blockages）
- 定义布局 Bounds（Placement Bounds）
- 定义布局吸引（Placement Attractions）
- 为合法化定义单元间距约束

2.10.1 定义 Keepout Margins

Defining Keepout Margins

Keepout margin 是 block 中固定单元边界周围的区域（如图 2.14 阴影部分），其中不放置其他单元。

Outer keepout margin 是单元边界外的区域；**inner keepout margin** 是单元边界内的区域。固定单元各侧 keepout margin 的宽度可以相同或不同，取决于如何定义。此外，keepout margin 可定义为 hard 或 soft。将单元排除在此类区域之外可避免拥堵与 net 绕行，并获得更好的 QoR。

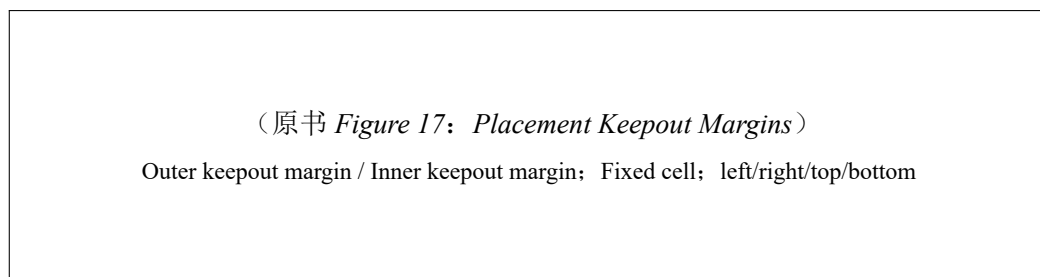


图 2.14: 布局 Keepout Margins

要用 `create_keepout_margin` 定义 keepout margin。默认创建 hard keepout margin；要创建 soft keepout margin，使用 `-type soft`。

定义 Outer Keepout Margin

Defining an Outer Keepout Margin

可在硬宏、层次单元或叶子单元上定义 outer keepout margin。定义时，可显式指定 keepout 距离；对硬宏，也可让工具根据宏 pin 数量推导 keepout 距离。

要显式指定 outer keepout margin，用 `-outer` 为各侧指定边距。按 `{lx by rx ty}` 格式指定左、下、右、上边距。值为 0 表示该侧无 keepout margin。

例如，为名为 `my_macro` 的宏创建各侧边距为 10 的 hard outer keepout margin：

```
icc2_shell> create_keepout_margin -outer {10 10 10 10} my_macro
```

要让工具根据硬宏的 pin 数量推导 outer keepout 距离，用 `-tracks_per_macro_pin` 指定 track 与 pin 之比。使用该选项时，工具根据 track 宽度、宏 pin 数量以及指定的 track-to-pin 比计算 keepout margin，该比通常设为接近 0.5 的值。较大值产生较大 keepout margin。推导的 keepout margin 始终为 hard；`-type` 设置被忽略。推导的边距受 `-min_padding_per_macro` 与 `-max_padding_per_macro` 指定的最小与最大值约束。

例如，以 track-to-pin 比 0.6、最小 keepout 距离 0.1、最大 keepout 距离 0.2 为名为 `my_macro` 的宏推导 outer keepout margin：

```
icc2_shell> create_keepout_margin -tracks_per_macro_pin 0.6 \
-min_padding_per_macro 0.1 -max_padding_per_macro 0.2 my_macro
```

定义 Inner Keepout Margin

Defining an Inner Keepout Margin

可在层次单元上定义 inner keepout margin，但不能在硬宏或叶子单元上定义。定义 inner keepout margin 时必须显式指定 keepout 距离。

要用 `-inner` 为各侧指定边距，格式为 `{lx by rx ty}`。值为 0 表示该侧无 keepout margin。

例如，为名为 `my_hcell` 的层次单元创建各侧边距为 10 的 hard inner keepout margin：

```
icc2_shell> create_keepout_margin -inner {10 10 10 10} my_hcell
```

2.10.2 定义基于区域的布局障碍

Defining Area-Based Placement Blockages

基于区域的布局障碍是矩形区域，其中不能放置单元，或对单元类型或数量加以限制。IC Compiler II 支持下列类型的基于区域的布局障碍：

- **Hard:** 在粗布局、优化与合法化期间阻止在指定区域内放置标准单元与硬宏。
- **Hard macro:** 在粗布局、优化与合法化期间阻止在指定区域内放置硬宏。
- **Soft:** 在粗布局期间阻止在指定区域内放置标准单元与硬宏，但允许优化与合法化在指定区域内放置单元。
- **Partial:** 在粗布局期间限制指定区域内的单元密度，对优化与合法化无影响。

要用 `create_placement_blockage` 定义布局障碍。至少须指定布局障碍的坐标。要创建矩形布局障碍，用 `-boundary` 指定矩形左下角与右上角坐标；要创建直角多边形布局障碍，用 `-boundary` 指定多边形坐标。也可通过将边界多边形指定为 `geo_mask` 或物理对象集合来创建多个布局障碍。若结果区域解析为多个非连续多边形，命令为每个多边形创建一个布局障碍。

默认情况下，`create_placement_blockage` 创建 **hard** 布局障碍。要创建其他类型，用 `-type` 指定障碍类型。单次 `create_placement_blockage` 只能创建一种类型的布局障碍。

可用 `-name` 为布局障碍指定名称，以便按名查询或移除。

定义 **Hard** 布局障碍

Defining a Hard Placement Blockage

要定义 **hard** 布局障碍，指定边界，并可选择指定名称。

例如，在角点为 (10,20) 与 (100,200) 的矩形区域内创建 **hard** 布局障碍：

```
create_placement_blockage -boundary {10 20 100 200}
```

注意

Hard 布局障碍在布局、合法化、优化与时钟树综合期间均被遵从。

Hard 布局障碍也可在 DEF 中定义，如 Example 13。

```
# Example 13: Placement Blockages in DEF
BLOCKAGES 2 ;
- PLACEMENT
  RECT ( 0 327600 ) ( 652740 327660 ) ;
- PLACEMENT
  RECT ( 0 327600 ) ( 652740 327660 ) ;
END BLOCKAGES
```

定义 **Hard Macro** 布局障碍

Defining a Hard Macro Placement Blockage

要定义 **hard macro** 障碍，指定边界、类型 (`-type hard_macro`)，并可选择指定名称。

例如，定义角点为 (120,75) 与 (230,200) 的 **hard macro** 障碍：

```
create_placement_blockage -boundary {120 75 230 200} \
    -type hard_macro
```

注意

Hard macro 布局障碍在布局、合法化与优化期间被遵从。这是硬宏布局所遵从的唯一布局障碍类型。

定义 **Soft** 布局障碍

Defining a Soft Placement Blockage

要定义 soft 障碍，指定边界、类型（**-type soft**），并可选择指定名称。

例如，定义角点为 (120,75) 与 (230,200) 的 soft 障碍：

```
create_placement_blockage -boundary {120 75 230 200} \
    -type soft
```

Soft 障碍阻止初始布局在指定区域内放置单元，但允许合法化、优化与时钟树综合这样做。然而，在布局与优化之后的后续增量布局中，工具可将合法化、优化与时钟树综合期间添加的单元移出 soft blockage 区域。要防止此行为，使用下列应用选项设置：

```
set_app_options -name place.coarse.enable_enhanced_soft_blockages \
    -value true
```

注意

Soft 布局障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义 **Partial** 布局障碍

Defining a Partial Placement Blockage

要定义 partial 障碍，指定边界、类型（**-type partial**）、障碍百分比（**-blocked_percentage**），并可选择指定名称。

例如，定义最大允许单元密度为 60%（blocked percentage 为 40）、角点为 (10,20) 与 (100,200) 的 partial 障碍：

```
create_placement_blockage -boundary {10 20 100 200} \
    -type partial -blocked_percentage 40
```

要允许无限制使用 partial blockage 区域，将障碍百分比指定为 0（**-blocked_percentage 0**）。

注意

Partial 布局障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义预定义类别的障碍

Define a Blockage of a Predefined Category

类别障碍（category blockage）是一种特殊的 partial 障碍，用于控制预定义单元类别在 partial blockage 内的布局。

要创建类别障碍：

1. 用 `define_user_attribute` 为类别定义用户属性，并用 `set_attribute` 将其应用到受影响的单元参考或实例。

要阻止单元放入类别障碍，将属性设为 `true`；要允许放入，设为 `false`。

定义与应用属性时遵守下列规则：

- 一个障碍只能由单个属性控制；
- 多个障碍可由同一属性控制；
- 一个单元可有多个属性，从而影响其在多个类别障碍中的布局；
- 同一属性可应用到参考与实例；
- 若单元实例与其参考的属性不同，实例上的属性优先。

2. 用 `create_placement_blockage` 配合 `-type category`、`-blocked_percentage` 与 `-category` 创建类别障碍。`-category` 的参数为用户属性名。

下例定义阻止某类单元放入其中的障碍：

```
define_user_attribute -type boolean \
  -classes lib_cell dr_cells
set_attribute [get_lib_cells mylib/dr*] dr_cells true
create_placement_blockage -boundary {{4300 2400} {4500 2700}} \
  -type category -blocked_percentage 45 -category dr_cell
```

下例定义仅允许某类单元放入其中的障碍：

```
define_user_attribute -type boolean \
  -classes lib_cell non_iso
set_attribute [get_lib_cells mylib/*] non_iso true
set_attribute [get_lib_cells mylib/iso*] non_iso false
create_placement_blockage -boundary {{4300 2400}{4500 2700}} \
  -type category -blocked_percentage 30 -category non_iso
```

注意

类别障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义排除寄存器的障碍

Defining Blockages That Exclude Registers

要定义排除寄存器的 partial 障碍，指定边界、类型（`-type register`）、障碍百分比（`-blocked_percentage`），并可选择指定名称。

例如，定义排除寄存器、但对其他单元允许 50% 单元密度的 partial 障碍：

```
create_placement_blockage -boundary {{10 20} {100 200}} \
    -type register -blocked_percentage 50
```

注意

寄存器障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义排除相对布局组的障碍

Defining Blockages That Exclude Relative Placement Groups

要定义排除相对布局组（relative placement groups）的 partial 障碍，指定边界、类型（-type rp_group）、障碍百分比（-blocked_percentage），并可选择指定名称。

例如，定义排除相对布局组、但对所有其他单元允许 100% 单元密度的 partial 障碍：

```
create_placement_blockage -boundary {{10 20} {100 200}} \
    -type rp_group -blocked_percentage 0
```

注意

相对布局组障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义仅允许相对布局单元的障碍

Defining Blockages That Allow Relative Placement Cells Only

要定义仅允许相对布局单元的 partial 障碍，指定边界、类型（-type allow_rp_only）、障碍百分比（-blocked_percentage），并可选择指定名称。

例如，定义仅允许相对布局单元、最大允许单元密度 80%（blocked percentage 为 20）、角点为 (10, 20) 与 (100, 200) 的 partial 障碍：

```
create_placement_blockage -name rp_only -type allow_rp_only \
    -boundary {10 20 100 200} -blocked_percentage 20
```

这些障碍允许工具放置相对布局组（通常是大型结构），而不干扰其他单元的布局。工具布局并合法化相对布局组后，可移除这些障碍并允许工具在其中放置其他单元。

注意

仅相对布局单元障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义仅允许缓冲器的障碍

Defining Blockages That Allow Buffers Only

要定义仅允许缓冲器的 partial 障碍，指定边界、类型（-type allow_buffer_only）、障碍百分比（-blocked_percentage），并可选择指定名称。

例如，定义仅允许放置缓冲器与反相器、单元密度 30% 的 partial 障碍：

```
create_placement_blockage -boundary {{10 20} {100 200}} \
    -type allow_buffer_only -blocked_percentage 70
```

注意

仅缓冲器障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

查询布局障碍

Querying Placement Blockages

要用 `get_placement_blockages` 返回当前 block 中匹配特定条件的布局障碍集合。

移除布局障碍

Removing Placement Blockages

要用 `remove_placement_blockages` 从 block 移除布局障碍。要用 `-all` 移除全部布局障碍；要移除特定布局障碍，指定布局障碍名称。

2.10.3 定义布局 Bounds

Defining Placement Bounds

布局 bound 是控制成组单元布局的约束。它允许你将单元分组以最小化线长，并将单元放在最合适的位置。例如，你可能希望为时钟门控单元或极关键时序单元组定义 bound，以保证它们不会因其他逻辑的布局而被打散。

表 2.3 列出 IC Compiler II 支持的布局 bound 类型。

要用 `create_bound` 定义布局 bound。定义时必须用 `-name` 指定其名称。

一般还须指定要包含在 bound 中的单元与 port。若包含层次单元，则子设计中的全部单元属于该 bound。但也可创建空 bound，之后用 `add_to_bound` 指定内容；可用 `remove_from_bound` 从 bound 移除对象。

还必须指定创建所需 bound 类型所需的选项。下列主题说明如何创建各类 move bound：

- 定义 Move Bounds
- 定义 Group Bounds
- 查询布局 Bounds
- 移除布局 Bounds

定义 Move Bounds

Defining Move Bounds

Move bound 是放置一组单元的固定区域。它由一个或多个矩形或直角多边形 shape 组成，这些 shape 可以邻接、不相交或重叠。要用 `-boundary` 定义这些 shape 的边界：创建新 move bound 时用 `create_bound`；向已有 move bound 添加 shape 时用 `create_bound_shape`。注意：`create_bound` 可一次指定一个或多个 shape，而每次 `create_bound_shape` 只能指定单个 shape。

表 2.3: 布局 Bound 类型

Bound 类型	说明
Soft move bound	工具在粗布局期间设法将 move bound 中的单元放在指定区域内，但不保证单元一定放在 bound 内。Soft move bound 在粗布局期间被遵从，合法化期间不被遵从。
Hard move bound	工具必须将 move bound 中的单元放在指定区域内。Hard move bound 在粗布局与合法化期间均被遵从，但时钟树综合期间不被遵从。
Exclusive move bound	工具必须在粗布局与合法化期间将指定单元放在指定区域内，并将所有其他单元放在排他性 move bound 之外。排他性 move bound 中新添加缓冲器的布局由 <code>opt.buffering.exclusive_hard_bound_buffering_mode</code> 控制。默认在缓冲期间，若能改善 QoR，工具可向排他性 hard move bound 添加新单元。若将该应用选项设为 0，工具遵从排他性 hard move bound。
Soft group bound	工具设法将 group bound 中的单元放在浮动区域内；但不保证单元一定放在 bound 内。
Hard group bound	工具必须将 group bound 中的单元放在浮动区域内，其实际坐标由工具确定。
Dimensionless group bound	工具根据用于使 group bound 内单元彼此靠近的努力程度，确定 group bound 的形状与位置。

- 指定矩形边界时，用下列格式给出左下角与右上角：

```
{{llx lly} {urx ury}}
```

- 指定直角多边形边界时，用下列格式给出各顶点坐标：

```
{{x1 y1} {x2 y2} {x3 y3} {x4 y4} ...}
```

Move bound 可以是 hard、soft 或 exclusive。

- 定义 soft move bound 的语法：

```
create_bound -name name [-type soft] [-effort effort_level] \
  -boundary {coordinates} [bound_objects]
```

默认 effort 为 ultra；也可指定 low、medium 或 high。

例如，为 INST_1 单元实例定义左下角 (100,100)、右上角 (200,200) 的矩形 soft move bound：

```
icc2_shell> create_bound -name b1 -boundary {100 100 200 200} INST_1
```

- 定义 hard move bound 的语法：

```
create_bound -name name -type hard -boundary {coordinates} \
  [bound_objects]
```

例如，为 INST_1 定义矩形 hard move bound：

```
icc2_shell> create_bound -name b2 -type hard \
    -boundary {100 100 200 200} INST_1
```

- 定义 exclusive move bound 的语法:

```
create_bound -name name -exclusive -boundary {coordinates} \
    [bound_objects]
```

例如, 为 INST_1 定义矩形 exclusive move bound:

```
icc2_shell> create_bound -name b3 -exclusive \
    -boundary {100 100 200 200} INST_1
```

要用 create_bound_shape 向已有 move bound 添加 shape; 用 remove_bound_shapes 从已有 move bound 移除 shape。

定义 Group Bounds

Defining Group Bounds

Group bound 是放置一组单元的浮动区域。Group bound 可以是 hard、soft 或 dimensionless。

- 定义 soft group bound 的语法:

```
create_bound -name name [-type soft] [-effort effort_level] \
    [-dimensions {width height} bound_objects]
```

默认 effort 为 ultra; 也可指定 low、medium 或 high。

例如, 为 INST_1 与 INST_2 定义宽 100、高 100 的 soft group bound:

```
icc2_shell> create_bound -name b4 -dimensions {100 100} \
    {INST_1 INST_2}
```

- 定义 hard group bound 的语法:

```
create_bound -name name -type hard -dimensions {width height} \
    [bound_objects]
```

例如, 为 INST_1 与 INST_2 定义宽 100、高 100 的 hard group bound:

```
icc2_shell> create_bound -name b5 -type hard -dimensions {100 100} \
    {INST_1 INST_2}
```

- 定义 dimensionless group bound 的语法:

```
create_bound -name name [-effort effort_level] \
    [bound_objects]
```

默认 effort 为 medium; 也可指定 low、high 或 ultra。

例如, 为 INST_1 与 INST_2 单元实例定义无量纲 group bound, 并要求工具以 high effort 使单元在 bound 内更紧密放置, 可用:

```
icc2_shell> create_bound -name b6 -effort high {INST_1 INST_2}
```

2.10.4 查询布局 Bound

Querying Placement Bounds

要报告 block 中的布局 bound，使用 `report_bounds` 命令。

要返回当前 block 中匹配指定条件的布局 bound 集合，使用 `get_bounds` 命令。

要返回与一个或多个 move bound 关联的 bound shape 集合，使用 `get_bound_shapes` 命令。

2.10.5 移除布局 Bound

Removing Placement Bounds

要从 block 中移除布局 bound，使用 `remove_bounds` 命令。要移除 block 中全部布局 bound，使用 `-all` 选项；要移除特定布局 bound，指定 bound 名称。

2.10.6 定义布局 Attraction

Defining Placement Attractions

布局 attraction 是一种约束，用于指定一大组单元应放置在布局区域的同一邻近范围内。它是软约束，工具在初始布局时会考虑；但在后续增量布局阶段影响较小。若要对较小单元组施加更严格的布局约束，请用 `create_bound` 创建 move bound 或 group bound。

定义布局 attraction 使用 `create_placement_attraction` 命令。例如，指定子块 `add1` 与 `mult1` 中的单元放在同一邻近范围：

```
icc2_shell> set add1_cells [get_flat_cells add1/*]
icc2_shell> set mult1_cells [get_flat_cells mult1/*]
icc2_shell> create_placement_attraction -name add1_mult1 \
"$add1_cells $mult1_cells"
```

可用 `-region` 选项指定：

- 放置单元的区域：给出区域左下角与右上角坐标，例如：

```
icc2_shell> set U1_cells [get_flat_cells U1/*]
icc2_shell> create_placement_attraction -name U1 \
-region {{0 0} {2000 1500}} $U1_cells
```

- 沿某直线放置单元（例如宏单元边缘）：给出直线两端坐标，例如：

```
icc2_shell> set U2_cells [get_flat_cells U2/*]
icc2_shell> create_placement_attraction -name U2 \
-region {{1000 200} {1000 800}} $U2_cells
```

- 围绕某位置放置单元（例如 port 位置）：给出该位置坐标，例如：

```
icc2_shell> set U3_cells [get_flat_cells U3/*]
icc2_shell> create_placement_attraction -name U3 \
-region {{0 700}} $U3_cells
```

下表列出用于修改、报告与移除布局 attraction 的其他命令。更多信息见各命令 man 页。

表 2.4: 与布局 Attraction 相关的命令（原书 Table 5）

目的	使用的命令
定义布局 attraction	create_placement_attraction
向已有布局 attraction 添加单元	add_to_placement_attraction
从已有布局 attraction 移除单元	remove_from_placement_attraction
报告布局 attraction	report_placement_attractions
查找已有布局 attraction	get_placement_attractions
移除已有布局 attraction	remove_placement_attractions

参见

定义 Move Bounds。

2.10.7 定义合法化单元间距约束

Defining Cell Spacing Constraints for Legalization

单元间距约束控制标准单元与另一标准单元或边界之间的间距（芯片边界、硬宏、硬宏 keep-out margin、硬布局 blockage，或 voltage area guard band）。将库单元分配到组（边界属于预定义组 SNPS_BOUNDARY），再定义这些组中单元之间的所需间距。

默认情况下，合法化期间标准单元之间没有间距约束。为提升良率，可定义标准单元之间，或标准单元与边界之间的有效间距。

注意

标准单元与电源/地 net 之间间距约束的支持，取决于该 net 在 block 中的表示方式。若电源/地 net 定义为完整 blockage，合法化器与 check_legality 命令会忽略标准单元与该电源/地 net 之间的间距规则违例。若定义为部分 blockage，则合法化器与 check_legality 会检查标准单元与电源/地 net 之间的间距规则违例。

单元间距约束的实现方式是：在库单元左右两侧（假定单元为 north 朝向）附着类似属性的 label，并规定这些 label 之间的无效间距。

定义单元间距约束的步骤：

1. 使用 set_placement_spacing_label 命令，为有间距约束的库单元添加 label。
每个 label 须指定：

- label 名称（-name 选项）
- 应用该 label 的库单元（-lib_cells 选项）
- 应用该 label 的库单元侧面（-side 选项，可取 right、left 或 both）

label 定义可累加；同一 label 可用于某些单元的右侧与另一些单元的左侧。例如，将名为 x 的 label 赋给 cellA 的右侧，以及 cellB、cellC 的左侧：

```
icc2_shell> set_placement_spacing_label -name X \  
-lib_cells {cellA} -side right
```

```
icc2_shell> set_placement_spacing_label -name X \
-lib_cells {cellB cellC} -side left
```

也可为一侧分配多个 label。例如，将 Y 与 Z 赋给 cellB 的右侧：

```
icc2_shell> set_placement_spacing_label -name Y \
-lib_cells {cellB} -side right
icc2_shell> set_placement_spacing_label -name Z \
-lib_cells {cellB} -side right
```

2. 使用 set_placement_spacing_rule 命令定义 label 之间的间距要求。

每条规则须指定：

- 受约束的 label（-labels 选项）

每次 set_placement_spacing_rule 必须恰好指定两个 label。可为 set_placement_spacing_label 定义的任意 label，或预定义的 SNPS_BOUNDARY label（包含芯片边界、硬宏边界、硬宏 keepout margin、硬布局 blockage 与 voltage area guard band）。两个 label 可相同也可不同。

- 无效间距范围，以 unit tile 个数计。

例如，要求 label X 与 Y 之间至少有一个 unit tile（不可 abut）：

```
icc2_shell> set_placement_spacing_rule -labels {X Y} {0 0}
```

要求 X label 与任意边界之间至少有一个 unit tile：

```
icc2_shell> set_placement_spacing_rule \
-labels {X SNPS_BOUNDARY} {0 0}
```

要求两个 X label 之间不能恰好间隔两个 unit tile：

```
icc2_shell> set_placement_spacing_rule -labels {X X} {2 2}
```

要求 label X 与 Z 的间距小于 2 个或大于 4 个 unit tile：

```
icc2_shell> set_placement_spacing_rule -labels {X Z} {2 4}
```

注意

Caution: 单元间距约束不会随 block 保存；仅作用于当前会话，每个会话都必须重新定义。

2.10.8 报告单元间距约束

Reporting Cell Spacing Constraints

报告单元间距规则使用 report_placement_spacing_rules 命令。该命令报告当前会话中定义的单元间距 label、其所应用的库单元（及侧面），以及为其定义的规则。

2.10.9 移除单元间距约束

Removing Cell Spacing Constraints

移除间距规则使用 `remove_placement_spacing_rules` 命令，必须指定要移除的规则：

- 移除全部间距规则：使用 `-all` 选项。
- 移除特定 `label` 及其关联的全部规则：用 `-label` 指定该 `label`。
- 移除两 `label` 之间的规则：用 `-rule` 指定与该规则关联的两个 `label`。

2.11 指定布局设置

Specifying Placement Settings

工具在布线前流程的多个阶段执行粗布局（`coarse placement`）。以下主题说明如何控制粗布局：

- 在早期阶段使用不精确约束进行布局
- 为时钟门控单元自动生成 Group Bound
- 控制布局密度
- 控制布局期间的拥塞驱动重构
- 降低拥塞
- 布局时考虑宽单元密度
- 布局时考虑单元 Pin 的影响
- 布局时考虑时钟 Net 非默认布线规则带来的拥塞影响
- 布局时考虑时序阵列中时钟门控单元的影响
- 布局时考虑合法化影响
- 布局时考虑 DFT 连接
- 布局时考虑动态功耗 QoR
- 执行 IR-Drop 感知布局
- 布局时分散 Repeater 单元

2.11.1 在早期阶段使用不精确约束进行布局

Performing Placement With Inaccurate Constraints at Early Stages

设计周期早期，布局约束可能不精确，导致工具在粗布局时退出。若因区域过小或被 `blockage` 覆盖而过利用率，仍希望继续布局，可将应用选项 `place.coarse.handle_early_data` 设为 `true`。启用后工具会发出如下警告示例：

```
Warning: Utilization of move_bound_0 is 109%. (PLACE-089)
Warning: Placement continues with over utilized regions in the design.
(PLACE-083)
Warning: Overutilized regions for move_bound_0 are modified by removal of
partial and full blockages. (PLACE-084)
```

启用该功能后，若某区域利用率超过 90%，工具会打印警告。要更改该阈值，使用应用选项 `place.coarse.utilization_warning_threshold`。

2.11.2 为时钟门控单元自动生成 Group Bound

Generating Automatic Group Bounds for Clock Gating Cells

工具可自动为集成时钟门控单元及其驱动的时序单元生成 group bound。设置如下：

```
icc2_shell> set_app_options -name place.coarse.icg_auto_bound \
-value true
```

启用后，工具在布局开始时创建自动 group bound，并在布局结束时移除。已属于其他 group bound 的单元不会纳入自动 group bound。

要限制自动 bound 可包含的最大 fanout 数，使用 `place.coarse.icg_auto_bound_fanout_limit`，例如：

```
icc2_shell> set_app_options \
-name place.coarse.icg_auto_bound_fanout_limit -value 30
```

默认 fanout 限制为 40。

2.11.3 控制布局密度

Controlling the Placement Density

可按下表控制 block 的布局密度。

表 2.5: 控制布局密度的应用选项（原书 Table 6）

目的	使用的应用选项
指定最大密度，控制非拥塞驱动布局时单元可放置的密集程度	<code>place.coarse.auto_density_control</code> （默认 <code>-enhanced</code> ；若未将 <code>place.coarse.auto_density_control</code> 设为 <code>false</code> ，工具在流程中按预设计划工作。禁用该选项或手动将 <code>place.coarse.max_density</code> 设为 0 会使单元在 core 区域均匀铺开）
指定最大利用率，控制高拥塞区域周围低拥塞区域的单元可放置密集程度，以便高拥塞区单元可向外摊开以降低拥塞	<code>place.coarse.congestion_driven_max_util</code> （默认 0.93）

指定最大密度或最大利用率时，取值应在 1 与 block 总体利用率之间。例如，若利用率 40%，可在 1 与 0.4 之间选择，如下：

```
icc2_shell> set_app_options -name place.coarse.max_density -value 0.6
icc2_shell> set_app_options \
    -name place.coarse.congestion_driven_max_util -value 0.8
```

若不指定最大密度或最大利用率，且 `place.coarse.auto_density_control` 未设为 `false`，工具会根据设计流程阶段自动推导数值。

默认情况下 `place.coarse.auto_density_control` 已启用且为 `enhanced`，有助于改善总功耗与线长。

注意

无需禁用 `place.coarse.auto_density_control` 即可覆盖 `max_density` 与 `congestion_driven_max_util`；用户设置始终优先于工具推导值。

布局全程工具会打印 `PLACE-027` 信息消息，说明所用取值：

```
Information: Automatic density control has selected the following
settings: max_density 0.60, congestion_driven_max_util 0.77. (PLACE-027)
```

工具将最大密度与拥塞驱动最大利用率设置独立应用于每个可放置区域（如 `voltage area` 或 `exclusive move bound`），而非对整个 `block` 取平均。

2.11.4 控制布局期间的拥塞驱动重构

Controlling Congestion-Driven Restructuring During Placement

默认情况下，工具在下列阶段对 `net` 进行重构以降低拥塞：

- `create_placement` 命令（使用 `-timing_driven` 时除外）
- `place_opt` 命令的 `initial_place` 阶段

可如下控制拥塞驱动重构：

- 将 `place.coarse.cong_restruct_effort` 设为 `low`、`medium`（默认）、`high` 或 `ultra` 以指定 `effort`。
- 将 `place.coarse.cong_restruct_depth_aware` 设为 `true`，防止路径深度增加超过三级逻辑。

2.11.5 降低拥塞

Reducing Congestion

要降低 `block` 拥塞，可使用：

- 将 `place.coarse.congestion_layer_aware` 设为 `true`，按层分别考虑拥塞并提高粗布局中拥塞缓解的精度。

默认工具在布局时合并所有层的拥塞信息。按层考虑后，工具可识别并修复 `pin` 密度高、但下层资源不足以完成 `pin` 连接的区域。

- 将 `place.coarse.increased_cell_expansion` 设为 `true`，根据局部布线资源需求在布局时扩大单元的虚拟面积。

在布线资源短缺区域增大虚拟面积，可降低单元密度与拥塞。

默认工具在水平方向扩展单元,有助于降低垂直布线层拥塞。若拥塞主要在水平布线层,将 `place.coarse.congestion` 设为 `both` (默认为 `horizontal`)。

默认工具用全局布线拥塞图识别需扩展的高拥塞区,但不考虑 `soft routing rule` 带来的拥塞。要考虑 `soft routing rule` 的拥塞效应,将 `route.global.export_soft_congestion_maps` 设为 `true`。

2.11.6 布局时考虑宽单元密度

Considering Wide Cell Density During Placement

若单元无法跨垂直电源 strap,且单元宽度超过电源 strap 间距的一半,则两条电源 strap 之间只能放置一个此类单元。因此布局时必须尽量降低此类宽单元密度,以确保合法化时无需大位移即可放置。

对含宽单元的先进工艺节点,将 `place.coarse.wide_cell_use_model` 设为 `true`,以在布局中启用宽单元建模。

2.11.7 布局时考虑单元 Pin 的影响

Considering the Effects of Cell Pins During Placement

当用 `set_technology -node` 将工艺节点设为 7、7+、5、s5 或 s4 时,可将 `place.coarse.pin_cost_aware` 设为 `true` (默认 `false`),使布局使用工艺相关的 pin-cost 模型。启用后,工具用该模型预测访问各 pin 所需布线资源,以改善 pin 可达性。

对其余工艺节点,可将 `place.coarse.pin_density_aware` 设为 `true` (默认 `false`) 以控制布局时的最大局部 pin 密度。

表 2.6: 控制布局期间 Pin 影响的设置 (原书 Table 7)

<code>set_technology -node</code>	<code>pin_cost_aware</code>	<code>pin_density_aware</code>	工具行为
7, 7+, 5, s5, or s4	<code>true</code>	<code>true</code> 或 <code>false</code>	执行工艺相关的 pin-cost 感知布局
7, 7+, 5, s5, or s4	<code>false</code>	<code>true</code>	执行 pin 密度感知布局
7, 7+, 5, s5, or s4	<code>false</code>	<code>false</code>	布局时不考虑单元 pin 影响
其他任意设置	<code>true</code> 或 <code>false</code>	<code>true</code>	执行 pin 密度感知布局
其他任意设置	<code>true</code> 或 <code>false</code>	<code>false</code>	布局时不考虑单元 pin 影响

2.11.8 布局时考虑时钟 Net 非默认布线规则带来的拥塞影响

Considering the Congestion Effects Due to the Nondefault Routing Rules of Clock Nets

为改善时序并降低串扰,时钟 net 常大量使用非默认布线规则 (NDR)。这些规则需要额外空间,可能在时钟布线后增加拥塞。

要将时钟 net NDR 的影响纳入布局考虑,将 `place.coarse.ndr_area_aware` 设为 `true`。

2.11.9 布局时考虑时序阵列中时钟门控单元的影响

Considering the Effects of Clock Gating Cells of Sequential Arrays During Placement

时序阵列（sequential array）常见于存储类器件，是按二维阵列排列的一组寄存器；同一行寄存器的时钟 pin 通常由单个时钟门控单元驱动。

要将时序阵列的时钟门控单元纳入布局考虑，将 `place.coarse.seq_array_icg_aware` 设为 `true`，有助于降低时钟 net 引起的拥塞。

2.11.10 布局时考虑合法化影响

Considering Legalization Effects During Placement

为减少合法化时大量单元被大幅搬移，粗布局应了解合法化限制。例如，因电源 strap 位置，某宽单元不能放在特定位置。

要将合法化效应纳入布局，将 `place.coarse.enhanced_legalizer_driven_placement` 设为 `true`。适用于合法化时大量单元有小到中等位移的 block；可用合法化报告的 RMS 位移值判断是否降低了位移。

2.11.11 布局时考虑 DFT 连接

Considering DFT Connections During Placement

要在仍优化功能布局的同时，增强工具对 DFT 连接的感知以改善 DFT 逻辑布局，将 `place.coarse.enable_dft_m` 设为 `auto`（默认 `false`）。

2.11.12 布局时考虑动态功耗 QoR

Considering the Dynamic Power QoR During Placement

要在布局时考虑动态功耗 QoR，执行：

1. 按「标注开关活动」所述，为设计标注 switching activity。
 2. 用 `set_scenario_status -dynamic_power true` 确保至少一个 scenario 启用动态功耗优化。
 3. 用下列方法之一，为 `create_placement`、`refine_placement`、`place_opt` 或 `clock_opt` 启用功耗驱动布局：
 - 将 `place.coarse.enhanced_low_power_effort` 设为 `none`、`low`、`medium` 或 `high`（默认 `low`），以启用动态功耗驱动布局。
- 动态功耗驱动布局中，工具尝试改善时序关键 net 的时序与功耗，以及其他 net 的功耗，从而在不影响时序 QoR 的前提下改善功耗 QoR。

2.11.13 执行 IR-Drop 感知布局

Performing IR-Drop-Aware Placement

布局时，工具可用单元的电压（IR）drop 值识别高功耗密度区域，并摊开高压降单元以降低该区域功耗密度。

执行 IR-drop 感知布局的步骤：

1. 对 block 进行布局并合法化。
2. 设置 RedHawk Fusion，并用 `analyze_rail -voltage_drop` 做静态或动态压降分析，例如：

```
icc2_shell> source redhawk_setup.tcl
icc2_shell> analyze_rail -voltage_drop static -nets {VDD VSS}
```

更多信息见「执行压降分析」。

3. 将 `place.coarse.ir_drop_aware` 设为 `true`，启用 IR-drop 感知布局。
4. (可选) 按「控制 IR-Drop 感知布局」指定附加设置。
5. 重新运行布局。

注意

执行 IR-drop 感知布局需要 Digital-AF 与 SNPS_INDESIGN_RH_RAIL license key。

控制 IR-Drop 感知布局

Controlling IR-Drop-Aware Placement

将 `place.coarse.ir_drop_aware` 设为 `true` 后，工具按单元总数创建三类单元。

(原书 *Figure 18: Default Cell Categories for IR-Drop-Aware Placement*)

图 2.15: IR-Drop 感知布局的默认单元类别

默认按图 2.15 所示准则选择三类单元：

- 上层类别：压降最高的约 1% 单元；布局时摊开力度最大。
- 中层类别：接下来约 5% 单元；摊开力度小于上层。
- 下层类别：其余单元；工具不对其进行摊开。

可用 `place.coarse.ir_drop_default_target_high_percentage` 与 `place.coarse.ir_drop_default_target_low_percentage` 更改上层与中层的百分比。例如，上层取前 2%，中层取接下来 6%：

```
icc2_shell> set_app_options \
-name place.coarse.ir_drop_default_target_high_percentage \
-value 2.0
icc2_shell> set_app_options \
-name place.coarse.ir_drop_default_target_low_percentage \
-value 6.0
```

也可不用单元总数百分比,而用压降相对电源电压的百分比定义类别:将 `place.coarse.ir_drop_default_target_high_population` 与 `place.coarse.ir_drop_default_target_low_population` 设为 `false` (默认为 `true`)。

例如, 压降大于电源电压 8% 的单元归入上层, 4%–8% 归入中层:

```
icc2_shell> set_app_options \
  -name place.coarse.ir_drop_default_target_high_population \
  -value false
icc2_shell> set_app_options \
  -name place.coarse.ir_drop_default_target_high_percentage \
  -value 8.0
icc2_shell> set_app_options \
  -name place.coarse.ir_drop_default_target_low_population \
  -value false
icc2_shell> set_app_options \
  -name place.coarse.ir_drop_default_target_low_percentage \
  -value 4.0
```

也可混合: 上层用单元总数百分比, 中层用压降相对电源电压百分比, 或反之。例如上层取前 2% 单元, 中层取压降大于电源电压 4% 的后续单元:

```
icc2_shell> set_app_options \
  -name place.coarse.ir_drop_default_target_high_population \
  -value true
icc2_shell> set_app_options \
  -name place.coarse.ir_drop_default_target_high_percentage \
  -value 2.0
icc2_shell> set_app_options \
  -name place.coarse.ir_drop_default_target_low_population \
  -value false
icc2_shell> set_app_options \
  -name place.coarse.ir_drop_default_target_low_percentage \
  -value 4.0
```

可用 `set_placement_ir_drop_target` 为特定 voltage area 设置约束。例如, 将 VA1 中前 1.5% 单元归入上层、接下来 4.5% 归入中层:

```
icc2_shell> set_placement_ir_drop_target VA1 high 1.5
icc2_shell> set_placement_ir_drop_target VA1 low 4.5
```

将 VA2 中压降大于电源电压 10% 的单元归入上层, 5%–10% 归入中层:

```
icc2_shell> set_placement_ir_drop_target VA2 high 10 -irdrop
icc2_shell> set_placement_ir_drop_target VA2 low 5 -irdrop
```

获取、报告与重置基于 voltage area 的约束, 分别使用 `get_placement_ir_drop_target`、`report_placement_ir_drop_target` 与 `reset_placement_ir_drop_target`。

2.11.14 布局时分散 Repeater 单元

Spreading Repeater Cells During Placement

若 block 在宏单元或 blockage 边缘/角落有大量 repeater 链 (buffer、inverter 或流水线寄存器), 工具可能沿边缘/角落聚集单元从而加剧拥塞。可在运行 `create_placement`、`place_opt` 或 `clock_opt`

前将 `place.coarse.spread_repeater_paths` 设为 `true`；工具会沿正交方向摊开 `repeater`，以降低边缘与角落拥塞。

2.12 指定合法化设置

Specifying Legalization Settings

工具在设计流程多个阶段执行合法化。以下主题说明如何控制合法化：

- 合法化期间最小化大位移
- 合法化期间优化 Pin Access
- 启用高级 PG Net 检查
- 启用高级合法化算法
- 设置 Variant 感知合法化
- 检查库单元是否可合法放置

2.12.1 合法化期间最小化大位移

Minimizing Large Displacements During Legalization

要最小化合法化时单元大位移，可启用：

- 朝向优化：将 `place.legalize.optimize_orientations` 设为 `true`。
工具会考虑翻转单元朝向以减小合法化位移。
- Stream placement：将 `place.legalize.stream_place` 设为 `true`。
工具将许多单元小幅移动，避免单颗单元大幅移动。可用 `place.legalize.stream_effort` 与 `place.legalize.stream_effort_limit` 进一步控制。

2.12.2 合法化期间优化 Pin Access

Optimizing Pin Access During Legalization

对先进工艺节点，为改善高 pin 密度区域的可布线性，可通过重分布单元启用 pin 优化：将 `place.legalize.optimize_pin_access_using_cell_spacing` 设为 `true`。

2.12.3 启用高级 PG Net 检查

Enabling Advanced PG Net Checks

对 12 nm、7 nm 或更小等先进节点，可将 `place.legalize.enable_advanced_prerouted_net_check` 设为 `true`，启用单元与预布线 PG net 之间的高级物理 DRC 检查。

启用后，可用 `place.legalize.advanced_layer_access_check` 与 `place.legalize.advanced_libpin_access_c` 进一步控制标准单元 pin 的可达性检查；对多数设计，默认行为已适用。

2.12.4 启用高级合法化算法

Enabling Advanced Legalization Algorithms

要将用于 2D 规则检查与单元交互的高级合法化算法启用(可缩短合法化 runtime),将 `place.legalize.enable_a` 设为 `true`。

要指定未自动检测的附加高级合法化规则,设置 `place.legalize.enable_advanced_legalizer_rules`。

启用高级合法化算法后,默认在 `pin access` 优化时对齐垂直 `pin shape` 以改善可布线性。也可通过 `place.legalize.optimize_pin_access_strategies` 指定附加策略,见下表。

表 2.7: `place.legalize.optimize_pin_access_strategies` 设置 (原书 Table 8)

目的	使用的设置
对齐水平 <code>pin shape</code>	<code>horizontal_align</code>
将单元移离高 <code>pin</code> 密度区域	<code>avoid_high_pin_density</code>
同时做以上两者	<code>horizontal_align avoid_high_pin_density</code>

高级合法化与合法性检查期间,工具可同时检查多单元与 `PG net shape` 之间的 `DRC`。启用:将 `place.legalize.enable_multi_cell_pnet_checks` 设为 `true`。

要在最后一次 `route_opt` 或 `route_auto` 之后启用多单元检查,将下列应用选项设为 `true` (须先启用 `place.legalize.enable_multi_cell_pnet_checks`),有助于在考虑 `QoR` 的同时改善 `runtime`:

- `place.legalize.enable_multi_cell_access_check`: 考虑邻接单元与附近预布线 `PG net shape` 对单元 `pin access` 的影响。默认 `false`。
- `place.legalize.enable_multi_cell_track_capacity_check`: 分析该单元 `pin` 与邻接单元的 `track` 容量。`Track capacity` 估计该区域布线 `track` 是否足以访问全部指定 `pin`。启用本选项前须先启用 `place.legalize.enable_multi_cell_access_check`。默认 `false`。

启用多线程高级合法化:

1. 将 `place.legalize.enable_threaded_advanced_legalizer` 设为 `true`。
2. 按「配置多线程」完成多线程配置。

执行多线程高级合法性检查:

1. 将 `place.legalize.enable_advanced_legalizer` 设为 `true`。
2. 按「配置多线程」完成多线程配置。
3. 用 `check_legality` 执行合法性检查。

2.12.5 设置 Variant 感知合法化

Setting Up for Variant-Aware Legalization

某些库提供功能等效、但可在不同位置合法化的单元集合。工具可通过用 `variant` 替换单元来合法化,而不必搬移单元。

下图示出 `pin` 位置不同的功能等效单元。使用 `variant` 感知合法化时,若单元 `pin` 未对齐 `track`,工具可尝试用其 `variant` 使 `pin` 对齐 `track`。

(原书 *Figure 19: Equivalent Cells With Different Pin Locations*)

图 2.16: Pin 位置不同的等效单元

下图示出 pin 颜色不同的功能等效单元，须对齐同色 track。若 pin 未对齐对应颜色 track，工具可尝试使用不同颜色 pin 的 variant 并对齐到合适颜色的 track。

(原书 *Figure 20: Equivalent Cells With Pins of Different Colors*)

图 2.17: Pin 颜色不同的等效单元

设置参考库与设计以支持 variant 感知合法化，需完成：

- 定义等效单元组
- 启用 Variant 感知合法化

定义等效单元组

Defining Equivalent Cell Groups

Library Compiler 支持从 Liberty 源文件创建 variant-ready 库。Liberty 中的 `physical_variant_cells` 属性定义单元的 variant；若该属性存在，编译后的库即为 variant ready。

若逻辑库非 variant ready，可在 Library Manager 的库准备阶段用 `create_cell_groups` 定义等效单元组。例如向已有参考库添加等效单元组：

```
lm_shell> create_workspace -flow edit LIB.ndm
lm_shell> create_cell_group -name A [get_lib_cells MY_LIB/A_*]
lm_shell> create_cell_group -name B [get_lib_cells MY_LIB/B_*]
lm_shell> check_workspace
lm_shell> commit_workspace
```

在 Library Manager 或 IC Compiler II 中，可用 `report_cell_groups` 报告库中的等效单元组；该命令既报告 variant-ready 逻辑库 Liberty 中定义的组，也报告 Library Manager 中用 `create_cell_groups` 定义的组。

启用 Variant 感知合法化

Enabling Variant-Aware Legalization

将 `place.legalize.enable_variant_aware` 设为 true 以启用。启用后，工具优先用 variant 交换违例单元来修复合法化 DRC，而非搬移；若无 variant，再移到合法位置。

Variant 感知合法化中的 pin-track 与 pin-color 对齐，须将下列应用选项设为 **true**:

- `place.legalize.enable_prerouted_net_check`
- `place.legalize.enable_pin_color_alignment_check`

并用 `place.legalize.pin_color_alignment_layers` 指定要对齐的层。

2.12.6 检查库单元是否可合法放置

Checking if Library Cells Are Legally Placeable

指定全部布局与合法化约束及设置后，可用 `analyze_lib_cell_placement -lib_cells` 检查特定库单元能否在 `block` 中合法放置。

- 用 `-region` 将分析限制在 `core` 的特定区域（默认搜索整个 `core`）。
- 用 `-trials` 限制分析的 `site` 数量（默认随机分析 1000 个 `site`）。设为 0 则分析全部空闲 `site`，可能增加 `runtime`。
- 用 `-no_pdc` 或 `-no_adv` 在分析时忽略物理设计约束或高级设计规则。
- 用 `-threshold` 仅报告未达指定阈值的相对布局组：仅当单元可放置 `site` 占所分析 `site` 的百分比低于阈值时才报告。
- 用 `-max_cells` 限制报告单元数（默认 100，从通过率最差的库单元开始）。

若 `block` 中某些库单元通过率极低，合法化时会导致大位移与更长 `runtime`。此时应检查电源计划、单元版图与合法化设置以提高通过率。

下列示例分析当前 `block` 中有实例的全部库单元：

```
icc2_shell> analyze_lib_cell_placement -lib_cells [add_to_collection \
  -unique "" [get_attribute [get_cells -physical_context] ref_phys_block]]
```

Analyzing 215 lib cell(s).

Warning: Routing direction of metal layer P0 is neither "horizontal" nor "vertical". PDC checks will not be performed on this layer. (PDC-003)

PDC app_options settings =====

```
place.legalize.enable_prerouted_net_check: 1
place.legalize.num_tracks_for_access_check: 1
place.legalize.use_eol_spacing_for_access_check: 0
place.legalize.allow_touch_track_for_access_check: 1
place.legalize.reduce_conservatism_in_eol_check: 0
place.legalize.preroute_shape_merge_distance: 0.0
```

Layer M1: cached 0 shapes out of 869 total shapes.

Layer M2: cached 773 shapes out of 773 total shapes.

Cached 41757 vias out of 99466 total vias.

0.0%...2.0%...4.0%...6.0%...8.0%...10.0%.....

Lib Cell	Pass Rate
saed32_lvt_lsup:LSUPX1_LVT.frame	0.4450

saed32_lvt_1sup:LSUPX8_LVT.frame	0.4780
saed32_lvt_1sup:LSUPX2_LVT.frame	0.4810
saed32_lvt_1sup:LSUPX4_LVT.frame	0.5040
.....	
saed32_rvt_std:HADDX1_RVT.frame	0.8600
saed32_lvt_std:OA21X1_LVT.frame	0.8610

2.13 控制单元、Net、Pin 与 Port 的优化

Controlling the Optimization of Cells, Nets, Pins, and Ports

以下主题说明如何控制单元、net、pin 与 port 的优化：

- 优化期间保留单元与 Net
- 将优化限制为仅做单元尺寸调整
- 优化期间保留网络
- 标记时钟网络
- 禁用设计规则检查（DRC）
- 尺寸调整期间保留 Pin 名称
- 保留已有层次的 Port
- 隔离输入与输出 Port
- 修复多 Port Net
- 控制向 Module、层次单元与 Voltage Area 添加新单元
- 为优化指定单元名前缀

2.13.1 优化期间保留单元与 Net

Preserving Cells and Nets During Optimization

要用 `set_dont_touch` 在优化期间保留设计对象。该命令对当前设计中的单元、net、reference 与子设计设置 `dont_touch` 属性，防止优化时修改或替换。

下表说明工具如何在优化期间保留各类设计对象。

表 2.8: 优化期间 `dont_touch` 对各设计对象的行为

设计对象	工具行为
单元	将单元排除在优化之外。
Net	保留各 pin 到该 net 的连接。
Module、设计、库单元	阻止优化所有引用它们的实例。
层次单元或 module	将该单元及其全部子单元排除在优化之外。

使用 `set_dont_touch` 时请注意：

- 该命令阻止层次 ungrouping。
- 子单元必须继承其父 module 的同一 dont_touch 属性。例如，父 module 已标 dont_touch 时，不能单独移除子单元上的该属性。
- 仅对完全映射（fully mapped）的逻辑使用，包括单元、module、设计，以及周围逻辑已完全映射的 net。
- dont_touch 优先级高于 boundary_optimization 与 size_only。
报告带 dont_touch 的设计对象，使用 report_dont_touch。
下列命令在单元 A 上设置 dont_touch：

```
icc2_shell> set_dont_touch [get_cells A] true
```

移除该属性可用 remove_attributes，或将 set_dont_touch 设为 false：

```
icc2_shell> remove_attributes [get_cells A] dont_touch
icc2_shell> set_dont_touch [get_cells A] false
```

下表汇总可对 module、单元、net 与库单元设置、移除或报告 dont_touch 的命令支持情况。Yes 表示支持，No 表示不支持。

表 2.9: dont_touch 属性的命令支持（原书 Table 9）

命令	Module	单元（实例）	Net	库单元
set_dont_touch	Yes	Yes	Yes	Yes
set_attribute	Yes	Yes	Yes	Yes
define_user_attribute	No	No	No	No
get_attribute	Yes	Yes	Yes	Yes
remove_attributes	Yes	Yes	Yes	Yes
report_dont_touch	Yes	Yes	Yes	No
set_dont_touch_unmapped_object	No	No	Yes	NA

2.13.2 将优化限制为仅做单元尺寸调整

Restricting Optimization to Cell Sizing Only

要允许优化期间仅对单元或实例做尺寸调整，使用 set_size_only。该命令对指定单元或实例设置 size_only 属性，防止除 sizing 外的修改或替换。

查询该属性可用 report_attributes、get_attribute 或 report_cells；移除可用 remove_attributes，或将 set_size_only 设为 false。

在特定单元上设置：

```
icc2_shell> set_size_only [get_cells cell_name] true
```

查询：

```
icc2_shell> get_attribute [get_cells cell_name] size_only
```

报告当前设计全部 size_only 信息：

```
icc2_shell> report_size_only -all
```

2.13.3 优化期间保留网络

Preserving Networks During Optimization

可用 `set_dont_touch_network` 在优化期间保留时钟树等网络。该命令对时钟、pin 或 port 设置 `dont_touch`，防止网络传递扇出中的单元与 net 被修改或替换。

默认同时保留时钟路径与 clock-as-data 路径，并在网络层次中传播 `dont_touch`。仅保留时钟路径时指定 `-clock_only`；从网络移除该属性时指定 `-clear`。

在 CLK 时钟网络上设置：

```
icc2_shell> set_dont_touch_network [get_clocks CLK]
```

仅对 CLK 时钟路径：

```
icc2_shell> set_dont_touch_network [get_clocks CLK] -clock_only
```

移除：

```
icc2_shell> set_dont_touch_network [get_clocks CLK] -clear
```

2.13.4 标记时钟网络

Marking the Clock Networks

若 block 尚无时钟树，应使用 ideal clock 进行优化。将全部时钟网络标为 ideal：

```
foreach_in_collection mode [all_modes] {
  current_mode $mode
  set_ideal_network [all_fanout -flat -clock_tree]
}
```

为建模时钟树对布局的影响，还应为每个时钟用 `set_clock_uncertainty`、`set_clock_latency` 与 `set_clock_transition` 定义 `uncertainty`、`latency` 与 `transition` 约束。

在执行时钟树综合前，必须用 `remove_ideal_network` 移除时钟树扇出上的 ideal 设置。

2.13.5 禁用设计规则检查（DRC）

Disabling Design Rule Checking (DRC)

可用 `set_auto_disable_drc_nets` 禁用时钟、常数、scan enable 与 scan clock net 上的设计规则检查。

用该命令的选项控制当前设计中特定 net 的 DRC，见表 2.10。

例如，对全部此类 net 启用 DRC：

```
icc2_shell> set_auto_disable_drc_nets -none
```

对连接时钟与常数的 net 禁用 DRC：

表 2.10: set_auto_disable_drc_nets 的选项（原书 Table 10）

选项	作用
-none	对全部 net 启用 DRC。
-all	对全部时钟、常数、scan enable 与 scan clock net 禁用 DRC。
-constant true false	对常数 net 禁用（true）或启用（false）DRC。
-on_clock_network true false	对时钟网络禁用或启用 DRC。
-scan true false	对 scan enable 与 scan clock net 禁用或启用 DRC。

```
icc2_shell> set_auto_disable_drc_nets \  
-on_clock_network true -constant true
```



图 2.18: 已禁用 DRC 的时钟与常数 Net

2.13.6 尺寸调整期间保留 Pin 名称

Preserving Pin Names During Sizing

默认情况下，若结果电路功能等价，优化可在 sizing 时更改叶子单元的 pin 名称。IC Compiler II 会自动更新内部约束版本以反映异常约束中涉及的新 pin 名。若发生此情况，用于约束 block 的原始约束不能再用于 signoff；必须从 IC Compiler II 写出约束并随结果 block 一并使用。

该行为为优化引擎选择单元与改善代价函数提供最大灵活性。可通过设置 opt.common.preserve_pin_names 限制 sizing，使约束在优化中保持不变。默认值为 never，可取 never 或 always。

限制为仅 pin 名等价的单元：

```
icc2_shell> set_app_options \  
-name opt.common.preserve_pin_names -value always
```

恢复默认：

```
icc2_shell> set_app_options \  
-name opt.common.preserve_pin_names -value never
```

2.13.7 保留已有层次的 Port

Preserving Ports of Existing Hierarchies

优化与时钟树综合期间,工具可在已有层次 block 上创建新 port。要阻止此行为,使用 `set_freeze_ports`,并将对应单元实例的 `freeze_clock_port` 属性设为 `true`。

可用 `-clock`、`-data` 或 `-all` 分别阻止添加时钟 port、数据 port 或两者。例如,阻止在 MBX22 上创建额外时钟 port:

```
icc2_shell> set_freeze_ports -clock [get_cells MBX22] true
```

报告 freeze-port 设置使用 `report_freeze_ports`。

2.13.8 隔离输入与输出 Port

Isolating Input and Output Ports

可隔离输入与输出 port 以提高时序模型精度。在指定输入或输出 port 插入隔离逻辑,使用 `set_isolate_ports`。隔离逻辑可以是 buffer 或一对 inverter; 默认放置 buffer 以隔离:

- 输入 port 与其扇出网络
- 输出 port 与其驱动

使用 `-force` 时,工具确保 `-driver` 指定的库单元不会被 sizing。

工具不会在下列 port 上放置隔离逻辑:

- 双向 port
- 定义为时钟源或电源 pin 的 port
- 连接到带 `dont_touch` 属性的 net 的 port

示例

Example

下列命令插入隔离逻辑:

- 输入 port in3: buffer
- 输出 port out1: buffer
- 输出 port out3: LIBCELL_BUFF 库单元
- 输出 port out4: 一对 inverter

```
icc2_shell> set_isolate_ports {in3 out1}
icc2_shell> set_isolate_ports -driver LIBCELL_BUFF out3 -force
icc2_shell> set_isolate_ports -type inverter out4
```

(原书插图: *Without port isolation / After running the port isolation commands*)
 示出插入 buffer 与 inverter 对前后的设计对比。

图 2.19: Port 隔离逻辑插入前后对比

2.13.9 修复多 Port Net

Fixing Multiple-Port Nets

多 port net 包括:

- Feedthrough net: 输入 port 直接馈入输出 port
- 连接到多个输出 port 的 net (逻辑等价输出)

(原书示意: *Feedthrough net / Logically equivalent outputs*)

图 2.20: 多 Port Net 类型示意

门级网表中以 `assign` 语句表示的多 port net, 可能在后续流程导致设计规则违例。默认优化期间工具不修复多 port net。

要将 `opt.port.eliminate_verilog_assign` 设为 `true` 以修复 (默认 `false`)。修复时工具依次:

1. 尽可能重连, 使每条 net 只连到一个层次 port。
 工具优先重连以避免不必要的 buffering; 因默认启用 `boundary optimization`, 也会跨层次重连。
2. 对未能通过重连修复的 net 插入 buffer 或 inverter。

2.13.10 控制向 Module、层次单元与 Voltage Area 添加新单元

Controlling the Addition of New Cells to Modules, Hierarchical Cells, and Voltage Areas

控制向下列对象添加新单元:

- Module 或层次单元: 使用 `set_allow_new_cells`
- Voltage area: 在 `create_voltage_area_rule` 中使用 `-allow_new_cells`

可用这些设置阻止向 abut 设计的顶层层次与 voltage area 添加新单元。设置保存在设计库中, 并在整个实现流程中被工具遵守。

阻止向顶层 module 添加:

```
icc2_shell> set_allow_new_cells \  
[get_attribute [current_block] top_module] false
```

阻止向 M1 module 添加:

```
icc2_shell> set_allow_new_cells [get_modules M1] false
```

阻止向 U22 层次单元添加:

```
icc2_shell> set_allow_new_cells [get_cells U22] false
```

阻止向 VA1 voltage area 添加:

```
icc2_shell> create_voltage_area_rule -name VA1_rule \
    -allow_new_cells false -voltage_areas VA1
```

2.13.11 为优化指定单元名前缀

Specifying a Cell Name Prefix for Optimization

可用 `opt.common.user_instance_name_prefix` 为优化期间在数据 net 上添加的单元指定名前缀。

下列示例为数据 net 上添加的单元指定名前缀:

- `place_opt` 期间: `PO_`
- `clock_opt` 期间: `CO_`
- `route_opt` 期间: `RO_`

```
icc2_shell> set_app_options \
    -name opt.common.user_instance_name_prefix -value "PO_"
icc2_shell> place_opt
icc2_shell> set_app_options \
    -name opt.common.user_instance_name_prefix -value "CO_"
icc2_shell> clock_opt
icc2_shell> set_app_options \
    -name opt.common.user_instance_name_prefix -value "RO_"
icc2_shell> route_opt
```

为时钟树综合期间在时钟网络上添加的单元指定名前缀,使用 `cts.common.user_instance_name_prefix`,见「为时钟单元定义名前缀」。

2.14 指定布线前优化设置

Specifying Settings for Preroute Optimization

为性能、功耗与面积 (PPA) 优化设计是工具的主要目标之一。以下主题说明如何在布线前阶段指定性能、功耗与面积优化设置:

- 控制 `place_opt` 的 DFT 优化
- 指定布线前优化的寄生参数估计设置
- 指定布线前优化的自动 Via Ladder 插入设置

- 在高利用率区域启用面积回收
- 启用高级逻辑重构

2.14.1 控制 place_opt 的 DFT 优化

Controlling DFT Optimization for the place_opt Command

若 block 含 scan chain，默认 create_placement、place_opt 与 clock_opt 会执行物理 DFT 优化。初始布局时，工具忽略 scan chain，专注功能 net 的 QoR；初始布局后，基于初始布局对 scan chain 重新分区（repartition）并重排序（reorder），以进一步改善 QoR。

利用物理信息对 scan chain 重分区与重排序可：

- 缩短 scan chain 线长
- 最小化拥塞并改善可布线性

对含 scan chain 的 block 运行上述命令时，block 必须包含 SCANDEF 信息。用 read_def 标注 SCANDEF：

```
icc2_shell> read_def ./myscan.def
```

注意

若此前已在 block 上标注 scan chain 信息，且 SCANDEF 中某 chain 与已有 scan chain 同名，则保留已有 chain 定义。

SCANDEF 文件指定 scan 路径所用的 scan chain 组件及其 pin，以及重排序约束。为获最佳效果，应使用 TestMAX DFT 做 DFT 插入，并用 Design Compiler topographical mode 生成 SCANDEF。DFT 插入见 *TestMAX DFT User Guide*；生成 SCANDEF 见 *Design Compiler User Guide*。

工具在 DFT 优化中使用 SCANDEF 信息。可按下表控制 DFT 优化。

表 2.11: 控制 DFT 优化的设置（原书 Table 11）

目的	使用的设置
对无 SCANDEF 信息的 block 运行 create_placement、place_opt 或 clock_opt	将 place.coarse.continue_on_missing_scandef 设为 true
禁用 DFT 优化	将 opt.dft.optimize_scan_chain 设为 false
禁用 scan chain 重分区	将 opt.dft.do_repartition 设为 false
禁用基于时序的 scan chain 重排序	将 opt.dft.timing_friendly_reorder 设为 false
重排序时考虑 scan 路径上的 hold 违例（CTS 后使用，以减少 scan 路径 hold 违例）	将 opt.dft.clock_aware_scan_reorder 设为 true
重分区与重排序时考虑 scan 路径上的 hold 违例（CTS 后使用）	将 opt.dft.clock_aware_scan_repartition_reorder 设为 false
DFT 优化期间保留层次 port（启用后不做 timing-friendly 或 clock-aware DFT 优化）	将 opt.dft.hier_preservation 设为 true

报告 DFT 优化相关应用选项设置，使用 `report_app_options`。
对已布局 block 执行独立 DFT 优化，使用 `optimize_dft`。

2.14.2 指定布线前优化的寄生参数估计设置

Specifying Parasitic Estimation Settings for the Preroute Optimization

以下主题说明如何在布线前阶段控制寄生参数估计：

- 启用基于全局布线层（GRLB）的布线前优化
- 为布线前优化启用 Route-Driven Estimation（RDE）

启用基于全局布线层（GRLB）的布线前优化

Enabling Global-Route-Layer-Based (GRLB) Preroute Optimization

为改善与布线后阶段的相关性，可在 `place_opt` 与 `clock_opt` 期间启用基于全局布线层的 RC 估计。启用后，工具对全部 net 使用全局布线，识别每单位电阻最合适的层，并将 net 约束到最小/最大层以进行布线前 RC 估计。

用 `opt.common.use_route_aware_estimation` 启用：

- **auto**：仅当不同布线层的每单位电阻存在差异时启用
- **true**：无论层间每单位电阻是否差异均启用

若启用该功能，在执行布线前须用 `remove_route_aware_estimation` 移除全部基于全局布线的估计。

为布线前优化启用 **Route-Driven Estimation（RDE）**

Enabling Route-Driven Estimation (RDE) for Preroute Optimization

为改善与布线后相关性，工具可执行全局布线、基于全局布线做提取，并在优化中使用这些寄生信息。

若检测到设计所用工艺为小于 16nm 的先进节点，默认启用该功能。对任意工艺启用：将 `opt.common.enable_rde` 设为 **true**。

启用后，工具在 `place_opt` 与 `clock_opt` 的 `final_opt` 阶段执行 route-driven 寄生估计。寄生信息存入设计库，供后续优化步骤使用。启用后，应在设计流程后续全部布线前优化步骤中保持启用。

Route-driven estimation 期间，工具遵守 `set_extraction_options` 的 `-early_cap_scale`、`-late_cap_scale`、`-early_res_scale`、`-late_res_scale` 所指定的电容/电阻缩放因子。

启用 RDE 时，工具忽略用于启用 GRLB RC 估计的 `opt.common.use_route_aware_estimation` 设置。

2.14.3 指定布线前优化的自动 Via Ladder 插入设置

Specifying Automatic Via Ladder Insertion Settings for Preroute Optimization

Via ladder 是从 pin 层向上延伸到上层布线层的堆叠 via。优化期间使用 via ladder 可改善设计的性能与电迁移鲁棒性。工具可在 `place_opt` 与 `clock_opt` 期间为时序关键路径上的单元 pin 自动插入 via ladder。

布线前优化中执行 via ladder 插入：

1. 确保已按「定义 Via Ladder 规则」定义 via ladder 规则。
2. 用 `set_via_ladder_candidate` 为特定库 pin 指定可用 via ladder，见「为库 Pin 指定 Via Ladder 候选」。
3. (可选) 将 `opt.common.enable_via_ladder_insertion` 设为 `true`，为关键路径启用高性能与电迁移 via ladder 插入。
4. (可选) 将 `route.global.insert_gr_via_ladders` 设为 `true`，在带 via ladder 约束的 pin 上启用基于全局布线的 via ladder 插入。
5. 用 `place_opt` 或 `clock_opt` 执行优化。

为库 Pin 指定 Via Ladder 候选

Specifying Via Ladder Candidates for Library Pins

用 `set_via_ladder_candidate` 指定布线前优化中某库 pin 可用的有效 via ladder。每次命令只能指定一个 via ladder；要为同一库 pin 指定多个候选，多次调用，例如：

```
icc2_shell> set_via_ladder_candidate [get_lib_pins lib1/AND2/A1] \
-ladder_name "VP4"
icc2_shell> set_via_ladder_candidate [get_lib_pins lib1/AND2/A1] \
-ladder_name "VP2"
```

同一库 pin 上多个 via ladder 的指定顺序表示插入优先级；应从最高优先级开始指定。上例中 VP4 优先于 VP2。

注意

若同一 pin 同时适用 `pattern_must_join` 属性与 via ladder 候选，且两种方法估计延时相同时，via ladder 优先于基于 pattern 的 must-join 连接。

要指定某库 pin 需要电迁移 via ladder，将 pin 属性 `is_em_via_ladder_required` 设为 `true`：

```
icc2_shell> set_attribute [get_lib_pins lib1/INV4/A] \
is_em_via_ladder_required true
```

若将该属性设为 `true`，必须用 `set_via_ladder_candidate` 将某个电迁移 via ladder 指定为候选。可通过下列方式之一将 via ladder 标识为电迁移 via ladder：

- 在工艺文件中定义 via 规则时使用 `forElectromigration=1`，例如：

```
ViaRule "EMVP1" {
    ...
    forElectromigration=1
}
```

- 将 `for_electro_migration` 属性设为 `true`：

```
icc2_shell> set_attribute [get_via_rules EMVP1] \
for_electro_migration true
```

参见

定义 Via Ladder 规则。

2.14.4 在高利用率区域启用面积回收

Enabling Area Recovery in Regions of High Utilization

对因高利用率区域而无法合法化的设计，可将 `opt.common.small_region_area_recovery` 设为 `true`，让工具在高利用率区域执行面积回收。

启用后，工具在 `place_opt` 与 `clock_opt` 期间执行面积回收，但可能轻微劣化时序 QoR。

2.14.5 启用高级逻辑重构

Enabling Advanced Logic Restructuring

为改善面积、时序与功耗 QoR，可启用高级逻辑重构。对 `place_opt` 与 `clock_opt` 的 `final_opto` 阶段，用 `opt.common.advanced_logic_restructuring_mode` 启用，见下表。

表 2.12: `opt.common.advanced_logic_restructuring_mode` 设置（原书 Table 12）

目的	使用的设置
不执行重构	<code>none</code> （默认）
改善面积 QoR	<code>area</code>
改善时序 QoR	<code>timing</code>
改善功耗 QoR	<code>power</code>
改善面积与时序 QoR	<code>area_timing</code>
改善时序与功耗 QoR	<code>timing_power</code>
改善面积与功耗 QoR	<code>area_power</code>
改善面积、时序与功耗 QoR	<code>area_timing_power</code>

重构时，工具不会：

- 跨逻辑层次重构
- 考虑带 `dont_touch`、`size_only` 或 `fixed` 属性的单元
- 在线长增加时接受结果

但若设计无拥塞或布线问题，可将 `opt.common.advanced_logic_restructuring_wirelength_costing` 设为 `medium` 或 `none`（默认 `high`），使工具在线长增加时仍接受重构结果。

注意

该功能需要 Digital-AF license key。

2.15 设置功耗相关功能

Setting Up for Power-Related Features

以下主题说明为功耗相关功能设置设计所需的任务：

- 标注开关活动（Switching Activity）
- 为 `place_opt` 与 `clock_opt` 启用功耗优化
- 通过限制低阈值电压（LVT）单元百分比改善良率
- 更新 Activity 以改善功耗优化
- 启用电源完整性功能

2.15.1 标注开关活动

Annotating the Switching Activity

执行低功耗布局与动态功耗优化时，若为设计标注 `switching activity`，可获得更好的功耗节省。标注方式：

- 用 `read_saif` 读取开关活动文件（SAIF）。
`Switching activity` 与 `scenario` 相关；使用该命令时，确保当前 `scenario` 已启用动态功耗优化。
- 用 `set_switching_activity` 设置 `switching activity`。
可用 `-mode`、`-corner` 或 `-scenario` 为特定 `mode`、`corner` 或 `scenario` 设置；并确保所指定（或对应）的 `scenario` 已启用动态功耗优化。

若未指定 `switching activity`，工具对主输入与 `black box` 输出应用默认 `toggle rate`，并在设计中传播。

报告 block 的 `switching activity` 使用 `report_activity`；移除特定 `net`、`pin`、`port` 或整个 block 的 `switching activity` 使用 `reset_switching_activity`。

结合 **Name-Mapping** 文件使用 **RTL Switching Activity**

Using RTL Switching Activity With a Name-Mapping File

要在 IC Compiler II 中使用 RTL SAIF 文件，须先在 Design Compiler 中完成：

1. 用 `read_saif` 读入 RTL SAIF。
2. 优化前用 `saif_map -start` 创建 `name-mapping` 数据库，跟踪优化中对象名变更。
3. 执行优化及其他设计变更。
4. 用 `saif_map -write_map` 生成将 RTL 对象名映射到门级对象的 `name-mapping` 文件。

然后在 IC Compiler II 中：

1. 用 `saif_map -read_map` 读入 Design Compiler 的 `name-mapping` 文件，创建 `name-mapping` 数据库并开始跟踪对象名变更。
若不读 `name-mapping` 文件、但希望跟踪 IC Compiler II 内的网表变更，可用 `saif_map -start` 创建数据库并开始跟踪。

2. (可选) 按「控制名称映射」手动修改名称映射。
3. 用 `read_saif` 读入同一 RTL SAIF (不影响 name-mapping 数据库)。
4. 执行优化及其他设计变更。

控制名称映射

Controlling the Name Mapping

在 SAIF-map 流程中, name-mapping 数据库跟踪优化期间 pin、port、单元与 net 名称变更, 保存各对象的原始名称, 使优化后仍可应用原始 SAIF。

可用 `saif_map` 的选项手动控制数据库中的名称映射, 见下表。

表 2.13: 手动控制名称映射的选项 (原书 Table 13)

目的	saif_map 选项
为对象指定名称映射并覆盖数据库中可能已有的映射	-set_name
向数据库已有映射追加名称映射	-add_name
将 -set_name 或 -add_name 指定的映射应用于逻辑取反对象	与 -set_name/-add_name 联用 -inverted
读入 SAIF 时用 name rule 更改对象名	-change_name
报告全部手动名称映射设置	-report
移除特定对象的名称映射设置	-remove_name
清除全部手动创建的名称映射与 name rule 以重置	-reset
从数据库检索名称映射设置	-get_saif_names、-get_object_names

缩放开关活动

Scaling the Switching Activity

若 SAIF 文件中的时钟频率与 IC Compiler II 中不同, 可按下列步骤缩放 switching activity:

1. 用 `read_saif` 读入 SAIF。
2. 将 `power.use_generated_clock_scaling_factor` 设为 on 以启用缩放。
3. 用 `set_power_clock_scaling` 缩放 switching activity。须指定:
 - 与待缩放 switching activity 关联的时钟对象
 - 用 -period 指定 SAIF 中使用的时钟周期, 或用 -ratio 指定 SAIF 周期与 IC Compiler II 周期之比

另可用 `-scenario` 指定应用缩放的 scenario。

下列示例读入 SAIF、启用缩放, 将 CLK1/CLK2 关联活动按比例 5 缩放, 将 CLK3 按比例 2 缩放:

```
icc2_shell> read_saif top.saif
icc2_shell> set_app_options -list \
```

```
{power.use_generated_clock_scaling_factor true}
icc2_shell> set_power_clock_scaling -ratio 5 {CLK1 CLK2}
icc2_shell> set_power_clock_scaling -ratio 2 {CLK3}
```

若对同一时钟再次运行 `set_power_clock_scaling`，工具会在已缩放结果上继续缩放。
使用该命令时，工具仅缩放由 `read_saif` 应用的 switching activity；不缩放：

- 由 `set_switching_activity` 应用的 switching activity
- block abstract 内的 switching activity

缩放后的 switching activity 在设计中持久保存，可用 `write_saif` 写出并用于后续流程。

为供电网络指定开关概率

Specifying Switching Probability for Supply Nets

block 的漏电功耗会按其 supply net 的开关概率缩放，该概率表示 supply net 处于 on 状态的时间比例。用 `set_supply_net_probability -static_probability` 为一个或多个 supply net 指定开关概率。默认应用于当前 scenario 及对应 mode 与 corner；用 `-modes`、`-corners` 或 `-scenarios` 指定作用范围。获取用 `get_supply_net_probability`，重置用 `reset_supply_net_probability`。

默认工具经电源开关传播 supply net activity，并根据 UPF 电源开关约束确定开关后 supply net 的静态概率。例如：

```
create_power_switch my_switch \
  -output_supply_port {vout VDDS} \
  -input_supply_port {vin VDD} \
  -control_port {ms_sel ctrl1} \
  -control_port {ms_ctrl ctrl2} \
  -on_state {on vin {ms_ctrl && !ms_sel}}
```

工具根据电源开关处于 on 的概率推导连接到开关输出的 VDDS supply net 静态概率，依据包括：

- `-on_state` 指定的布尔函数 (`ms_ctrl && !ms_sel`)，以及对应控制 port 所连 net (`ctrl1`、`ctrl2`) 的 switching activity（静态概率）
- `-on_state` 中输入供电 port 所连 supply net (`VDD`) 的开关概率

下列应用选项控制是否根据供电开关活动缩放动态与漏电功耗：

- `power.scale_dynamic_power_at_power_off`：是否缩放动态功耗。默认 `false`（不缩放）。
- `power.scale_leakage_power_at_power_off`：是否缩放漏电功耗。默认 `true`（缩放）。

2.15.2 为 place_opt 与 clock_opt 启用功耗优化

Enabling Power Optimization for the place_opt and clock_opt Commands

IC Compiler II 可优化动态与静态（漏电）功耗。

- 动态功耗：设计对象（单元、pin、net）中电压或逻辑翻转所耗散的能量，与翻转次数及频率成正比。
- 静态（漏电）功耗：电路无翻转时仍耗散的能量，亦称 `leakage power`，取决于器件特性。主要贡献是亚阈值漏电。在较低工艺节点，漏电在总功耗中占显著份额。

以下主题说明如何启用工具执行的各类功耗优化：

- 执行传统漏电功耗优化
- 执行动态功耗优化
- 执行总功耗优化

执行传统漏电功耗优化

Performing Conventional Leakage-Power Optimization

布线前优化阶段设置传统漏电功耗优化：

1. 用 `set_scenario_status -leakage_power true` 确保至少一个 scenario 启用漏电功耗优化；并用 `set_qor_strategy -metric leakage_power | total_power` 决定功耗流程目标。
2. (可选) 用 `set_qor_strategy -mode` 将 effort 设为 `balanced` 或 `extreme_power` (默认 `balanced`)，作为功耗流程的 effort 控制。

使用这些设置后，工具在 `place_opt` 与 `clock_opt` 期间执行传统漏电功耗优化，但不做动态功耗优化。

执行动态功耗优化

Performing Dynamic-Power Optimization

布线前优化阶段设置动态功耗优化：

1. 按「标注开关活动」为设计标注 `switching activity`。
2. 用 `set_scenario_status -dynamic_power true` 确保至少一个 scenario 启用动态功耗优化。

使用这些设置后，工具在 `place_opt` 与 `clock_opt` 期间执行动态功耗优化，但不做漏电功耗优化。

执行总功耗优化

Performing Total-Power Optimization

总功耗优化在优化中同时考虑漏电与动态功耗代价。设置步骤：

1. 设置漏电功耗优化：用 `set_scenario_status -leakage_power true` 确保至少一个 scenario 启用。
2. 设置动态功耗优化：
 - (a) 按「标注开关活动」标注 `switching activity`。
 - (b) 用 `set_scenario_status -dynamic_power true` 确保至少一个 scenario 启用。

使用该设置后，工具在 `place_opt` 与 `clock_opt` 期间执行总功耗优化。

2.15.3 通过限制 LVT 单元百分比改善良率

Improving Yield By Limiting the Percentage of Low-Threshold-Voltage (LVT) Cells

IC Compiler II 可执行基于 LVT 百分比的优化。限制 block 中 LVT 单元百分比可改善良率；也可在早期设计阶段用于评估 RTL 质量。

设置基于 LVT 百分比的优化：

1. 指定 LVT 库单元组，例如：

```
icc2_shell> remove_attributes -quiet \
[get_lib_cells -quiet */*] threshold_voltage_group
icc2_shell> set_attribute [get_lib_cells -quiet *lvt*/*] \
threshold_voltage_group LVT
icc2_shell> set_threshold_voltage_group_type \
-type low_vt LVT
```

确保所指定的 LVT 库单元：

- 已用 `set_lib_cell_purpose` 定义有效 purpose，并包含在 `set_target_library_subset` 定义的相应 target-library subset 中
 - 未设置 `dont_touch` 或 `dont_use`
2. 用 `set_multi_vth_constraint -low_vt_percentage` 指定 LVT 单元百分比上限。
以单元数量百分比计用 `-cost cell_count`；以单元面积百分比计用 `-cost area`。
重置用 `reset_multi_vth_constraint`，报告用 `report_multi_vth_constraint`。

设置完成后，运行 `place_opt`、`clock_opt` 或 `route_opt` 时，工具对 block 中数据路径单元使用该 LVT 百分比约束。为尽量减小 QoR 扰动，`route_opt` 期间的基于 LVT 百分比优化力度有限，因此应尽早并在流程中启用。

注意

LVT 单元延时更小，但漏电功耗更高。限制 LVT 数量可降低漏电；要进一步降低漏电，请在流程各阶段启用漏电功耗优化。

2.15.4 更新 Activity 以改善功耗优化

Updating Activity for Improved Power Optimization

IC Compiler II 中的 PrimePower In-Design 流程使用 RTL FSDB 的输入波形信息，执行基于时间的分析以更新 activity，提高功耗估计精度。

用 `set_indesign_primepower_options` 设置 In-Design 流程的 PrimePower 分析选项。必须用 `-fsdb` 指定 RTL activity 文件，用 `-pwr_shell` 指定 PrimePower 可执行文件路径。其他选项可指定 FSDB 细节、分布式/并发处理设置、scenario 与库设置，以及输出文件。

用 `update_indesign_activity` 执行分析，包括：

- 调用 PrimePower，读入网表、库、PVT、约束、寄生、必要 name mapping、UPF 等设计数据。库细节来自 IC Compiler II 内存中的设计数据库。
- 将 RTL FSDB 读入 PrimePower，执行基于时间的分析以生成门级 SAIF。

- 将门级 SAIF 读回 IC Compiler II。

启动 `update_indesign_activity` 前，将功耗 scenario 设为当前 scenario；activity 对当前 scenario 刷新。

若用 `set_indesign_primepower_options` 的 `-scenarios` 指定多个 scenario，则从 PrimePower 刷新的门级 SAIF 会读回所有指定 scenario。

PrimePower In-Design UI 允许用原生 ZeBu 数据库 (ZTDB) 传递输入波形。`set_indesign_primepower_options` 的 `-ztdb` 选项配置 PrimePower In-Design，以用 `update_indesign_activity` 创建脚本；该脚本使用 PrimePower 的 `read_ztdb` 做基于时间的分析，并生成自动回标到 IC Compiler II 的门级 SAIF。

指定分布式分析

Specifying Distributed Analysis

为加快 runtime，可使用分布式分析（PrimePower 中默认禁用）。

在 `set_indesign_primepower_options` 中指定：

- `-num_processes`：启动的主机数，须大于 1。
- `-submit_command` 或 `-host_names`：主机配置。
- `-max_cores`：最大核数，默认 4。

选项有效时工具打印：

```
Information: PrimePower is being called in distributed mode
```

2.15.5 启用电源完整性功能

Enabling the Power Integrity Features

IC Compiler II 提供下列技术保障设计电源完整性：

- 动态功耗整形（Dynamic Power Shaping, DPS）：利用并发时钟与数据优化（useful skew）降低瞬态功耗。
- 压降感知布局：用单元电压（IR）drop 值识别高功耗密度区，并摊开高压降单元以降低该区功耗密度。

设置推荐电源完整性流程：

1. 将 `opt.common.power_integrity` 设为 `true` 以启用。启用后，IC Compiler II 在不同阶段使用：
 - (a) `place_opt` 的 `final_opto` 阶段：动态功耗整形（DPS）
 - (b) `clock_opt`：压降感知布局
2. （可选）将 `opt.common.power_integrity_effort` 设为 `low`，降低电源完整性 effort，使工具不以牺牲时序 QoR 为代价降低压降。
默认为 `high`：默认工具尝试将压降降至电源电压的 8% 以下，即使牺牲时序 QoR。
3. （可选）用 `opt.common.ir_drop_threshold` 指定最大压降阈值（默认电源电压的 8%）。若将 `opt.common.power_integrity_effort` 设为 `low`，工具忽略该阈值。

4. 按「设置动态功耗整形」指定 DPS 设置。
5. 按「设置压降感知布局」指定相关设置。
6. (可选) 启用 IR 驱动 sizing: 用 RedHawk 动态压降分析结果识别压降违例相关单元, 并尝试用更小漏电流的单元替换。除将 `opt.common.power_integrity` 设为 `true` 外, 还将 `clock_opt.flow.enable_irdriven` 设为 `true`。

也可不使用推荐电源完整性流程设置, 而按「手动启用动态功耗整形与压降感知布局」为 `place_opt` 或 `clock_opt` 手动启用。

设置动态功耗整形

Setting Up for Dynamic Power Shaping

1. 确保为多电源 net 指定了 UPF 设置。
2. 确保已创建并激活 `setup` 与 `hold scenario`。
3. (可选) 如下控制 DPS 期间的功耗分析与优化:
 - 用 `ccd.dps.focus_power_scenario` 指定关注的 `scenario`。默认关注全部活动 `scenario`; 设计有大量活动 `scenario` 时, 指定一两个可缩短 `runtime`, 例如:

```
icc2_shell> set_app_options -name \
    ccd.dps.focus_power_scenario -value {{S1 S2}}
```

- 用 `ccd.dps.use_case_type` 控制内部功耗分析类型。
默认 `autovector`: 以 20% 翻转触发器概率运行内部功耗分析, 且所有触发器每周期有效时钟; 不建模时钟门控。
设为 `activity` 时, 工具根据 `activity` 选择翻转概率与时钟门控模型。
- 用 `ccd.dps.optimize_power_target_modes` 定义总体动态功耗优化目标:
 - `global_peak`: 降低全局峰值电流
 - `stepped_psgs` (默认): 在设计上步进窗口降低局部峰值电流, 以降低压降违例
- 用 `ccd.dps.optimize_setup_tradeoff_level` 指定功耗与 `setup` 时序的折中: `low`、`medium` (默认) 或 `high`。
- 用 `ccd.dps.optimize_hold_tradeoff_level` 指定功耗与 `hold` 时序的折中: `low`、`medium` (默认) 或 `high`。

还可用 `ccd.dps.use_cases`、`ccd.dps.auto_targets`、`ccd.dps.auto_target_constraint_parameters` 等更复杂的应用选项进一步控制; 详见对应 `man` 页。

设置压降感知布局

Setting Up for Voltage-Drop-Aware Placement

1. 用 `rail.analysis_type` 指定压降分析类型。有效值: `static`、`dynamic` (默认)、`dynamic_vcd`、`dynamic_vectorless`。
2. 按「执行压降分析」指定 RedHawk 压降分析所需设置。

手动启用动态功耗整形与压降感知布局

Manually Enabling Dynamic Power Shaping and Voltage-Drop-Aware Placement

手动启用:

- 对 place_opt 启用 DPS:

```
icc2_shell> set_app_option -name opt.common.power_integrity \  
-value false  
icc2_shell> set_app_option -name place_opt.flow.enable_dps \  
-value true
```

- 对 place_opt 启用压降感知布局:

```
icc2_shell> set_app_option -name opt.common.power_integrity \  
-value false  
icc2_shell> set_app_option -name place_opt.flow.enable_irap \  
-value true
```

- 对 clock_opt 启用压降感知布局:

```
icc2_shell> set_app_option -name opt.common.power_integrity \  
-value false  
icc2_shell> set_app_option -name clock_opt.flow.enable_irap \  
-value true
```

2.16 指定布线资源

Specifying the Routing Resources

可为 block（全局层约束）、特定 net（net 专用层约束）以及未综合的时钟 net（时钟树层约束）指定最小与最大布线层。若同时指定全局层约束与 net 专用层约束，net 专用约束覆盖全局约束。

除约束布线层外，还可为每层指定优选布线方向。布线层约束用于 RC 估计、拥塞分析与布线。因其同时影响 RC 估计、拥塞分析与布线，应在对 block 执行布局前设置。

以下主题说明如何指定布线层约束、优选布线方向与 via ladder:

- 指定全局层约束
- 指定 Net 专用层约束
- 指定时钟树层约束
- 启用基于 Level 的时钟非默认布线规则
- 为层设置优选布线方向

2.16.1 指定全局层约束

Specifying the Global Layer Constraints

用 set_ignored_layers 指定全局层约束。默认全局层约束用于 RC 估计、拥塞分析，并作为布线的软约束。可用下列选项设置约束并更改默认行为。

- 用 `-min_routing_layer` 与 `-max_routing_layer` 指定最小与最大布线层（使用工艺文件中的层名）。

例如，布线、RC 估计与拥塞分析使用 M2 至 M7：

```
icc2_shell> set_ignored_layers \
-min_routing_layer M2 -max_routing_layer M7
```

- 仅允许为 pin 连接使用超出最小/最大布线层的层：将 `route.common.global_min_layer_mode` 与 `route.common.global_max_layer_mode` 设为 `allow_pin_connection`。
- 将约束改为硬约束：将上述两应用选项设为 `hard`。
- 用 `-rc_congestion_ignored_layers` 指定 RC 估计与拥塞分析中额外忽略的层。所指定层必须在最小与最大布线层之间。例如，布线用 M2–M7，RC 估计与拥塞分析用 M3–M7：

```
icc2_shell> set_ignored_layers \
-min_routing_layer M2 -max_routing_layer M7 \
-rc_congestion_ignored_layers {M2}
```

- 更改已有层约束时，只需重新设置该选项。
重置某选项仅覆盖该选项现有值，其他选项不变。
例如，已将最小层设为 M2、最大层设为 M7，要将最大层改为 M8 并保留其他设置：

```
icc2_shell> set_ignored_layers -max_routing_layer M8
```

报告全局层约束

Reporting Global Layer Constraints

用 `report_ignored_layers` 报告被忽略的层，例如：

```
icc2_shell> report_ignored_layers
*****
Report : Ignored Layers
Design : my_design
Version: J-2014.12
Date   : Wed Oct 22 15:58:23 2014
*****
Layer Attribute          Value
-----
Min Routing Layer       M2
Max Routing Layer       M7
RC Estimation Ignored Layers  P0 M1 M2 M8 M9 MRDL
1
```

移除全局层约束

Removing Global Layer Constraints

用 `remove_ignored_layers` 移除全局层约束，必须指定下列一个或多个选项：

- **-min_routing_layer**: 移除最小布线层设置；同时移除低于该层、用于 RC 估计与拥塞分析的忽略层。
- **-max_routing_layer**: 移除最大布线层设置；同时移除高于该层、用于 RC 估计与拥塞分析的忽略层。
- **-all**: 移除最小与最大布线层之间、用于 RC 估计与拥塞分析的忽略层。
- **-rc_congestion_ignored_layers layer_list**: 移除指定的 RC/拥塞忽略层（须在最小与最大布线层之间）。

2.16.2 指定 Net 专用层约束

Specifying Net-Specific Layer Constraints

用 `set_routing_rule` 指定 net 专用层约束；用 `-min_routing_layer` 与 `-max_routing_layer` 指定最小与最大布线层（工艺文件层名）。

例如，布线 n1 时使用 M2 至 M7：

```
icc2_shell> set_routing_rule [get_nets n1] \
-min_routing_layer M2 -max_routing_layer M7
```

默认 net 专用最小层约束为软约束，最大层约束为硬约束。可如下更改默认行为：

- 用 `route.common.net_min_layer_mode` 与 `route.common.net_max_layer_mode` 更改全部 net 专用层约束的默认强度。
设为 `soft` 表示软约束，`allow_pin_connection` 允许仅对 pin 连接使用更低层，`hard` 表示硬约束。
- 用 `set_routing_rule` 定义约束时的 `-min_layer_mode` 与 `-max_layer_mode` 更改单个 net 专用约束的强度（取值同上）。
- 用 `route.common.net_min_layer_mode_soft_cost` 与 `route.common.net_max_layer_mode_soft_cost` 设置全部 net 专用软约束违例代价。
取值为 `low`、`medium`（默认）或 `high`。代价控制路由器避免违例的 effort；`high` 可减少违例但增加 runtime，`low` 可缩短 runtime 但可能增加违例。
- 用 `set_routing_rule` 的 `-min_layer_mode_soft_cost` 与 `-max_layer_mode_soft_cost` 为单个 net 专用约束设置软约束违例代价（取值同上）。

移除 Net 专用布线层约束

Removing Net-Specific Routing Layer Constraints

用 `set_routing_rule -clear` 移除 net 专用布线层约束。注意：该命令也会移除赋给指定 net 的任何非默认布线规则。

2.16.3 指定时钟树层约束

Specifying Clock-Tree Layer Constraints

用 `set_clock_routing_rules` 指定时钟树层约束;用 `-min_routing_layer` 与 `-max_routing_layer` 指定最小与最大布线层（工艺文件层名）。

默认该命令将指定的层约束赋给全部时钟树。限制赋给范围的选项见表 2.14。

表 2.14: 限制时钟树层约束（原书 Table 14）

将层约束赋给	使用的选项
特定时钟树	<code>-clocks clocks</code>
连接到时钟 root 的 net ¹	<code>-net_type root</code>
连接到一个或多个时钟 sink 的 net ¹	<code>-net_type sink</code>
时钟树中的内部 net（除 root 与 sink 外全部） ¹	<code>-net_type internal</code>
特定时钟 net	<code>-nets nets</code>

¹ 可与 `-clocks` 联用进一步限制赋值；不可与 `-nets` 联用。

图 2.21 示出 CTS 后时钟树的 root、internal 与 sink net。默认 root-net 层约束应用于从时钟 root 起、直至时钟树分支到扇出大于 1 之前的全部单扇出时钟 net；此后直至 sink net 应用 internal-net 层约束。

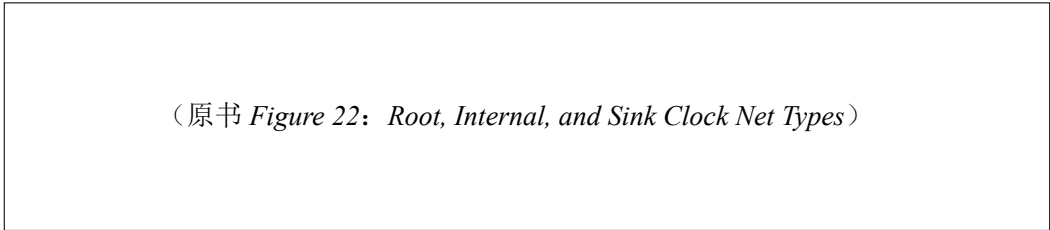


图 2.21: Root、Internal 与 Sink 时钟 Net 类型

例如，布线 CLK1 的 root net 时使用 M4 至 M7：

```
icc2_shell> set_clock_routing_rules -clocks CLK1 -net_type root \  
-min_routing_layer M4 -max_routing_layer M7
```

识别 rootnet 时可用 `set_clock_tree_options -root_ndr_fanout_limit` 指定传递扇出上限。例如，传递扇出超过 300 个 sink 的时钟 net 视为 root net：

```
icc2_shell> set_clock_tree_options -root_ndr_fanout_limit 300
```

图 2.22 示出指定传递扇出上限为 300 时同一时钟树的 root、internal 与 sink net。

计算传递扇出以识别 root net 时，工具仅计入有效时钟 sink，不包括 ignore pin。若被识别为 root 的 net 短于 10 micron，工具对该 net 使用 internal-net 层约束。

注意

用较小的 `set_clock_tree_options -root_ndr_fanout_limit` 值会增加被赋 root-net 层约束的时钟 net 数量，可能加剧布线拥塞。

(原书 *Figure 23: Using a Fanout Limit for Selecting Root Nets*)

图 2.22: 用扇出限制选择 Root Net

移除该传递扇出限制，使用 `remove_clock_tree_options -root_ndr_fanout_limit`。

2.16.4 启用基于 Level 的时钟非默认布线规则

Enabling Level-Based Clock Nondefault Routing Rule

时钟树综合期间，可按时钟树不同 level 为 net 分配连续的非默认布线规则。工具仅在初始 CTS 时实施变更，并按下列方式分配 NDR:

- 仅连接到纯 sink 的 net: sink NDR
- 同时连接到 sink 与中间单元 (repeater 或时钟门) 的 net: internal NDR
- 从 root 级直至下游分支点的 net: root NDR

(原书 *Figure 24: Level-Based Clock Nondefault Routing Rule*)

图 2.23: 基于 Level 的时钟非默认布线规则

如图所示，CTS 后:

- 未启用基于 level 的时钟 NDR: n1 与 n3 因至少连接一个 sink，编译时钟树后须获得 sink NDR; 这会使不同 level 上出现不连续的 NDR。
- 启用基于 level 的时钟 NDR: 时钟树路径上全部 NDR 须按拓扑从时钟 root 到 sink 排序，即 Root NDR > Internal NDR > Sink NDR。因此 n1、n2 赋 internal NDR，n3 赋 sink NDR。

启用该功能:

```
icc2_shell> set_app_options -list {cts.compile.topological_ndr true}
```

2.16.5 为层设置优选布线方向

Setting the Preferred Routing Direction for Layers

IC Compiler II 要求为工艺文件中定义的布线层指定优选布线方向。通常该信息在单元库中定义。要设置或更改某层的优选布线方向，按下列语法设置其 `routing_direction` 属性:

```
set_attribute -objects layers
  -name routing_direction
  -value vertical | horizontal
```

用工艺文件中的层名指定布线层。该属性设置的层方向仅作用于当前 block。

例如，将 M5 与 M7 的优选方向设为 vertical：

```
icc2_shell> set_attribute -objects [get_layers {M5 M7}] \
  -name routing_direction -value vertical
```

注意

用 `create_routing_guide -switch_preferred_direction` 在 routing guide 覆盖区域内更改优选方向的设置，会覆盖 `routing_direction` 属性设置。

报告一个或多个布线层的用户定义优选方向,使用 `get_attribute`;移除则使用 `remove_attributes`。

参见

准备布线层。

2.17 使用 Early Data Check Manager 处理设计数据

Handling Design Data Using the Early Data Check Manager

设计周期早期迭代中，设计数据可能不完整或不正确，从而妨碍探索性实现等。IC Compiler II Early Data Check Manager 允许指定工具在下列实现任务中如何处理数据检查：

- 设计规划
- 多电压设计实现
- 优化
- 布局
- 合法化
- 含物理层次设计的层次化实现

下表列出用于控制数据检查处理方式及生成相关信息的 Early Data Check Manager 命令。

应在开始实现前指定数据检查处理策略。

下列示例指定对 SCANDEF 信息检查采用严格策略，因此因缺少 SCANDEF，工具不会继续执行 `create_placement`：

```
icc2_shell> set_early_data_check_policy -policy strict \
  -check place.coarse.missing_scan_def
icc2_shell> create_placement
Information: Policy for early data check 'place.coarse.missing_scan_def'
is 'error'. (EDC-001)
Error: No valid scan def found. (PLACE-042)
```

表 2.15: Early Data Check Manager 命令（原书 Table 15）

目的	使用的命令
指定一个或多个数据检查的处理策略	set_early_data_check_policy
获取数据检查违例报告	report_early_data_checks
获取可在其他命令中使用的违例 Tcl 集合	get_early_data_check_records
将设置保存到文件供日后使用	write_early_data_check_config
清除数据检查设置以便下一轮迭代	remove_early_data_check_records

```
Information: Ending 'create_placement' (FLW-8001)
icc2_shell> report_early_data_checks -check place.coarse.missing_scan_def
Check                                Policy Strategy Fail Count
-----
place.coarse.missing_scan_def  error                                1
-----
```

下列示例对所有与粗布局相关的数据检查采用宽松策略，因此即使无 SCANDEF，工具仍继续 create_placement：

```
icc2_shell> set_early_data_check_policy -policy lenient \
    -check place.coarse.*
icc2_shell> create_placement
Information: Policy for early data check 'place.coarse.missing_scan_def'
    is 'tolerate'. (EDC-001)
Continuing without valid scan def.
Start transferring placement data.
Creating placement from scratch.
...
```

更多信息见 *IC Compiler II Data Model User Guide*。

2.18 应用 Mega-Switch 命令设置

Applying Mega-Switch Command Settings

可用 mega-switch 命令，以单一命令为工具流程不同阶段应用适用于特定情形的设置：

- 应用先进工艺节点所需设置
- 应用高性能核所需设置
- 应用改善特定 QoR 指标所需设置

2.18.1 应用先进工艺节点所需设置

Applying Required Settings for Advanced Technology Nodes

要对 12 nm 或 7 nm 工艺节点应用特定于布局、合法化、布线与提取的设置，使用 set_technology 的 -node 12 或 -node 7。

使用该命令时，在布局、优化与布线前执行：

1. 加载并 link 设计。
2. 用 `set_technology -node` 应用工艺节点所需设置，例如：

```
icc2_shell> set_technology -node 7
```

3. (可选) 用 `set_technology -report_only` 报告工艺节点设置。
4. 应用设计专用应用选项或命令设置，以覆盖 `set_technology` 的通用设置。

用 `save_lib` 保存设计库时，工艺节点设置一并保存。

注意

用 `set_technology` 指定工艺节点后，不可再更改。

该命令支持哪些 ASIC 工艺，请联系 Synopsys Support。

2.18.2 应用高性能核所需设置

Applying Required Settings for High Performance Cores

可用 `set_hpc_options` 应用为实现高性能核最佳 QoR 所需的工具设置。所应用设置影响布局、合法化、优化、时钟树综合、布线、提取与时序分析。

- 列出支持的高性能核类型与实现阶段：用 `-list`
- 指定高性能核类型：用 `-core`
- 指定实现流程阶段：用 `-stage`

若设计使用 12 nm 或 7 nm 节点，须在运行 `set_hpc_options` 前先用 `set_technology` 应用节点专用设置，示例如下：

```
set_technology -node 7
set_hpc_options -core A72 -stage clock_opt_cts
...
clock_opt -from build_clock -to route_clock
...
set_hpc_options -core A72 -stage clock_opt_opto
...
clock_opt -from final_opto
...
set_hpc_options -core A72 -stage route_auto
...
route_auto
...
set_hpc_options -core A72 -stage route_opt
...
route_opt
...
```

2.18.3 应用改善特定 QoR 指标所需设置

Applying Required Settings for Improving Specific QoR Metrics

要用 `set_qor_strategy -stage pnr` 应用改善特定 QoR 指标所需的工具设置。

用 `-metric` 指定要改善的 QoR 指标。为获得最佳：

- 时序：使用 `timing`
- 漏电功耗与时序：使用 `leakage_power`
- 总功耗与时序：使用 `total_power`。
为进一步改善总功耗(以 `runtime` 为代价),可与 `-metric total_power` 联用 `-mode extreme_power`。
用 `-mode` 指定设计流程目标模式。支持取值：
 - **balanced**: 默认模式，针对 `-metric` 指定的目标指标优化设计。
 - **early_design**: 适用于设计流程早期、侧重短周转时间原型的阶段。
该模式可以 QoR 为代价缩短 `runtime`。
 - **extreme_power**: 建议用于以 `runtime` 为代价进一步改善总功耗。
仅应与 `-metric leakage_power` 或 `-metric total_power` 联用。若与 `-metric timing` 联用，工具忽略该 `mode` 并使用 `balanced`。

使用该命令时，工具应用影响优化、布局、合法化、时钟树综合、布线、提取与时序分析的应用选项设置。

查看所需设置可用：

- 用 `-output` 生成包含这些设置的 Tcl 脚本
- 用 `-report_only` 报告设置
- 用 `-diff_only` 仅报告未设为所需值的设置

Chapter 3

布局与优化

Placement and Optimization

要了解如何在 IC Compiler II 中执行布局与优化，请参阅：

- 执行布局与优化
- 识别优化期间无法修复的问题
- 分析布局
- 分析时序
- 分析功耗
- 对比 QoR 数据

3.1 执行布局与优化

Performing Placement and Optimization

完成设计规划与电源规划后，即可对设计执行布局（placement）、优化（optimization）与合法化（legalization）。以下主题说明如何进行：

- 执行独立布局与合法化
- 使用 `place_opt` 命令执行布局与优化
- 使用 Design Compiler 工具的物理引导（physical guidance）
- 执行多比特寄存器优化
- 执行 Magnet Placement
- 细化布局（Refining Placement）
- 对多电压 block 执行布局与优化
- 重建缓冲树（Buffer Trees）

- 添加与移除 tie cell

3.1.1 执行独立布局与合法化

Performing Standalone Placement and Legalization

要执行粗布局（coarse placement），使用 `create_placement` 命令。若 block 含有通过读取 SCAN-DEF 文件标注的 scan chain，粗布局器还会执行 scan chain 优化。

可通过 `-effort` 选项控制运行时间与 QoR 之间的权衡。

要启用：

- 时序驱动布局：使用 `-timing_driven` 选项
- 考虑高扇出 net 与长 net，并计入优化阶段随后在这些 net 上添加的 buffer 的时序驱动布局：使用 `-buffering_aware_timing_driven` 选项
- 拥塞驱动布局：使用 `-congestion` 选项
要控制用于拥塞消除的 CPU 努力程度，使用 `-congestion_effort` 选项
- 拥塞驱动重构：使用 `-congestion_driven_restructuring` 选项
使用该选项时，工具会进行多轮布局与逻辑重构，以帮助降低拥塞

注意

在使用 `create_placement` 运行布局之前，应先运行 `merge_clock_gates` 合并 clock-gating 逻辑。

要执行合法化，使用 `legalize_placement` 命令。有关可指定的合法化约束与设置的更多信息，见「指定合法化设置」。

注意

按「启用高级合法化算法」所述启用 advanced legalizer 时，不支持 `legalize_placement` 的 `-cells` 选项。

3.1.2 使用 `place_opt` 命令执行布局与优化

Performing Placement and Optimization With the `place_opt` Command

要用单一命令执行粗布局、物理优化与合法化，使用 `place_opt` 命令。

该命令支持多线程，并使用由 `set_host_options -max_cores` 指定的线程数。

`place_opt` 命令包含以下阶段：

1. 初始布局（`initial_place`）
此阶段工具合并 clock-gating 逻辑并执行粗布局。若 block 含有通过读取 SCANDEF 标注的 scan chain，工具还会执行 scan chain 优化。
2. 初始 DRC 违例修复（`initial_drc`）
此阶段工具移除已有缓冲树，并执行高扇出 net 综合（high-fanout-net synthesis）与电气 DRC 违例修复。

3. 初始优化 (**initial_opto**)

此阶段工具执行时序、面积、拥塞与漏电功耗优化。

4. 最终布局 (**final_place**)

此阶段工具执行增量布局以改善时序与拥塞，并对设计合法化。

5. 最终优化 (**final_opto**)

此阶段工具进一步优化并合法化，以改善时序与拥塞。

运行 **place_opt** 时，默认执行全部布局与优化阶段。

要将 **place_opt** 限制为仅运行其中一个或多个阶段，使用 **-from** 指定起始阶段，使用 **-to** 指定结束阶段（之后停止）。

若不指定 **-from**，工具从 **initial_place** 阶段开始；若不指定 **-to**，工具一直运行到 **final_opto** 阶段完成。

例如，要跳过初始布局，但对其余优化与布局阶段在已完成初始布局的设计上运行，可使用：

```
icc2_shell> place_opt -from initial_drc
```

为布局与优化创建临时时钟树

Creating a Temporary Clock Tree for Placement and Optimization

默认情况下，**place_opt** 在布局与优化期间使用理想时钟（ideal clock）信息。但可通过将应用选项 **place_opt.flow.trial_clock_tree** 设为 **true**，让工具构建临时时钟树，并在 **place_opt** 期间使用传播时钟（propagated clocks）。

布局与优化之后，使用 **clock_opt** 或 **synthesize_clock_trees** 执行时钟树综合时，工具会移除临时时钟树并构建实际时钟树。为使临时时钟树与实际时钟树相关，应在运行 **place_opt** 之前指定时钟树综合设置。

优化 **Clock-Gating** 单元

Optimizing Clock-Gating Cells

默认情况下，工具在以下时机合并 clock-gating 单元：

- 未启用 Synopsys physical guidance 流程时，在 **place_opt** 的 **initial_place** 阶段开始处；
- 通过将 **place_opt.flow.do_spg** 设为 **true** 启用 Synopsys physical guidance（SPG）流程时，在 **place_opt** 的 **initial_opt** 阶段开始处。

但若在运行 **place_opt** 之前已运行 **merge_clock_gates**，则 **place_opt** 期间不再合并 clock gate。可按如下方式控制 clock-gating 单元的合并：

- 仅合并处于时钟树同一层级的 clock-gating 单元：将 **cts.icg.merge_cross_level** 设为 **false**。默认情况下，工具可合并处于时钟树不同层级的 clock-gating 单元。
- 禁用 clock-gating 单元合并：将 **place_opt.flow.merge_clock_gates** 设为 **false**。

若 clock gate 的 enable 引脚处于关键时序路径上，**place_opt** 还可进一步优化 clock-gating 单元。要启用该功能，将 **place_opt.flow.optimize_icgs** 设为 **true**。

在 clock-gating 单元优化期间，工具会：

- 执行 clock-aware 布局（无论 `place_opt.flow.clock_aware_placement` 如何设置），并将关键 clock-gating 单元及其扇出放置在更优位置；
- 构建临时时钟树（无论 `place_opt.flow.trial_clock_tree` 如何设置），并用该时钟树识别时序关键的 clock-gating 单元；
- 若 clock gate 的 enable 引脚处于关键时序路径上，则拆分（split）clock-gating 单元。
可通过 `place_opt.flow.optimize_icgs_critical_range` 指定临界范围，以控制拆分程度。

对 **Clock Gate** 使用更准确的 **Latency**

Using Accurate Latencies for Clock Gates

默认情况下，工具在整个 `place_opt` 流程中估计并更新 clock-gate latency。这确保在时钟树综合之前、时钟仍为理想时钟时，数据通路优化与 `useful-skew` 优化能使用更准确的 clock-gate latency。

要提高估计 clock-gate latency 的准确性，应在运行 `place_opt` 之前指定时钟树综合设置，例如时钟树 `buffer/inverter`、时钟非默认布线规则（`nondefault routing rules`）等。

注意

若已按「为布局与优化创建临时时钟树」启用 `trial clock tree synthesis`，或按「优化 Clock-Gating 单元」启用 clock-gate 优化（二者在 CTS 之前用更准确的方法建模时钟树效应），则工具不再执行 clock-gate latency 估计与 `clock-gate-latency-aware` 布局。

工具不会修改你对 clock-gating 单元指定的任何 `set_clock_latency` 约束。工具估计的 clock-gate latency 值通过 `set_clock_latency -offset` 存储。可用 `write_script -format icc2` 查看这些估计值，并在生成输出中查找 `set_clock_latency -offset` 命令。`write_sdc` 生成的 SDC 文件不会捕获这些估计的 latency offset。

要阻止工具估计特定 clock-gate 的 latency，对其时钟引脚设置 `dont_estimate_clock_latency` 属性，例如：

```
icc2_shell> set_attribute \
    [get_pins {ICG21/CK}] dont_estimate_clock_latency true
```

启用基于全局布线的高扇出综合

Enabling Global Route Based High-Fanout Synthesis

工具执行高扇出综合与电气 DRC 违例修复时，默认使用虚拟布线（`virtual routes`）。

要在高扇出综合与电气 DRC 违例修复期间使用全局布线（`global routes`），在运行 `place_opt` 之前设置 `place_opt.initial_drc.global_route_based`：

```
icc2_shell> set_app_options \
    -name place_opt.initial_drc.global_route_based -value 1
```

要启用包含 `trial` 高扇出综合的集成两遍初始布局流程，在运行 `place_opt` 之前设置：

```
icc2_shell> set_app_options \
    -name place_opt.initial_drc.global_route_based -value 1
icc2_shell> set_app_options \
    -name place_opt.initial_place.two_pass -value true
```

该集成两遍初始布局流程包括：线长驱动布局、**trial** 高扇出综合，以及时序驱动布局。工具丢弃 **trial** 高扇出综合结果，但保留时序驱动布局，并继续进行基于全局布线的高扇出综合。

对 **floorplan** 碎片化严重的设计，启用集成两遍初始布局与基于全局布线的高扇出综合，有助于改善时序并降低拥塞。

启用缓冲树重建

Enabling the Rebuilding of Buffer Trees

在 **place_opt** 的 **initial_drc** 阶段，工具默认执行高扇出 **net** 综合，为高扇出 **net** 构建缓冲树。**initial_drc** 之后，若能改善 **setup** 时序、逻辑 DRC 违例、线长或面积，工具可移除并重建整棵缓冲树或部分缓冲树。

要在以下阶段启用缓冲树重建：

- 在 **initial_opto** 阶段针对多电压 **net**：将 **opt.buffering.enable_mv_rebuffering** 设为 **true**
- 在 **final_opto** 阶段针对任意 **net**：将 **opt.buffering.enable_rebuffering** 设为 **true**

更改拥塞努力程度

Changing the Congestion Effort

默认情况下，**place_opt** 执行拥塞优化以改善设计的可布线性。可通过 **place_opt.place.congestion_effort** 禁用或控制该优化的努力程度；默认值为 **medium**。

对关键 **Net** 使用非默认布线规则

Using of Nondefault Routing Rules for Critical Nets

为改善时序 QoR，工具可在布线前优化期间对时序关键 **net** 使用非默认布线规则（**nondefault routing rules**）。此外，工具可引导路由器将这些非默认规则分配作为软约束加以遵守。

要为 **place_opt** 启用该能力，将 **place_opt.flow.optimize_ndr** 设为 **true**：

```
icc2_shell> set_app_options -name place_opt.flow.optimize_ndr \
    -value true
```

启用路径优化

Enabling Path Optimization

路径优化（**path optimization**）是一种增量优化能力，专注于改善时序路径的 QoR。也可在 **place_opt** 期间通过将 **place_opt.flow.do_path_opt** 设为 **true** 来启用：

```
icc2_shell> set_app_options -name place_opt.flow.do_path_opt -value true
```

在 **place_opt** 期间执行 **IR-Drop-Aware** 布局

Performing IR-Drop-Aware Placement During the place_opt Command

布局期间，工具可利用单元的电压（**IR**）**drop** 值识别高功耗密度区域，并分散高压降单元，从而降低此类区域的功耗密度。

要在 `place_opt` 期间执行 IR-drop-aware 布局，按下列步骤：

1. 用 `place_opt -to final_place` 运行到最终布局阶段。
2. 为 RedHawk Fusion 做好设置，并用 `analyze_rail -voltage_drop` 执行静态或动态压降分析，例如：

```
icc2_shell> source redhawk_setup.tcl
icc2_shell> analyze_rail -voltage_drop static -nets {VDD VSS}
```

更多信息见「执行压降分析」。

3. 将 `place.coarse.ir_drop_aware` 设为 `true`，以启用 IR-drop-aware 布局。
4. (可选) 按「控制 IR-Drop-Aware 布局」指定额外设置。
5. 用 `place_opt -from final_place` 重新运行最终布局阶段。

在 `place_opt` 期间执行并发时钟与数据优化

Performing Concurrent Clock and Data Optimization During the place_opt Command

在数据通路优化中应用 `useful skew` 技术，利用正松弛并调整寄存器的时钟到达时间以改善时序 QoR，称为并发时钟与数据（concurrent clock and data, CCD）优化。

默认情况下，工具在 `place_opt` 的最终优化阶段执行 CCD。可通过将 `place_opt.flow.enable_ccd` 设为 `false` 来禁用。

`place_opt` 期间导出的时钟 latency 调整以 `clock-latency offset` 形式通过 `set_clock_latency -offset` 存储，并以 `clock-balance-point-delay offset` 形式通过 `set_clock_balance_point -offset` 存储。可用 `write_script -format icc2` 查看这些 `clock-latency offset`（在输出中查找 `set_clock_latency -offset`）。`write_sdc` 生成的 SDC 不会捕获这些 `offset`。可用 `report_clock_balance_point` 报告 `clock-balance-point-delay offset`。

要为 `place_opt` 期间的 CCD 设置 block，执行下列步骤：

1. 确保为 `clock_opt` 设为 `active` 的 `scenario`，对 `place_opt` 也保持 `active`。
2. 通过下列两种设置之一，指定工具在 `place_opt` 期间构建临时时钟树并使用传播时钟：

```
icc2_shell> set_app_options -name place_opt.flow.trial_clock_tree \
    -value true
```

```
icc2_shell> set_app_options -name place_opt.flow.optimize_icgs \
    -value true
```

或者，工具可用理想时钟做 `useful skew` 计算；但必须为所有 `clock-gating` 单元与时序单元（含 `macro`）的时钟引脚指定准确的理想时钟 latency。

可通过下列可选步骤改变 `place_opt` 期间 CCD 的默认行为：

1. (可选) 限制 CCD 的 latency 调整值，见「限制 Latency 调整值」。
2. (可选) 控制边界寄存器的 latency 调整，见「排除边界路径」。
3. (可选) 在 CCD 中忽略特定 `path group`，见「排除特定 Path Group」。

4. (可选) 在 CCD 中忽略特定 scenario, 见「排除特定 Scenario」。
5. (可选) 在 CCD 中忽略特定 sink, 见「排除特定 Sink」。
6. (可选) 分析 CCD 效果, 见「报告并发时钟与数据时序」。
7. (可选) 对带内部延时的 macro 导出自动零平衡点 (automatic zero balance point)。
8. (可选) 将 latch 与 discrete clock gate 按相同 offset 一起 skew, 见「Skewing Latch and Discrete Clock Gates」。

3.1.3 使用 Design Compiler 工具的物理引导

Using Physical Guidance From the Design Compiler Tool

使用 place_opt 执行布局与优化时, 可将 Design Compiler Graphical 工具的 Synopsys physical guidance 信息作为起点。

使用 Design Compiler Graphical 布局的好处包括:

- 缩短布局步骤的运行时间;
- 改善 Design Compiler Graphical 与 IC Compiler II 之间的相关性。

要从 Design Compiler Graphical 传递设计, 使用 Verilog 格式的门级网表以及约束信息。

为保持一致性并改善相关性, 必须:

- 在两个工具中使用相同的逻辑约束 (含时序与 UPF 信息)。
但须将 Design Compiler 的 scenario 约束转换为 IC Compiler II 支持的 mode、corner 与 scenario 格式。
- 在 IC Compiler II 中使用 Design Compiler Graphical 布局, 并在两个工具中使用相同的 floorplan 与物理约束。
不过, 可提供 Design Compiler Graphical 当前不支持、但 IC Compiler II 物理实现所需的额外 floorplan 信息。

要从 Design Compiler Graphical 传递物理信息 (布局、floorplan 与物理约束), 执行下列任务:

- 在 Design Compiler Graphical 中用 write_def 生成 DEF 文件, 并在 IC Compiler II 中用

```
read_def -add_def_only_objects {cells} -convert_sites
```

读入。

该 DEF 包含:

- Die area、site rows 与 tracks
- Port 的位置与形状
- Macro、标准单元与 physical-only 单元的位置
- Route guide 与预布线 net
- Placement blockage
- 创建 Tcl 文件, 包含 DEF 未捕获的物理约束, 并在 IC Compiler II 中应用到设计。
DEF 未捕获的物理约束包括:

- Voltage area 定义
- Layer 约束
- 带利用率的特殊 route guide
- 带 blocked layers 的特殊 blockage

要在 IC Compiler II 中使用 Synopsys physical guidance，在运行 place_opt 之前设置：

```
icc2_shell> set_app_options -name place_opt.flow.do_spg -value true
```

下列示例脚本展示如何将 Synopsys physical guidance 与 place_opt 配合使用：

```
# Read in the netlist
read_verilog design_spg.v

# Apply the timing constraints
source timing_const.tcl

# Apply the UPF constraints
load_upf design.upf

# Apply the DEF file generated from the Design Compiler Graphical tool
read_def -add_def_only_objects {cells} -convert_sites design_dcg.def

# Apply the placement constraints not supported by DEF
source placement_const.tcl

# Enable the Synopsys placement guidance flow
set_app_options -name place_opt.flow.do_spg -value true

# Perform placement and optimization
place_opt
```

更多信息见 *Design Compiler User Guide* 中的 Synopsys physical guidance。

3.1.4 执行多比特寄存器优化

Performing Multibit Register Optimization

IC Compiler II 可将单比特寄存器或较小的多比特寄存器组合 (bank)，并替换为等效的更大多比特寄存器。例如，可将八个 1-bit 寄存器或四个 2-bit 寄存器 bank 组合，替换为一个 8-bit 寄存器 bank 或两个 4-bit 寄存器 bank。工具仅在单比特寄存器具有相同时序约束时才合并它们，并将时序约束复制到结果多比特寄存器。

用多比特寄存器替换单比特寄存器可减少：

- 面积（因共享晶体管与优化的晶体管级版图）；
- 时钟树总网线长度；
- 时钟树 buffer 数量与时钟树功耗。

若能改善总负松弛 (total negative slack)，工具也可将大多比特寄存器拆分 (debank) 为较小的多比特寄存器或单比特寄存器。

(原书 *Figure 25: Replacing Multiple Single-Bit Register Cells With a Multibit Register Cell*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 3.1: 用多比特寄存器单元替换多个单比特寄存器单元

某些逻辑库具有混合驱动强度 (`mixed-drive-strength`) 的多比特寄存器，部分 bit 驱动强度高于其他 bit。对此类单元，若违例路径经过较低驱动强度的 bit，工具可对混合驱动强度多比特单元重新布线 (`rewire`)，使违例路径经过较高驱动强度的 bit。

以下主题说明如何执行多比特寄存器优化：

- 执行集成多比特寄存器优化
- 使用离散命令执行多比特寄存器优化
- 对多比特保持寄存器 (`retention registers`) 做 `banking`

执行集成多比特寄存器优化

Performing Integrated Multibit Register Optimization

要在 `place_opt` 期间执行多比特寄存器优化：

1. 将 `place_opt.flow.enable_multibit_banking` 设为 `true`，启用多比特寄存器 `banking`。
2. (可选) 将 `place_opt.flow.enable_multibit_debanking` 设为 `true`，启用 `debanking`。
3. (可选) 将 `multibit.banking.across_equivalent_icg` 设为 `true`，允许对由等效 `integrated-clock-gating` (ICG) 单元驱动的寄存器做 `banking`。
默认情况下，工具仅对由同一时钟 `net` 驱动的寄存器做 `banking`。启用后，工具会对等效 `clock-gating` 单元驱动的寄存器做 `banking`，并合并这些 `clock-gating` 单元。但这样做可能丢失施加在连接至 `clock-gating` 单元的 `net` 上的非默认布线规则。因此优化后请检查并重新应用这些规则。
4. (可选) 将 `place_opt.flow.enable_multibit_rewiring` 设为 `true`，启用多比特寄存器 `rewiring`。
5. (可选) 用 `set_multibit_options` 指定 `banking` 设置。例如，将 `reg[1]` 排除在多比特优化之外：

```
icc2_shell> set_multibit_options -exclude [get_cells "reg[1]"]
```

6. (可选) 按表 3.1 控制 `banking` 与 `debanking`。
7. 用 `place_opt` 执行布局与优化。
8. 用 `report_multibit` 报告多比特组件。

表 3.1: 控制多比特寄存器 Banking 与 Debanking 的应用选项

目的	将下列应用选项设为 true
Banking 时忽略未包含在 SCANDEF 信息中的 scan flip-flop	<code>multibit.banking.enable_strict_scan_check</code>
Banking 时考虑带多个 scan-in 引脚与内部 scan-in 引脚的多比特寄存器库单元	<code>multibit.banking.lcs_consider_multi_si_cells</code>
Banking 与 debanking 时忽略用户指定的 size-only 单元	<code>multibit.common.ignore_sizeonly_cells</code>

使用离散命令执行多比特寄存器优化

Performing Multibit Register Optimization Using Discrete Commands

可用下列离散步骤执行多比特 banking:

1. 用 `create_placement` 执行初始布局。
2. (可选) 用 `set_multibit_options` 指定 banking 设置。
3. 用 `identify_multibit` 识别可替换为多比特单元的单元组，见「识别多比特 Bank」。
4. 用 `place_opt -from initial_drc` 优化设计。
5. (可选) 用 `report_multibit` 报告 banking 信息。
6. (可选) 若 banking 未改善 QoR，用 `split_multibit` 将特定多比特 bank 拆分为单独寄存器或更小的多比特 bank，见「拆分多比特 Bank」。

识别多比特 Bank

Identifying Multibit Banks 要用 `identify_multibit` 识别可替换为多比特单元的寄存器、isolation cell 或 level-shifter 组，必须指定：

- 如何应用 banking 信息，任选其一：
 - 用 `-apply` 让工具将 banking 信息应用到设计；
 - 用 `-output_file` 生成包含 `create_multibit` 命令的输出文件，以便手动应用。
可查看、必要时编辑该输出，再用 `source` 应用到设计。
- 要 bank 的单元类型，任选其一：
 - 用 `-register` 识别可 bank 的寄存器组；
 - 用 `-mv_cell` 识别可 bank 的 isolation 或 level-shifter 组。

使用 `-register banking` 寄存器时，可选地：

- 用 `-input_map_file` 指定输入映射文件，控制单比特到多比特 bank 的映射。该文件包含哪些多比特寄存器 bank 替换哪些单比特寄存器，例如：

```
reg_group_1 {REGX1 REGX2 REGX4}
2 {1 MREG2}
4 {1 MREG4}
6 {1 MREG2}{1 MREG4}
```

该映射文件包含名为 `reg_group_1` 的功能组，说明如何将参考单元类型为 REGX1、REGX2、REGX4 的单比特寄存器组合为多比特寄存器：

- 第一行：两个单比特寄存器可组合成一个由一个 MREG2 参考单元构成的寄存器 bank；
 - 第二行：四个单比特寄存器可组合成一个由一个 MREG4 参考单元构成的寄存器 bank；
 - 第三行：六个单比特寄存器可组合成由一个 MREG2 与一个 MREG4 构成的寄存器 bank。
- 用 `-slack_threshold` 按松弛排除寄存器。松弛小于指定阈值的寄存器被忽略。默认考虑所有寄存器。

此外，可选地通过下列方式控制被 bank 的单元：

- 用 `-cells` 指定要考虑的实例列表（使用后不考虑其他单元）；
- 用 `-exclude_instance` 指定要排除的实例列表；
- 用 `-exclude_library_cells` 指定要排除其全部实例的库单元列表。

默认情况下，`identify_multibit` 执行 DFT 优化，重新划分 scan chain，使由同一时钟驱动器或 gating 单元驱动的寄存器进入同一 scan chain，从而增加可组合成多比特寄存器 bank 的单比特寄存器数量。若不希望重新划分 scan chain，对 `identify_multibit` 使用 `-no_dft_opt`。

拆分多比特 Bank

Splitting Multibit Banks 为改善局部拥塞或路径松弛，可用 `split_multibit` 将多比特 bank 拆分为更小的多比特 bank 或单比特单元。默认情况下，该命令拆分负松弛时序路径上的所有多比特单元。

要拆分：

- 特定单元：指定单元实例名。可用 `-lib_cells` 指定拆分后使用的较小单元的参考名。
- 下例将名为 `reg_gr0` 的四比特寄存器 bank 拆分为两个两比特多比特 bank：

```
icc2_shell> split_multibit reg_gr0 -lib_cells {lib1/MREG2 lib1/MREG2}
```

- 松弛小于特定值的路径上的多比特单元：使用 `-slack_threshold`。
- 下例拆分松弛差于 -1 的路径上的多比特单元：

```
icc2_shell> split_multibit -slack_threshold -1
```

- 特定时序 path group 中的多比特单元：使用 `-path_groups`。
- 使用该选项时，工具拆分指定 path group 中负松弛路径上的多比特单元。可与 `-slack_threshold` 联用为该 path group 指定松弛阈值。
- 下例拆分 path group CLKA 中松弛差于 -0.5 的路径上的多比特单元：

```
icc2_shell> split_multibit -path_groups CLKA \
-slack_threshold -0.5
```

还可进一步控制拆分：

- 用 `-exclude_instance` 排除特定单元；
- 用 `-cells` 仅考虑特定单元。

`split_multibit` 仅拆分由 `create_multibit` 或 `identify_multibit` 创建的多比特单元。

对多比特保持寄存器做 **Banking**

Banking Multibit Retention Registers

工具可将单比特保持寄存器组合为多比特保持寄存器，若满足：

- 单比特保持寄存器关联同一 `retention strategy`；
- 其 `retention` 引脚具有相同驱动器；
- 单元库中有匹配的多比特保持寄存器；
- 该多比特保持寄存器的库单元已通过 `map_retention_cell -lib_cells` 指定为该 `retention strategy` 的库单元。

工具可拆分多比特保持寄存器，若：

- 单元库中有匹配的单比特保持寄存器；
- 这些单比特保持寄存器的库单元已通过 `map_retention_cell -lib_cells` 指定为该 `retention strategy` 要使用的库单元。

注意

若使用 `identify_multibit` 识别多比特保持寄存器，请使用 `-register`，而非 `-mv_cell`。

3.1.5 执行 Magnet Placement

Performing Magnet Placement

为改善复杂 floorplan 的拥塞或改善设计时序，可使用 **magnet placement**：将固定对象指定为磁体（magnet），并让工具将连接到该磁体对象的所有标准单元靠近其放置。磁体对象可为固定 macro 单元、固定 macro 的引脚，或 I/O port。

为获得最佳结果，应在标准单元布局之前执行 **magnet placement**。

执行 **magnet placement** 时，使用 `magnet_placement` 命令，并指定磁体及所需特殊功能的选项。表 3.2 列出 `magnet_placement` 选项。

下列示例使用图 3.2 所示数据通路。

（原书 Figure 26: *Example of Pulling Cells in a Contiguous Data Path*）

图 3.2: 在连续数据通路中拉取单元的示例

表 3.2: magnet_placement 命令选项

目的	使用该选项
指定要拉向磁体对象的单元	-cells cell_list
允许用 -cells 指定层次单元，并说明如何处理这些层次单元	-hierarchy_mode default block all
允许移动标记为 fixed 的单元	-move_fixed
允许移动标记为 legalize-only 的单元	-move_legalize_only
Magnet placement 后将移动过的单元标记为 fixed 或 legalize-only	-mark_fixed / -mark_legalize_only
指定距磁体的逻辑级数，在该范围内考虑 magnet placement	-logical_levels number
确定逻辑级数时排除 buffer 与 inverter	-exclude_buffers
查找要拉取的单元时，在时序单元输入处停止路径追踪	-stop_by_sequential_cells
查找要拉取的单元时，在时序单元输出处停止路径追踪	-stop_on_sequential_cells
查找要拉取的单元时，在指定 pin、port 或单元之外停止路径追踪	-stop_points
防止将单元放在 soft blockage 上（默认仅避开 hard blockage）	-avoid_soft_blockage
磁体为多个 port 或长 port 时，指定如何放置单元	-multiple_long_port_mode auto nearby
仅报告将被拉向磁体的单元，但不实际拉取	-only_report_magnet_cells
创建正在被拉向磁体的单元集合	-get_collection

下列命令将所有单元 C1 至 C8 拉向磁体对象 C0:

```
icc2_shell> magnet_placement C0
```

下列命令将 C6、C7、C8 拉向磁体对象 C2:

```
icc2_shell> magnet_placement C2 -cells {C6 C7 C8}
```

尽管 C3、C4、C5 形成连续数据通路，但它们与 C0 隔离，因此下列命令不会将任何单元拉向 C0:

```
icc2_shell> magnet_placement C0 -cells {C3 C4 C5}
```

下列命令也不会将任何单元拉向 C0，因为 C3、C6、C8 不形成连续数据通路:

```
icc2_shell> magnet_placement C0 -cells {C3 C6 C8}
```

3.1.6 细化布局

Refining Placement

要细化布局并最小化拥塞，使用 `refine_placement` 命令。默认对该命令对整个设计执行增量布局。

- 要细化特定区域的布局，使用 `-coordinates` 并指定区域坐标。
- 要更改布局努力程度，使用 `-effort`，取值为 `low`、`medium` 或 `high`；默认为 `medium`。
- 要更改最小化拥塞的努力程度，使用 `-congestion_effort`，取值为 `low`、`medium` 或 `high`；默认为 `medium`。
- 要控制对现有布局的扰动程度，使用 `-perturbation_level`，取值为 `min`、`medium`、`high` 或 `max`；默认为 `medium`。

3.1.7 对多电压 Block 执行布局与优化

Performing Placement and Optimization on Multivoltage Blocks

布局与优化会自动考虑多电压约束，例如 `voltage area`。

布局期间，工具会：

- 将单元放置在其关联的 `voltage area` 或 `voltage area shape` 内；
- 将 `level shifter` 与 `isolation cell` 靠近 `voltage area` 边界放置。

优化期间插入 `buffer` 时，工具会：

1. 考虑 `net` 的驱动器与负载的相关供电，并用下列之一对电源状态做动态分析：
 - 用 `create_pst` 创建的 UPF power state table；
 - 用 `add_power_state` 创建的 UPF power state groups。
2. 根据电源状态与被缓冲 `voltage area` 中的可用供电，选择不会引入新的多电压违例的缓冲供电。优先选用单轨（`single-rail`）`buffer` 而非双轨（`dual-rail`）`buffer`，以节省功耗与面积并改善可布线性。

要了解特定 net 如何或是否被缓冲，使用 `check_bufferability` 命令，见「分析 Net 的可缓冲性」。

3.1.8 重建缓冲树

Rebuilding Buffer Trees

在 `place_opt` 的 `initial_drc` 阶段，工具移除已有缓冲树并执行高扇出综合。运行 `place_opt` 之后，要用 `create_buffer_trees` 按特定需求重建特定缓冲树。

默认情况下，`create_buffer_trees` 会：

- 为所有高扇出 net 重建缓冲树。
要重建特定高扇出 net，使用 `-from`。
- 在重建前移除已有缓冲树。
要保留已有 buffer 并增量构建缓冲树，使用 `-incremental`。
- 使用虚拟布线拓扑与提取构建缓冲树。
要基于全局布线构建缓冲树，使用 `-global_route_based`。
- 不合法化新添加的单元。
要合法化添加的单元，在 `create_buffer_trees` 之后运行 `legalize_placement`。

下例仅为 n33 net 重建缓冲树，且仅使用 buf4x 与 buf8x 类型 buffer：

```
icc2_shell> set_lib_cell_purpose -include none {tech_lib/*}
icc2_shell> set_lib_cell_purpose \
  -include optimization {tech_lib/buf4x tech_lib/buf8x}
icc2_shell> create_buffer_trees -from n33 -global_route_based
```

注意

`create_buffer_trees` 仅应在设计流程的布线前（`preroute`）阶段使用。

3.1.9 添加与移除 Tie Cell

Adding and Removing Tie Cells

运行 `place_opt` 或 `clock_opt` 时，工具可在优化期间添加 tie cell。此外，可用 `add_tie_cells` 为特定 pin 或 port 添加 tie cell。`place_opt`、`clock_opt` 或 `add_tie_cells` 首次向 block 添加 tie cell 时，工具会优化 tie cell 连接：移除所有已有 tie cell，并添加经最优聚类的 tie cell。

要移除特定 tie cell，使用 `remove_tie_cells`。默认移除当前 block 中的所有 tie cell。要移除特定实例或某库单元的全部实例，使用 `-objects`。

3.2 识别优化期间无法修复的问题

Identifying Issues That Cannot Be Fixed During Optimization

IC Compiler II 提供下列能力，用于识别优化期间无法修复的问题：

- 分析 Net 的可缓冲性
- 分析无法修复的违例

3.2.1 分析 Net 的可缓冲性

Analyzing the Bufferability of Nets

要分析当前 block 中的 net 是否可被缓冲，或报告为何无法缓冲，使用 `check_bufferability`。该命令检查是否存在：

- 在指定 voltage area 中可用于缓冲该 net 的 supply net；
- 可用于缓冲该 net 的合适库单元；
- 阻止缓冲的 net 设置，例如 `ideal-network` 或 `don't-touch` 约束。

使用该命令时必须指定：

- 要分析的 net，任选其一：
 - 用 `-nets` 指定 net 名；
 - 用 `-driver` 与 `-loads` 指定驱动器与负载，并用 `-hierarchy` 指定进行分析的逻辑层次。
- 用 `-voltage_area` 指定分析可缓冲性的 voltage area。

例如，要分析名为 n34 的 net 在 PD1 电源域中是否可添加 buffer：

```
icc2_shell> check_bufferability -nets n34 -voltage_area PD1
Information: Can insert dual rail buffers (8 lib_cells) and inverters (6
lib_cells) with supply nets (power: VDDSW, ground: VSS). (MV-454)
```

要分析在 PD_TOP voltage area 的 top 层次中，是否可在驱动引脚 `top_reg_1/Q` 与负载引脚 `top_U1_9/B` 之间添加 buffer：

```
icc2_shell> check_bufferability -driver top_reg_1/Q \
-loads top_U1_9/B -hierarchy / -voltage_area PD_TOP
Information: Cannot insert buffers or inverters, because required supply
nets are not available. (MV-459)
```

要获取关于 net 可缓冲性的更多信息，使用 `-verbose`。

3.2.2 分析无法修复的违例

Analyzing Violations That Cannot Be Fixed

要分析 block 并生成包含无法修复违例信息的报告，使用 `analyze_design_violations`。报告先给出分析摘要，再按降序列出违例明细。违例按无法修复的原因分类。

使用该命令时，必须用 `-type` 指定要分析的违例类型：

- `max_trans`：分析最大 transition 违例
- `setup`：分析 setup 违例
- `hold`：分析 hold 违例

- **static_noise**: 分析静态噪声违例
- **only_lib**: 分析影响优化的库设置

此外，可指定：

- 用 **-fanout** 指定识别高扇出 net 的阈值。默认将扇出大于 30 的 net 视为高扇出 net。
- 用 **-slack** 指定识别小的时序违例的松弛阈值。默认将小于 5 ps 的松弛违例视为小违例。
- 用 **-drc_nets** 指定用于最大 transition 分析的 net。默认分析所有 net。
- 用 **-endpoints** 指定 setup/hold 分析的端点。默认分析所有端点。
- 用 **-output** 指定输出文件名。工具会在你指定的文件名后添加 .txt 扩展名。默认生成名为 **analyze_design_violations** 的文件。

3.3 分析布局

Analyzing the Placement

IC Compiler II 提供下列基于 GUI 与报告的能力，用于分析布局：

- 报告利用率 (Utilization)
- 报告布局 QoR
- 查询与更改布局状态
- 在 GUI 中分析布局

3.3.1 报告利用率

Reporting Utilization

Block 的利用率按下列公式计算，其中 **demand** 为对象占用的总面积，**capacity** 为总可用面积：

$$\text{Utilization} = \text{Demand} / \text{Capacity}$$

报告利用率的步骤：

1. (可选) 用 **create_utilization_configuration** 创建利用率报告配置。

下例为当前 block 创建名为 **config1** 的配置，用 **core area** 计算 **capacity**，并排除 **hard** 与 **soft placement blockage**：

```
icc2_shell> create_utilization_configuration -scope block config1 \
          -capacity core_area -exclude {hard_blockages soft_blockages}
```

要用 **remove_utilization_configurations** 移除利用率配置。

2. 用 **report_utilization** 报告利用率。

用 **-config** 与 **-scope** 指定配置及其存储位置。**-scope** 的有效值为 **tech**、**lib** 与 **block**。若与 **-config** 联用时未指定 **-scope**，工具依次在当前 **block**、设计库、逻辑库中搜索该配置。

要用 **-of_objects** 报告特定对象的利用率，并指定一个或多个 **block**、**voltage area**、**site array** 或 **bound**。所有对象必须为同一类型。

3.3.2 报告布局 QoR

Reporting the Placement QoR

`report_placement` 显示当前 block 的布局 QoR 信息。默认报告 block 中所有 net 的总半周长包围盒（half-perimeter boundary box）线长，以及任何布局违例，例如：

```
icc2_shell> report_placement
...
...

Wire length report (all)
=====
wire length in design leon3mp: 12386597.894 microns.

Physical hierarchy violations report
=====
Violations in design leon3mp:
    0 cells have placement violation.
```

要用 `-hierarchical` 同时报告子 block 与当前 block。

要用 `-wirelength` 及相应关键字，将线长计算限制为特定类型的 net：

- `-wirelength interface`：仅包含跨越物理边界的 net；
- `-wirelength hard_macro`：仅包含连接到 hard macro 的 net；
- `-wirelength IO`：仅包含连接到 I/O pin 或 pad 的 net；
- `-wirelength none`：抑制线长报告。

要用 `-physical_hierarchy_violations` 及相应关键字，限制报告的布局违例类型：

- `-physical_hierarchy_violations internal`：仅报告放置在当前 block 布局区域之外的单元；
- `-physical_hierarchy_violations external`：仅报告与子 block 重叠的单元；
- `-physical_hierarchy_violations none`：抑制布局违例报告。

还可报告下列布局度量：

- **Swimming pool area**

Swimming pool 是由子 block、hard macro 与 placement blockage 围成的区域。此类区域可能使布局或布线难以成功，应覆盖 placement blockage 或予以消除。

- 用 `-swimming_pool_area` 报告当前 block 中的总 swimming pool 面积；
- 用 `-maximum_swimming_pool_area` 报告面积小于特定阈值的 swimming pool 区域。

- **Thin channel area**

Thin channel 是当前 block 的子 block（无论是 hard macro 还是物理层次）之间的狭窄区域。此类通道可能使布局或布线难以成功，应覆盖 placement blockage 或加宽。

- 用 `-thin_channel_area` 报告当前 block 中的总 thin channel 面积；
- 用 `-maximum_thin_channel_width` 或 `-maximum_thin_channel_height` 报告宽度或高度小于特定阈值的 thin channel。

- **Hard macro** 上方布线的线长
用 `-hard_macro_route_over` 报告当前 block 中在 hard macro 上方布线的 net 线长。该选项报告每个存在 over-macro 布线的 hard macro 的水平与垂直线长，以及 over-macro 布线的总线长。
- **Hard macro** 重叠数量
用 `-hard_macro_overlap` 报告当前 block 中重叠 hard macro 的总数。

3.3.3 查询与更改布局状态

Querying and Changing the Placement Status

单元实例的 `status` 属性指示其当前布局状态。表 3.3 列出可能取值。

表 3.3: 布局状态关键字

布局状态	说明	DEF 语法
unplaced	单元实例尚未布局。注：要用 <code>reset_placement</code> 将所有已布局单元恢复为 <code>unplaced</code> 。该命令仅修改状态为 <code>placed</code> 的单元；不修改 <code>fixed</code> 或 <code>locked</code> 的单元。	+UNPLACED
placed	单元实例已布局，但后续操作仍可移动。	+PLACED
fixed	单元实例已布局，后续操作可调整尺寸但不可移动。注：要防止这些单元被 <code>sizing</code> ，须设置 <code>dont_touch</code> 属性，见「优化期间保留单元与 Net」。	+FIXED
legalize_only	单元实例已布局，仅可由合法化器移动。	N/A
locked	单元实例已布局，不可手动或由工具移动。	+COVER

要获取单元实例的布局状态，用 `get_attribute` 查询其 `status` 属性。例如，获取名为 INST1 的单元实例：

```
icc2_shell> get_attribute -objects [get_cells INST1] -name status
```

要更改单元实例的布局状态，使用 `set_placement_status`。例如，将 INST1 设为 `fixed`，使其在后续优化与布局中不被移动：

```
icc2_shell> set_placement_status fixed [get_cells INST1]
```

3.3.4 在 GUI 中分析布局

Analyzing the Placement in the GUI

IC Compiler II 图形用户界面（GUI）提供下列工具，用于分析与可视化布局后的结果质量：

- **Global Route Congestion Map:** 可视化布局在避免布线拥塞方面的质量。
- **Cell Density Map:** 识别 block 中或矩形区域内的高单元密度区。
- **Pin Density Map:** 识别 block 中或矩形区域内的高引脚密度区。

有关在 GUI 中使用这些分析工具的更多信息，见 *IC Compiler II Graphical User Interface User Guide* 中的「Using Map and Visual Modes」。

3.4 分析时序

Analyzing Timing

IC Compiler II 会在需要时自动更新时序与延时估计结果。但可用 `update_timing` 显式更新时序。可用下列命令为 block 生成时序报告：

- `report_timing`: 报告当前 block 的最差时序路径。
- `report_qor`: 显示当前 block 的 QoR 信息与统计。
- `report_constraints`: 报告当前 block 的逻辑 DRC 违例。

有关生成这些报告的更多信息，见 *IC Compiler II Timing Analysis User Guide* 中的「Generating Reports」。

3.5 分析功耗

Analyzing Power

要计算并报告 block 的功耗，使用 `report_power`。

`report_power` 支持下列漏电功耗计算方法：

- **Average**: 基于所有状态的等权重概率。
- **Unconditional**: 基于 `default_cell_leakage_power` 库属性或 `cell_leakage_power` 库单元属性。
- **State-dependent**: 基于单元各状态条件的加权和。计算取决于状态相关漏电网模型表征与静态概率。若状态相关漏电信息不可用，则功耗计算使用 `unconditional` 方法。

要为 `report_power` 选择漏电功耗计算方法，将 `power.leakage_mode` 设为 `average`、`unconditional` 或 `state`。要不报告漏电功耗，设为 `off`。默认为 `state`：

```
icc2_shell> set_app_options -name power.leakage_mode -value average
```

要在功耗分析与报告期间使用复合电流源（CCS）接收端模型电容，将 `power.use_ccs_rcv_cap` 设为 `true`。

下列示例给出默认功耗报告。

```
...
...
Mode: my_mode
Corner: bc_corner
Scenario: bc_scenario
Voltage: 1.160000
Temperature: 125.000000
Voltage unit: 1.00V
Temperature unit: 1.00C
Power unit: 1.00pW
Cell Leakage Power          = 5125495.58 uW
Power Group      Leakage Power      Total Power      (    % )
-----
```

```

---
io_pad          0.00e+00      0.00e+00      ( 0.00%)
memory          0.00e+00      0.00e+00      ( 0.00%)
black_box       0.00e+00      0.00e+00      ( 0.00%)
clock_network   3.32e+10      3.32e+01      ( 0.65%)
register        1.62e+12      1.62e+03      ( 31.67%)
sequential      1.16e+09      1.16e+00      ( 0.02%)
combinational   3.47e+12      3.47e+03      ( 67.66%)
-----
---
Total          5.13e+12 pW      5.13e+03 mW
1

```

代码 3.1: 默认功耗报告

3.5.1 为报告创建功耗组

Creating Power Groups for Reporting

运行 `report_power` 且未指定单元列表时,工具报告默认功耗组的功耗信息。可用 `set_power_group` 创建由单元实例或库单元组成的功耗组。`report_power` 会同时报告默认与用户指定功耗组的信息。

若单元属于多个功耗组,用于报告目的的归属按下列优先级确定:

1. 用 `set_power_group` 为单元实例指定的功耗组;
2. 用 `set_power_group` 为其库单元指定的功耗组;
3. 该单元的默认功耗组。

要用 `get_power_group` 获取特定单元的功耗组:

- 用 `-default` 获取其默认功耗组;
- 用 `-user` 获取其可能所属的用户定义功耗组;
- 不带选项则获取用于报告目的的功耗组。

要用 `report_power_groups` 报告 `block` 中的所有功耗组。要用 `get_power_group_objects` 获取特定功耗组中的单元集合。

要用 `reset_power_group` 移除功耗组,或从特定功耗组中移除单元。

3.5.2 报告基于引脚的时钟网络功耗

Reporting Pin-Based Clock Network Power

`report_power` 将时钟网络功耗作为名为 `clock_network` 的默认功耗组的一部分进行报告。

默认情况下,报告的时钟网络功耗基于单元:若单元的某一输出在时钟网络上,则该单元的功耗(含其所有输入与输出的功耗)计入时钟网络。但可通过下列应用选项,将 `report_power` 报告的时钟网络功耗限制为仅包含时钟网络上的引脚:

```
icc2_shell> set_app_options -list {power.clock_network pin_based}
```

默认为 `cell_based`。

`report_clock_qor -type power` 报告的功耗信息是基于引脚（pin-based）的。

3.6 对比 QoR 数据

Comparing QoR Data

可按下列步骤生成基于 Web 的报告，以用 QORsum 报告查看并对比 QoR 数据：

1. (可选) 用 `set_qor_data_options` 配置后续步骤捕获与显示的 QoR 数据。

- 用 `-leakage_scenario` 与 `-dynamic_scenario` 指定设计中最关键的功耗 scenario。
工具用你指定的功耗 scenario 生成 QORsum 报告中功耗 QoR 的高层摘要。若未指定，则对漏电与动态 scenario 均使用总功耗最高的 active 功耗 scenario。
这些设置仅用于功耗 QoR 摘要；工具使用所有 active 功耗 scenario 的功耗信息来捕获并报告 QORsum 中的详细功耗信息。
- 用 `-clock_name` 与 `-clock_scenario` 指定最关键的时钟名与时钟 scenario。
工具用你指定的时钟名与 scenario 生成时钟 QoR 的高层摘要。若未指定，工具识别最关键的时钟并用于摘要。
这些设置仅用于时钟 QoR 摘要；工具使用所有时钟生成详细时钟 QoR 信息。
- 用 `-run_name` 为 QORsum 报告中的该次运行指定标识名。
默认工具用编号命名各次运行（如 Run1、Run2）。也可用更有意义的名称。在步骤 3 用 `compare_qor_data` 的 `-run_names` 也可指定运行名；若这样做，工具忽略 `set_qor_data_options -run_name` 指定的名称。

下例指定漏电功耗 scenario、动态功耗 scenario、时钟 scenario 以及用于对应摘要的时钟：

```
icc2_shell> set_qor_data_options \
-leakage_scenario S1 -dynamic_scenario S2 \
-clock_scenario S3 -clock_name sys_clk
```

2. 用 `write_qor_data` 收集报告用的 QoR 数据。该命令捕获与时序、功耗、运行时间及其他度量相关的 QoR 数据。

可在希望捕获 QoR 数据的每个设计流程阶段多次运行 `write_qor_data`。用 `-label` 指定值以标明捕获所处的流程阶段。

默认 `write_qor_data` 捕获与已布局并优化设计相关的数据。可用 `-report_group` 及五个支持值之一（unmapped、mapped、placed、cts、routed）按流程状态调整捕获的报告集。更细粒度控制可用 `-report_list` 显式指定要生成的每个报告。

`write_qor_data` 从多数常用工具报告（如 `report_qor`、`report_power`）向磁盘捕获标准 QoR 数据集。若有不属于命令集合的额外自定义数据要捕获，用 `capture_qor_data` 捕获完全自定义的 QoR 数据或 GUI 版图图像，以纳入基于 Web 的 QORsum 报告。这可用于补充 `write_qor_data` 捕获的标准 QoR 数据，或创建完全自定义的 QORsum 报告。相关命令包括：`define_qor_data_panel`（在 QORsum 中创建新面板以查看 QoR 数据），以及 `set_qor_data_metric_properties`（自定义任意 QORsum 面板中任意度量列的属性，例如着色样式或对比数据时的着色阈值）。

3. 用 `compare_qor_data` 生成 QORsum 报告。

该命令取一次或多次 `write_qor_data` 运行捕获的数据，并创建基于 Web 的报告以查看与对比这些结果。

必须用 `-run_locations` 指定上一步各次 `write_qor_data` 输出的位置；每个路径对应要对比的运行结果。

可用 `-run_names` 为 QORsum 中的运行指定标识名；若这样做，工具忽略 `set_qor_data_options -run_name`。默认用编号命名（如 `run1`、`run2`）。

可用 `-output` 指定输出目录；默认写入名为 `compare_qor_data` 的目录。要用 `-force` 覆盖已有输出数据；默认不覆盖。

4. 用 `view_qor_data` 查看生成的 QORsum 报告。

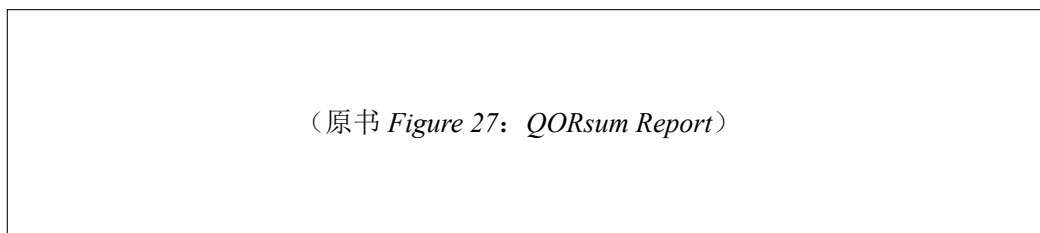


图 3.3: QORsum 报告

有关探索对比数据的更多信息，见下列主题：

- 设置基线运行（Baseline Run）
- 更改 QoR 显示样式
- 排序与过滤数据
- 探索详细对比数据

3.6.1 设置基线运行

Setting Your Baseline Run

默认情况下，数据的第一行是基线运行，其他所有运行与之对比。基线运行的名称以金色高亮，标记为「golden」或基线结果。

非基线单元格的着色指示相对基线的差异方向与程度：

- 红色表示相对基线变差；绿色表示相对基线改善；
- 较浅着色表示差异较小；较深着色表示差异较大。

要将光标悬停在列标题上，可查看各色阶对应的 `delta` 阈值。

要更改基线运行，单击 **Runs** 按钮，并从 **Baseline Run** 列选择新的基线运行。

参见

更改 QoR 显示样式；排序与过滤数据；探索详细对比数据。

3.6.2 更改 QoR 显示样式

Changing the QoR Display Style

默认情况下，对比表显示所有运行的 QoR 值。例如，某次运行的 NVE 数为 2374。

在表中任意位置右键单击，可循环切换不同显示样式：

- 相对基线的百分比 **delta**；
- 相对基线的绝对 **delta**（以斜体显示）。

例如，按百分比 **delta**，NVE 显示为比基线少 2.7% 的失败端点；按绝对 **delta**，NVE 显示为比基线少 65 个失败端点。

参见

排序与过滤数据；探索详细对比数据。

3.6.3 排序与过滤数据

Sorting and Filtering the Data

可对运行数据排序与过滤，以揭示结果中的模式，并确定要进一步探索的参数。

相关主题：

- 排序数据
- 过滤度量（Metrics）
- 过滤运行（Runs）
- 分析示例

参见

探索详细对比数据。

排序数据

Sorting the Data

单击任意列标题，按该度量排序。第一次单击升序，第二次单击降序。

例如，要使最差 TNS 位于表顶、最佳 TNS 位于表底，单击 TNS 列标题；再次单击则相反。

过滤度量

Filtering Metrics

要控制表中显示哪些度量，单击 **Metrics** 按钮，并在 **Metrics** 对话框中相应勾选或取消勾选。

例如，要仅显示 **setup** 时序、网表面积与单元计数，按图 3.4 选择度量。

(原书 *Figure 28: Metrics Dialog Box*)

图 3.4: Metrics 对话框

过滤运行

Filtering Runs

要控制表中显示哪些运行，单击 **Runs** 按钮，并从 **Visible Runs (Summary)** 列相应勾选或取消勾选探索运行。

例如，要仅显示前四次运行，按图 3.5 选择。

(原书 *Figure 29: Choose Baseline and Visible Runs Dialog Box*)

图 3.5: 选择基线与可见运行对话框

分析示例

Example Analysis

下例演示如何排序与过滤数据，以缩小要进一步探索的运行范围。

假设打开图 3.6 所示对比报告，并将 `util60_aspect1%1_layerM9` 运行设为基线。

(原书 *Figure 30: Comparison Report*)

图 3.6: 对比报告

可先查看 TNS，并将数据从最佳 TNS 排到最差 TNS。注意到最佳 TNS 运行使用 M9 顶层金属，最差使用 M7，说明限制金属层对时序影响显著。

随后可通过关闭 M7 运行的可见性，将分析限制为 M9 运行。在得到最佳 TNS 运行后，可按 GRCOverflow 列排序以将最差 overflow 置于顶部，从而对比拥塞。

注意到较高利用率运行的拥塞高于较低利用率运行。可按图 3.7 关闭较高利用率运行的可见性，留下数量可控、更能满足时序与拥塞目标的探索运行子集。

(原书 *Figure 31: Displaying Lower-Utilization Runs*)

图 3.7: 显示较低利用率运行

3.6.4 探索详细对比数据

Exploring the Detailed Comparison Data

启动 QORsum 报告时，应用打开 QOR Summary 表，汇总各次运行的高层时序、功耗与拥塞度量。该视图及其他摘要视图有助于排序与过滤数据，以定位要进一步探索的运行。

用摘要表识别最佳候选运行后，可用详细表获得更深洞察。摘要表给出所有运行的高层设计 QoR（如每次运行的整体设计时序 WNS、TNS、NVE），而详细表可显示设计中所有 path group 基于 path-group 的时序。详细表允许查看某一 path group 在所有运行上的时序，或一次查看最多六个运行的所有 path group 时序。所有详细表命名约定以「Details」结尾。例如，Path Type Details 表显示基于 path-type 的时序信息。

默认情况下，详细表从 Focused View 开始，如图 3.8。

(原书 *Figure 32: Focused View*)

图 3.8: Focused View

Focused View 一次显示一个 path group。要用顶部下拉菜单更改焦点。例如，可如图 3.9 指定 scenario 与 path group。

(原书 *Figure 33: Focused View Drop-down Menu*)

图 3.9: Focused View 下拉菜单

Focused View 聚焦于所有运行的某一特定属性。也可查看较少运行的全部属性。

例如，可同时查看所有 path group 以识别最差 path group。要切换到传统详细视图，单击顶部菜单的 Focused View 切换按钮，如图 3.10。

要查看特定度量的详细对比数据，单击左侧面板中的某个详细视图。例如，要查看基于 path-group 的时序详情，选择 Path Group Details，如图 3.11。

摘要表的每一行显示一次流程的结果，而详细表的每一行显示诸如 path group 名称等详细信息。各流程在每个度量下并排显示为列。例如，WNS 列组（以及每个度量的列组）显示标为 1、2、5、6、9、10 的相同六列。这些列代表各流程结果，用流程 ID 而非完整流程名，以避免列过宽。每个度量

(原书 *Figure 34: Focused View Toggle Button*)

图 3.10: Focused View 切换按钮

(原书 *Figure 35: Path Group Details*)

图 3.11: Path Group Details

下的第一列是基线，其余五列为与之对比的测试流程。表上方有图例：金色框中的流程 ID 表示基线，蓝色框表示测试流程，后跟流程名。可单击 **Legend** 按钮展开或折叠图例。也可在单击 **Runs** 按钮打开的「Choose Baseline and Visible Runs」对话框中查看流程 ID。

详细表视图一次最多可显示六个运行（基线运行，加上在 Choose Baseline and Visible Runs 对话框中选择的前五个其他运行）。要更改显示的运行：

1. 单击 **Runs** 按钮，打开 Choose Baseline and Visible Runs 对话框。
2. 在 **Visible Runs (Summary)** 列中相应勾选或取消勾选运行。这会同步选中或取消选中 **Visible flows (Detailed)** 列中的那些运行。

过滤详细数据

Filtering the Detailed Data

详细视图提供过滤器以聚焦特定数据。某些视图有默认自动显示的过滤器。例如，下列详细视图具有默认的 scenario 与 path group 过滤器：Path Type Details、Path Group Details 与 Logic Level Details。

可通过下列方式修改默认过滤器：

- 单击过滤器名称前的 X 符号以移除过滤器；
- 更改过滤器值；
- 单击过滤器上除 X 符号或值字段以外的任意位置，以启用或禁用过滤器。

也可对详细数据应用自定义过滤器。创建自定义过滤器：

1. 单击 **Filter** 按钮，显示 Add Filter 字段。
2. 通过定义过滤条件并选择要纳入结果的数据集来定义过滤器。
选择列与比较类型，再指定比较值。
 - 数值比较：选择 =、!=、<、<=、> 或 >=，并在 Value 字段中指定数值。
 - 字符串比较：选择 contains，并指定字符串值。
 - 正则表达式比较：选择 regexp，并指定正则表达式。

- 基于所选列可用值的过滤：选择 **enum**，**Value** 字段变为包含可用值的下拉菜单，再启用一个或多个显示的值。启用时，值前显示勾选标记。

要启用或禁用某值：单击或用上下箭头导航到该值使其高亮，再按 **Enter** 切换状态。也可在 **Value** 字段中输入字符串以过滤下拉菜单中的可用值。单击 **X** 符号可取消已选值。

3. 单击绿色勾选标记应用过滤器。

例如，要过滤 **Path Group Details** 视图以仅显示 **func@cworst scenario** 中的 **path group**，按相应界面定义过滤器。

Chapter 4

时钟树综合

Clock Tree Synthesis

要在 IC Compiler II 中执行时钟树综合（CTS），请参阅以下主题：

- CTS 前提条件
- 定义时钟树
- 验证时钟树
- 设置时钟树设计规则约束
- 指定 CTS 设置
- 实现时钟树并执行 CTS 后优化
- 实现多源时钟树（Multisource Clock Trees）
- 分析时钟树结果

4.1 CTS 前提条件

Prerequisites for Clock Tree Synthesis

在对 block 运行 CTS 之前，应满足以下要求：

- 已用 `create_clock` 或 `create_generated_clock` 标识时钟源。
- block 已完成布局与优化。

使用 `check_legality -verbose` 确认布局合法。对布局不合法的 block 运行 CTS 可能导致运行时间过长、QoR 下降。

开始 CTS 前，估计的 QoR 应已满足要求，包括：

– 拥塞（Congestion）

若 CTS 前拥塞未解决，插入时钟树可能使拥塞恶化。若 block 拥塞，可用 `-congestion` 与 `-congestion_effort high` 重新运行 `create_placement`，但运行时间可能较长。

- 时序 (Timing)
 - 最大电容 (Maximum capacitance)
 - 最大转换时间 (Maximum transition time)
- 电源与地网已预布线 (prerouted)。
 - 高扇出网络 (如 scan enable) 已用缓冲器综合。
 - 已定义 active scenario。
默认情况下, 工具对所有为 setup 或 hold 分析启用的 active scenario 中的全部时钟进行综合与优化。

4.2 定义时钟树

Defining the Clock Trees

运行 CTS 前, 应分析 block 中每棵时钟树并确定:

- 时钟根 (clock root) 是什么;
- 所需的时钟 sink 与时钟树例外 (exceptions);
- 时钟树是否包含既有单元 (如 clock-gating 单元);
- 时钟树是否自收敛 (**convergent clock path**) 或与另一时钟树重叠 (**overlapping clock path**);
- 是否与其他时钟树存在时序关系 (如 interclock skew 要求)。

据此定义时钟树并确认工具得到的定义正确。工具根据 block 中已定义的时钟推导时钟树; 若不满足需求, 可按下列主题定义:

- 推导时钟树
- 定义时钟树例外
- 限制时钟网络上的优化
- 跨 mode 复制时钟树例外
- 由理想时钟延迟推导时钟树例外
- 处理存在平衡冲突的端点

4.2.1 推导时钟树

Deriving the Clock Trees

工具从时钟根出发, 沿传递扇出 (transitive fanout) 追踪至时钟端点以推导时钟树。一般在遇到时序单元或宏的时钟引脚时终止; 但对集成时钟门控 (ICG) 单元, 或其扇出驱动 generated clock 的时序单元, 工具会继续穿越。

若门控逻辑使用非单极性 (non-unate) 单元 (如 XOR/XNOR), 追踪时会同时使用正单极性与负单极性时序弧。若与 block 功能模式不符, 可用 set_case_analysis 将单元非时钟输入保持为常量, 迫使其进入 CTS 所需功能模式。

例如，图 4.1 所示门控逻辑中，可用下列命令使 XOR 门（U0）进入功能模式：

```
icc2_shell> set_case_analysis 0 U0/B
```

（原书 *Figure 36: Non-Unate Gated Clock*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 4.1: 非单极性门控时钟

识别时钟根

Identifying the Clock Roots

工具将 `create_clock` 定义的时钟源（输入端口或内部层次引脚）作为时钟根。

对由 `create_generated_clock` 定义的嵌套时钟树，工具将 `master-clock` 源视为时钟根，嵌套树端点视为该 `master-clock` 源的端点。例如如图 4.2 中，工具将 CLK 端口视为 `genclk1` 的时钟根。

（原书 *Figure 37: Nested Clock Tree With a Generated Clock*）

图 4.2: 带 generated clock 的嵌套时钟树

若 block 含 `generated clock`，须正确界定 `master-clock` 源，否则 skew 与时序 QoR 会变差：

- 若无法回溯到 `master-clock` 源，则无法将该 `generated clock` 的 sink 与其源的 sink 一并平衡；
- 若 `master-clock` 源并非由 `create_clock/create_generated_clock` 定义的时钟源，则无法为其或其源综合时钟树。

用 `check_clock_trees` 验证 `master-clock` 源定义，见「验证时钟树」。

指定时钟根时序特性

Specifying the Clock Root Timing Characteristics

为获得真实 CTS 结果，须正确建模时钟根时序特性：

- 若时钟根为无 I/O pad 的输入端口，须准确指定 `driving cell`。
若驱动过弱，工具可能插入额外缓冲以满足最大转换时间、最大电容等 DRC。
未指定 `driving cell`（或驱动强度）时，工具假定端口具有无限驱动能力。
例如，CLK1 为 CLK1 时钟树根，设驱动为 `mylib/CLKBUF`：

```
icc2_shell> set_driving_cell -lib_cell mylib/CLKBUF [get_ports CLK1]
```

- 若时钟根为已插入 I/O pad 的输入端口，须准确指定输入转换时间。例如上升 0.3、下降 0.2：

```
icc2_shell> set_input_transition -rise 0.3 [get_ports CLK1]
icc2_shell> set_input_transition -fall 0.2 [get_ports CLK1]
```

识别时钟端点

Identifying the Clock Endpoints

推导时钟树时，工具识别两类端点：

- **Sink pins**（亦称 **balancing pins**）：用于延迟平衡。工具为所有 sink 赋予零插入延迟并用于平衡。CTS 中 sink 参与 DRC 与时钟时序（skew、insertion delay）的计算与优化。
- **Ignore pins**：排除在时钟时序计算与优化之外；仅参与 DRC 计算与优化。CTS 时在 ignore pin 前插入 guide buffer 以隔离；该点之外不再做 skew/insertion delay 优化，但仍做 DRC 修复。

工具将下列端点识别为 sink pins：

- 时序单元（latch 或 flip-flop）的时钟引脚（除非该单元驱动 generated clock）；
- 宏单元的时钟引脚。

工具将下列端点识别为 ignore pins：

- 位于另一时钟扇出中的时钟树源引脚。例如图 4.3 中，被驱动时钟 clk2 的源引脚是驱动时钟 clk1 的 ignore pin；被驱动时钟的 sink 不视为驱动时钟的 sink。

（原书 Figure 38: Cascaded Clock With Two Source Clocks）

图 4.3: 双源级联时钟

- 时序单元的非时钟输入引脚；
- 三态使能引脚；
- 输出端口；
- 定义不正确的时钟引脚（例如无触发沿信息，或无到输出的时序弧）；
- 用 set_case_analysis 保持常量的缓冲器/反相器输入引脚；

注意

若时钟源被 set_case_analysis 保持为常量，工具不会综合该时钟树。

- 无扇出、或无已启用时序弧的组合单元/ICG 输入引脚。

可在 GUI 中检查 sink/ignore 识别是否正确（见「在 GUI 中分析时钟树」）。若默认正确，时钟树定义即完成；否则先排查禁用时序弧与 case analysis（report_disable_timing、report_case_analysis），

删除导致错误定义的设置。必要时用 `set_clock_balance_points` 覆盖默认 `balancing/ignore` 设置，见「定义时钟树例外」。

4.2.2 定义时钟树例外

Defining Clock Tree Exceptions

时钟树例外是用户对工具为特定时钟推导的默认端点（`balancing` 与 `ignore pins`）所做的更改。工具支持：

- 用户定义 **sink pins**：在工具推导之外额外定义 sink（例如将工具推导的 `ignore pin` 定义为 sink）。见「定义 Sink Pins」。
- 用户定义插入延迟要求：为 sink（推导或用户指定）指定特殊 `insertion delay`。见「定义插入延迟要求」。
- 用户定义 **ignore pins**：排除工具推导为 sink 的端点（例如排除组合逻辑之后的所有分支）。见「定义 Ignore Pins」。

定义 Sink Pins

Defining Sink Pins

语法：

```
set_clock_balance_points
  -clock clock
  -consider_for_balancing true
  -balance_points pins
  [-corners corners]
```

`-clock` 可选；省略则对所有时钟生效。用户指定 sink 的默认 `insertion delay` 为 0；覆盖方法见「定义插入延迟要求」。默认应用于所有 corner；用 `-corners` 可限定。

例如将 U2/A 指定为当前 corner 下 CLK 的 sink：

```
icc2_shell> set_clock_balance_points -clock [get_clocks CLK] \
  -consider_for_balancing true -balance_points [get_pins U2/A]
```

报告用 `report_clock_balance_points`（同时报告用户定义的 sink 与 `ignore`）。移除用 `remove_clock_balance_po`

```
icc2_shell> remove_clock_balance_points -clock [get_clocks CLK] \
  -balance_points [get_pins U2/A]
```

定义插入延迟要求

Defining Insertion Delay Requirements

用 `set_clock_balance_points` 的 `-delay` 覆盖 sink 默认零相位延迟。工具将指定延迟（正或负）加到到达该 sink 的计算插入延迟上。语法：

```
set_clock_balance_points
  [-clock clock]
  -delay delay
```

```
[-rise] [-fall] [-early] [-late]
-balance_points pins
[-corners corners]
```

默认用于最长/最短路径以及上升/下降路径；用 **-rise/-fall/-early/-late** 控制适用范围。默认所有 corner；用 **-corners** 限定。

例如对 U2/CLK 在上升最短路径使用 2.0 ns：

```
icc2_shell> set_clock_balance_points -clock [get_clocks CLK] \
  -rise -early -delay 2.0 -balance_points [get_pins U2/CLK]
```

用 **report_clock_balance_points** 报告用户定义插入延迟、带用户插入延迟的推导 sink，以及用户定义 ignore pins。

定义 Ignore Pins

Defining Ignore Pins

语法：

```
set_clock_balance_points
[-clock clock]
-consider_for_balancing false
-balance_points pins
```

-clock 可选。对图 4.4 中 CLK 树，若要忽略 U2 之后的所有分支：

```
icc2_shell> set_clock_balance_points -clock [get_clocks CLK] \
  -consider_for_balancing false -balance_points [get_pins U2/A]
```

(原书 Figure 39: User-Defined Ignore Pin)

图 4.4: 用户定义的 ignore pin

报告用 **report_clock_balance_points**。CTS 时工具添加 guide buffer 隔离 ignore pin；后续数据通路优化可修复 guide buffer 之外的 DRC。移除：

```
icc2_shell> remove_clock_balance_points -clock [get_clocks CLK] \
  -balance_points [get_pins U2/CLK]
```

确保时钟树例外有效

Ensuring Clock Tree Exceptions are Valid

工具通过引脚属性 **is_clock_used_as_clock** 是否为 true 判断引脚是否在时钟网络上。若对属性为 false 的引脚用 **set_clock_balance_points** 指定例外，工具会接受但在 CTS 中忽略。

考虑图 4.5 的时钟网络：默认情况下，U2/A、U2/Y、FF3/D 的 `is_clock_used_as_clock` 为 `false`，U2/A 之后视为数据网络；U2/A 为 `ignore pin`，其扇出排除在 CTS 之外。CTS 会在 U2/A 前加 `guide buffer`；后续数据通路优化可修 DRC。

(原书 Figure 40: *Specifying Exceptions on Pins Beyond the Clock Network*)

图 4.5: 在时钟网络之外的引脚上指定例外

若希望 CTS 用时钟树约束一直修到 FF3/D 的 DRC:

1. 将 FF3/D 显式设为 `ignore pin`:

```
icc2_shell> set_clock_balance_points -clock [get_clocks CLK] \
    -consider_for_balancing false -balance_points [get_pins FF3/D]
```

2. 用 `set_sense -clock_leaf` 强制将其视为时钟网络引脚:

```
icc2_shell> set_sense -clock_leaf [get_pins FF3/D]
```

该命令会：将扇入到该 `clock leaf` 的时钟分支上所有引脚的 `is_clock_used_as_clock` 设为 `true`；将该 `leaf` 扇出上所有引脚设为 `false`，且时钟不继续传播。

检查属性:

```
icc2_shell> get_attribute [get_pins FF3/D] is_clock_used_as_clock
```

4.2.3 限制时钟网络上的优化

Restricting Optimization on the Clock Network

可通过以下方式限制时钟网络部分区域的优化:

- **Don't touch**: 标识不做 CTS/优化的部分，可防止特定引脚之外被修改、特定 `net` 被缓冲、特定单元被改尺寸。见「设置 Don't Touch」。
- **Size-only**: 仅允许单元改尺寸；除非 `placement status` 为 `fixed`，否则仍可搬移。见「设置 Size-Only」。

设置 Don't Touch

Setting Don't Touch Settings

```
icc2_shell> set_dont_touch_network -clock_only [get_pins pin_name]
```

`-clock_only` 仅作用于时钟网络；省略则同时作用于数据与时钟路径。清除:

```
icc2_shell> set_dont_touch_network -clock_only [get_pins pin_name] -clear
```

也可对具体对象用 `set_dont_touch`:

```
icc2_shell> set_dont_touch [get_cells cell_name] true
icc2_shell> set_dont_touch [get_nets -segments net_name] true
```

报告: `report_dont_touch -all`。

注意

若单元 placement status 为 fixed, CTS 中按 don't touch 单元处理。

设置 Size-Only

Setting Size-Only Settings

```
icc2_shell> set_size_only [get_cells cell_name] true
icc2_shell> report_size_only -all
```

4.2.4 跨 Mode 复制时钟树例外

Copying Clock Tree Exceptions Across Modes

用 `set_clock_tree_options` 的 `-copy_exceptions_across_modes`、`-from_mode`、`-to_mode`, 指定将例外从某一 mode 复制到一个或多个等价 mode。例如将 M1 的例外复制到 M2、M3:

```
icc2_shell> set_clock_tree_options -copy_exceptions_across_modes \
    -from_mode M1 -to_mode {M2 M3}
```

4.2.5 由理想时钟延迟推导时钟树例外

Deriving Clock Tree Exceptions From Ideal Clock Latencies

若已对特定 sink 设置理想时钟延迟, 可用 `derive_clock_balance_points` 将其转换为时钟树例外: 把 `set_clock_latency` 在 sink 上的理想延迟转为 `set_clock_balance_points`。应在设计仍为理想时钟、CTS 之前运行。

默认行为:

- 为所有启用 setup/hold (或两者) 的 active scenario 中的所有理想 primary clock 生成 balance point; 不为 generated clock 生成。可用 `-clocks/-corners` 限定。
- 计算每个 primary ideal clock 的参考延迟 = `set_clock_latency` 在时钟上指定的理想 source 延迟 + network 延迟。可用 `-reference_latency` 指定不同参考值。
- 将推导的约束应用到设计; 用 `-output` 可改为输出含 `set_clock_balance_points` 的文件。

推导的 balance point 按时钟与 corner 区分。须确保 CTS 所用各 mode、至少 worst corner 上对所有时钟正确设置了 `set_clock_latency`。

公式:

$$\text{sink 处 clock balance point 延迟} = \text{时钟参考延迟} - \text{sink 处总理想时钟延迟}$$

其中总理想延迟 = 时钟 source latency + sink 的理想 network latency。

示例约束:

```
icc2_shell> set_clock_latency -source 0.5 [get_clocks clk]
icc2_shell> set_clock_latency 1.0 [get_clocks clk]
icc2_shell> set_clock_latency 1.5 [get_pins reg1/CK]
icc2_shell> set_clock_latency 0.5 [get_pins reg2/CK]
```

运行 `derive_clock_balance_points` 后得到:

```
icc2_shell> set_clock_balance_points -delay -0.5 \
    -balance_points [get_pins reg1/CK] -clock clk -corners worst
icc2_shell> set_clock_balance_points -delay 0.5 \
    -balance_points [get_pins reg2/CK] -clock clk -corners worst
```

为避免冲突，不要对已用该命令推导 balance point 的时钟再手动施加 balance point。

4.2.6 处理存在平衡冲突的端点

Handling Endpoints With Balancing Conflicts

某些 block 中存在结构上无法平衡的端点。工具可自动检测并推导 ignore pin 以解决冲突。对 `synthesize_clock_trees` 与 `clock_opt` 默认启用;设 `cts.common.enable_auto_exceptions` 为 false 可关闭。

可检测并修复的冲突类型:

- 含组合时钟路径模块的内部 **sink**: 若模块同时含内部 sink 与组合时钟路径，无法平衡内部 sink; 工具将其定义为 ignore pin。例如图 4.6(a) 网表对应 (b) 中带 check pin 的 ETM: ETM 内部 check pin 延迟来自 ETM 时序库，CTS 视其为 sink 并与顶层其他 sink 平衡; 若到 check pin 为最短路径，顶层 CTS 无法在最短路径上插缓冲，导致大 skew。将其设为 ignore 可解决。

(原书 Figure 41: Module With Combinational Clock Path)

图 4.6: 含组合时钟路径的模块

- 无法同时平衡的 **sink**: 两时钟驱动公共 sink，且其一因 generated clock 成为另一 sink 的 parent 时，无法同时将 parent、child 与公共 sink 平衡。工具将 parent 时钟引脚定义为 ignore pin。例如图 4.7 中，工具将 FF_gen/CLK 定义为 clkb 的 ignore pin。

(原书 Figure 42: Clock Pins That Cannot Be Balanced Simultaneously)

图 4.7: 无法同时平衡的时钟引脚

4.3 验证时钟树

Verifying the Clock Trees

综合前用 `check_clock_trees` 验证定义。例如：

```
icc2_shell> check_clock_trees -clocks [get_clocks CLK]
```

未指定 `-clock` 时检查当前 block 全部时钟。检查项包括：

- 无 sink 的时钟（master 或 generated）；
 - 时钟网络成环；
 - 重叠时钟使多时钟到达同一寄存器，但未启用 `multiple-clocks-per-register` 传播；
 - 被忽略的时钟树例外；
 - 在输出引脚上定义的 stop/float pin；
 - 时钟树 reference 中使用了具有多条时序弧的缓冲器；
 - 导致空缓冲列表的情形；
 - generated clock 无有效 master-clock 源，包括：
 - `create_generated_clock` 中指定的 master 不存在；
 - master 未驱动 generated clock 的源引脚；
 - 源引脚由多时钟驱动，但部分 master 未在 `create_generated_clock` 中指定。
- 例如图 4.8：GEN_REG/Q 由 CLKA、CLKB 驱动，若仅指定 CLKA 为 master，则 GEN_CLK 无有效 master。

（原书 Figure 43: *Generated Clock With Invalid Master Clock Source*）

图 4.8: master 源无效的 generated clock

对多角多模（MCMM）设计，还检查：

- 冲突的 per-clock 例外设置；
- 冲突的 balancing 设置。

实现前应手动修复报告问题；每条消息有 man 页说明修复方法。修复全部问题可改善时钟树与时序 QoR。

4.4 设置时钟树设计规则约束

Setting Clock Tree Design Rule Constraints

CTS 支持以下设计规则约束：

- 最大电容：用 `set_max_capacitance` 的 `-clock_path` 指定。未指定时 CTS 默认为 0.6 pF。
- 最大转换时间：用 `set_max_transition` 的 `-clock_path` 指定。未指定时 CTS 默认为 0.5 ns。

默认作用于当前 mode 关联的所有 corner。对特定 mode：用 `get_clocks -mode`；对特定 corner：用约束命令的 `-corners`。须跨 mode 及其 corner 正确施加。

例如对当前 mode 所有 corner 的 CLK 路径设最大转换 0.20 ns：

```
icc2_shell> set_max_transition 0.20 -clock_path [get_clocks CLK]
```

对不同 corner 设不同值：

```
icc2_shell> set_max_transition 0.15 -corners corner1 \
-clock_path [get_clocks -mode mode1]
icc2_shell> set_max_transition 0.10 -corners corner2 \
-clock_path [get_clocks -mode mode1]
```

对 mode 关联的不同 corner 设相同最大电容：

```
icc2_shell> set_max_capacitance 0.6 \
-clock_path [get_clocks -mode mode2] \
-corners [get_corner \
[get_attribute [get_modes mode2] associated_corners]]
```

4.5 指定 CTS 设置

Specifying the Clock Tree Synthesis Settings

CTS 选项指导时钟树实现，包括以下主题：

- 指定时钟树 References
- 设置 Skew 与 Latency 目标
- 启用局部 Skew 优化
- 指定 CTS 主 Corner
- 阻止综合特定时钟
- 保留既有时钟树
- 启用时钟树降功耗技术
- 启用线长感知的功耗驱动搬移
- 降低电迁移（EM）
- 处理不准确约束

- 定义时钟单元间距规则
- 创建 Skew Groups
- 定义时钟单元名前缀
- 初始 CTS 使用全局路由器
- 为时钟网络指定约束
- 为特定端点构建更快时钟树
- 改进延迟驱动的时钟门分裂
- 降低时钟网络上的信号完整性影响
- 指定时钟延迟调整设置
- 报告时钟树设置

4.5.1 指定时钟树 References

Specifying the Clock Tree References

可用于构建时钟树的缓冲器/反相器，以及既有门的 reference cell，称为时钟树 references。用 `set_lib_cell_purpose -include cts` 指定。列表须满足：

- 至少含一个缓冲器或一个反相器；
- 包含既有门的库单元（否则无法在 CTS/优化中对其改尺寸）。可用 `derive_clock_cell_references` 自动推导非缓冲/反相既有单元的等价单元，见「为既有门推导 References」；
- 多电压且需 always-on 缓冲时，列表须含 always-on 单元；
- 列表中的库单元不得有 `dont_touch`（即便列入 reference 也不会被 CTS 使用）。

为确保仅使用指定单元、且这些单元不被 CTS 以外的优化使用：

```
icc2_shell> set cts_cells list_of_cells
icc2_shell> set_lib_cell_purpose -exclude cts [get_lib_cells]
icc2_shell> set_lib_cell_purpose -include none [get_lib_cells $cts_cells]
icc2_shell> set_lib_cell_purpose -include cts [get_lib_cells $cts_cells]
```

为既有门推导 References

Deriving Clock Tree References for Preexisting Gates

须将等价单元指定为 clock references，否则影响 QoR。`derive_clock_cell_references` 会为非缓冲/反相的既有时钟单元在等价库单元上设置 `valid_purpose`；不推导缓冲器与反相器的等价单元，须手动指定：

```
icc2_shell> set_lib_cell_purpose -include cts \
    {tech_lib/clk_buf* tech_lib/clk_inv*}
icc2_shell> derive_clock_cell_references
icc2_shell> synthesize_clock_trees
```

用 `-output` 可生成 Tcl 脚本而非直接应用：

```
icc2_shell> derive_clock_cell_references -output cts_leq_cells.tcl
icc2_shell> set_lib_cell_purpose -include cts \
    {tech_lib/clk_buf* tech_lib/clk_inv*}
icc2_shell> source cts_leq_cells.tcl
icc2_shell> synthesize_clock_trees
```

限制所用目标库

Restricting the Target Libraries Used

用 `set_target_library_subset -clock` 可限制顶层或逻辑层次下级所用库；须将 `opt.common.enable_target_` 设为 1。例如全局允许 HVT/LVT 的 buf，但 TSK_BLK 仅用 LVT_lib：

```
icc2_shell> set_lib_cell_purpose -include cts \
    {HVT_lib/buf1 HVT_lib/buf2 LVT_lib/buf1 LVT_lib/buf2}
icc2_shell> set_target_library_subset -clock {LVT_lib} \
    -objects [TOP/TSK_BLK]
icc2_shell> set_app_options \
    -name opt.common.enable_target_library_subset_opt -value 1
```

4.5.2 设置 Skew 与 Latency 目标

Setting Skew and Latency Targets

默认 CTS 追求所有时钟的最佳 skew 与 latency，对低频时钟可能带来面积、功耗与运行时间开销。

- skew 目标: `set_clock_tree_options -target_skew`
- latency 目标: `set_clock_tree_options -target_latency`

默认作用于所有时钟与所有 corner；用 `-clocks/-corners` 限定。报告: `report_clock_tree_options`。
移除: `remove_clock_tree_options (-all 或相应选项)`。

4.5.3 启用局部 Skew 优化

Enabling Local Skew Optimization

默认最小化全局 skew（最长与最短时钟路径之差）。若最长/最短路径寄存器之间无时序路径，优化全局 skew 未必改善 QoR；优化局部 skew（时序路径 launch/capture 寄存器间最坏 skew）可改善 QoR。局部 skew 优化针对所有违例的 setup/hold 路径。

对 `synthesize_clock_trees` 与 `clock_opt` 的综合与优化阶段启用：

```
icc2_shell> set_app_options -list {cts.compile.enable_local_skew true}
icc2_shell> set_app_options -list {cts.optimize.enable_local_skew true}
```

启用后默认由工具推导有助于 QoR 的 skew 目标，并忽略 `set_clock_tree_options -target_skew`。若要阻止自动推导：

```
icc2_shell> set_app_options \  
-name cts.common.enable_auto_skew_target_for_local_skew -value false
```

4.5.4 指定 CTS 主 Corner

Specifying the Primary Corner for Clock Tree Synthesis

初始 CTS 默认识别时钟延迟最差的 corner，并在该 corner 的所有 mode 中插入缓冲以平衡延迟。用 `cts.compile.primary_corner` 指定主 corner。

4.5.5 阻止综合特定时钟

Preventing Specific Clocks From Being Synthesized

默认综合所有 `create_clock/create_generated_clock` 定义的时钟。用 `cts.common.skip_cts_clocks` 排除：

```
icc2_shell> set_app_options \  
-name cts.common.skip_cts_clocks -value CK_B
```

4.5.6 保留既有时钟树

Preserving Preexisting Clock Trees

默认 CTS 会移除所有既有时钟缓冲器与反相器。将 `cts.compile.remove_existing_clock_trees` 设为 `false` 可阻止。要保留特定既有缓冲/反相器，对其施加 `don't touch` 或 `size-only` 例外。

4.5.7 启用时钟树降功耗技术

Enabling Clock Tree Power Reduction Techniques

用 `cts.compile.power_opt_mode` 启用，见表 4.1。

表 4.1: `cts.compile.power_opt_mode` 设置

目的	设置
将 clock-gating 单元移近驱动以缩短高翻转输入网线长	gate_relocation
用聚类技术改善功耗	low_power_targets
同时使用门控搬移与聚类	all

4.5.8 启用线长感知的功耗驱动搬移

Enabling Wire Length Aware Power-Driven Relocation

工具支持改进的活动驱动时钟门搬移:控制线长劣化,并在搬移前考虑功耗影响。将 `cts.compile.power_opt_mode` 设为 `all` 或 `gate_relocation`。下列情况不搬移：

- 搬移 ICG 功耗收益很小或恶化功耗，同时恶化线长；
- 搬移线长影响大而功耗收益小。

4.5.9 降低电迁移

Reducing Electromigration

时钟单元功耗较高；局部聚集会提高电源轨电流密度，增加 EM 风险。可通过设置时钟单元间距，避免沿标准单元电源轨在垂直 strap 之间局部堆积：

1. 用 `set_clock_cell_spacing` 定义反相器、缓冲器、ICG 的间距规则（见「定义时钟单元间距规则」）；
2. 用 `synthesize_clock_trees` 或 `clock_opt` 执行 CTS；

注意

`balance_clock_groups` 也遵守时钟单元间距规则。见「平衡不同时钟树之间的 Skew」。

3. 用 `check_legality -verbose` 检查间距违例；设置合适约束后应无违例。

4.5.10 处理不准确约束

Handling Inaccurate Constraints During Clock Tree Synthesis

实现早期约束可能不准确或过严，直接 CTS 可能导致长运行时间或差 QoR。可设：

```
icc2_shell> set_app_options \
    -name cts.common.enable_dirty_design_mode -value true
```

此时工具会：

- 忽略时钟网上的 `dont_touch/dont_touch_network`；
- 移除过紧的最大转换/最大电容约束；
- 移除小于 50 microns 的最大网长约束；
- 若水平间距 $> 3 \times \text{site height}$ 或垂直间距 $> 20 \times \text{site height}$ ，忽略时钟单元间距规则；
- 若 NDR 的 `width+spacing` $> 10 \times$ （默认 `width+spacing`），忽略 NDR；
- 报告大到足以增加时钟插入延迟的 `balance point` 延迟值。

4.5.11 定义时钟单元间距规则

Defining Clock Cell Spacing Rules

用 `set_clock_cell_spacing`；至少指定 x 或 y 方向最小间距（通常两者都指定）。

- `-x_spacing`：微米，通常依赖驱动强度与时钟频率；
- `-y_spacing`：通常小于半行高。

注意

间距过大因单元远离目标 sink 簇而增加 skew。

默认作用于所有时钟单元；用 `-lib_cells/-clocks` 限定。相邻两单元若同方向都有规则，间距为两者之和。可将 `cts.placement.cell_spacing_rule_style` 设为 `maximum`，改为取较大值而非求和。

`synthesize_clock_trees`、`balance_clock_groups`、`clock_opt` 均遵守这些规则。报告：`report_clock_cell_spacing`。移除：`remove_clock_cell_spacings`（默认全部；`-lib_cells` 可限定）。

4.5.12 创建 Skew Groups

Creating Skew Groups

若希望一组 sink 仅在组内平衡，用 `create_clock_skew_group`，以 `-objects` 指定 sink。可选：`-clock`、`-mode`（默认当前 mode）、`-name`。例如：

```
icc2_shell> create_clock_skew_group -name sg1 \
    -objects {reg1/CP reg2/CP reg3/CP}
```

报告：`report_clock_skew_groups`。移除：`remove_clock_skew_groups`。

4.5.13 定义时钟单元名前缀

Defining a Name Prefix for Clock Cells

用 `cts.common.user_instance_name_prefix`：

```
icc2_shell> set_app_options \
    -name cts.common.user_instance_name_prefix -value "CTS_"
```

数据网上优化（含 `clock_opt`）的单元前缀用 `opt.common.user_instance_name_prefix`。

4.5.14 初始 CTS 使用全局路由器

Using the Global Router During Initial Clock Tree Synthesis

默认初始 CTS 用虚拟路由器；复杂 floorplan 可能产生拥塞热点并在时钟布线后劣化 QoR。将 `cts.compile.global_route_aware_buffering` 设为 `true`，可在 `synthesize_clock_trees/clock_opt` 的初始 CTS 阶段使用全局路由器。

4.5.15 为时钟网络指定约束

Specifying Constraints for Clock Nets

默认用默认布线规则与可用布线层。可按「指定布线资源」设置时钟网最小/最大布线层，用于 RC 估计、拥塞分析与布线。为降低线延迟，可用宽线与高层金属（NDR）；须先定义规则并赋给时钟网（见「使用非默认布线规则」）。

4.5.16 为特定端点构建更快时钟树

Constructing Faster Clock Tree to Specific Endpoints

可为指定端点构建无额外绕线/延迟平衡的更快时钟树以改善 latency。须先为所需端点创建用户 skew group，再运行 `synthesize_clock_trees` 或 `clock_opt` 的 `build_clock` 阶段：

```
icc2_shell> create_clock_skew_group -objects <endpoints>
```

注意

- 若快速端点之间有平衡要求，须放入同一 skew group；
- 若无平衡要求，须为每个端点单独建 skew group；
- 所需 skew group 端点的上游时钟逻辑与默认组或其他用户 skew group 共享，可能导致为平衡其他组端点而绕线。

(原书 *Figure 44: Faster Clock Tree to Specific Endpoints*)

图 4.9: 到特定端点的更快时钟树

若一门同时驱动 skew group 与非 skew group 端点，skew group 端点可能因非组端点的平衡要求而变慢。除建 skew group 外，还须用 `split_clock_cells -to` 沿路径克隆门，以从时钟源到目标端点建立独立路径（在 CTS 之前）：

```
icc2_shell> split_clock_cells -to $endpoints
```

也可用 `split_clock_cells -to <$sinks> -latency_driven` 自动识别最优克隆方案；端点须同时用 `create_clock_skew_group` 建组。

4.5.17 改进延迟驱动的时钟门分裂

Improving Latency-Driven Clock-Gate Splitting

延迟驱动的 CG 分裂现可自动识别并评估不同克隆搬移，以改善插入延迟并满足最大级数约束。启用：将 `cts.icg.latency_driven_cloning` 设为 `true`。

(原书 *Figure 45: Latency Driven Clock-Gate Splitting*)

图 4.10: 延迟驱动的时钟门分裂

4.5.18 降低时钟网络上的信号完整性影响

Reducing Signal Integrity Effects on Clock Nets

可通过推导更大间距的 NDR 并赋给无拥塞区域的时钟网改善时序。启用：

```
icc2_shell> set_app_options \
    -name cts.optimize.enable_congestion_aware_ndr_promotion -value true
```

新 NDR 间距取决于原规则：

- 原 NDR 间距 $\leq 2 \times$ 默认间距：加倍，名称加后缀 `_ext_spacing`；
- 原 NDR 间距 $> 2 \times$ 默认：增加一个默认间距，后缀同上；
- 默认规则：间距加倍，新名为 `default_rule_equivalent_ndr_double_spacing`。

4.5.19 指定时钟延迟调整设置

Specifying Settings for Clock Latency Adjustments

I/O 端口通常由真实或虚拟时钟约束。CTS 后须更新约束 I/O 的时钟延迟以保持边界约束准确。用 `clock_opt` 时工具自动更新；可用 `set_latency_adjustment_options` 控制。例如按 `PC_CLK` 更新 `VCLK1/VCLK2`，排除 `VCLK3`：

```
icc2_shell> set_latency_adjustment_options \
    -reference_clock PC_CLK -clocks_to_update {VCLK1 VCLK2} \
    -exclude_clocks VCLK3
```

用 `synthesize_clock_trees` 时不会自动更新；CTS 后运行 `compute_clock_latency`（遵守上述设置）。

4.5.20 报告时钟树设置

Reporting the Clock Tree Settings

用 `report_clock_settings`。默认报告当前 block 所有时钟的配置（DRC、目标 skew/latency）、references、单元间距规则、NDR。用 `-clock` 限定时钟；用 `-type`：

- `configurations`：DRC、目标 skew/latency；
- `references`：时钟树 references；
- `spacing_rules`：单元间距；
- `routing_rules`：时钟网 NDR。

4.6 实现时钟树并执行 CTS 后优化

Implementing Clock Trees and Performing Post-CTS Optimization

执行 CTS 前应保存 block，以便必要时以同一起点 refinement 目标后重跑。可用方法：

- 独立 CTS（`synthesize_clock_trees`）

- 用 `clock_opt` 综合、优化并布线时钟树
- 控制并发时钟与数据（CCD）优化
- 分裂时钟单元
- 平衡不同时钟树之间的 Skew
- 使用机器学习数据的基于全局布线优化
- 布线时钟树
- 在 CTS、优化与时钟布线中插入 Via Ladder
- CTS 后将时钟标记为 propagated
- 执行布线后时钟树优化
- 执行电压优化
- 将时钟树标记为已综合
- 移除时钟树

4.6.1 独立时钟树综合

Performing Standalone Clock Trees Synthesis

```
icc2_shell> synthesize_clock_trees
```

该命令执行：

1. **CTS**：此前对时钟网做虚拟布线并用 RC 估计确定时序；
2. 时钟树优化：此前对时钟网做全局布线并用 RC 提取；优化中对被修改的时钟网做增量全局布线。

注意

该命令不对时钟树做详细布线。

默认作用于所有为 setup/hold 启用的 active scenario 中的全部时钟。用 `-clock` 限定，例如：

```
icc2_shell> synthesize_clock_trees -clocks [get_clocks CLK]
```

CTS 后，工具将所有 active scenario 各 mode 中的时钟设为 propagated。

4.6.2 用 `clock_opt` 综合、优化并布线时钟树

Synthesizing, Optimizing, and Routing Clock Trees With the `clock_opt` Command

`clock_opt` 用一条命令完成综合、布线与进一步优化，阶段为：

1. `build_clock`：综合并优化所有 active scenario 各 mode 的全部时钟；之后设为 propagated；
2. `route_clock`：对已综合时钟网做详细布线；

3. **final_opto**: 进一步优化、时序驱动布局与合法化, 然后全局布线并做广泛的基于全局布线的优化 (含增量合法化与 route patching)。

用 **-from/-to** 限制连续阶段; 省略 **-from** 从 **build_clock** 开始, 省略 **-to** 跑到 **final_opto** 完成。例如仅综合优化并详细布线:

```
icc2_shell> clock_opt -to route_clock
```

因 **final_opto** 做全局布线, 运行前应设好所有路由器相关设置 (布线规则、route guide、工艺节点应用选项等)。每个 **block** 应只跑一次 **final_opto**; 完成后信号网均已全局布线, 后续 **route_auto/route_global/route** 会跳过全局布线。为避免后续 track assignment/详细布线出错, **clock_opt** 后不要改动任何全局布线形状。

CTS 中考虑压降信息

Considering Voltage Drop Information During Clock Tree Synthesis

默认 **clock_opt** 插入时钟缓冲时不考虑压降, 可能把缓冲放在高 PG 电阻处。可用 RedHawk Fusion 分析 PG 网络:

1. **analyze_rail -min_path_resistance**;
2. **open_rail_result** 加载结果;
3. 将 **clock_opt.flow.enable_voltage_drop_aware** 设为 **true**;
4. 运行 **clock_opt**。

优化中对关键网络使用 **NDR**

Using Nondefault Routing Rules for Critical Nets During Optimization

布线前优化中可对时序关键网使用 **NDR**, 并以软约束引导路由器。对 **clock_opt** 启用:

```
icc2_shell> set_app_options -name clock_opt.flow.optimize_ndr \
-value true
```

在 **clock_opt** 中执行 **CCD** 优化

Performing Concurrent Clock and Data Optimization During the clock_opt Command

在数据通路优化中利用正松弛调整寄存器时钟到达时间以改善时序 QoR, 称为并发时钟与数据 (CCD) 优化。**clock_opt** 默认启用; 将 **clock_opt.flow.enable_ccd** 设为 **false** 可关闭。可选控制步骤: 限制延迟调整值、排除边界路径、排除特定 path group/scenario/sink、控制时序/hold 优化努力、控制 I/O 时钟延迟调整、用 **report_ccd_timing** 分析效果 (详见「控制 CCD 优化」各小节)。

控制 **clock_opt** 中的多比特优化

Controlling Multibit Optimization Performed During the clock_opt Command

- 混合驱动强度多比特单元重连: 违例路径经过较弱驱动位时, 可重连使路径走较强驱动位。启用: **clock_opt.flow.enable_multibit_rewiring=true**。

- **CTS 后多比特 debanking:** 在 `final_opto` 阶段，若不会引入 hold 违例则可 `debank` 以改善 QoR。
启用: `clock_opt.flow.enable_multibit_debanking=true`。

在时钟网络上执行功耗或面积回收

Performing Power or Area Recovery on the Clock Network

启用 CCD 时，工具在 `final_opto` 对时钟单元与寄存器做时钟功耗回收。未启用 CCD 时可：

```
icc2_shell> set_app_options \
-name clock_opt.flow.enable_clock_power_recovery \
-value power
```

此前须：用 `set_scenario_status` 的 `-dynamic_power/-leakage_power` 启用功耗优化 scenario；（可选）用 `read_saif` 提供翻转活动，否则用默认活动做动态功耗回收。面积回收则将值设为 `area`。

在 `clock_opt` 中执行 **IR-Drop** 感知布局

Performing IR-Drop-Aware Placement During the clock_opt Command

1. `clock_opt -to route_clock;`
2. 配置 RedHawk Fusion 并用 `analyze_rail -voltage_drop` 做静态/动态压降分析；
3. 将 `place.coarse.ir_drop_aware` 设为 `true`；
4. （可选）按「控制 IR-Drop 感知布局」做附加设置；
5. `clock_opt -from final_opto`。

4.6.3 控制并发时钟与数据优化

Controlling Concurrent Clock and Data Optimization

CCD 对 `place_opt` 与 `clock_opt` 默认启用；对 `route_opt` 须将 `route_opt.flow.enable_ccd` 设为 `true`。可控制：

- 限制延迟调整值
- 排除边界路径 / 特定 `path group` / `scenario` / `sink`
- 控制时序与 hold 优化努力
- 控制 I/O 时钟延迟调整
- 在 `route_opt` 中做动态压降驱动的 CCD
- 在布线前/布线后阶段指定优化目标
- 布线后启用缓冲移除
- 报告 CCD 时序；将 CCD offset 缩放到新 scenario
- 对 latch 与离散时钟门一并 skew

限制延迟调整值

Limiting the Latency Adjustment Values

- 提前量上限: `ccd.max_prepone`
- 推迟量上限: `ccd.max_postpone`

无默认值，单位为库时序单位。例如：

```
icc2_shell> set_app_options -list {ccd.max_prepone 0.2}
icc2_shell> set_app_options -list {ccd.max_postpone 0.1}
```

排除边界路径

Excluding Boundary Paths

默认对 block 内所有路径做 CCD。边界寄存器为输入端口传递扇出、输出端口传递扇入中的寄存器。将 `ccd.optimize_boundary_timing` 设为 `false` 可排除边界路径。可用 `ccd.ignore_ports_for_boundary_identification` 使部分端口的边界路径仍参与 CCD：

```
icc2_shell> set_app_options -name ccd.optimize_boundary_timing \
    -value false
icc2_shell> set_app_options \
    -name ccd.ignore_ports_for_boundary_identification \
    -value {IN_A OUT_A}
```

即便禁用边界寄存器 CCD，若改善共享时钟路径的内部寄存器 QoR，工具仍可优化其时钟树扇入锥。将 `ccd.optimize_boundary_timing_upstream` 设为 `false` 可阻止，但会严重限制 CCD 范围。上述设置影响 `place_opt/clock_opt/route_opt`。

排除特定 Path Groups / Scenarios / Sinks

Excluding Specific Path Groups, Scenarios, and Sinks

```
icc2_shell> set_app_options -name ccd.skip_path_groups \
    -value {CLK1 {CLK2 scnA}}
icc2_shell> set_app_options -name ccd.ignore_scenarios \
    -value {scnB}
```

排除特定 sink：

1. 对 sink 设 `cts_fixed_balance_pin`：

```
icc2_shell> set_attribute -objects reg21/CK \
    -name cts_fixed_balance_pin -value true
```

2. 将 `ccd.respect_cts_fixed_balance_pins` 设为 `true`（阻止调整这些 sink 的延迟；其他 sink 的 CCD 仍可能间接改变其延迟）。设为 `upstream` 可阻止从这些 sink 到时钟根的时钟路径被改动。

控制时序与 Hold 优化努力

Controlling Timing and Hold Optimization Effort

- `ccd.timing_effort`: low/high, 默认 medium (影响 `clock_opt` 的 `final_opto` 与 `route_opt`);
- `ccd.hold_control_effort`: medium/high/ultra, 默认 low。提高 hold 优先级会减少可修复的 setup 违例, 仅在 hold 关键时使用。

控制 I/O 时钟延迟调整

Controlling the Adjustment of I/O Clock Latencies

`clock_opt` 做 CCD 时默认调整约束边界端口的 I/O 时钟延迟; 随后运行 `compute_clock_latency` 时也使用 CCD 技术。不更新的情形包括: `ccd.adjust_io_clock_latency=false`、禁用边界路径 CCD、用 `ccd.skip_path_groups` 排除对应 path group、用 `set_latency_adjustment_options -exclude_clocks` 排除特定 I/O 时钟。

route_opt 中动态压降驱动的 CCD

Performing Dynamic-Voltage-Drop-Driven CCD During route_opt

将 `route_opt.flow.enable_voltage_drop_opt_ccd` 设为 true。工具用 RedHawk Fusion 识别动态压降热点与可搬移的 cell row, 在优化中搬移或缩小尺寸以降低峰值动态压降且不伤害时序 QoR。可选阈值:

- `ccd.voltage_drop_voltage_threshold`: 相对供电电压的压降百分比;
- `ccd.voltage_drop_population_threshold`: 按最差违例选取的单元数百分比。

另可将 `route_opt.flow.enable_irdrivenopt` 与上述选项同时设为 true, 启用 IR 驱动 sizing (用更小漏电单元替换)。

布线前阶段指定优化目标

Specifying Optimization Targets at the Preroute Stage

对 `place_opt/clock_opt` 的 CCD, 可提高下列目标的 WNS 优先级:

- 特定 path group: `ccd.targeted_ccd_path_groups` (若同时在 `ccd.skip_path_groups` 中则仍跳过);
- 端点文件: `ccd.targeted_ccd_end_points_file` (同时指定 path group 与端点时, 仅优化指定组内的指定端点);
- 最差 300 条路径: `ccd.enable_top_wns_optimization=true`。

布线后阶段指定优化目标

Specifying Optimization Targets at the Postroute Stage

1. 将 `route_opt.flow.enable_targeted_ccd_wns_optimization` 设为 true;

2. 用 `ccd.targeted_ccd_path_groups` 和/或 `ccd.targeted_ccd_end_points_file` 指定目标（至少其一）；
3. （可选）`ccd.targeted_ccd_select_optimization_moves`: `auto`（默认，含 `buffering`）、`size_only`、`equal_or_smaller_sizing`、`footprint_sizing`；
4. （可选）`ccd.targeted_ccd_wns_optimization_effort`: `high`（默认）/ `medium` / `low`；
5. （可选）`ccd.targeted_ccd_threshold_for_nontargeted_path_groups`: 非目标 path group 可被劣化的松弛阈值（默认正松弛可劣化至零）。

可同时启用常规 CCD（`route_opt.flow.enable_ccd`）与 targeted CCD；此时先常规后 targeted。示例脚本（设计已布线并做过布线后优化）：

```
open_lib design1
open_block blk.post_route
set_app_options -name route_opt.flow.enable_ccd -value false
set_app_options \
  -name route_opt.flow.enable_targeted_ccd_wns_optimization -value true
set_app_options -name ccd.targeted_ccd_path_groups -value clkA
set_app_options -name ccd.targeted_ccd_end_points_file -value clkA_ep.txt
set_app_options \
  -name ccd.targeted_ccd_select_optimization_moves -value size_only
set_app_options -name ccd.targeted_ccd_wns_optimization_effort \
  -value medium
route_opt
```

布线后启用缓冲移除

Enabling Buffer Removal at the Postroute Stage

```
icc2_shell> set_app_options -name route_opt.flow.enable_ccd -value true
icc2_shell> set_app_options -name ccd.post_route_buffer_removal -value true
icc2_shell> route_opt
```

报告 CCD 时序

Reporting Concurrent Clock and Data Timing

用 `report_ccd_timing`。默认报告 block 中五个最关键端点寄存器的最差 capture（D-slack）与 launch（Q-slack）的 setup/hold。-type: fanin/fanout、stage、chain（见图 4.11）。默认报告 setup; -hold 报告 hold。-scenarios 限定 scenario; -pins 限定端点。可用 -prepone/-postpone 与 -annotate_cell_delay/-annotate 做 what-if 分析（仅报告，不保存）。

将 CCD Offset 缩放到新 Scenario

Scaling CCD Offsets to New Scenarios for Reporting Timing

用 `copy_useful_skew` 将 CCD offset 复制到新 scenario，以获得一致的时序/QoR 报告。当 pin 的 CTS balance point offset 仅在部分 active scenario 中推导时，用该命令复制到无 offset 的 inactive/active

(原书 *Figure 46: Previous, Current, and Next Path Stages*)

图 4.11: 前一、当前与下一路径阶段

scenario,并确保在计算新 offset 的所有 pin 与 scenario 上设置相应 `set_clock_latency`。先对有 balance point offset 的 scenario 取平均,再按各 scenario corner 的缩放因子缩放。

对 **Latch** 与离散时钟门一并 **Skew**

Skewing Latch and Discrete Clock Gates

在 `compile_fusion` 或 `place_opt` 的 `final_opto` 前将 `ccd.skew_opt_merge_dcg_cells_by_driver` 设为 `true` (默认 `false`),使 latch 与门控元件使用相同 offset,保持同级并改善离散时钟门的 hold。

4.6.4 分裂时钟单元

Splitting Clock Cells

```
icc2_shell> split_clock_cells -cells [get_cells U1/ICG*]
```

不分裂的情形:无 DRC 违例;有 don't-touch/size-only/fixed-placement;须在电源域或 `set_freeze_ports` 标识的 block 边界 punch port。分裂后新单元命名为 < 原名 >_split_< 整数 >,并复制原单元全部设置与约束。也可用 `-loads` 指定由同一驱动驱动的负载集合:

```
icc2_shell> set loads1 [get_pins I1/reg*/CK]
icc2_shell> set loads2 [get_pins I2/reg*/CK]
icc2_shell> split_clock_cells -loads [list $loads1 $loads2]
```

4.6.5 平衡不同时钟树之间的 Skew

Balancing Skew Between Different Clock Trees

参与延迟平衡的时钟集合称为 clock balance group;可定义多个组,并为组内时钟定义延迟 offset;合称 interclock delay balancing 约束。

注意

工具无法在 generated clock 与其他时钟之间平衡 skew。

步骤:生成约束(手动或自动)→运行平衡。

定义 **Interlock** 延迟平衡约束

Defining the Interlock Delay Balancing Constraints

用 `create_clock_balance_group`: 至少 `-name` 与 `-objects` (可用 `get_clocks -mode`)。默认目标为零 `offset`, 以组内最长插入延迟平衡。用 `-offset_latencies` 指定相对 `offset`; `-corner` 限定 `corner`。例如 `clk3` 比 `clk1/clk2` 少 100:

```
icc2_shell> create_clock_balance_group -name group3 \
  -objects [get_clocks {clk1 clk2 clk3}] \
  -offset_latencies {0 0 -100}
```

报告: `report_clock_balance_groups`。移除: `remove_clock_balance_groups` (指定组或 `-all`)。

自动生成约束

Generating Interlock Delay Balancing Constraints Automatically

```
icc2_shell> derive_clock_balance_constraints
```

将有 `interlock` 时序路径的时钟放入同一 `balance group`。可用 `-slack_less_than` 仅考虑松弛小于阈值的路径, 例如:

```
icc2_shell> derive_clock_balance_constraints -slack_less_than -0.2
```

运行 **Interlock** 延迟平衡

Running Interlock Delay Balancing

```
icc2_shell> balance_clock_groups
```

对 MCMM, 对所有 `active scenario` 执行。

4.6.6 使用机器学习数据的基于全局布线优化

Performing Global-Route-Based Optimization Using Machine Learning Data

对布线前/布线后时序相关性差的设计, 可在 CTS 后用详细布线后得到的 ML 数据做优化。术语: `Labels` (预测输出)、`Features` (影响输入)、`Model` (映射关系)、`Training` (学习过程)。

迭代 **N** (采集与训练):

1. 完成 `clock_opt` 的 `build_clock` 与 `route_clock`;
2. 设 `est_delay.ml_delay_opto_dir` 为输出目录;
3. 将 `est_delay.ml_delay_gre_mode` 设为 `feature`;
4. `clock_opt -from final_opto`;
5. 详细布线;
6. 详细布线后、布线后优化前: `estimate_delay -training_labels`;
7. `estimate_delay -train_model`;

8. 继续布线后优化与后续流程。

迭代 **N+1** (应用模型): 同样完成时钟综合与布线 → 指定 ML 目录 → 将 `est_delay.ml_delay_gre_mode` 设为 `enable` → `clock_opt -from final_opto` → 详细布线 → `estimate_delay -disable_model` → 继续流程。

须保证各步骤/迭代中 `active scenario` 相同; 设计、环境或流程变更时须重建模型。

4.6.7 布线时钟树

Routing Clock Trees

```
icc2_shell> route_group -all_clock_nets -reuse_existing_global_route true
```

`-reuse_existing_global_route true` 使详细路由器使用既有时钟全局布线, 相关性更好。或:

```
icc2_shell> clock_opt -from route_clock -to route_clock
```

4.6.8 在 CTS、优化与时钟布线中插入 Via Ladder

Inserting Via Ladders During CTS, Optimization, and Clock Routing

Via ladder 是从引脚层堆叠到上层、供路由器连接的通孔叠层, 可降低通孔电阻, 改善性能与 EM 鲁棒性。流程:

1. 定义 via ladder 规则;
2. 用 `set_via_ladder_candidate` 指定库引脚可用的 via ladder;
3. 用 `set_via_ladder_rules` 定义约束;
4. 将 `opt.common.enable_via_ladder_insertion` 设为 `true`;
5. `clock_opt -to build_clock`;
6. `insert_via_ladders`;
7. `clock_opt -from route_clock`。

4.6.9 CTS 后将时钟标记为 Propagated

Marking Clocks as Propagated After Clock Tree Synthesis

运行 `synthesize_clock_trees` 或 `clock_opt` 后, 工具移除所有 `active scenario` 中已综合时钟的 `ideal` 设置, 并将对应寄存器时钟引脚设为 `propagated`。若有 CTS 前未设为 `active` 的 `scenario`:

```
icc2_shell> set_scenario_status -active true [all_scenarios]
icc2_shell> synthesize_clock_trees -propagate_only
```

`-propagate_only` 仅移除 `ideal` 设置, 不执行 CTS。

4.6.10 执行布线后时钟树优化

Performing Postroute Clock Tree Optimization

详细布线时钟/信号网后，因全局布线与详细布线差异以及耦合/串扰，时钟 QoR 可能下降。用 `synthesize_clock_trees -postroute`，并以 `-routed_clock_stage` 指明布线阶段：

```
icc2_shell> synthesize_clock_trees -postroute \
-routed_clock_stage detail
icc2_shell> synthesize_clock_trees -postroute \
-routed_clock_stage detail_with_signal_routes
```

注意

启用 CCD（含用该技术的功耗/面积回收）时，布线后时钟树优化不优化 latency/skew，仅修复时钟网络上的逻辑 DRC。

4.6.11 执行电压优化

Performing Voltage Optimization

电压优化尝试在更低电压下获得更好的 PPA。需要：不同电压的 scaled library panes；以及带 scaled library 设置的 signoff 方法。高层流程：

1. 原电压 place_opt；
2. 原电压 clock_opt；
3. define_scaling_lib_group 关联相关 library panes；
4. set_vopt_range 指定 corner 电压范围；
5. set_vopt_target 指定 TNS/功耗目标与容差（可用 `-excludeIO`）；
6. voltage_opt；
7. 优化电压下 route_auto；
8. 优化电压下 route_opt。

4.6.12 将时钟树标记为已综合

Marking Clock Trees as Synthesized

用 `mark_clock_trees` 防止工具修改既有时钟树，见表 4.2。工具遍历时钟树直至遇到例外引脚或默认 sink。

4.6.13 移除时钟树

Removing Clock Trees

`remove_clock_trees` 从根到 sink 遍历并移除缓冲器与反相器（保留带 dont_touch/size_only 的单元）。例外之外的单元仍视为时钟树一部分；clock-to-data 引脚之外视为数据通路不移除。同时

表 4.2: 使用 mark_clock_trees 命令

目的
仅标记特定时钟（默认标记所有 active scenario 各 mode 中 create_clock/create_generated_clock 定义的时钟）
标记为已综合（默认）
清除 synthesized 属性
施加 dont_touch
清除 dont_touch
传播 set_clock_routing_rules 的 NDR
传播 set_clock_cell_spacing 间距规则
将 sink 标记为 fixed
冻结时钟网布线

重置 CTS 相关属性。

注意

remove_clock_trees 不支持 clock mesh 网。

默认移除所有 mode 的所有时钟树；用 -clocks 限定（默认当前 mode；可用 get_clocks -mode）。结构对移除的影响见表 4.3。默认同时移除被删时钟网的详细布线形状；可将网的 shape_use 设为 user_route 以保留。

表 4.3: 时钟树移除行为

对象	对移除的影响
Boundary cell	移除
Don't touch net 上的单元	保留
Don't touch / Fixed cell	保留
Generated clock	若定义在缓冲/反相器引脚上则保留；遍历与移除继续穿过
Guide buffer	移除
ICG	保留；遍历继续穿过
Block abstraction model	保留；遍历继续穿过
Inverter	仅成对移除；单独反相器不移除
Isolation / Level shifter	保留
Three-state buffer	保留；遍历在此停止
例外之外添加的缓冲/反相器	移除

4.7 实现多源时钟树

Implementing Multisource Clock Trees

本节介绍多源时钟树结构类型及实现流程：

- 多源时钟树结构简介
- 实现常规多源时钟树
- 使用集成 Tap Assignment 的常规多源时钟树
- 仅含 H-Tree 全局结构的常规多源时钟树
- 实现 MSCTS 多重 H-Tree
- 实现功耗感知柔性 H-Tree
- 实现结构型多源时钟树（SMSCTS）
- 使用集成子树综合的 SMSCTS
- 改善 SMSCTS 与 CCD 的相关性
- 插入时钟驱动器；综合全局时钟树；创建时钟 strap；布线到 strap；分析 clock mesh
- 自动 Tap 插入与 H-Tree 综合；指定 Tap Assignment 选项；构建本地子树

4.7.1 多源时钟树结构简介

Introduction to Multisource Clock Trees Structures

多源时钟树对片上偏差更宽容，跨 corner 性能优于传统结构，由以下组成：

- 全局时钟结构：时钟根、通常为 H-tree 的全局时钟树、mesh drivers、clock mesh；
- 本地子树，驱动方式可为：
 - 由连接到 mesh 的预定义 tap drivers 驱动（图 4.12）——子树用常规 CTS(synthesize_clock_trees/clock_opt) 构建，称常规多源时钟树；
 - 直接从 mesh 多点驱动（图 4.13）——保留用户结构并通过合并/分裂优化，称结构型多源时钟树。

（原书 Figure 47: Regular Multisource Clock Tree）

图 4.12: 常规多源时钟树

(原书 *Figure 48: Structural Multisource Clock Tree*)

图 4.13: 结构型多源时钟树

4.7.2 实现常规多源时钟树

Implementing a Regular Multisource Clock Tree

1. 指定时钟树约束与设置;
2. 多电压设计: 启用 `cts.multisource.enable_full_mv_support`; 启用 `opt.common.allow_physical_feedthrough` 运行 `check_mv_design` 并修复;
3. 启用 `cts.multisource.cell_em_aware`;
4. `create_clock_drivers` 插入 tap drivers;
5. `create_clock_straps` 创建 clock mesh;
6. `create_clock_drivers` 插入 mesh drivers;
7. `synthesize_multisource_global_clock_trees` 构建全局树;
8. `route_clock_straps` 连接 mesh 与 tap;
9. `analyze_subcircuit` 分析 mesh;
10. 指定 tap assignment 选项;
11. `synthesize_multisource_clock_taps` (合并等价时钟单元、将端点赋给最近 tap、沿路径分裂、复制 UPF/SDC/属性);
12. `clock_opt -from build_clock -to route_clock` 构建由 tap 驱动的本地子树, 并修复 ignore 例外之外的 DRC;
13. 分析时钟树结果。

4.7.3 使用集成 Tap Assignment 的常规多源时钟树

Implementing a Regular Multisource Clock Tree Using Integrated Tap Assignment

若在 `place_opt` 前插入 tap drivers, 工具可在布局优化中分配 sink 以改善总体时序与时钟门控路径。流程与常规类似, 但在 mesh 分析后: 将 `place_opt.flow.enable_multisource_clock_trees` 设为 true 并运行 `place_opt`, 再 `clock_opt -from build_clock -to route_clock`。

4.7.4 仅含 H-Tree 全局结构的常规多源时钟树

Implementing a Regular Multisource Clock Tree With an H-Tree-Only Global Clock Tree Structure

全局结构无 mesh, H-tree 直接驱动 tap (图 4.14)。用 `set_regular_multisource_clock_tree_options` 与 `synthesize_regular_multisource_clock_trees` 完成 tap 插入与 H-tree 综合, 再做 tap assignment 与 `clock_opt`。

(原书 Figure 49: Regular Multisource Clock Tree With an H-Tree-Only Global Clock Structure)

图 4.14: 仅含 H-Tree 全局结构的常规多源时钟树

4.7.5 实现 MSCTS 多重 H-Tree

Implementing MSCTS Multiple H-Tree

用 `set_regular_multisource_clock_tree_options` 的 `-htree_sections` 在 floorplan 不同区域构建多个全局树 (图 4.15), 可用对称 tap 配置或坐标指定 tap 位置。

(原书 Figure 50/51: MSCTS Multiple H-Tree)

图 4.15: MSCTS 多重 H-Tree

4.7.6 实现功耗感知柔性 H-Tree

Implementing Power Aware Flexible H-Tree

在柔性 H-Tree 综合中加入功耗感知以优化总时钟功耗与总体 latency。须至少有一个动态功耗 scenario 在 H-tree 综合时为 active; 多 scenario 时取最差 active scenario 做功耗代价。初始 tap 配置基于 floorplan、sink 分布、布局与布线约束等估计各子树 latency/功耗并比较。启用: `cts.multisource.power_aware_flexib`

4.7.7 实现结构型多源时钟树

Implementing a Structural Multisource Clock Tree

子树由 mesh 或预定义驱动直接驱动; 通过合并、分裂、改尺寸与搬移优化, 同时保留用户定义的级数结构。主要步骤: MV/EM 设置 → `create_clock_straps` → 插入 mesh drivers → 综合全局树 → 用 `set_annotated_delay/set_annotated_transition` 建模 mesh 延迟 (避免子树综合时看到过大延

迟) → `synthesize_multisource_clock_subtrees` → `route_clock_straps` → `analyze_subcircuit`
→ `clock_opt -from build_clock -to route_clock`。

4.7.8 使用集成子树综合的 SMSCTS

Implementing a Structural Multisource Clock Tree Using Integrated Subtree Synthesis

用 `place_opt` 综合优化本地子树可改善时序 QoR。额外注意:可启用 `cts.multisource.enable_multi_corner_synthesis`。用 `set_multisource_clock_subtree_options` 指定 `-clock`、`-driver_objects` 及可选的 `-dont_merge_cells`、`-balance_levels`、`-target_level`、`-max_total_wire_delay`、`-enable_icg_reordering` 等;启用 `place_opt.flow.enable_subtree_synthesis` 后运行 `place_opt`;布线到 strap 并分析 mesh 后,启用 `clock_opt.flow.enable_multisource_clock_trees` 再跑 `clock_opt` 以增量优化子树。

4.7.9 改善 SMSCTS 与 CCD 的相关性

Improving the Correlation Between the SMSCTS and CCD

将 `cts.multisource.enable_subtree_synthesis_aware_ccd` 设为 `true`,可在 `place_opt` 与 `clock_opt` 的 `build_clock` 阶段改善 CCD balance point 实现与 SMSCTS 后时序,降低长短路径违例。

4.7.10 插入时钟驱动器

Inserting Clock Drivers

插入前:指定全部 CTS 设置(含布线规则);可将 `cts.multisource.enable_clock_driver_snapping` 设为 `true` 使驱动与 mesh strap 对齐。`create_clock_drivers` 可用于 mesh drivers (常规/结构型)与 tap drivers (常规)。要点:

- `-loads`: 负载网或同网引脚/端口;
- `-location` 精确位置,或 `-boxes` 网格(可配 `-boundary`);
- `-lib_cells` 或 `-template`;
- mesh drivers 用 `-short_outputs` 将输出短接驱动 mesh;
- `-configuration` 可插入多层驱动形成 H-tree (图 4.16)。

(原书 *Figure 52: Clock Drivers Placed in an H-Tree Structure*)

图 4.16: 按 H-Tree 结构放置的时钟驱动器

插入后驱动标记为 `fixed` 与 `don't touch`;避免与 `fixed` 单元、`blockage`、宏重叠,但可与非 `fixed` 单元重叠。命令不做合法化,需 `legalize_placement` (多次插入后统一合法化以节省时间)。布线 H-tree 时使用时钟网规则允许的最高层;为防驱动置于预布线下方导致引脚可达性问题,会为同层预布线创建临

时 `placement blockage`, 布线后删除。多行高单元放置困难时可将 `cts.multisource.enable_pin_accessibility_for_` 设为 `false`。移除用 `remove_clock_drivers`。

多级物理层次: 用 `set_editability` 启用下级编辑; 设 `cts.multisource.enable_mlp_flow=true`; 插入驱动; 用 `synthesize_multisource_global_clock_trees -roots -leaves -use_zroute_for_pin_connections` 布线。下级优先复用既有时钟端口, 否则在允许时创建新端口 (`set_freeze_ports`)。

4.7.11 综合全局时钟树

Synthesizing the Global Clock Trees

`synthesize_multisource_global_clock_trees` 综合并详细布线 H-tree 风格全局树, 尽量最小化端点 skew。须指定 `-nets` 与 `-lib_cells`。默认用 Custom Router; 引脚连接 DRC 难解时用 `-use_zroute_for_pin_connections` 可在靠近引脚的最高可用金属层停止。也可仅对已综合结构做详细布线 (`-roots/-leaves`)。移除用 `remove_multisource_global_clock_trees`。

4.7.12 创建时钟 Strap

Creating Clock Straps

`create_clock_straps` 创建单层直线金属形状, 可实现:

- **Clock mesh:** 水平与垂直层上的二维网格, 交叉点用 via 连接 (图 4.17);
- **Clock spine:** 一维或二维; 二维由一维 spine 连接正交方向多条 stripe, 不同 spine 的 stripe 不相连, 最小间距称 backoff (图 4.18)。

(原书 Figure 53: Clock Mesh Structure)

图 4.17: Clock Mesh 结构

(原书 Figure 54: Clock Spine Structure)

图 4.18: Clock Spine 结构

主要选项: `-net`、`-boundary`、`-keepouts` (`-allow_splitting false` 可禁止跨越 keepout 时分裂)、`-layers`、`-width`、`-grid` (单条距离或 start/end/step 迭代列表)、`-margins`、`-create_ends`、`-allow_floating`、`-type` (`user_route/stripe/detect`)、spine 的 `-length/-backoff/-spine_direction`、屏蔽相关 `-bias/-bias_to_nets/-bias_margins`、`-clear`。

Mesh 示例:

```
icc2_shell> create_clock_straps -nets [get_nets clk1_mesh] \
  -layers {M7 M8} -widths {2.4 2.4} -types {stripe stripe} \
  -grids {{20 1200 100} {60 980 150}} -boundary {{0 0} {1200 980}}
```

4.7.13 布线到时钟 Strap

Routing to Clock Straps

`route_clock_straps -nets` 将驱动与负载连到 `shape_use=stripe` 的 strap（不连 `user_route`）。
`-topology`:

- **fishbone**（默认）：各驱动单独连最近 stripe；多负载用 comb 连到 finger 再连 stripe（图 4.19）。可设 `-fishbone_fanout/-fishbone_span/-fishbone_sub_span/-fishbone_layers`（图 4.20、4.21）。
- **comb**：每引脚直接连最近 stripe；曼哈顿距离超过 comb distance（默认两全局布线格，可用 `route.common.comb_dist`）时用 Steiner 连最近 net shape（图 4.22）。适合大量引脚直接位于 stripe 下，但堆叠 via 可能引起物理 DRC（图 4.23）。
- **sub_strap**：在中间层创建平行于 stripe 的附加 strap，减少堆叠 via（图 4.24）。须 `-sub_strap_layers`；可选 `-sub_strap_max_delay`（默认 2 ps）。

（原书 *Figure 55: Fishbone Topology*）

图 4.19: Fishbone 拓扑

（原书 *Figure 56: Fanout of a Finger*）

图 4.20: Fishbone finger 的扇出

（原书 *Figure 57: Span and Subspan*）

图 4.21: Fishbone 的 span 与 subspan

(原书 *Figure 58: Comb Topology*)

图 4.22: Comb 拓扑

(原书 *Figure 59: Vias in the Comb Topology*)

图 4.23: Comb 拓扑中的 Via

4.7.14 分析 Clock Mesh

Analyzing the Clock Mesh

Mesh 需要比传统结构更高的时序精度。用 `analyze_subcircuit` 做晶体管级仿真并回标时序。前提: mesh 已详细布线; 有门级电路模型与晶体管模型; 可访问 NanoSim/FineSim/HSPICE。须指定 `-net` (或 `-to`) 与 `-name`; 多时钟时用 `-clock`。可选在端口/起点标注转换 (时钟须为 `propagated`)。步骤: RC 提取 → 生成 SPICE → 仿真 → 生成含 `set_disable_timing/set_annotated_delay/set_annotated_transition` 的注释文件 (多驱动 mesh 禁用除 `anchor driver` 外的驱动) → 应用注释。各步可用独立选项单独运行。

4.7.15 自动 Tap 插入与 H-Tree 综合

Performing Automated Tap Insertion and H-Tree Synthesis

1. 用 `set_regular_multisource_clock_tree_options` 指定 `-clock`、`-topology` `htree_only`、`-tap_boxes`、`-tap_lib_cells`、`-htree_lib_cells` 等 (库单元须已 `set_lib_cell_purpose -include cts`)。可报告/移除对应 options。
2. (可选) 启用柔性 H-tree: `cts.multisource.flexible_htree_synthesis=true`; 对称 `tap:cts.multisource.tap_synthesis=true`。
3. `synthesize_regular_multisource_clock_trees` (阶段 `tap_synthesis` 与 `htree_synthesis`; 可用 `-from/-to`)。

(原书 *Figure 60: Vias in the Substrap Topology*)

图 4.24: Substrap 拓扑中的 Via

4.7.16 指定 Tap Assignment 选项与设置

Specifying Tap Assignment Options and Settings

运行 `synthesize_multisource_clock_taps` 前：

1. 移除不必要的 `dont_touch/size_only/fixed/locked`，以免阻碍合并/分裂；
2. 用 `set_multisource_clock_tap_options` 指定 `-clock`、`-driver_objects`、`-num_taps`（须等于驱动数）等；
3. （可选）用 `create_multisource_clock_sink_group` 保持 skew group 对象在同一 tap 下；
4. （可选）设置合并/分裂命名相关的 `cts.multisource.subtree_*` 前缀/后缀。

4.7.17 构建本地时钟子树结构

Building the Local Clock Subtree Structures

结构型多源树中，用 `synthesize_multisource_clock_subtrees` 综合由 mesh 直接驱动的本地子树。前置：建好全局结构；用真实 annotated delay/transition 建模 mesh；清除不必要的保护属性；用 `set_multisource_clock_subtree_options` 配置。可选：多角优化、忽略驱动上 `max_cap/max_fanout`（`cts.multisource.ignore_drc_on_subtree_driver`）、fishbone 子树布线（`cts.multisource.subtree_routing_mode`）等。

命令步骤：合并等价单元 → 分裂/改尺寸/搬移并 snap 第一级到 strap → 布线（默认可全局布线；启用 fishbone 则混合详细 fishbone 与全局 comb，见图 4.25）→ 基于布线细化。可用 `-from/-to` 运行 `merge/optimize/route_clock/refine`。由此构建的结构型子树在后续 CTS 命令中不会被更改或删除。

（原书 Figure 61: Detail Fishbone Routes and Global Comb Routes）

图 4.25: 详细 Fishbone 布线与全局 Comb 布线

4.8 分析时钟树结果

Analyzing the Clock Tree Results

综合后分析是否满足要求，通常包括：生成时钟树 QoR 报告；分析时钟时序；在 GUI 中验证时钟实例布局。

4.8.1 生成时钟树 QoR 报告

Generating Clock Tree QoR Reports

用 `report_clock_qor`，可生成：

- **Summary** (默认): latency、skew、DRC、面积、缓冲计数;
- **Latency** (-type latency): 每时钟最长/最短路径;
- **DRC violators** (-type drc_violators);
- **Robustness** (-type robustness, 须 -robustness_corner): 当前 corner 与指定 robustness corner 的 latency 比;
- **Balance groups** (-type balance_groups): 含指定与实际 latency offset;
- **Local skew** (-type local_skew): 每时钟/skew group 汇总及最大/最小局部 skew;
- **Power** (-type power): 按时钟/scenario 的漏电、内部、sink、网络翻转、动态与总功耗。

可用 -csv/-output 输出 CSV。直方图类型: -histogram_type 为 latency/transition/capacitance/local_skew/robustness。

可用 -from/-to/-through、-clock、-skew_group、-mode、-corner、-scenario 限定范围 (-scenario 与 -mode/-corner 互斥)。

4.8.2 报告时钟树功耗

Reporting Clock Tree Power

用 report_clock_power。可用 -clocks/-modes/-corners/-scenario 限定; -type per_segment 或 -type per_subtree:

- **Segment**: 从一个 ICG 输出到其扇出中下一 ICG 或 sink 的缓冲树;
- **Subtree**: 从 ICG 一直到其扇出全部 sink 的时钟树 (图 4.26)。

(原书 Figure 62: Segments and Subtrees of a Clock Tree)

图 4.26: 时钟树的 Segment 与 Subtree

4.8.3 创建时钟网络引脚集合

Creating Collections of Clock Network Pins

get_clock_tree_pins 默认返回所有 scenario 全部时钟网上的引脚。可用 -clocks、-scenarios/-modes/-corners -from/-through/-to、-filter 限定。例如:

```
icc2_shell> set clk1_icg_pin \
[get_clock_tree_pins -filter is_on_ICG -clocks clk1]
```

4.8.4 分析时钟时序

Analyzing Clock Timing

用 report_clock_timing, 须指定 -type:

- **skew:** 单时钟局部 skew;
- **interclock_skew:** 时钟间 skew;
- **latency:** 延迟;
- **transition:** 转换时间。

4.8.5 在 GUI 中分析时钟树

Analyzing Clock Trees in the GUI

IC Compiler II GUI 提供 Clock Tree Visual Mode, 可将时钟树信息叠加到版图视图, 或在原理图视图中查看结构。详见 *IC Compiler II Graphical User Interface User Guide* 中的 Using Map and Visual Modes。

Chapter 5

布线与布线后优化

Routing and Postroute Optimization

本章介绍 IC Compiler II 路由器 Zroute 的布线能力，以及布线后优化功能。Zroute 面向多核硬件架构，可高效处理 45 nm 及以下先进工艺设计规则与可制造性设计（DFM）任务。

本章包括以下主题：

- Zroute 简介（Introduction to Zroute）
- 基本 Zroute 流程（Basic Zroute Flow）
- 布线前提条件（Prerequisites for Routing）
- 定义 Via（Defining Vias）
- 插入 Via Ladder（Inserting Via Ladders）
- 检查可布线性（Checking Routability）
- 布线约束（Routing Constraints）
- 布线应用选项（Routing Application Options）
- 时钟网络布线（Routing Clock Nets）
- 关键网络布线（Routing Critical Nets）
- 次级电源/地 pin 布线（Routing Secondary Power and Ground Pins）
- 信号网络布线（Routing Signal Nets）
- 网络屏蔽（Shielding Nets）
- 执行布线后优化（Performing Postroute Optimization）
- 分析并修复信号电迁移违例（Analyzing and Fixing Signal Electromigration Violations）
- 执行 ECO 布线（Performing ECO Routing）
- 在 GUI 中布线（Routing Nets in the GUI）

- 清理已布线网络 (Cleaning Up Routed Nets)
- 分析布线结果 (Analyzing the Routing Results)
- 保存布线信息 (Saving Route Information)
- 推导掩模颜色 (Deriving Mask Colors)
- 插入与移除 Cut Metal 形状 (Inserting and Removing Cut Metal Shapes)

5.1 Zroute 简介

Introduction to Zroute

Zroute 包含五个布线引擎：全局布线 (global routing)、轨道分配 (track assignment)、详细布线 (detail routing)、ECO 布线与布线验证。全局布线、轨道分配与详细布线可通过任务专用命令或自动布线命令调用；ECO 布线与布线验证则使用任务专用命令。

Zroute 主要特性包括：

- 全布线步骤 (含全局布线、轨道分配、详细布线) 支持多核多线程；
- 真实连通模型：矩形相触即视为电气连通，无需导线中心线重合；
- 动态迷宫网格：可 off-grid 连接 pin，同时保留网格路由器的速度优势；
- 多边形管理器：识别多边形，并理解 DRC 面向多边形；
- 详细布线阶段并发优化设计规则、天线规则、导线与 via；
- 详细布线阶段并发插入冗余 via；
- 全局/轨道/详细布线内建 soft rule 支持；
- 时序与串扰驱动的全局布线、轨道分配、详细布线与 ECO 布线；
- 智能设计规则处理 (合并冗余违例、智能收敛)；
- 带层约束与非默认布线规则 (NDR) 的 net group 布线；
- 时钟布线；
- 布线验证；
- 以 soft rule 方式优化 DFM 与可制造性 (DFY)；
- 支持先进设计规则，如多重图案化 (multiple patterning)。

5.2 基本 Zroute 流程

Basic Zroute Flow

图 5.1 给出基本 Zroute 流程，包括时钟布线、信号布线、DFM 优化与布线验证。

(原书 *Figure 63: Basic Zroute Flow*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 5.1: 基本 Zroute 流程

5.3 布线前提条件

Prerequisites for Routing

运行 Zroute 前，block 与物理库须满足：

- 库要求：Zroute 从 `technology file` 获取全部设计规则信息，布线前须确保规则已在 `technology file` 中定义。详见 *Synopsys Technology File and Routing Rules Reference Manual*。
- **Block 要求：**
 - 电源/地网络已在设计规划后、布局前布线完成（见 *IC Compiler II Design Planning User Guide*）；
 - 已完成时钟树综合与优化（见第 四 章）；
 - 估计拥塞可接受；
 - 估计时序可接受（约 0 ns slack）；
 - 估计最大电容与 transition 无违例。

后三项可通过 `check_routability` 验证布局可布线性（见「检查可布线性」）。

5.4 定义 Via

Defining Vias

路由器支持以下 via 定义类型：

- **Simple via 与 simple via array：** simple via 为单 cut via，由 cut 层及 cut/金属层矩形的高宽指定；simple via array 为多 cut 的 simple via。可用于：带 NDR 的时钟/信号布线、冗余 via 插入、带先进 via 规则的 PG 布线。
- **Custom via：** 由任意 Manhattan 多边形集合构成的多 cut、异形 via；仅可用于冗余 via 插入。

Via 定义来源：

- 设计库关联 `technology file` 的 `ContactCode` 段；
- LEF 文件中的 via rule `GENERATE` 语句（定义 simple via array）；
- 用户自定义 via（`create_via_def`）。

若希望 Zroute 将某 via 定义用于信号布线,须满足:is_default 为 true,且 is_excluded_for_signal_routing 为 false。此外, Zroute 还会使用 technology file 的 via table 或 create_routing_rule 所定义 NDR 中显式指定的非默认 via。

5.4.1 从 LEF 读取 Via 定义

Reading Via Definitions from a LEF File

使用 read_tech_lef 读取 LEF 中 via rule GENERATE 语句。支持如下 LEF 语法:

```
via_rule_name GENERATE [DEFAULT]
  LAYER lower_layer_name
    ENCLOSURE lower_overhang1 lower_overhang2
  LAYER upper_layer_name
    ENCLOSURE upper_overhang1 upper_overhang2
  LAYER cut_layer_name
    RECT llx lly urx ury
    SPACING x_spacing BY y_spacing
```

不支持 WIDTH 与 RESISTANCE 语句。

5.4.2 创建 Via 定义

Creating a Via Definition

使用 create_via_def 创建 via 定义; 创建后可在设计中任意处使用 (类似 technology file 的 ContactCode)。用户 via 定义保存在设计库中, 仅该设计可用。可定义 simple via、simple via array 与 custom via。

定义 Simple Via

Defining Simple Vias

```
create_via_def
  -cut_layer layer
  -cut_size {width height}
  -upper_enclosure {width height}
  -lower_enclosure {width height}
  [-min_rows number_of_rows]
  [-min_columns number_of_columns]
  [-min_cut_spacing distance]
  [-cut_pattern cut_pattern]
  [-is_default]
  [-force]
  via_def_name
```

无需指定 enclosure 层: 工具根据 technology file 中与 via 层相邻的金属层自动确定。默认创建单 cut via; 定义 via array 时指定 -min_rows/-min_columns 与 -min_cut_spacing。默认 cut pattern 为满阵列; 可用 -cut_pattern 修改。覆盖已有定义须用 -force。

```
icc2_shell> create_via_def design_via1_HV \
  -cut_layer VIA12 -cut_size {0.05 0.05} \
  -lower_enclosure {0.02 0.0} -upper_enclosure {0.0 0.2}

icc2_shell> create_via_def design_via12 -cut_pattern "01 10"
```

用 report_via_defs 报告用户 via 定义。

定义 Custom Via

Defining Custom Vias

```
create_via_def
  -shapes { {layer {coordinates} [mask_constraint]} ... }
  [-lower_mask_pattern alternating | uniform]
  [-upper_mask_pattern alternating | uniform]
  [-force]
  via_def_name
```

-shapes 须为一个 via 层与两个金属 enclosure 层指定形状；每层可有多个形状。

```
icc2_shell> create_via_def design_H_shape_via \
  -shapes { {VIA12 {0.035 -0.035} {0.100 0.035}}
           {VIA12 {-0.100 -0.035} {-0.035 0.035}}
           {METAL1 {-0.130 -0.035} {0.130 0.035}}
           {METAL2 {-0.100 -0.065} {-0.035 0.065}}
           {METAL2 {0.035 -0.065} {0.100 0.065}}
           {METAL2 {-0.100 -0.035} {0.100 0.035}} }
```

双图案化时，可对形状或 enclosure 层分配 mask constraint，二者不可同时使用：

- 对形状：在 -shapes 中用 mask_one/mask_two/mask_three；
- 对 enclosure 层：用 -lower_mask_pattern/-upper_mask_pattern (uniform 或 alternating)，创建 via 时再指定首个形状的 mask。

```
icc2_shell> create_via_def VIA12 \
  -shapes { {M1 {-0.037 -0.010} {-0.017 0.01} mask_two}
           {M1 {0.017 -0.010} {0.037 0.01} mask_one}
           {VIA1 {-0.037 -0.01} {0.037 0.01} mask_two}
           {M2 {-0.044 -0.010} {0.044 0.010} mask_one} }
icc2_shell> create_via -net n1 -via_def VIA12 \
  -origin {100.100 200.320}
```

5.5 插入 Via Ladder

Inserting Via Ladders

Via ladder（亦称 via pillar）是自 pin 层向上堆叠至上层、供路由器连接的堆叠 via。Via ladder 可降低 via 电阻，改善性能与电迁移鲁棒性。

使用 via ladder 前须：

- 在设计的工艺数据中定义 via ladder 规则（见「定义 Via Ladder 规则」）；
- 在全局布线之前插入 via ladder。

插入方法：

- 布线前优化中的自动插入；
- 基于约束的插入：insert_via_ladders；
- 手动插入：create_via_ladder。

参见

查询 Via Ladder；移除 Via Ladder；控制 Via Ladder 连接。

5.5.1 定义 Via Ladder 规则

Defining Via Ladder Rules

Via ladder 规则定义各级的行数与每行 cut 数，可在 technology file 中定义，或用 create_via_rule 创建。以下三种方式可创建等价规则：

方式 1：在 technology file 的 ViaRule 段中定义（含 cutLayerNameTbl、numCutRowsTbl、numCutsPerRowTbl、upperMetalMinLengthTbl、cutXMinSpacingTbl、cutYMinSpacingTbl、maxNumStaggerTracksTbl、forHighPerformanceForElectromigration 等）。详见 *Synopsys Technology File and Routing Rules Reference Manual*。

方式 2：命令行详细规格 + 属性：

```
icc2_shell> create_via_rule -name VL1 -cut_layer_names {VIA1 VIA2} \
    -cut_names {CUT1A CUT2A} -cut_rows {2 2} -cuts_per_row {2 2}
icc2_shell> set_attribute [get_via_rules VL1] \
    upper_metal_min_length_table {L1 L2}
icc2_shell> set_attribute [get_via_rules VL1] \
    for_high_performance false
icc2_shell> set_attribute [get_via_rules VL1] \
    for_electro_migration true
```

方式 3：先创建空规则再设属性（cut_layer_name_table_size、cut_layer_name_table、num_cuts_per_row_table 等）。

无论创建方式如何，工具对 via rule 一视同仁。可用 get_via_rules、remove_via_rules、report_via_rules、list_attributes -application -class via_rule，以及 write_tech_file 保存。

5.5.2 为电迁移 Via Ladder 生成规则

Generating Via Ladder Rules for Electromigration Via Ladders

可用 generate_via_ladder_template -config_file 根据 XML 配置生成规则与约束脚本，输出：

- 模板脚本（默认 auto_gen_via_ladder_template.tcl，可用 -template_file 改名）：含 create_via_rule 与 set_attribute；

- 关联脚本（默认 `auto_gen_via_ladder_association.tcl`，可用 `-association_file` 改名）：含 `set_via_ladder_candidate`，并将 `pin` 的 `is_em_via_ladder_required` 设为 `true`。

配置文件基本结构：

```
<EmRule>
  <Template name="rule_name">
    <Layer name="layer_name" row_number="cuts_per_row"
      [max_stagger_tracks="count"] ... />
  </Template>
  <Pin name="pin_name">
    <Template name="rule_name"/>
  </Pin>
</EmRule>
```

工具从 `technology file` 推导 `cut` 层、`cut` 名与各金属层最小长度。

注意

运行 `generate_via_ladder_template` 前须打开 `block`，以确保可访问工艺数据。

```
icc2_shell> generate_via_ladder_template -config_file vl_config
```

5.5.3 为性能 Via Ladder 生成规则

Generating Via Ladder Rules for Performance Via Ladders

使用 `setup_performance_via_ladder` 生成性能 via ladder 规则与关联。默认：

- 生成至 M5 的规则（`-max_layer` 可改）；输出 XML（默认 `auto_perf_via_ladder_rule.xml`）与规则脚本（默认 `auto_perf_via_ladder_rule.tcl`）；也可用 `generate_via_rules_for_performance`；
- 将规则关联到无 `dont_use` 的库单元 `pin`（`-dont_use` 可包含 `dont_use` 单元；`-lib_pins` 可限定 `pin`）；输出关联脚本（默认 `auto_perf_via_ladder_association.tcl`）；也可用 `associate_performance_via_ladder`；
- 运行所生成的脚本。

5.5.4 基于约束的 Via Ladder 插入

Constraint-Based Via Ladder Insertion

步骤：

1. 定义 via ladder 规则；
2. 用 `set_via_ladder_rules` / `set_via_ladder_constraints` 定义约束；
3. `insert_via_ladders` 插入；
4. `verify_via_ladders` 验证。

定义 Via Ladder 约束

Defining Via Ladder Constraints

约束指定在哪些 pin 上插入何种模板；可定义全局与 instance 级约束，冲突时 instance 级优先。另可用 `set_via_ladder_candidate` 为特定 pin 指定候选（布线前优化在时序关键路径上插入时使用）。用 `report_via_ladder_constraints` 报告。

全局约束(via ladder rules):`set_via_ladder_rules`。映射选项:`-master_pin_map/-master_pin_map_file` (格式 `{{lib_cell/pin {via_ladder_list}}...}`)、`-default_ladders`。作用范围:`-all_instances_of、-all_clock_outputs、-all_clock_inputs、-all_pins_driving`。报告/移除:`report_via_ladder_rules、remove_via_ladder_rules`。

Instance 级约束:

```
icc2_shell> set_via_ladder_constraints -pins {u1/i1 u2/i2} \
    {VL1 VL2 VL3}
```

移除用 `remove_via_ladder_constraints`。

插入 Via Ladder

Inserting Via Ladders

`insert_via_ladders` 默认行为与常用选项:

- 默认不删除已有 ladder；用 `-clean true` 先清除；
- 默认针对约束中全部 pin（信号与时钟）；用 `-nets` 限定网络；
- 每个目标 pin 插入一个 ladder；多模板时选用首个不导致 DRC 的模板；全导致 DRC 时默认选 DRC 代价最低者；
- `-ignore_routing_shape_drcs true`：检查时忽略详细布线形状；
- `-relax_line_end_via_enclosure_rule true`：放松线端 cut enclosure；
- `-relax_pin_layer_metal_spacing_rules true`：ladder 完全包在 pin 内时放松 pin 层间距；
- `-allow_drcs false`：禁止产生 DRC 的插入；
- `-strictly_honor_cut_table true`：仅用模板中的 cut；
- `-shift_vias_on_transition_layers true`：过渡层 cut 偏离 track 以改善 DRC；
- `-ndr_on_top_layer_only true`：仅顶层遵循非默认宽度；
- `-connect_within_metal/-connect_within_metal_for_via_ladder`：要求 pin 层上方 via 完全落在 pin（含延伸）内；
- `-allow_patching true`：允许 enclosure 打补丁以满足最小长度；
- `-auto_stagger true`：规则未定义最大 stagger track 时仍允许 stagger；
- `-verbose true/-user_debug true`：报告插入状态与失败详情（内部/硬/软 DRC）。

注意

若 pin 过小无法容纳任一指定 ladder，则不插入。

保护 Via Ladder**Protecting Via Ladders**

```
icc2_shell> set_app_options -name design.enable_via_ladder_protection \
-value true
```

为 true 时, remove_shapes、remove_vias、remove_objects、set_attribute、set_via_def、add_to_edit_group 仅处理不属于 via ladder 的对象；属于 ladder 的对象会警告并跳过。

验证与更新**Verifying and Updating Via Ladders**

verify_via_ladders 检查 ladder 是否匹配约束并正确连到 pin；可用 -nets 限定。若插入时用了 -shift_vias_on_transition_layers true, 验证时也必须使用, 否则会误报。-report_all_via_ladders false 可将每类报告限制为 40 条。

更新方式: refresh_via_ladders; 或在 route_group/route_auto/route_eco 中由 route.auto_via_ladder.update (默认 true) 自动更新。相关应用选项包括 route.auto_via_ladder.connect_within_metal、route.auto_via_ladder.report_all_via_ladders、route.auto_via_ladder.update_on_route_group_nets_only 等。

手动插入**Manual Via Ladder Insertion**

1. 用 create_shape/create_via 创建组成形状；
2. 用 create_via_ladder -shapes 创建 ladder (名称形如 VIA_LADDER_n)。

可选属性选项: -via_rule、-electromigration、-high_performance、-pattern_must_join、-pin。

查询: get_via_ladders、report_via_ladders。移除:

```
icc2_shell> remove_via_ladders *
```

-nets 可限定网络。

5.6 检查可布线性

Checking Routability

布局完成后, 用 check_routability 检查 block 是否具备详细布线条件。默认检查:

- 被阻塞的标准单元端口: 物理 pin 均不可达则视为 blocked。可达判定: pin 内有伸向邻层的 via、pin 层上存在不少于搜索范围的路径, 或较短路径止于邻层 via。默认搜索范围为 2 倍 layer pitch (-standard_cell_search_range, 最大 10)。-connect_standard_cells_within_pins true 检查 via 是否完全在 pin 内。-check_standard_cell_blocked_ports false 可关闭。

- 被阻塞的顶层/宏端口: 默认搜索距离为 10 倍 pitch (`-blocked_range/-blocked_range_via_side`, 最大 40)。`-obey_direction_preference true` 要求沿优选方向可达。`-check_non_standard_cell_blocked_port` `false` 可关闭。
- 越界 **pin** (`-check_out_of_boundary false` 可关);
- 最小网格违例 (`-check_min_grid false` 可关);
- 非法 **via** 定义 (未着色 via array 行列不超过 20; custom/非对称 simple via 不超过 1000 cut; via 定义总数不超过 65535——不可关闭);
- 非法 **real metal via cut blockage** (`-check_via_cut_blockage false` 可关);
- 最小宽度设置 (NDR 最小宽/屏蔽宽不超过 technology 最大宽——不可关闭)。

可选检查 (需显式打开): `-check_pg_blocked_ports`、`-check_redundant_pg_shapes`、`-check_routing_track_spacing`、`-check_frozen_net_blocked_ports`、`-check_no_net_pins`、`-check_real_metal_blockage_overlap_pin`、`-check_lib_via_cut_blockage`、`-check_shield`、`-via_ladder`。

Pin 连接控制: `-obey_access_edges`、`-access_edge_whole_side`、`-report_no_access_edge`、`-allow_via_rotation`。

Blocked port 检查会考虑 `set_ignored_layers/set_routing_rule/set_clock_routing_rules` 的软/硬层约束及 `route.common.freeze_layer_by_layer_name` 等; `-honor_layer_constraints false` 可忽略层约束。

错误数据默认名为 `check_routability.err` (`-error_data` 可改), 随 block 保存; 可用 error browser 查看。

5.7 布线约束

Routing Constraints

布线约束在布线过程中提供指导。表 5.1 概述各类约束。

5.7.1 定义 Routing Blockage

Defining Routing Blockages

Routing blockage 定义特定层上禁止布线的区域, Zroute 视其为硬约束。用 `create_routing_blockage` 创建, 至少指定边界与层。

- 矩形: `-boundary { {l1x l1y} {urx ury} }`;
- 直角多边形: `-boundary { {x1 y1} {x2 y2} ... }`, 也可由 `geo_mask` 等物理对象并集生成 (多连通则生成多个 blockage);
- 层: `-layers` (金属、via 或 poly)。

默认名称 `RB_objId`, 阻止边界内所有网络; 按 `real metal` 参与 RC 提取与 DRC, 须满足最小间距。常用选项: `-name`、`-cell`、`-zero_spacing` (零间距且不作 `real metal`; 防止 via 插入时须用此选项)、`-net_types` (`signal/clock/power/ground` 等)。

```
icc2_shell> create_routing_blockage \
  -boundary { {0.0 0.0} {100.0 100.0} } \
```

表 5.1: 布线约束类型

指导内容	说明
控制布线层	<code>set_ignored_layers</code> 等, 见「指定布线资源」
更严格最小宽/间距	定义并应用 NDR (<code>create_routing_rule</code>)
控制穿越 voltage area	电压域布线规则
控制布线方向	层优选方向; 区域用 <code>preferred-direction-only</code> / <code>switch-preferred-direction routing guide</code>
限制非优选方向边数	<code>maximum-number-of-pattern routing guide</code>
控制 off-grid 布线	见「控制 Off-Grid 布线」
禁止特定 net 布线	<code>physical_status</code> 设为 <code>locked</code>
禁止特定区域布线	<code>routing blockage</code>
为顶层布线预留空间	<code>corridor routing blockage</code>
限制布线到特定区域	<code>routing corridor</code>
区域优先级	<code>access preference routing guide</code>
控制布线密度	<code>utilization routing guide</code>
鼓励 river routing	<code>single-layer river routing guide</code>
改善硬宏 pin 可布线性	<code>pin access routing guides</code>
控制 block 边界周围布线	<code>perimeter constraint objects</code>
Metal cut allowed/forbidden 规则	<code>boundary</code> <code>cell</code> 插入时自动创建, 或 <code>derive_metal_cut_routing_guides</code>
控制 pin 连接	<code>must-join</code> 、 <code>pin connection</code> 类型、 <code>pin tapering</code>
控制 via ladder 连接	见相关小节
限制重布线范围	<code>rerouting mode</code>

```
-net_types signal -layers M1
icc2_shell> create_routing_blockage \
  -boundary { {0.0 0.0} {100.0 100.0} } \
  -net_types signal -layers V1 -zero_spacing
icc2_shell> create_routing_blockage \
  -boundary { {0.0 0.0} {100.0 100.0} } \
  -net_types {power ground} -layers M2
```

为顶层布线预留空间：创建时加 `-reserve_for_top_level_routing`。块级实现时作普通 `blockage`；`frame view` 提取时移除以允许顶层布线。

查询：`get_routing_blockages`、`get_objects_by_location -classes routing_blockage`。移除：`remove_routing_blockages (-all 移除全部 corridor)`。

5.7.2 定义 Routing Guide

Defining Routing Guides

`Routing guide` 为区域提供布线指令。用户定义的 `guide` 由 `Zroute` 遵从，但 `Advanced Route` 工具不遵从。

注意

用户定义的 `routing guide` 由 `Zroute` 遵从；`Advanced Route` 工具不遵从。

用 `create_routing_guide` 创建，须指定矩形边界 `-boundary {{llx lly} {urx ury}}` 及用途相关选项。表 5.2 列出用户定义类型。

表 5.2: 用户定义 Routing Guide

用途	选项
仅在优选方向布线	<code>-preferred_direction_only</code>
切换优选方向	<code>-switch_preferred_direction</code>
限制特定布线模式出现次数	<code>-max_patterns</code>
控制布线密度	<code>-horizontal_track_utilization / -vertical_track_utilization</code>
区域优先级	<code>-access_preference</code>
鼓励 river routing	<code>-river_routing</code>

默认名称 `RD#n`；可用 `-name`、`-layers`、`-cell`。

控制布线方向

Using Routing Guides to Control the Routing Direction

```
icc2_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
  -layers {M4} -preferred_direction_only
icc2_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
  -layers {M1 M2} -switch_preferred_direction
```

注意

若 switch-preferred-direction 与 preferred-direction-only 重叠，前者优先。

限制非优选方向边数

Using Routing Guides to Limit Edges in the Nonpreferred Direction

```
icc2_shell> create_routing_guide -boundary {{50 50} {200 200}} \
    -max_patterns {{M2 non_pref_dir_edge 2} {M4 non_pref_dir_edge 3}} \
    -layers {M2 M4}
```

当前支持的 pattern 为 non_pref_dir_edge。超过阈值时 Zroute 报告违例。

控制密度与优先级、**River Routing**

Density, Access Preference, and River Routing

```
icc2_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
    -horizontal_track_utilization 50 -vertical_track_utilization 30
icc2_shell> create_routing_guide -boundary {{0 0} {300 300}} \
    -access_preference {M1 {wire_access_preference {{0 0} {2 1}} 0.2}
        {via_access_preference {{0 0} {5 5}} 1.0}
        {via_access_preference {{40 40} {45 45}} 0.5}}
icc2_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
    -river_routing -layers {M2}
```

强度 0–1；强度为 1 时视为硬约束，小于 1 的区域被忽略。用 report_routing_guides 报告。查询/移除：get_routing_guides、remove_routing_guides -all。

5.7.3 推导 Routing Guide**Deriving Routing Guides****Pin Access Routing Guide****Deriving Pin Access Routing Guides**

在更小工艺节点复用硬宏时，可用 derive_pin_access_routing_guides 改善宏 pin 可达性：

```
icc2_shell> derive_pin_access_routing_guides -cells myMacro \
    -layers {M2 M3} -x_width 0.6 -y_width 0.7
```

为指定宏在指定层创建 routing guide，并在宏周围创建带 pin 开孔的 metal blockage 及相邻 via 层 blockage。默认仅在优选方向开孔；-nonpreferred_direction 可在非优选方向也开孔；-pin_extension 可延伸开孔。命令会将宏 pin 的 is_rectangle_only_rule_waived 设为 true。

移动宏后须重新生成；不可手动修改。

(原书 *Figure 64: Pin Access Routing Guides and Blockages*)

图 5.2: Pin Access Routing Guide 与 Blockage

Metal Cut Routing Guide

Deriving Metal Cut Routing Guides

若 technology file 定义了 metal cut allowed / forbidden preferred grid extension 规则且已有 boundary cell, 可用:

```
derive_metal_cut_routing_guides -add_metal_cut_allowed
```

-check_only 可检查; 错误数据默认 block_name.err (-error_view 可改)。

5.7.4 控制 Block 边界周围布线

Controlling Routing Around the Block Boundary

用 derive_perimeter_constraint_objects 沿边界放置金属形状、routing guide 与 blockage。边界变更后须重新生成。

- -perimeter_objects metal_preferred: 沿优选方向边插入浮空金属 (连续形状或用 -stub 的金属 stub);
- -short_metal_length/-metal_spacing: 垂直于非优选边的短金属;
- -perimeter_objects mprg_nonpreferred: 沿非优选边插入最大 pattern guide (阈值为 0);
- 另可沿边界插入 routing blockage。

常用选项: -hierarchical、-layers、-spacing_from_boundary、-width_nonpreferred、-spacing_nonpreferred、-check_only。

(原书 *Figure 65: Continuous Metal Shape Perimeter Constraint Objects*)

图 5.3: 连续金属形状边界约束对象

(原书 *Figure 66: Metal Stub Perimeter Constraint Objects*)

图 5.4: Metal Stub 边界约束对象

(原书 *Figure 67: Metal Shape Perimeter Constraint Objects With Additional Short Shapes*)

图 5.5: 带短金属的边界约束对象

5.7.5 在特定区域内布线 (Routing Corridor)

Routing Nets Within a Specific Region

Routing corridor 将布线限制在指定区域。全局布线视其为硬约束；轨道分配与详细布线视其为软约束（为修 DRC 可能略微越界）。

用 `create_routing_corridor` 创建, `add_to_routing_corridor` 将 net 加入, `check_routing_corridors` 检查。

5.7.6 使用非默认布线规则 (NDR)

Using Nondefault Routing Rules

用 `create_routing_rule` 定义比 technology file 更严格的宽度、间距、via 间距、via 选择、mask、屏蔽等规则，再用 `set_routing_rule` 或 `set_clock_routing_rules` 赋给网络。

最小线宽与间距

Defining Minimum Wire Width and Spacing Rules

```
create_routing_rule rule_name \
  -widths {layer1 width1 ...} \
  -spacings {layer1 spacing1 ...}
```

(原书 *Figure 68: Routing Guide Perimeter Constraint Objects*)

图 5.6: Routing Guide 边界约束对象

(原书 *Figure 69: Using a Routing Corridor*)

图 5.7: 使用 Routing Corridor

也可定义 soft spacing (带 weight):

```
-soft_spacings { {layer spacing weight} ... }
```

(原书 *Figure 70: Ignoring Nondefault Spacing Rule Violations*)

图 5.8: 忽略非默认间距规则违例

为 soft spacing 分配修复努力:

```
icc2_shell> set_app_options \
-name route.common.soft_rule_weight_to_effort_level_map \
-value { {low low} {medium medium} {high high} }
```

effort 取值: off/low/medium/high (low weight 不可用 medium/high effort)。

最小 Via 间距与非默认 Via

Via Spacing and Nondefault Vias

```
icc2_shell> create_routing_rule via_spacing_rule \
-via_spacings {{V1 V1 2.3} {V1 V2 3.2}}
```

指定非默认 via: -cuts (由工具按 technology 选择) 或 -vias (显式列出 via 类型、cut 数与方向 R/NR)。

Mask 约束与屏蔽规则

Mask Constraints and Shielding Rules

```
icc2_shell> create_routing_rule clock_mask1 \
-mask_constraints {M4 mask_one M5 mask_one}
icc2_shell> create_routing_rule shield_rule \
-shield_widths {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3} \
-shield_spacings {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3}
```

注意

为确保 DRC 收敛，双图案化 mask 约束应仅用于极少数时序关键网络。

报告: report_routing_rules -verbose (-output 可写出 Tcl)。移除: remove_routing_rules。
修改须先删后重建。

将 NDR 赋给网络

Assigning Nondefault Routing Rules to Nets

- set_clock_routing_rules: CTS 前赋给时钟网，CTS 会传播到新建时钟网；
- set_routing_rule: 赋给信号网，或 CTS 后的时钟网。

表 5.3 列出限制时钟规则分配的选项。

表 5.3: 限制时钟布线规则分配

赋给	选项
特定时钟树	-clocks
连到 clock root 的网络	-net_type root
连到 clock sink 的网络	-net_type sink
时钟树内部网络	-net_type internal
特定时钟网络	-nets

(原书 Figure 71: Root, Internal, and Sink Clock Net Types)

图 5.9: Root / Internal / Sink 时钟网络类型

```
icc2_shell> set_clock_routing_rules -rules NDR1 -net_type root
icc2_shell> set_clock_routing_rules -rules NDR2 -net_type internal
icc2_shell> set_clock_routing_rules -rules NDR3 -net_type sink
icc2_shell> set_clock_tree_options -root_ndr_fanout_limit 300
```

(原书 Figure 72: Using a Fanout Limit for Selecting Root Nets)

图 5.10: 用扇出限制选择 Root 网络

注意

较小的 `-root_ndr_fanout_limit` 会增加使用 root NDR 的时钟网数量，可能加剧拥塞。

多条 NDR 时优先级: `set_routing_rule > set_clock_routing_rules -nets > set_clock_routing_rules -clocks > 全局 set_clock_routing_rules`。

信号网示例:

```
icc2_shell> set_routing_rule -rule WideMetal [get_nets CLK]
icc2_shell> set_routing_rule -default_rule [get_nets CLK]
```

另有 `-no_rule`、`-clear`。报告: `report_routing_rules -of_objects`、`report_clock_routing_rules`。

5.7.7 控制 Off-Grid 布线

Controlling Off-Grid Routing

默认仅当 technology 中 `onWireTrack/onGrid` 为 1 时才要求对齐 track。可用以下选项覆盖:

```
icc2_shell> set_app_options \
-name route.common.wire_on_grid_by_layer_name \
-value {{M2 true} {M3 true}}
icc2_shell> set_app_options \
-name route.common.via_on_grid_by_layer_name \
-value {{V2 true}}
icc2_shell> set_app_options \
-name route.common.extra_via_off_grid_cost_multiplier_by_layer_name \
-value {{M2 0.5}}
```

有效代价 = 基础代价 $\times (1 + \text{multiplier})$, multiplier 范围 0.0–20.0。

5.7.8 Must-Join Pin 布线

Routing Must-Join Pins

Must-join pin 的各 terminal 作为同一网络连接。默认随机结构 (图 5.11); 为改善电磁特性, 可用 pattern-based must-join (经 via ladder 能力实现, 图 5.12)。

(原书 Figure 73: Default Must-Join Pin Connection)

图 5.11: 默认 Must-Join 连接

多层结构 (图 5.13): 将 `route.auto_via_ladder.pattern_must_join_over_pin_layer` 设为 2。当库 pin 有 `pattern_must_join` 且运行 `route_auto/route_group/route_eco` 时自动使用。

```
icc2_shell> set_app_options -name \
route.auto_via_ladder.pattern_must_join_max_number_stagger_tracks \
-value {3 0}
```

(原书 *Figure 74: Single-Level Pattern-Based Must-Join Pin Connection*)

图 5.12: 单层 Pattern-Based Must-Join

(原书 *Figure 75: Multi-Level Pattern-Based Must-Join Pin Connection*)

图 5.13: 多层 Pattern-Based Must-Join

无法创建时报告“Needs pattern must join pin connection”DRC。可用 `route.auto_via_ladder.update_pattern_mu` 控制是否在每次布线时更新。

5.7.9 控制 Pin 连接与 Tapering

Controlling Pin Connections and Pin Tapering

```
icc2_shell> set_app_options \
  -name route.common.connect_within_pins_by_layer_name \
  -value {{M1 via_all_pins}}
```

Mode: off (默认)、via_standard_cell_pins、via_wire_standard_cell_pins、via_all_pins、via_wire_all_pins。

Pin tapering: 默认基于距离; 也可用层基 tapering (-taper_over_pin_layers/-taper_under_pin_layers 等)。二者互斥, 同时指定时采用层基设置。相关应用选项: route.detail.pin_taper_mode、route.detail.use_wide_w route.detail.use_wide_wire_to_input_pin 等。

Via ladder 连接: 默认连到 ladder 最顶层。route.detail.via_ladder_upper_layer_via_connection_mode、route.detail.allow_default_rule_nets_via_ladder_lower_layer_connection 可调整。

(原书 *Figure 76: Pattern-Based Must-Join Pin Connection With Staggering*)

图 5.14: 带 Stagger 的 Pattern-Based Must-Join

(原书 *Figure 77–79: Pin Connection Restrictions*)

图 5.15: Pin 连接限制示意 (无限制 / via / via+wire)

5.7.10 设置重布线模式

Setting the Rerouting Mode

将 net 的 `physical_status` 设为 `locked` (禁止重布)、`minor_change` (仅小改) 或 `unrestricted` (默认)。

```
icc2_shell> set_attribute -objects [get_nets net1] \
    -name physical_status -value locked
```

5.7.11 定义与使用 Routing Corridor (详述)

Defining and Using Routing Corridors

Routing corridor 将特定网络的全局布线限制在一组相连矩形所定义的区域内, 并可对每个矩形指定最小/最大布线层。主要用于在信号布线前布关键网。若 routing guide 与 corridor 冲突, corridor 优先。

```
icc2_shell> create_routing_corridor -name corridor_a \
    -boundary { {10 10} {20 35} } \
    -min_layer_name M2 -max_layer_name M4
icc2_shell> create_routing_corridor_shape -routing_corridor corridor_a \
    -boundary { {20 25} {40 35} } \
    -min_layer_name M2 -max_layer_name M4
icc2_shell> add_to_routing_corridor corridor_a [get_nets {n1 n2}]
icc2_shell> check_routing_corridors RC_0
```

也可按路径创建: `-path + -width`; 端帽样式 `-start_endcap / -end_endcap (flush/full_width/half_width)`。一个 net/supernet 只能属于一个 corridor, 且 corridor 须覆盖其全部 pin。

修改: `create_routing_corridor_shape`、`remove_routing_corridor_shapes`、`add_to_routing_corridor`、`remove_from_routing_corridor`。报告: `report_routing_corridors (-output 可写出 Tcl)`、`get_routing_corridors -of_objects`。移除: `remove_routing_corridors -all`。

5.7.12 布线层资源与冻结层 (章内相关)

Routing Layer Resources and Freeze Layers

全局/网络/时钟的最小最大布线层分别由 `set_ignored_layers`、`set_routing_rule` 与 `set_clock_routing_rule` 的 `-min_routing_layer / -max_routing_layer` 设置; 软/硬模式由 `route.common.global_min_layer_mode`、`route.common.net_min_layer_mode` 等控制。冻结层由 `route.common.freeze_layer_by_layer_name`

与 `route.common.freeze_via_to_frozen_layer_by_layer_name` 指定。电压域穿越与层优选方向的详细设置见第 2 章相关主题。

5.7.13 天线规则与冗余 Via（章内要点）

Antenna Rules and Redundant Vias

详细布线阶段并发处理天线规则，并在报告中给出天线违例与冗余 via 转换率。用 `check_routes` 的 `-antenna` 可启用/关闭天线检查。冗余 via 可在信号布线期间并发插入；详细布线结束报告含“Redundant via conversion report”（按层与 `weight` 的优化转换率）。更完整的天线修复与冗余 via 专项流程见原书交叉引用主题及第 6 章芯片收尾相关内容。

5.8 布线应用选项

Routing Application Options

工具提供影响全部三个布线引擎的选项，以及分别影响全局布线、轨道分配、详细布线的选项。详见 `man` 页：`route.common_options`、`route.global_options`、`route.track_options`、`route.detail_options`。

执行布线命令时，已设置（或工具代设）的选项会写入布线日志。要显示全部布线选项（含未设置者）：

```
icc2_shell> set_app_options \
    -name route.common.verbose_level -value 1
```

5.9 时钟网络布线

Routing Clock Nets

在布线其余网络前先布时钟网，使用 `route_group`：

```
icc2_shell> route_group -all_clock_nets
```

也可用 `-nets` 指定部分时钟网。该命令对时钟网依次执行全局布线、轨道分配与详细布线。

- 普通时钟网使用 A-tree 布线以最小化各 pin 到 driver 的距离；
- 时钟 mesh 默认 Steiner；设 `route.common.clock_topology` 为 `comb` 可改用 `comb`；
- 默认忽略已有全局布线；`-reuse_existing_global_route true` 可增量复用；
- 若已有详细布线则做增量详细布线；
- 默认最多 40 次 search-and-repair；`-max_detail_route_iterations` 可改。

注意

若违例已全部修复或判定无法再修，Zroute 会在达到最大迭代前提前停止。

5.10 关键网络布线

Routing Critical Nets

```
icc2_shell> route_group -nets collection_of_critical_nets
icc2_shell> route_group -from_file file_name
```

若网络关联 routing corridor，则在该区域内布线。全局布线视 corridor 为硬约束；轨道分配与详细布线视之为软约束。

默认忽略已有全局布线，复用 dangling wire 与已有详细布线。-reuse_existing_global_route true 做增量全局布线（与 corridor 不兼容）；-utilize_dangling_wires false 禁用 dangling 复用；-stop_after_global_route true 在全局布线后停止。

默认不修 soft DRC；可设：

```
icc2_shell> set_app_options \
-name route.common.post_group_route_fix_soft_violations \
-value true
```

5.11 次级电源与地 Pin 布线

Routing Secondary Power and Ground Pins

步骤：

1. 确认标准单元 frame view 中次级 PG pin 属性正确；
2. 设置布线约束；
3. 用 route_group 布线。

5.11.1 验证属性

Verifying the Secondary Power and Ground Pin Attributes

必需属性：is_secondary_pg=true；port_type=power 或 ground。

```
icc2_shell> report_attributes -application \
[get_lib_pins -of_objects reflib/cell/frame -all \
-filter "name == attr_name"]
```

5.11.2 设置约束并布线

Setting Constraints and Routing

可与信号布线类似设置层约束、单连接、route.common.connect_within_pins_by_layer_name、route.common.number_of_secondary_pg_pin_connections（簇内最大 pin 数，0 表示不限）、NDR 等。若要下次级 PG 与 tie-off 分离约束，设 route.common.separate_tie_off_from_secondary_pg 为 true。

Topology ECO 努力由 `route.detail.topology_eco_effort_level` 控制: `off` (默认) / `low` (1 次) / `medium` (4 次) / `high` (8 次)。

```
icc2_shell> route_group -nets VDD1
icc2_shell> add_buffer {TOP/U1001/Z} {libA/BUFHVT} \
    -new_cell_names mynewHVT
icc2_shell> legalize_placement
icc2_shell> connect_pg_net -automatic
icc2_shell> route_group -nets {VDD2}
```

5.12 信号网络布线

Routing Signal Nets

信号布线前，全部时钟网须已布线且无违例。方法：

- 独立命令: `route_global`、`route_track`、`route_detail` (适合定制流程或大块分步)；每条命令开始读入、结束更新 block，不检查输入数据完整性；
- 自动布线: `route_auto` (依次全局、轨道、详细；适合验证收敛、拥塞与 DRC QoR)。

信号布线可插入冗余 via (见「在信号网上插入冗余 Via」)。

5.12.1 全局布线

Global Routing

布线前设置 `route.common_options` 与 `route.global_options`。默认非时序驱动。全局路由器将 block 划分为 global routing cell (默认宽度等于标准单元高度，与 row 对齐)，按 blockage、pin、track 计算容量与需求，报告 overflow。

先进节点可开 via 密度建模：

```
set_app_options -name route.global.via_cut_modeling -value true
```

两阶段: phase 0 初始布线；随后多轮 rip-up/reroute 减拥塞。Effort: `minimum/low/medium` (默认，最多 3 轮) / `high` (4) / `ultra` (6)。`route.global.force_full_effort` 可强制跑满轮次。

结束后库中保存 g-link/g-via (供后续步骤: 轨道分配与详细布线后移除) 与拥塞数据。默认仅硬拥塞；`route.global.export_soft_congestion_maps=true` 可保存软拥塞。

设计规划阶段探索: `route_global -floorplan true`; 层次化可用 `-virtual_flat all_routing` 或 `top_and_interface_routing_only`。

时序驱动全局布线

Timing-Driven Global Routing

```
set_app_options -name route.global.timing_driven -value true
```

流程: 算延时 → phase0 → 两轮减拥塞 → 更新时序 → 识别新关键网 → 再多轮重布关键网。Effort 与 `route.global.advance_node_timing_driven_effort` (`low/medium/high`) 共同决定重布轮数 (见表 5.4)。`route.global.timing_driven_effort_level` 默认 `high`，偏重时序 QoR。

表 5.4: 时序驱动全局布线重布轮数（示意）

-effort_level	advance=low	medium	high（默认）
low	2	2	2
medium（默认）	3	3	5
high	4	4	6
ultra	6	6	8

串扰驱动与增量全局布线

Crosstalk-Driven and Incremental Global Routing

```
set_app_options -name route.global.crosstalk_driven -value true
set_app_options -name time.si_enable_analysis -value true
# 增量:
route_global -reuse_existing_global_route true
```

5.12.2 轨道分配

Track Assignment

须先完成全局布线。route_track 为各全局布线分配实际 track，并重布重叠导线。完成后不再保留全局布线。

注意

轨道分配用实际金属形状替换全局布线后，block 中不再包含全局布线。

时序/串扰驱动:route.track.timing_driven、route.track.crosstalk_driven(及 time.si_enable_analysis)。

5.12.3 详细布线

Detail Routing

须先完成全局布线与轨道分配。详细路由器按分区查找 DRC，rip-up/reroute 修复，并并发处理天线规则、优化 via 数与线长。

默认：整块布线（-coordinates 可限区域）；首轮均匀分区、后续按违例分布调整；最多 40 轮（-max_number_iterations）；非时序驱动。相关选项:route.detail.optimize_partition_size_for_drc、route.detail.drc_convergence_effort_level、route.detail.force_max_number_iterations、route.detail.timing_driven、route.common.rc_driven_setup_effort_level、route.detail.repair_shorts_over_macros_effort_level、route.common.post_detail_route_fix_soft_violations。

增量详细布线须用 -incremental true（否则从 iteration 0 重开）:

```
icc2_shell> route_detail -incremental true
```

注意

增量详细布线不修复 open net; 须用 ECO 布线 (route_eco)。

Early mode: set_qor_strategy -mode early -stage pnr。可用 route.detail.save_after_iterations 在指定轮次保存 block。route.detail.report_ignore_drc 可从汇总中排除特定 DRC。

5.12.4 自动布线

Routing Signal Nets by Using Automatic Routing

```
icc2_shell> route_auto
```

默认忽略已有全局布线, 顺序执行全局、轨道、详细, 步骤间不保存。选项: -reuse_existing_global_route、-stop_after_track_assignment、-max_detail_route_iterations、-save_after_global_route/-save_after_track、-save_cell_prefix。

时序驱动: 将 route.global.timing_driven、route.track.timing_driven、route.detail.timing_driven 均设为 true。串扰驱动: 另开 route.global.crosstalk_driven、route.track.crosstalk_driven 与 time.si_enable_analysis。

5.13 网络屏蔽

Shielding Nets

Zroute 按屏蔽规则中的宽度与间距生成屏蔽导线。除同层屏蔽外, 还可对上一层/下一层做同轴屏蔽 (coaxial shielding, 图 5.16), 隔离更好但更耗布线资源。

(原书 Figure 80: Coaxial Shielding)

图 5.16: 同轴屏蔽

- 布线前屏蔽 (preroute): 覆盖更好, 但可能造成信号布线拥塞; 常用于关键时钟;
- 布线后屏蔽 (postroute): 对可布线性影响极小, 保护较弱。

(原书 Figure 81: Zroute Shielding Flow)

图 5.17: Zroute 屏蔽流程

5.13.1 定义并分配屏蔽规则

Defining and Assigning Shielding Rules

用 `create_routing_rule` 的 `-shield_widths/-shield_spacings` 定义;时钟网用 `set_clock_routing_rules` (仅 CTS 前), 信号网用 `set_routing_rule`。建议仅对高频/关键时钟用屏蔽, 低频时钟用双倍间距。

5.13.2 布线前屏蔽

Performing Preroute Shielding

时钟树布线后、信号布线前, 用 `create_shields` 按规则加屏蔽。默认: 对所有有屏蔽规则且未 frozen 的网; 忽略短于 4 pitch 的导线 (`route.common.min_shield_length_by_layer_name` 可按层改); 屏蔽接到地网 (`-with_ground` 或 `route.common.shielding_nets`); 同层屏蔽。

同轴屏蔽: `-coaxial_above/-coaxial_below`, 配合 `-coaxial_above_skip_tracks` 等或 `-coaxial_skip_tracks_o` 或用户间距选项 (不可混用)。`-align_to_shape_end`、`-preferred_direction_only` 等可进一步控制。

```
icc2_shell> create_shields -nets $clock_nets \
    -coaxial_below true -coaxial_below_skip_tracks 0 \
    -with_ground VSS
```

命令报告基于网络与线长的平均屏蔽率。若与 `report_shields` 略有差异, 以 `report_shields` 为准 (通常 <3%, 最多约 5%)。

5.13.3 信号布线中的软屏蔽规则

Soft Shielding Rules During Signal Routing

有屏蔽规则但未预屏蔽的网, Zroute 在全局/轨道/详细全程预留屏蔽空间, 详细布线结束报告 soft spacing (shielding) 违例; 并不真正插入屏蔽, 须事后再跑 `create_shields`。仅预留同层屏蔽空间。若希望有 PG 可用时不预留, 设 `route.common.allow_pg_as_shield=true`。

5.13.4 布线后屏蔽与增量屏蔽

Postroute and Incremental Shielding

布线后同样用 `create_shields`。若用 `-nets` 且无 NDR, 则用 technology 默认宽间距。产生 DRC 时自动做最多 5 轮详细布线; 仍有违例可 `route_detail -incremental true`; 若破坏 tie-off, 改用 `route_eco`。

增量屏蔽: 将 `route.common.resield_modified_nets` 设为 `unshield` 或 `resield` (默认 off)。仅支持工具内修改的网络, DEF 外部修改不支持。

查询屏蔽形状: `get_shapes -shield_only`。报告: `report_shields (-per_layer true)`。布线中启用屏蔽检查: `route.common.check_shield=true`。

```
# 示例: 时钟预屏蔽 + 关键网后屏蔽
create_routing_rule shield_rule \
    -shield_widths {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3} \
    -shield_spacings {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3}
```

```
set_clock_routing_rules -clocks CLK -rules shield_rule
synthesize_clock_trees
route_group -all_clock_nets -reuse_existing_global_route true
create_shields -nets $clock_nets -with_ground VSS
set_routing_rule -rule shield_rule $critical_nets
route_auto
create_shields -nets $critical_nets -with_ground VSS
```

5.14 执行布线后优化

Performing Postroute Optimization

两类优化：

- 逻辑优化：改善时序/面积/功耗 QoR，修逻辑 DRC，并做合法化与 ECO 布线——route_opt；
- 可布线性优化：加大单元间距以修 pin access 引起的布线 DRC——optimize_routability。

若两者都跑，先逻辑后可布线性。

5.14.1 布线后逻辑优化

Performing Postroute Logic Optimization

前置：设计已完全布线并合法化，逻辑/布线 DRC 不过多。用 check_legality、check_routes 验证。推荐流程：

1. compute_clock_latency 更新时钟延时；
2. 运行两次 route_opt（第一次合法化/ECO 可能影响时序，第二次再优化）。

route_opt 步骤：提取并更新时序（默认 PrimeTime 延时引擎，需 O-2018.06 及以后 Library Manager 生成的库；可用 check_consistency_settings）→ 启用的优化 → 合法化 → ECO 布线。

默认优化 data path 的 setup/hold/面积/逻辑 DRC。额外优化见表 5.5。

表 5.5: 启用 route_opt 额外优化的应用选项

优化	应用选项
并发时钟与数据（CCD）及时钟逻辑 DRC	route_opt.flow.enable_ccd=true
时钟逻辑 DRC（未开 CCD 时）	route_opt.flow.enable_cto=true
时钟树面积回收	route_opt.flow.enable_clock_power_recovery=area
时钟树功耗回收	同上 = power
功耗优化	route_opt.flow.enable_power=true
机器学习路径优化	route_opt.flow.enable_ml_opto=true 且 PBA=path

注意

`route_opt` 仅遵从 `route_opt` 应用选项；除 `opt.common.allow_physical_feedthrough` 外不遵从 `opt` 选项。

PBA: `time.pba_optimization_mode=path` (推荐) 或 `exhaustive`。ECO 阶段默认 5 轮详细布线 (`route.detail.eco_max_number_of_iterations`)；`route_opt.eco_route.mode=track` 可改做轨道分配。

签核时序违例可用 `set_route_opt_target_endpoints (-setup_timing/-hold_timing)` 指定目标后跑 `route_opt`，再 `-reset`。Size-only 模式：`route_opt.flow.size_only_mode` 取 `true_footprint/footprint/equal`。

5.14.2 使用 `hyper_route_opt`

Performing Postroute Optimization Using `hyper_route_opt`

可用单次 `hyper_route_opt` 减少迭代次数；设置与 `route_opt` 相同。可选 Tcl 回调 `snps_hyper_route_opt_post_e` 控制 metal fill、PG augmentation 等。之后仍可用 `set_route_opt_target_endpoints+route_opt` 收尾，或 `route_detail -incremental` 修剩余布线 DRC。

5.14.3 修复 Pin Access 引起的 DRC

Fixing DRC Violations Caused by Pin Access Issues

```
optimize_routability
# 预览:
optimize_routability -check_drc_rules
# 完成后移除 keepout:
optimize_routability -remove_keepouts
```

分析邻接单元违例，在违例侧加 `keepout` (默认 site 宽，`-keepout_width` 可指定；`-flip` 可尝试翻转)。须随后 `route_eco` 或 `-route`；`route.common.eco_fix_drc_in_changed_area_only=on` 可仅在变更区修 DRC。

5.15 分析并修复信号电迁移违例

Analyzing and Fixing Signal Electromigration Violations

信号电迁移由线宽缩小与高速开关导致电流密度升高引起，可造成短路或开路。流程：

1. `read_signal_em_constraints` 加载约束 (ITF 或 ALF)；
2. `read_saif` 或 `set_switching_activity` 标注开关活动；
3. `report_signal_em` 分析；
4. `fix_signal_em` 修复。

```
open_block block1_routed
read_signal_em_constraints em.itf
read_saif block1.saif
```

```
report_signal_em -violated -verbose > block1.signal_em.rpt
fix_signal_em
```

ITF: -encrypted 可读加密文件。ALF: -format ALF。约束以查找表存入设计库。

分析计算 average / RMS / peak 电流。相关应用选项见表 5.6（如 em.net_delta_temperature、em.net_global_rms_relaxation_factor、em.net_min_duty_ratio、em.net_use_waveform_duration、em.net_violation_rule_types 等）。

表 5.6: 信号电迁移分析应用选项（节选）

选项	默认	说明
em.net_delta_temperature	5.0	RMS 查找用 ΔT
em.net_duration_condition_for_peak	0.0	peak 检查 duration
em.net_global_rms_relaxation_factor	1.0	RMS 全局松弛因子
em.net_use_waveform_duration	false	是否用波形 duration

fix_signal_em 通过加宽金属的 NDR 并做 ECO 布线修复；每调用一轮，有残留须再跑。可用 -nets 限定。

5.16 执行 ECO 布线

Performing ECO Routing

修改网络后须用 route_eco 重连，顺序执行全局、轨道、详细布线。默认考虑时序与串扰（会提取并更新时序）；若不需要可关闭各引擎的 timing/crosstalk_driven 选项以加速。

默认：处理全部 open net（-nets 可限定）；忽略已有全局布线（-reuse_existing_global_route true 可增量）；复用 dangling wire（-utilize_dangling_wires false 可关）；最多 40 轮详细布线；在整块修硬 DRC。

- -open_net_driven true: 仅在 open net 邻域修 DRC；
- -reroute modified_nets_only 或 modified_nets_first_then_others；
- route.common.post_eco_route_fix_soft_violations=true 修 soft DRC。

报告变更网络时默认前 100 条；-max_reported_nets -1 报告全部。

5.17 在 GUI 中布线

Routing Nets in the GUI

在活动 layout 视图用 Create Route 工具交互布线（Edit 工具栏按钮，或 **Create > Route**）。默认忽略 routing blockage（可在 Mouse Tool Options 中改为遵从）；使用 technology 宽间距而忽略 NDR（同样可在面板中开启遵从 NDR）。

注意

Create Route 工具不遵从 `create_routing_guide` 定义的 routing guide；也不使用 NDR 中的屏蔽宽/间距。

修改已布线网络的 GUI 工具：Area Push、Spread Wires、Stretch Connected、Quick Connect。面板上的 Help 可打开 man 页。

5.18 清理已布线网络

Cleaning Up Routed Nets

```
icc2_shell> remove_redundant_shapes
icc2_shell> remove_redundant_shapes -nets my_net \
    -remove_loop_shapes true
```

默认基于设计视图中已存 DRC 信息移除 dangling/floating 形状；`-initial_drc_from_input false` 则先跑 `check_routes`。可用 `-nets/-layers/-route_types` 限定。不改拓扑、不碰 terminal；不处理 open net 或有 DRC 的网。`-remove_dangling_shapes false`、`-remove_floating_shapes false`、`-remove_loop_shapes true`、`-report_changed_nets true`。清理后应重新做 DRC 验证。

5.19 分析布线结果

Analyzing the Routing Results

5.19.1 拥塞报告与拥塞图

Generating a Congestion Report and Map

```
icc2_shell> report_congestion
```

默认用库中拥塞图；若无则跑 `congestion-map-only` 全局布线。选项：`-rerun_global_router`、`-boundary`、`-layers`、`-mode global_route_cell_edge_based`、`-include_soft_congestion_maps`。

GUI: **View > Map > Global Route Congestion**。图 5.18、图 5.19 示意拥塞图与 GRC 上的 overflow（如 18/9 表示需求 18、供给 9）。

（原书 *Figure 82: Global Route Congestion Map*）

图 5.18: 全局布线拥塞图

拥塞计算模式: 各层 overflow 之和（默认，忽略 underflow），或总 demand 减总 supply（含 underflow，结果更乐观）。

(原书 *Figure 83: Global Route Overflow on Global Routing Cell*)

图 5.19: Global Routing Cell 上的 Overflow

5.19.2 用 Zroute 做设计规则检查

Performing Design Rule Checking Using Zroute

```
icc2_shell> check_routes
```

默认检查布线 DRC、open net、天线与 voltage area 违例（排除 user/frozen/PG 网）。可用 `-nets`、`-check_from_user_shapes`、`-check_from_frozen_shapes`、`-drc/-open_net/-antenna/-voltage_area` 可分别关闭。`-coordinates` 做区域检查（与 `-nets` 互斥；区域检查不做 open/天线/tie-to-rail）。

错误写入 `zroute.err`。可用 DRC query 命令或 error browser 分析。基于 `check_routes` 结果的增量详细布线：

```
icc2_shell> route_detail -incremental true \
    -initial_drc_from_input true
```

5.19.3 签核 DRC、外部工具与 LVS

Signoff DRC, External Tools, and LVS

- 签核: `signoff_check_drc` / `signoff_fix_drc`（需 IC Validator 许可证）；
- 外部 Calibre: `read_drc_error_file` 导入扁平 Calibre 错误文件；
- LVS: `check_lvs` 检查 short/open/floating（`-checks`、`-nets`、`-max_errors`、`-treat_terminal_as_voltage_source`、`-open_reporting_detailed`、`-report_floating_pins` 等）。

(原书 *Figure 84: Layout With Power and Ground Terminals*)

图 5.20: 含电源/地 Terminal 的版图

5.19.4 报告布线结果与线长

Reporting Routing Results and Wire Length

`report_design -routing`: 各层信号/PG 形状数与线长、HV 分布、via 统计与 double via 转换率等（`-hierarchical` 可含整棵物理层次）。

`report_wirelength`: 按层与方向汇总线长（含 global/detail、Virtual、BestAvailable；支持斜线）。

5.19.5 DRC 查询命令

Using the DRC Query Commands

```
icc2_shell> get_drc_error_data -all
icc2_shell> open_drc_error_data zroute.err
icc2_shell> get_drc_error_types -error_data zroute.err
icc2_shell> get_drc_errors -error_data zroute.err \
  -filter {type_name == "Diff net spacing"}
```

属性示例: `type_name`、`bbox`、`objects`、`shape`。完整列表: `list_attributes -application -class drc_error`。

5.20 保存布线信息

Saving Route Information

```
icc2_shell> write_routes
```

生成可复现金属形状与 via(含属性)的 Tcl 脚本;不包含 routing blockage(后者用 `write_floorplan`)。

5.21 推导掩模颜色

Deriving Mask Colors

布线命令通常会分配 mask 颜色。对未着色形状,用 `derive_mask_constraint` 从最近底层 track 推导(宽线占多 track 时取与下一空闲 track 相反颜色)。

```
icc2_shell> derive_mask_constraint \
  [get_shapes -of_objects [get_nets net1]]
icc2_shell> derive_mask_constraint -derive_cut_mask \
  [get_vias -within {{600 600} {700 700}}]
icc2_shell> derive_mask_constraint -follow_pin_mask \
  [get_shapes -of_objects [get_nets net1]]
icc2_shell> derive_mask_constraint -follow_wider_object_mask \
  [get_shapes -of_objects [get_nets net1]]
```

参见

第一章「多重图案化概念」。

5.22 插入与移除 Cut Metal 形状

Inserting and Removing Cut Metal Shapes

Cut metal 用于双图案化设计以减小线端最小间距。Cut metal 层及尺寸在 technology file 的 Layer 段由 `cutMetalWidth`、`cutMetalHeight/cutMetalExtension` 定义;与同 mask 号金属层对应(如 `cut-Metal1` ↔ `metal1`)。

布线定稿后：

```
icc2_shell> create_cut_metals  
icc2_shell> remove_cut_metals
```

插入条件：优选方向间距等于 `cutMetalWidth`、该处尚无 `cut metal`、金属层有对应 `cut metal` 层。
可插在 `net-net`、`net-PG`、`PG-obstruction` 及子单元内金属之间，并传播颜色。

写出 DEF 须 `write_def -version 5.8`；写出 GDSII/OASIS 须 `write_gds/write_oasis` 的 `-connect_below_cut_m`
布线变更后须先 `remove_cut_metals` 再重新插入。

Chapter 6

芯片收尾与可制造性设计

Chip Finishing and Design for Manufacturing

IC Compiler II 工具提供芯片收尾（chip finishing）以及可制造性设计（DFM）与良率（yield）相关能力，可在设计流程各阶段应用，以应对芯片制造中的工艺设计问题。

有关芯片收尾与可制造性设计功能，请参阅以下主题：

- 插入 Tap Cell（Inserting Tap Cells）
- 执行 Boundary Cell 插入（Performing Boundary Cell Insertion）
- 查找并修复天线违例（Finding and Fixing Antenna Violations）
- 插入冗余 Via（Inserting Redundant Vias）
- 优化线长与 Via 数量（Optimizing Wire Length and Via Count）
- 减小临界面积（Reducing Critical Areas）
- 插入金属-绝缘体-金属电容（Inserting Metal-Insulator-Metal Capacitors）
- 插入 Filler Cell（Inserting Filler Cells）
- 插入 Metal Fill（Inserting Metal Fill）

6.1 插入 Tap Cell

Inserting Tap Cells

Tap cell 是带有 well tie、substrate tie 或二者兼有的特殊非逻辑单元。当库中多数或全部标准单元不含 substrate/well tap 时，通常需要使用此类单元。一般设计规则会规定标准单元内每个晶体管到 well 或 substrate tap 的最大允许距离。

在全局布局之前（floorplanning 阶段），可在 block 中插入 tap cell，形成二维阵列结构，以确保后续放置的标准单元均满足最大 diffusion-to-tap 距离限制。插入后应目视检查，确认所有标准单元可布局区域均已被 tap cell 妥善保护。

IC Compiler II 提供两种插入 tap cell 阵列的方法：

- 基于 site row 以及 tap cell 之间的最大距离插入 tap cell 阵列
使用 `create_tap_cells` 命令，见「使用 `create_tap_cells` 命令」。
- 使用指定的 offset 与 pitch 插入 tap cell 阵列，并可选择在边界附近额外插入 tap cell
使用 `create_cell_array` 命令，见「使用 `create_cell_array` 命令」。

先进工艺节点常需额外插入 tap cell，以管理 substrate 与 well 噪声。在标准 tap cell 插入之后，还可使用以下能力：

- 插入 tap wall
Tap wall 是一行或一列无间隙线性排列的 tap cell。可在 block、macro 或 hard placement blockage 边界之外或之内插入。边界外的称为 exterior tap wall；边界内的称为 interior tap wall。
 - 插入 exterior tap wall：使用 `create_exterior_tap_walls`，见「插入 Exterior Tap Wall」。
 - 插入 interior tap wall：使用 `create_interior_tap_walls`，见「插入 Interior Tap Wall」。
- 插入 tap mesh
Tap mesh 是二维 tap cell 阵列，每个 mesh window 中有一个 tap cell。使用 `create_tap_meshes`，见「插入 Tap Mesh」。
- 插入 dense tap array
Dense tap array 的 tap 间距小于 `create_tap_cells` 所用间距。使用 `create_dense_tap_cells`，见「插入 Dense Tap Array」。

6.1.1 使用 `create_tap_cells` 命令

Using the `create_tap_cells` Command

要基于 site row 添加 tap cell 阵列，使用 `create_tap_cells`。必须指定用于插入的库单元名（`-lib_cell`）以及 tap cell 之间的最大距离（微米，`-distance`）。

例如：

```
icc2_shell> create_tap_cells -lib_cell myreflib/mytapcell -distance 30
```

注意

若应用选项 `place.legalize.enable_pin_color_alignment_check` 为 `true`（默认 `false`），命令在插入时会确保 tap cell 内部 pin 与金属对齐到相应颜色的 routing track，并移动 tap cell 以避免颜色对齐违例。为避免因移动导致 signoff DRC 错误，在该应用选项为 `true` 时应略微减小 tap 距离。

默认情况下，`create_tap_cells` 在整个 block 的每一行插入 tap cell。工具从行左边缘开始，按指定 tap 距离确定后续位置。此外，若在相邻于 block 边界、hard macro 或 hard placement blockage 的每个行边缘的最小 tap 距离（指定距离的一半）内不存在 tap cell，工具会额外插入一个 tap cell。图 6.1 给出采用 every-row 插入模式时的默认布局；注意 hard macro 右侧会额外加入 tap cell，以确保距 hard macro 边缘在最小 tap 距离内存在 tap cell。

可修改以下默认行为方面：

- 镜像（MX）行所用库单元
使用 `-mirrored_row_lib_cell` 指定镜像行所用库单元。

(原书 *Figure 85: Default Tap Cell Placement*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 6.1: 默认 Tap Cell 布局

- 插入图案

使用 `-pattern` 指定以下之一：

- `every_row` (默认): 每行都插入。此时 `-distance` 应约为工艺设计规则中最大 `diffusion-to-tap` 值的两倍。
- `every_other_row`: 仅在奇数行插入。`-distance` 应约为最大 `diffusion-to-tap` 值的两倍。
- `stagger`: 每行都插入, 偶数行相对奇数行偏移半个 `-offset`, 形成类似棋盘的图案。`-distance` 应约为最大 `diffusion-to-tap` 值的四倍。

- 相对行左边缘的偏移

使用 `-offset` 将图案起点向右平移指定微米距离。

- 固定单元的处理

默认在计算出的 `tap` 位置插入, 不论该处是否被固定单元占用。使用 `-skip_fixed_cells` 可避免在固定单元位置插入。默认将该固定单元视为 `blockage` 并在该处打断行, 从而可能在固定单元两侧各插入一个 `tap cell`。若要不打断行又避免重叠, 可同时使用 `-preserve_distance_continuity` 与 `-skip_fixed_cells`: 命令会平移 `tap cell` 以避免重叠; `-preserve_distance_continuity` 仅平移那些计算位置被所列库单元实例占用的 `tap cell`。

例如, 下列命令会平移计算位置被 `mylib/cell1` 固定实例占用的 `tap cell`:

```
icc2_shell> create_tap_cells -lib_cell mylib/tap -distance 30 \
-skip_fixed_cells -preserve_distance_continuity mylib/cell1
```

使用 `-skip_fixed_cells` 时, 还可使用 `-min_horizontal_periphery_spacing` 指定边界与位于 `blockage` 上方或下方的角部 `tap cell` 之间的最小水平间距 (以 `unit tile` 数为单位)。图 6.2 示出有无 `boundary cell` 时该选项对布局的影响。

- 额外 `tap cell` 的添加

可用下列一种或两种方法阻止额外插入：

- `-row_end_tap_bypass`: 声明 `boundary cell` 已含 `tap`, 故边界不再额外插入。注意命令不会验证 `block` 是否确有 `boundary cell`, 或其是否真含 `tap`。
- `-at_distance_only`: 仅允许在指定 `tap` 距离、半距离或四分之一距离 (仅 `stagger`) 处插入。注意使用该选项可能导致 DRC 违例。

- 插入区域

使用 `-voltage_area` 将插入限制在指定 `voltage area`。

(原书 *Figure 86: Tap Cell Boundary Row Spacing*)

请参阅原版 PDF 插图。

图 6.2: Tap Cell 边界行间距

- 命名约定

默认命名: `tapfiller!library_cell_name!number`。

用 `-separator` 可更改默认分隔符 “!”。

用 `-prefix` 可为特定运行加前缀, 命名变为: `tapfiller!prefix!library_cell_name!number`。

6.1.2 使用 `create_cell_array` 命令

Using the `create_cell_array` Command

要按特定布局添加 tap cell 阵列, 使用 `create_cell_array`。必须指定库单元名 (`-lib_cell`) 以及 x 、 y 方向的 pitch。

例如:

```
icc2_shell> create_cell_array -lib_cell myreflib/mytapcell \
-x_pitch 10 -y_pitch 10
```

默认在整个 block 的指定网格每个位置插入 tap cell。工具从 core 区域左下角开始, 按 x/y pitch 确定后续位置。插入的 tap cell 会 snap 到 site row, 并以该行默认方向放置。若位置被固定单元、hard macro、soft macro 或 power-switch cell 占用, 则不在该处插入。

可修改以下默认行为:

- 插入区域
 - `-voltage_area`: 限制到指定 voltage area。
 - `-boundary`: 限制到指定区域。
 若两者同时使用, 仅在重叠区域插入。
- 相对插入区域边界的偏移
 - `-x_offset`: 起点向右平移 (微米)。
 - `-y_offset`: 起点向上平移 (微米)。
- 插入图案
 - `-checkerboard`: 在网格每隔一个位置插入。
 - `-checkerboard even`: 左下角放置 tap cell。
 - `-checkerboard odd`: 左下角留空。

某些设计方法要求在棋盘图案中位于特定单元 (如 critical area breaker 或 boundary cell) 上方或下方的空位也插入 tap cell。可用 `-preserve_boundary_row_lib_cells` 指定相关库单元以启用

这些额外插入。

- **Snap** 行为

设置 `-snap_to_site_row false` 可禁用 snap 到 site row；此时默认方向为 R0。

- 方向

用 `-orient` 指定：R0、R90、R180、R270、MX、MY、MXR90 或 MYR90。

- 命名约定

默认：headerfooter__library_cell_name_R#_C#_number。

使用 `-prefix` 时：prefix__library_cell_name_R#_C#_number。

6.1.3 插入 Exterior Tap Wall

Inserting Exterior Tap Walls

Exterior tap wall 是放置在 block、macro 或 hard placement blockage 边界之外的一行或一列无间隙 tap cell。

使用 `create_exterior_tap_walls`，必须指定：

- **Tap wall 所用 tap cell** (`-lib_cell`)

默认以 R0 方向插入；可用 `-orientation` 指定其他方向。

若设计有 FinFET grid，选择 tap cell 时请遵循：

- 水平 tap wall：指定方向下的单元宽度须为 FinFET grid *x*-pitch 的整数倍；
- 垂直 tap wall：指定方向下的单元高度须为 FinFET grid *y*-pitch 的整数倍。

默认命名：extTapWall__library_cell_name_R#_C#_number。

使用 `-prefix` 时：prefix__library_cell_name_R#_C#_number。

- 插入侧 (`-side`)

每次只能指定一侧：top、bottom、right 或 left。多侧需多次运行。

- 插入区域 (`-bbox`)

指定包围盒左下与右上角：

```
-bbox {{llx lly}} {urx ury}}
```

包围盒必须覆盖指定侧上对象边缘的一部分。若覆盖该侧多条边，则沿最长边插入；若未覆盖任何边，则报错。

默认情况下，水平 tap wall 从包围盒左边缘开始并贴齐顶/底边界；垂直 tap wall 从包围盒底边缘开始并贴齐左/右边界。可用 `-x_margin` 与 `-y_margin` 指定边距。若含边距后的区域过小，则不插入。应确保插入区域不含 placement blockage、固定单元或 macro，否则可能导致 tap wall 不完整或缺失。

插入时命令不遵循标准单元 row/site，也不跨越对象边界。插入后会固定所插 tap cell 的布局。

6.1.4 插入 Interior Tap Wall

Inserting Interior Tap Walls

Interior tap wall 是放置在 block、macro、hard placement blockage 或非默认 voltage area 的标准单元布局区域边界之内的一行或一列 tap cell。

注意

仅可对具有单一 site 定义的设计插入 interior tap wall。

使用 `create_interior_tap_walls`，必须指定：

- **Tap cell** (`-lib_cell`)

须为与行高、site 宽匹配的单高 (`single-height`) 单元。

默认按行方向插入；可用 `-orientation` 更改。

默认命名：`tapfiller__library_cell_name_R#_C#_number`。

使用 `-prefix` 时：`prefix__library_cell_name_R#_C#_number`。

- 插入侧 (`-side`)

每次仅一侧；多侧需多次运行。

默认沿标准单元可布局区域指定侧的所有边无间隙放置。

- 用 `-bbox` 指定插入区域包围盒。命令沿包围盒所涵盖可布局区域在指定侧的最长边插入。对齐取决于该边段所含角点数量：

- 无角点：水平 wall 从左边、垂直 wall 从底边开始；

- 一个角点：从该角点开始；

- 指定侧上两个角点：水平从左边、垂直从底边开始。对含两角的水平 wall，末两单元间距可能缩短以对齐右角；空间不足时末单元可能不贴齐右角。

- 对水平 tap wall，可用 `-x_spacing` 指定单元间隙。若小于单元宽度则间距为 0；若非 site 宽整数倍则向下取整到 site 宽倍数。

插入时遵循标准单元 row 与 site；未定义则不插入。Placement blockage、固定单元与 macro 等障碍可能导致不完整或缺失的 tap wall。插入后固定布局。

6.1.5 插入 Tap Mesh

Inserting Tap Meshes

Tap mesh 是每个 mesh window 含一个 tap cell 的二维阵列。通常插在 P&R block 之外，也可插在完全位于 core 区域内或完全位于 core 区域外的任意区域。

使用 `create_tap_meshes`，必须指定：

- **Tap cell** (`-lib_cell`)

默认 R0；可用 `-orientation`。

默认命名：`headerfooter__library_cell_name_R#_C#_number`。

使用 `-prefix` 时：`prefix__library_cell_name_R#_C#_number`。

- 插入区域 (`-bbox`): `{{llx lly} {urx ury}}`

- **Mesh window** (`-mesh_window`)

以 x 、 y 方向 pitch 的整数值指定窗口大小：`-mesh_window {x_pitch y_pitch}`。

Mesh window 与插入区域左下角对齐。

插入时不遵循标准单元 row/site。在每个 mesh window 左下角放置 tap cell；若被阻挡，则取到该左下角曼哈顿距离最近的可用位置；若无可用位置则警告并跳过该窗口。插入后固定布局。

6.1.6 插入 Dense Tap Array

Inserting Dense Tap Arrays

Dense tap array 的 tap 间距小于 create_tap_cells 所用间距。

使用 create_dense_tap_cells，必须指定：

- **Tap cell** (-lib_cell)
须与运行 create_tap_cells 时相同。
默认命名：tapfiller!library_cell_name!number；可用 -separator、-prefix。
- **Tap 距离** (-distance)
必须小于 create_tap_cells 所用距离。
- **插入区域** (-bbox)

命令首先移除完全位于插入区域内的既有 tap cell，再按指定距离重新插入以形成更密阵列。为避免 tap 规则违例，应使用与 create_tap_cells 相同的插入图案。支持 -offset、-pattern、-skip_fixed_cells、-at_distance_only（详见 man 页）。插入后固定布局。

6.2 执行 Boundary Cell 插入

Performing Boundary Cell Insertion

在放置标准单元之前，可为 block 添加 boundary cell。Boundary cell 包括：加在单元行两端以及 core 区域、hard macro、blockage、voltage area 等对象边界周围的 end-cap cell；以及填充水平与垂直 end-cap 之间空隙的 corner cell。End-cap cell 通常为非逻辑单元（如电源轨去耦电容）。因工具接受任意标准单元作为 end-cap，须确保指定合适的单元。

插入 boundary cell 的步骤：

1. 用 set_boundary_cell_rules 指定插入要求（见「指定 Boundary Cell 插入要求」）。
2. 用 compile_boundary_cells 或 compile_targeted_boundary_cells 按规则插入（见「插入 Boundary Cell」）。
3. 用 check_boundary_cells 或 check_targeted_boundary_cells 验证布局（见「验证 Boundary Cell 布局」）。

每次运行仅支持单一 site 定义。compile_boundary_cells 与 compile_targeted_boundary_cells 根据 set_boundary_cell_rules 所指定库单元确定 site，并仅在使用该 site 定义的区域插入。若设计有多种 site 定义，须对每种各运行一次该流程。

6.2.1 指定 Boundary Cell 插入要求

Specifying the Boundary Cell Insertion Requirements

使用 set_boundary_cell_rules 指定：

- 用于 boundary cell 的库单元（见「指定 Boundary Cell 插入所用库单元」）；
- 放置规则（见「指定 Boundary Cell 放置规则」）；
- 插入单元的命名约定（见「指定 Boundary Cell 命名约定」）；
- 创建 routing guide 以遵守 metal cut allowed/forbidden preferred grid extension 规则（见「Boundary Cell 插入期间创建 Routing Guide」）；

注意

该功能仅由 `compile_boundary_cells` 支持，`compile_targeted_boundary_cells` 不支持。

- 创建 placement blockage 以遵守 minimum jog / minimum separation 规则（见「Boundary Cell 插入期间创建 Placement Blockage」）。

注意

该功能仅由 `compile_boundary_cells` 支持，`compile_targeted_boundary_cells` 不支持。

注意

`set_boundary_cell_rules` 指定的设置保存在设计库中。

参见

报告 Boundary Cell 插入要求：见「报告 Boundary Cell 插入要求」。

指定 **Boundary Cell** 插入所用库单元

Specifying the Library Cells for Boundary Cell Insertion

Boundary cell 包括左右上下边界的 end-cap，以及内/外角单元。可用 `set_boundary_cell_rules` 的下列选项为各类型指定不同库单元：

- `-left_boundary_cell`、`-right_boundary_cell`：分别指定左、右边界 end-cap 所用的单个库单元。
- `-top_boundary_cells`、`-bottom_boundary_cells`：分别指定顶、底边界 end-cap 的库单元列表。按指定顺序插入；若剩余空间小于当前单元，则插入列表中下一个能放入剩余空间的单元。
对垂直行 block，行自底向顶，故顶边界沿 block 左侧、底边界沿右侧。
对 double-back block 中的翻转行，顶边界单元用于底边界，底边界单元用于顶边界。
- `-top_tap_cell`、`-bottom_tap_cell`：分别指定顶、底边界行上的 tap cell，以确保端 cap 满足最大 diffusion-to-tap 限制。
- `-top_left_outside_corner_cell`、`-top_right_outside_corner_cell`、`-bottom_left_outside_corner_cell`、`-bottom_right_outside_corner_cell`：各外角位置的单个库单元。
- `-top_left_inside_corner_cells`、`-top_right_inside_corner_cells`、`-bottom_left_inside_corner_cells`、`-bottom_right_inside_corner_cells`：各内角位置的库单元列表。工具插入第一个与内角尺寸匹配的角单元；若无一精确匹配，则插入第一个可不违规则放置的单元。

图 6.3 示出含两个 hard macro/blockage 的水平 block 上 boundary 与 corner 单元位置；行翻转时位置也会翻转。

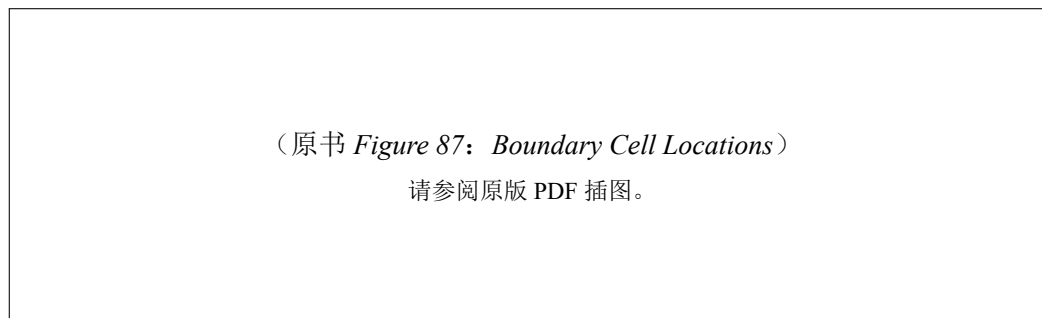


图 6.3: Boundary Cell 位置

指定 **Boundary Cell** 放置规则

Specifying Boundary Cell Placement Rules

默认：

- 运行 `compile_boundary_cells` 时，工具以默认方向在 `core` 区域、`hard macro` 与 `hard placement blockage` 周围放置 `boundary cell`；
- 运行 `compile_targeted_boundary_cells` 时，工具以默认方向在指定对象周围放置。

要翻转边界单元方向，可在 `set_boundary_cell_rules` 中使用一个或多个选项：`-mirror_left_boundary_cell`、`-mirror_right_boundary_cell`、`-mirror_left_outside_corner_cell`、`-mirror_right_outside_corner_cell`、`-mirror_left_inside_corner_cell`、`-mirror_right_inside_corner_cell`、`-mirror_left_inside_horizontal_abutment_cell`、`-mirror_right_inside_horizontal_abutment_cell`。不能翻转顶/底边界单元方向。

使用 `compile_boundary_cells` 时，还可修改：

- 翻转行上顶/底内角单元互换
`-do_not_swap_top_and_bottom_inside_corner_cell`：阻止在翻转行顶内角使用底内角单元、在底内角使用顶内角单元。
- 考虑 **voltage area**
`-at_va_boundary`：在 `voltage area` 边界两侧插入水平 `boundary cell`。
- 一单位 **tile** 间隙
`-no_1x`：防止插入产生一单位 `tile` 间隙。当行长等于两倍角单元宽加一单位 `tile` 宽时不插入；若等于两倍角单元宽则仍插入。
- **Tap cell** 间距
`-tap_distance`：顶/底边界行上 `tap cell` 间距（微米）。
- 短行上的插入
`-min_row_width`：行宽小于该阈值时不插入。
- 子 **block** 中的插入
`-insert_into_blocks`：递归在子 `block` 中插入（默认不插入）。

指定 **Boundary Cell** 命名约定

Specifying the Naming Convention for Boundary Cells

默认命名: `boundarycell!library_cell_name!number`。

用 `-separator` 更改分隔符;用 `-prefix` 加前缀后为:`boundarycell!prefix!library_cell_name!number`。

6.2.2 Boundary Cell 插入期间创建 Routing Guide

Creating Routing Guides During Boundary Cell Insertion

若 technology file 定义了 metal cut allowed 与 forbidden preferred grid extension 规则,可用 `-add_metal_cut_allowed` 在插入期间创建相应 routing guide。此时 `compile_boundary_cells` 会创建:

- Metal cut allowed routing guide: 覆盖全部可布局 site row 所占区域, 再按垂直收缩因子缩小 (为最小 site row 高度的 50%);
- Forbidden preferred grid extension routing guide: 覆盖直至 block 边界的剩余区域。

注意

仅 `compile_boundary_cells` 支持; `compile_targeted_boundary_cells` 不支持。

6.2.3 Boundary Cell 插入期间创建 Placement Blockage

Creating Placement Blockages During Boundary Cell Insertion

若工艺有 minimum jog 或 minimum separation 要求, 可在 `set_boundary_cell_rules` 中配合下列选项, 由 `compile_boundary_cells` 创建 placement blockage:

- `-min_horizontal_jog`: 若 macro (含 hard keepout)、hard placement blockage 或 voltage area (含 guard band) 的水平边小于指定值, 则用 `compile_boundary_cells -add_placement_blockage` 创建 blockage 以防违例。
- `-min_vertical_jog`: 对垂直边同理。
- `-min_horizontal_separation`: 若连续水平布局区域小于指定值, 则创建 blockage。
- `-min_vertical_separation`: 对垂直方向同理。

注意

默认 `compile_boundary_cells` 会检查指定规则但不创建 blockage; 须使用 `-add_placement_blockage` 才会创建。该功能仅 `compile_boundary_cells` 支持。

6.2.4 报告 Boundary Cell 插入要求

Reporting the Boundary Cell Insertion Requirements

使用 `report_boundary_cell_rules` 报告由 `set_boundary_cell_rules` 指定的要求。仅报告用户指定设置, 不报告默认值。

6.2.5 移除 Boundary Cell 插入要求

Removing Boundary Cell Insertion Requirements

使用 `remove_boundary_cell_rules` 移除一项或多项由 `set_boundary_cell_rules` 指定的要求。

6.2.6 插入 Boundary Cell

Inserting Boundary Cells

按插入目标选择命令：

- 在 core 区域、hard macro 与 placement blockage 周围插入： `compile_boundary_cells`;
- 仅在一个或多个 voltage area 周围： `compile_boundary_cells -voltage_area`;
- 仅在特定对象周围： `compile_targeted_boundary_cells -target_objects` (可针对特定 macro、voltage area、voltage area shape、placement/routing blockage 与 core 区域)。

`set_boundary_cell_rules` 同时配置两种 compile 命令；部分规则仅被 `compile_boundary_cells` 遵守，详见「指定 Boundary Cell 插入要求」。

6.2.7 验证 Boundary Cell 布局

Verifying the Boundary Cell Placement

用 `compile_boundary_cells` 插入后，用 `check_boundary_cells` 验证。用 `-error_view` 可生成供 error browser 查看的错误数据文件。该命令检查：

- 缺失的 **boundary** 或 **corner cell**

验证整个边界周围有 boundary/corner cell 且无间隙。图 6.4 示出有效布局及因缺失单元导致的错误。

注意

对成熟节点，boundary cell 可能仅插在左、右侧；此时命令只验证这两侧无间隙。

- 不正确的 **boundary/corner cell**

验证所用库单元与 `set_boundary_cell_rules` 指定一致。

- 不正确的方向

验证各 boundary cell 方向符合允许方向。

- 短行与短边

验证每行 boundary cell 宽于 `-min_row_width`，且各 blockage 水平边大于 `-min_horizontal_jog`。

(原书 *Figure 88: Check for Missing Boundary or Corner Cells*)

请参阅原版 PDF 插图。

图 6.4: 检查缺失的 Boundary/Corner Cell

6.3 查找并修复天线违例

Finding and Fixing Antenna Violations

芯片制造中，栅氧易被静电放电损伤。多层金属化过程中导线累积的静电荷可能损坏器件或导致芯片完全失效。静电电荷泄放入器件的现象称为天线（antenna）或电荷收集天线问题。

为防止天线问题，工具验证每个输入 pin 的金属天线面积除以栅极面积小于晶圆厂给出的最大天线比：

$$\frac{\text{antenna-area}}{\text{gate-area}} < \text{max-antenna-ratio}$$

该检查基于 block 的全局与层相关天线规则，以及单元的天线属性。

天线流程步骤：

1. 定义天线规则；
2. 指定 pin 与 port 的天线属性；
3. 分析并修复天线违例。

6.3.1 定义天线规则

Defining Antenna Rules

天线规则定义如何计算 block 中网络的最大天线比，以及 pin 的天线比。必须用 `define_antenna_rule` 定义全局天线规则；也可用 `define_antenna_layer_rule` 指定层相关规则。无层规则时使用全局规则。

相关主题：

- 计算最大天线比
- 计算 Pin 的天线比

计算最大天线比

Calculating the Maximum Antenna Ratio

须指定：

- 二极管提供多少保护（diode protection mode）——`define_antenna_rule` 的 `-diode_mode`（必选），见「设置 Diode Protection Mode」；

- 在二极管保护下如何计算天线比(diode ratio vector)——define_antenna_layer_rule 的 -diode_ratio (必选), 见「指定 Diode Ratio Vector」。

设置 Diode Protection Mode

Setting the Diode Protection Mode

Diode protection mode 规定二极管提供的保护程度。先进工艺设计常使用不同 gate oxide class 表示多种栅氧厚度。工具最多支持四种: gate class 0、1、2、3。无 gate class 数据的 pin 视为 class 0。对使用多种栅氧厚度的设计, 须使用支持多厚度的模式: 14、16、18。

用 define_antenna_rule -diode_mode 指定 (必选)。表 6.1 定义各模式如何影响网络最大天线比; 各二极管最大天线比的计算公式见表 6.3, 其中使用 -diode_ratio 向量值。

表 6.1: Diode Protection Mode 设置 (原书 Table 28)

模式	定义
0	二极管不提供任何保护。
1	二极管提供无限保护。此模式下最高金属层不检查天线违例, 因为最高层形成时输入-输出连接已完成。
2	保护有限; 多二极管时取各二极管最大天线比的最大值。
3	保护有限; 多二极管时取各二极管最大天线比之和。
4	保护有限; 多二极管时用各二极管 diode-protection 值之和计算最大天线比。
5	保护有限; 用所有二极管的最大 diode-protection 值计算等效栅极面积。
6	保护有限; 用所有二极管 diode-protection 值之和计算等效栅极面积。
7	保护有限; 用最大 diode-protection 值计算等效金属面积。
8	保护有限; 用 diode-protection 值之和计算等效金属面积。
9	保护有限; 按最大二极管保护做缩放。
10	保护有限; 按总二极管保护做缩放。
11	保护有限; 按最大二极管保护做缩放。
12	保护有限; 按总二极管保护做缩放。
13	已废弃。
14	保护有限; 两套独立计算公式。支持 gate class。
15	仅内部使用。
16	保护有限; 多二极管时用 diode-protection 值之和计算最大天线比。支持 gate class。
17	保护有限; 用 diode-protection 值之和计算等效栅极面积; 输入与输出 pin 分别计算。
18	保护有限; 多二极管时用 diode-protection 值之和计算最大天线比。支持 gate class, 且计算取决于 pin 是否连二极管。

指定 Diode Ratio Vector

Specifying the Diode Ratio Vector

用 `define_antenna_layer_rule -diode_ratio` 指定用于计算最大天线比的值。未指定时工具使用 `{0 0 1 0 0}`。实际用法取决于 `define_antenna_rule -diode_mode`。

表 6.2 给出各模式的向量格式；表 6.3 说明如何用向量值计算。其中：

- `dp`: 输出 pin 的 diode protection 值（见「指定天线属性」）；
- `layerMaxRatio`: 该层最大天线比,由 `define_antenna_layer_rule -ratio` 或全局规则的 `-metal_ratio/-cut_ratio` 指定。

示例见后文「模式 2/3/4」「模式 5/6」「模式 7/8」「模式 14 多栅氧厚度」各小节。

表 6.2: Diode Ratio Vector 格式（原书 Table 29）

模式	格式
2–10, 17	<code>{v0 v1 v2 v3 [v4]}</code> ; <code>v4</code> 为可选保护上限，省略视为 0（无上限）。
11, 12	<code>{{v0 v1 v2 v3 [v4]}{s0 s1 s2 s3 s4 s5}}</code> ; 第二向量为缩放值。
14	每个 gate class 一个 10 值向量（共 4 个）。
16	每个 gate class 一个 8 值向量（共 4 个）。
18	每个 gate class 一个 12 值向量（共 4 个）。

模式 2、3、4 示例

Example for Diode Modes 2, 3, and 4

假设天线面积按单层表面面积（antenna mode 1）计算，M1 的 diode ratio vector 为 `{0.7 0.0 200 2000}`，`layerMaxRatio=400`，同一 M1 网连接二极管 A（保护值 0.5）、B（1.0）、C（1.5）；未指定 `v4` 时按 0。

```
icc2_shell> define_antenna_rule -mode 1 -diode_mode diode_mode \  
-metal_ratio 400 -cut_ratio 20  
icc2_shell> define_antenna_layer_rule -mode 1 -layer "M1" \  
-ratio 400 -diode_ratio {0.7 0.0 200 2000}
```

各二极管最大天线比按表 6.3 与表 6.4 计算。

网络最大天线比：

- 模式 2：取各二极管最大值 = 2300；
- 模式 3：求和 = $400 + 2200 + 2300 = 4900$ ；
- 模式 4：用保护值之和代入公式， $(0.5 + 1.0 + 1.5) * 200 + 2000 = 2600$ 。

模式 5、6 示例

表 6.3: 基于 Diode Mode 的天线比计算（原书 Table 30）

模式	计
0, 1	不
2, 3, 4	若
若 $dp > v0$ 且 $v4 = 0$: $(dp + v1) * v2 + v3$;	
若 $dp \leq v0$: layerMaxRatio	
5	an
若 $max_diode_protection > v0$ 且 $v4 \neq 0$: $equi_gate_area = \min(((max_diode_protection + v1) * v2 + v3), v4)$;	
若 $v4 = 0$: $(max_diode_protection + v1) * v2 + v3$;	
若 $max_diode_protection \leq v0$: $equi_gate_area = 0$	
6	同
7	an
$equi_metal_area$ 由 $max_diode_protection$ 按与模式 5 相同的 $v0-v4$ 规则计算;	
若 $equi_metal_area > metal_area$ 则取 $metal_area$	
8	同
9	an
若 $max_diode_protection > v0$: $scale = \max(1 / ((max_diode_protection + v1) * v2 + v3), v4)$;	
否则 $scale = 1.0$	
10	同
11	an
按 $max_diode_protection$ 相对 $v0-v4$ 的区间选取 $s0-s5$	
12	同
13	N/
14	连
$(metal/via)/gate \leq (dp * v1 + v2)$ 且 $(metal/via)/(v5 * gate + v6 * dp) \leq v7$;	
不连二极管: $(metal/via)/gate \leq v3$ 且 $(metal/via)/(v8 * gate) \leq v9$	
15	N/
16	允
若 $gate-area \leq v5$: $antenna-ratio = antenna-area / (gate-area + v6 * dp)$;	
否则 $antenna-area / (gate-area + v6 * dp + v7)$	
17	同
18	连
	比
$antenna-ratio = antenna-area / gate-area$	

表 6.4: 各二极管最大天线比计算（原书 Table 31）

二极管	保护值	最大天线比
A	0.5	400（保护值 $< v0 = 0.7$, 用 layerMaxRatio）
B	1.0	2200 $((1.0 + 0.0) * 200 + 2000)$
C	1.5	2300 $((1.5 + 0.0) * 200 + 2000)$

Example for Diode Modes 5 and 6

假设天线面积按单层模式表面面积 (antenna mode 1) 计算, M1 的 diode ratio vector 为 {0 0 1 0} (默认), M1 的 layerMaxRatio 为 400, 同一 M1 网络连接二极管 A (保护值 0.5)、B (1.0)、C (1.5); 未指定 v4 时按 0 处理。

```
icc2_shell> define_antenna_rule -mode 1 -diode_mode diode_mode \
-metal_ratio 400 -cut_ratio 20
```

因使用全局 layerMaxRatio 与默认向量, 无需层相关规则。模式 5/6 的最大天线比为 $\text{metal_area} / (\text{gate_area} + \text{equi_gate_area})$, 其中 $\text{equi_gate_area} = (\text{diode_protection} + v1) * (v2 + v3)$ 。

- 模式 5: 用最大保护值 1.5, 得 $\text{metal_area} / (\text{gate_area} + 1.5)$;
- 模式 6: 用总保护值 $0.5 + 1.0 + 1.5 = 3.0$, 得 $\text{metal_area} / (\text{gate_area} + 3.0)$ 。

模式 7、8 示例

Example for Diode Modes 7 and 8

假设单层表面面积模式, M1 向量为 {0.7 0.0 150 800}, layerMaxRatio=400, 二极管保护值同前。

```
icc2_shell> define_antenna_rule -mode 1 -diode_mode diode_mode \
-metal_ratio 400 -cut_ratio 20
icc2_shell> define_antenna_layer_rule -mode 1 -layer "M1" \
-ratio 400 -diode_ratio {0.7 0.0 150 800}
```

模式 7/8 计算为 $(\text{metal_area} - \text{equi_metal_area}) / \text{gate_area}$:

- 模式 7: 最大保护 1.5, 得 $(\text{metal_area} - 1025) / \text{gate_area}$;
- 模式 8: 总保护 3.0, 得 $(\text{metal_area} - 1250) / \text{gate_area}$ 。

模式 14 (多栅氧厚度) 示例

Example for Diode Mode 14 With Multiple Gate Oxide Thicknesses

假设单层表面面积模式, 设计含两种栅氧厚度。gate class 0 的 M1 向量为 {1 0 1e9 1e9 0 1 2 285 1 285}, gate class 1 为 {1 0 1e9 1e9 1 0 2 165 1 28}, layerMaxRatio=285。模式 14 每个 gate class 需 10 个值; 即使仅用两类, 定义时仍须给出全部 40 个值。未用 class 填很大比率以禁用检查。

```
icc2_shell> define_antenna_rule -mode 1 -diode_mode 14 \
-metal_ratio 285 -cut_ratio 10
icc2_shell> define_antenna_layer_rule -mode 1 -layer "M1" \
-ratio 285 -diode_ratio {1 0 1e9 1e9 0 1 2 285 1 285 \
                        1 0 1e9 1e9 1 0 2 165 1 28 \
                        1 0 1e9 1e9 0 1 2 1e9 1 1e9 \
                        1 0 1e9 1e9 0 1 2 1e9 1 1e9 }
```

向量各分量含义:

- V0: 公式 1 使用的栅极面积范围 (0= 当前 class; 1= 全部 class 总和);

- V1: 连二极管时公式 1 的 diode protection 乘数;
- V2: 连二极管时公式 1 的偏移;
- V3: 不连二极管时公式 1 的比率要求;
- V4: 公式 2 的栅极面积范围 (同 V0);
- V5-V7: 连二极管时公式 2 的栅极乘数、保护乘数与比率要求;
- V8-V9: 不连二极管时公式 2 的栅极乘数与天线比要求。

模式 14 公式(表 6.3 续):若连二极管,须同时满足 $(\text{metal/via area})/\text{gate-area} \leq (\text{dp} \cdot \text{v1} + \text{v2})$ 与 $(\text{metal/via area})/(\text{v5} \cdot \text{gate-area} + \text{v6} \cdot \text{dp}) \leq \text{v7}$;若不连二极管,须同时满足 $(\text{metal/via area})/\text{gate-area} \leq \text{v3}$ 与 $(\text{metal/via area})/(\text{v8} \cdot \text{gate-area}) \leq \text{v9}$ 。

计算 Pin 的天线比

Calculating the Antenna Ratio for a Pin

工具支持多种天线比 (antenna-area/gate-area) 计算方式。通过指定下列信息控制:

- 如何计算天线面积 (antenna area mode);
- 计算时考虑哪些金属段 (antenna recognition mode)。

用 `define_antenna_rule` 与 `define_antenna_layer_rule` 的 `-mode` 指定 (二者的必选选项), 见「设置天线模式」。

6.3.2 设置天线模式

Setting the Antenna Mode

天线模式通过以下两方面控制天线比计算:

- 天线面积计算方式 (antenna area mode)
 - 表面面积: $W \times L$;
 - 侧壁面积: $(W + L) \times 2 \times \text{thickness}$ 。

注意

使用侧壁面积时, 须在 `technology file` 各 `Layer` 段中用 `unitMinThickness`、`unitNomThickness`、`unitMaxThickness` 定义金属厚度。

- 天线识别模式 (antenna recognition mode)
 - **Single-layer mode:** 仅考虑当前层金属段, 忽略下层。可布线性最好。
`antenna_ratio` = 该层相连金属面积 / 总栅极面积
 - **Accumulated-ratio mode:** 考虑当前层及与输入 pin 相连的下层段。
`antenna_ratio` = 当前层与下层各 `single-layer` 比率之和
 - **Accumulated-area mode:** 考虑当前层与下层金属面积。
`antenna_ratio` = 当前及下层相连金属面积 / 总栅极面积

用 `-mode` 指定天线模式（必选）。表 6.5 列出取值。

表 6.5: 天线模式设置（原书 Table 32）

天线模式值	面积计算方式	天线识别模式
1	Surface	Single-layer
2	Surface	Accumulated-ratio
3	Surface	Accumulated-area
4	Sidewall	Single-layer
5	Sidewall	Accumulated-ratio
6	Sidewall	Accumulated-area

天线识别模式示例

Antenna Recognition Mode Example

图 6.5 给出含侧视的布局示例；表 6.6 列出各识别模式下的天线比。

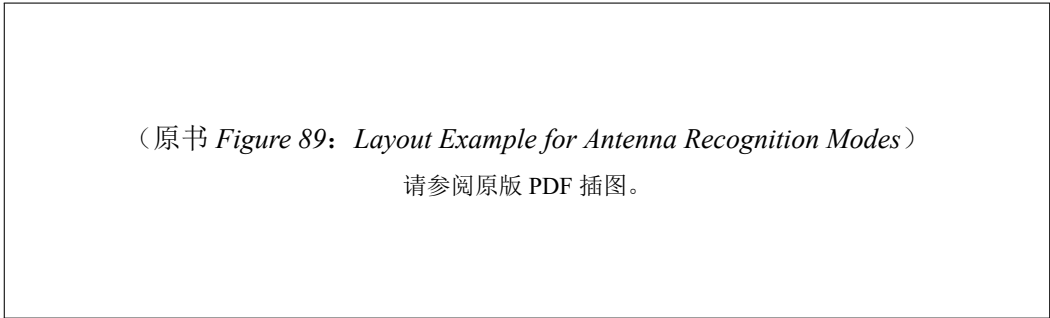


图 6.5: 天线识别模式布局示例

6.3.3 指定天线属性

Specifying Antenna Properties

标准单元与 hard macro 的天线属性一般在参考库的 frame view 中定义。可为未在库中定义属性的单元设置默认值：

- `route.detail.default_diode_protection`: 标准单元输出 pin 缺省 diode protection;
- `route.detail.default_gate_size`: 标准单元输入 pin 缺省 gate size;
- `route.detail.default_port_external_antenna_area`: 顶层 port 缺省天线面积;
- `route.detail.default_port_external_gate_size`: 顶层 port 缺省 gate size;
- `route.detail.macro_pin_antenna_mode`: macro pin 的天线处理方式;
- `route.detail.port_antenna_mode`: 顶层 port 的天线处理方式。

层次化流程中若为 block 创建 abstraction，须用 `derive_hier_antenna_property` 从 block 提取天线信息供上一层使用。

表 6.6: 天线识别模式与比率（原书 Table 33）

考虑的段	天线比
Single-layer M2: Gate1=M2b/Gate1; Gate2=M2d/Gate2; Gate3=M2e/Gate3。 M3: Gate1,2=(M3b+M3c)/(Gate1+Gate2); Gate3=M3d/Gate3。 M4: Gate1,2,3=(M4a+M4b)/(Gate1+Gate2+Gate3)。	M1: Gate1=M1b/Gate1; Gate2=M1c/Gate2; Gate3=M1d/Gate3。
Accumulated-ratio M2: Gate1=M1b/Gate1+M2b/Gate1（Gate2/3 类似累加）。 M3/M4: 在下层比率上继续累加当前层 single-layer 比率。	M1 同 single-layer。
Accumulated-area M2: Gate1=(M1b+M2b)/Gate1 等。 M3: Gate1,2 为多层金属面积之和 / (Gate1+Gate2); Gate3 类似。 M4: 全部金属面积 / (Gate1+Gate2+Gate3)。	M1 同 single-layer。

注意

若 macro 并非所有层都定义了层次化天线属性，Zroute 视为数据不完整，跳过天线分析并给出 ZRT-311 警告。参见 SolvNet 文章 027178。

参见

在 hard macro 上标注天线属性（Annotating Antenna Properties on Hard Macro Cells）。

6.3.4 分析并修复天线违例

Analyzing and Fixing Antenna Violations

若设计库含天线规则,Zroute 在详细布线期间自动分析并修复天线违例。要禁用,将 route.detail.antenna 设为 false。天线规则与其他设计规则一样在详细布线中并发检查与修正，以减少迭代、缩短总运行时间。

默认行为:

- 对全部时钟与信号网检查并修复；可用 route.detail.skip_antenna_fixing_for_nets 禁止对特定网修复（仍分析并报告）；
- 不对电源/地网检查或修复；将 route.detail.check_antenna_on_pg 设为 true 可启用；
- 在第二次迭代(初始布线完成且基本 DRC 已修复后)开始修复;可用 route.detail.antenna_on_iteration 更改；
- 通过 **layer hopping** 修复：把大金属多边形拆到更高层，或（对最高层有限保护情形）下移到更低层。

Layer hopping 类型:

- 用更高层金属段打断天线：对 metal-N 违例，在靠近栅极处插入小段 metal-(N+1)，降低剩余 metal-N 比率。不适用于最高层且输出保护有限的情况。
- 下移到更低层：主要用于最高层违例；可能增加 RC 与时序延迟。

也可插入二极管修复。将 `route.detail.insert_diodes_during_routing` 设为 `true` 启用。要强制仅用二极管，将 `route.detail.hop_layers_to_fix_antenna` 设为 `false`。若两者皆启用，默认先尝试 layer hopping；要将偏好改为二极管，设 `route.detail.antenna_fixing_preference` 为 `use_diodes`。

删除修复天线所不需要的冗余二极管：`route.detail.delete_redundant_diodes_during_routing=true`。Spare 二极管不删除；详细布线插入的 port 保护二极管也不删除。

每轮详细布线结束会报告天线违例，例如：

DRC-SUMMARY:

```
@@@@@@ TOTAL VIOLATIONS =      506
@@@ Total number of instance ports with antenna violations = 1107
```

检查天线违例：

```
icc2_shell> check_routes -antenna true
```

6.3.5 详细布线期间插入二极管

Inserting Diodes During Detail Routing

通过在靠近被保护栅极的网络上插入反向偏置二极管，为累积电荷提供放电通路，且正常工作时不应导通电流。

启用：`route.detail.insert_diodes_during_routing=true`。

控制选项：

- `route.detail.antenna_fixing_preference=use_diodes`：优先二极管而非 hopping；
- `route.detail.hop_layers_to_fix_antenna=false`：强制二极管修复；
- 默认可新增二极管或使用 spare；用 `route.detail.diode_preference` 设为 `new/spare/none`；
- `route.detail.diode_insertion_mode=new/spare/new_and_spare`：强制方法；

注意

要利用 spare 二极管，须在标准单元布局前/后、天线违例可能出现区域布线之前加入 spare。

- `route.detail.diode_libcell_names`：限制可用二极管（仅写单元名，勿含库名）；
- `route.detail.reuse_filler_locations_for_diodes=false`：禁止复用 filler 位置。

层次与 voltage area：

- Pin 违例：二极管插在与违例 pin 相同逻辑层次；
- 顶层 port 违例：默认顶层；若 port 属 voltage area，可将 `route.detail.use_lower_hierarchy_for_port_diodes` 设为 `true`。

详细布线后插入二极管

Inserting Diodes After Detail Routing

详细布线后可用 `create_diodes` 显式指定违例与约束。根据约束与 `route.detail.diode_insertion_mode`，插入新二极管或复用 `spare`。

须用列表指定：`port`、单元实例、二极管参考单元、数量、连接最高层、相对 `pin` 的最大距离。距离内无法插入/复用则跳过。

```
{port_name instance_name diode_reference number_of_diodes
  max_routing_layer max_routing_distance}
```

注意

将单元实例指定为顶层 `block` 名，可为顶层 `port` 插入二极管。

相关应用选项：`route.detail.diode_insertion_mode`、`route.detail.reuse_filler_locations_for_diodes`、`route.detail.use_lower_hierarchy_for_port_diodes`。

示例（CI 的 A port，2 个 Adiode，最高 M5，2.5 μm ）：

```
icc2_shell> create_diodes {{A CI Adiode 2 M5 2.5}}
```

6.4 插入冗余 Via

Inserting Redundant Vias

冗余 via 插入是 Zroute 在整个布线流程中支持的重要 DFM 功能。各布线阶段并发优化 via 数量与线长。结果以冗余 via 转换率（单 via 转为冗余 via 的百分比）衡量；亦应关注未优化单 via 数量——越少通常对 DFM 越有利。

相关主题：

- 在时钟网上插入冗余 Via
- 在信号网上插入冗余 Via

6.4.1 在时钟网上插入冗余 Via

Inserting Redundant Vias on Clock Nets

Zroute 可在时钟布线期间或之后插入冗余 via。布线后插入见「布线后冗余 Via 插入」。

时钟布线期间步骤：

1. 用 `create_routing_rule` 在非默认布线规则中指定冗余 via（见「指定非默认 Via」）。选择时须兼顾 DFM 与可布线性。NDR 还可定义更严线宽/间距与 `tapering` 距离。
2. 用 `set_clock_routing_rules` 将 NDR 赋给时钟网：

```
icc2_shell> set_clock_routing_rules -rules clock_via_rule
```

3. 运行 CTS：

```
icc2_shell> synthesize_clock_trees
```

4. 布线时钟网：

```
icc2_shell> route_group -all_clock_nets \  
-reuse_existing_global_route true
```

全局布线预留冗余 via 空间，详细布线插入冗余 via。

6.4.2 在信号网上插入冗余 Via

Inserting Redundant Vias on Signal Nets

可采用：

- 布线后冗余 via 插入；
- 并发 soft-rule 冗余 via 插入；
- 接近 100% 冗余 via 插入。

一般先做布线后插入；若转换率 $\geq 80\%$ 可再尝试并发 soft-rule；若 $\geq 90\%$ 可再尝试接近 100% 插入。

注意

转换率升高会使设计规则更难收敛，并可能降低信号完整性；接近 100% 仅用于确需极高转换率的 block，且可能需调整 floorplan utilization 以留出空间。

相关主题：查看默认 Via 映射表；定义定制 Via 映射表；布线后插入；并发 soft-rule；接近 100%；时序保持；报告转换率。

查看默认 Via 映射表

Viewing the Default Via Mapping Table

默认 Zroute 从 technology file 读取默认 contact code 并生成优化映射表。查看：

```
icc2_shell> add_redundant_vias -list_only true
```

多数情况下定制映射表优于默认表。若已用 add_via_mapping 创建定制表，该命令显示定制表。

示例（原书 Example 25）：

```
icc2_shell> add_redundant_vias -list_only  
...  
Redundant via optimization will attempt to replace the following vias:  
    VIA12SQ_C    -> VIA12SQ_C_2x1    VIA12SQ_C_2x1(r) VIA12SQ_C_1x2 ...  
    VIA12SQ_C(r) -> VIA12SQ_C_2x1    VIA12SQ_C_2x1(r) VIA12SQ_C_1x2 ...  
...
```

定义定制 **Via** 映射表

Defining a Customized Via Mapping Table

使用 `add_via_mapping`。至少用 `-from/-to` 指定源 via 与替换 via（须为 technology file 中的 via 或 `create_via_def` 创建的设计专用 via）。`-from` 须为 simple via 或 via array；`-to` 可为 simple、simple array 或 custom via。

细化选项：

- `-weight`：优先级 1–10（默认 1）；高权重优先使用；
- `-transform`：all（默认可旋转/翻转）、rotate、flip、none。

映射保存在设计库 via 映射表中。覆盖已有映射须 `-force`。

```
icc2_shell> add_via_mapping \
  -from {VIA12 1x1} -to {VIA12T 2x1} -weight 5
icc2_shell> add_via_mapping \
  -from {VIA12 1x1} -to {VIA12 2x1} -weight 1
```

用 `report_via_mapping` 查看（`-from/-to` 可过滤）。用 `remove_via_mappings` 移除（`-all/-from/-to`）。

使用 **Via** 映射表子集

Using a Subset of the Via Mapping Table for Redundant Via Insertion

通过应用选项按层选择权重组：

- `route.common.redundant_via_include_weight_group_by_layer_name`：仅用指定权重组；未列出的层不做插入；
- `route.common.redundant_via_exclude_weight_group_by_layer_name`：排除指定权重组；
- 两者同时设置时，在 include 集合中再排除 exclude 集合。

布线后冗余 **Via** 插入

Postroute Redundant Via Insertion

使用 `add_redundant_vias`：可将单 cut 换成多 cut 阵列、换成其他 contact code 的单 cut，或换成不同多 cut 阵列。插入时详细布线检查邻近 partition 的设计规则以减少违例。

默认对所有网插入；`-nets` 可限制网络。完成后重新检查并修复 DRC。

若转换率不足，可用 `-effort high`（约再提升 3–5%，通过移动 via 腾空间）；在 45 nm 及以下可能不利于光刻友好图案，此时宜改用并发 `soft-rule`。也可将 `route.detail.optimize_wire_via_effort_level` 设为 high 以减少单 via、缩短线长。

初始布线后插入完成后，设 `route.common.post_detail_route_redundant_via_insertion`，以便后续详细/ECO 布线后自动插入，维持转换率。

并发 **Soft-Rule** 冗余 **Via** 插入

Concurrent Soft-Rule-Based Redundant Via Insertion

在布线期间为冗余 via 预留空间以提高转换率；可用于初始与 ECO 布线。实际插入仍须事后运行 `add_redundant_vias`。

注意

预留空间会增加布线运行时间；仅当布线后方法转换率已 $\geq 80\%$ 且需进一步提高时使用。

步骤：

1. (可选) 定义 via 映射表；
2. 启用并发 soft-rule:

```
icc2_shell> set_app_options \
    -name route.common.concurrent_redundant_via_mode \
    -value reserve_space
```

用 `route.common.concurrent_redundant_via_effort_level` 控制努力（默认 low；high 也影响详细布线，可能影响 DRC 收敛）。若在 `place_opt` 前启用，拥塞估计会考虑冗余 via。

ECO：设 `route.common.eco_route_concurrent_redundant_via_mode=reserve_space`，并用对应 effort 选项。

注意

ECO 期间使用可能影响时序与 DRC 收敛；一般仅当初始布线用了接近 100% 插入时才在 ECO 用 soft-rule 维持转换率。

3. 布线 block（预留空间并修复硬规则）；
4. 运行布线后 `add_redundant_vias`。

接近 **100%** 冗余 Via 插入

Near 100 Percent Redundant Via Insertion

将冗余 via 作为硬设计规则处理。仅支持初始布线，不支持 ECO；初始用此法后，ECO 应改用 soft-rule 以保持转换率。

注意

对拥塞 block 可能大幅增加运行时间；仅当 soft-rule 已达 $\geq 90\%$ 且仍需提高时使用。

```
icc2_shell> set_app_options \
    -name route.common.concurrent_redundant_via_mode \
    -value insert_at_high_cost
```

布线期间插入冗余 via 并修复硬规则；优先级通常与其他硬规则相同，若详细布线 DRC 不收敛会自动放宽冗余 via 约束。

冗余 **Via** 插入时保持时序

Preserving Timing During Redundant Via Insertion

插入冗余 via 会改变时序：短网因电容增加变慢，长网因电阻下降变快。时序保持模式通过对关键网禁止插入来避免影响时序。

在 `add_redundant_vias` 上使用:

- `-timing_preserve_setup_slack_threshold`
- `-timing_preserve_hold_slack_threshold`
- `-timing_preserve_nets`

应在流程末尾、常规插入已收敛转换率与时序 QoR 之后使用；过早使用可能因关键网过多而严重降低转换率。

6.4.3 报告冗余 Via 率

Reporting Redundant Via Rates

插入后 Zroute 生成报告，包含:

- 非默认 via 的总优化转换率;
- 各层优化转换率 (含 double via 与 DFM-friendly bar via)，并分别相对总 via 数与可优化 routed via 数给出;

注意

使用 bar via 时优化转换率参考价值有限。

- 各层按 weight 的优化 via 分布 (高于某权重的转换率须对各权重求和); 无权重记为 “Un-optimized”;
- block 总 double via 转换率。

注意

`report_design` 的冗余 via 率与 `Zroute/check_routes` 不同: 前者含 PG 与信号 via, 后者仅信号 via, 且基于冗余 via 映射。

示例 (原书 Example 27):

```
Total optimized via conversion rate = 96.94% (1401030 / 1445268 vias)
Layer V01          = 41.89% (490617 / 1171301 vias)
  Weight 10        = 9.64% (112869 vias)
  Weight 5         = 32.25% (377689 vias)
  ...
Total double via conversion rate = 46.69% (2158006 / 4622189 vias)
```

6.5 优化线长与 Via 数量

Optimizing Wire Length and Via Count

详细布线时, Zroute 在存在 DRC 违例的区域优化线长与 via 数量, 但不优化无违例区域。

为提高良率, 在详细布线与冗余 via 插入之后, 可用 `optimize_routes` 独立优化线长与 via 数量。

默认按总体代价选择重布网络; 对所选网决定重布全部形状或部分形状。用 `-nets` 指定网络; 可用 `-reroute_all_shapes_in_nets` 强制重布该网全部形状。

默认最多 40 次详细布线迭代以修复优化后违例；可用 `-max_detail_route_iterations` 控制。

6.6 减小临界面积

Reducing Critical Areas

临界面积是这样一块区域：若随机颗粒缺陷中心落在该处，会导致电路失效、降低良率。导电缺陷造成短路，非导电缺陷造成开路。

相关主题：

- 通过 wire spreading 减小短路临界面积；
- 通过 wire widening 减小开路临界面积。

6.6.1 执行 Wire Spreading

Performing Wire Spreading

详细布线与冗余 via 插入后，可用 `spread_wires` 增大平均线间距，减小短路临界面积。默认对同层信号线：

- 在优选方向疏导半个 pitch（用 `-pitch` 以层 pitch 倍数修改，如 `-pitch 1.5`）；
- 最小 jog 长度为两倍层 pitch（`-min_jog_length`，层 pitch 整数倍）；
- 最小 jog 间距为最小层间距加半个层 pitch（`-min_jog_spacing_by_layer_name`，按层以微米指定）。

图 6.6 示出 jog 长度与间距的用法。

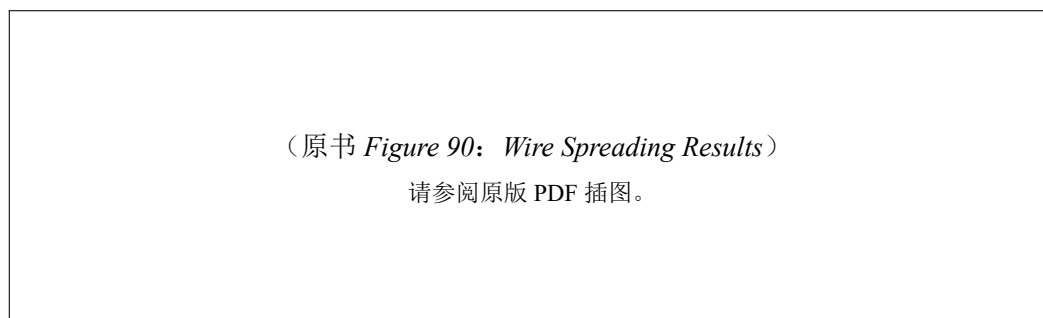


图 6.6: Wire Spreading 结果

```
icc2_shell> spread_wires -min_jog_length 3 \  
-min_jog_spacing_by_layer_name {{M1 0.07} {M2 0.08}}
```

疏导后自动进行详细布线迭代修复 DRC。时序保持模式选项：`-timing_preserve_setup_slack_threshold`、`-timing_preserve_hold_slack_threshold`、`-timing_preserve_nets`。仅对 slack 不小于阈值的网、`-timing_preserve_nets` 所列网，以及同层两 pitch 内的相邻网执行疏导。

6.6.2 执行 Wire Widening

Performing Wire Widening

在详细布线、冗余 via 与 wire spreading 之后，用 `widen_wires` 增大平均线宽以减小开路临界面积。默认将所有线加宽到原宽 1.5 倍。可用 `-widen_widths_by_layer_name` 为每层定义最多五种候选宽度：

```
icc2_shell> widen_wires \
-widen_widths_by_layer_name {{M1 0.07 0.06} {M2 0.08 0.07}}
```

加宽会减小邻线间距，可能削弱 spreading 带来的短路临界面积改善。用 `-spreading_widening_relative_weight` 权衡：默认等权；0.0–0.5 偏向 widening（开路）；0.5–1.0 偏向 spreading（短路）。

加宽后进行详细布线迭代修复 DRC；加宽导线不触发 fat wire 间距规则。时序保持选项同 spreading；不对 slack 小于阈值或指定保护网执行加宽。

6.7 插入金属–绝缘体–金属电容

Inserting Metal-Insulator-Metal Capacitors

MiM 电容由两层专用导电层夹绝缘体构成，插在两层常规金属之间，见图 6.7。

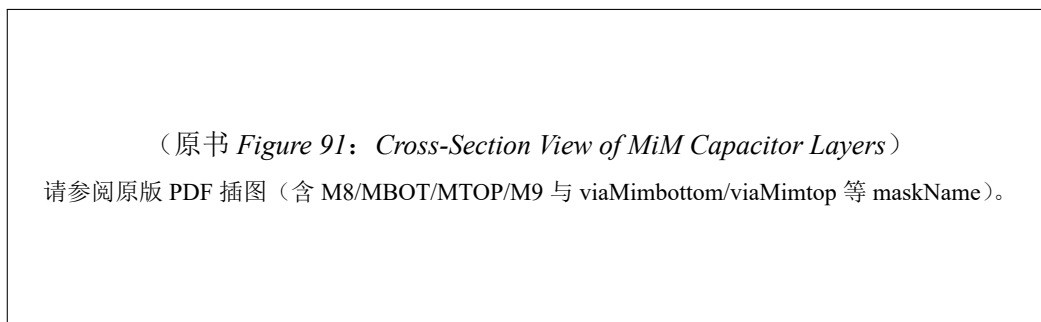


图 6.7: MiM 电容层剖面示意

MiM 电容通常接在电源与地之间，以在电气噪声下维持供电稳定。尺寸常按电源规划的 power strap 几何定制。

用 `create_mim_capacitor_array` 插入阵列并连接到电源轨。至少指定 MiM 库单元及阵列 *x/y* 增量：

```
icc2_shell> create_mim_capacitor_array \
-lib_cell my_lib/mim_ref_cell \
-x_increment 20 -y_increment 20
```

默认行为：

- 对整个 block 插入；可用 `-boundary` 限制到单个矩形/直角多边形区域。插入时不考虑标准单元、macro、placement blockage 或 voltage area；
- 方向 R0；可用 `-orientation`（作用于阵列全部单元）；
- 命名：mimcap!library_cell_name!number；可用 `-prefix`。

6.8 插入 Filler Cell

Inserting Filler Cells

为保证电源网连通，可在标准单元行空隙中插入 **filler**；亦常用于加入去耦电容以稳定供电。工具支持单高与多高 **filler**。

注意

插入前须用 `check_legality` 确认 **block** 已合法化。

两种主要方法：

- 有序 **filler** 参考列表（标准流程）：按列表顺序在每个间隙插入第一个能放入的单元。若工艺要求特定 **filler** 邻接标准单元，可先插入这些必选 **filler**，再用标准流程填剩余空隙；
- 阈值电压规则（VT-based）：按间隙两侧单元的 VT 类型用用户规则选 **filler**，多用于成熟晶圆厂节点。

6.8.1 标准 Filler Cell 插入

Standard Filler Cell Insertion

使用 `create_stdcell_fillers` 插入，用 `remove_stdcell_fillers_with_violation` 检查并移除违例 **filler**。插入时通过布局合法化器遵守先进节点规则与物理约束；若 `place.legalize.enable_advanced_legalize` 使用先进合法化算法做 2D 规则与单元交互检查，可缩短运行时间。

步骤：

1. 用 `check_legality` 确认合法。注意：`check_legality` 假定所有 `design_type==filler` 的标准单元都可用于填充，而 `create_stdcell_fillers` 仅插入 `-lib_cells` 所列单元，二者差异可能导致插入后仍有不可填间隙。
2. （可选）用 `create_left_right_filler_cells` 为特定标准单元两侧插入必选 **filler**（见「用特定 Filler 邻接标准单元」）。
3. （可选）`set_host_options -max_cores n` 启用多核（仅当 `technology file` 含最小阈值电压、OD 或 TPO 规则时生效）。
4. 用 `create_stdcell_fillers -lib_cells` 插入 **metal filler**（宜从大到小）。可用：

```
icc2_shell> get_lib_cells ref_lib/* -filter "design_type==filler"
icc2_shell> set FILLER_CELLS \
    [get_object_name [sort_collection -descending \
        [get_lib_cells ref_lib/* -filter "design_type==filler"] area]]
```

5. `connect_pg_net -automatic` 连接 PG。
6. `remove_stdcell_fillers_with_violation` 移除违例 **metal filler**（可能需多次，因移除会暴露新违例）。
7. 再插入 **nonmetal filler** 并连接 PG。

参见

Generic ECO Flow for Timing or Functional Changes; 移除 Filler Cell。

6.8.2 控制标准 Filler Cell 插入

Controlling Standard Filler Cell Insertion

默认填充整个 block 水平标准单元行的全部空隙。可控制：

- 按最低漏电选择：-leakage_vt_order 按漏电递减给出 VT 层；-leakage_vt_check 仅检查既有 filler 是否可换成更低漏电单元（最多报告数由 chf.create_stdcell_fillers.max_leakage_vt_order_violation 控制）。须启用 advanced legalizer，且 technology file 中 VT 层 maskType=implant。
- 填充百分比：-utilization；用 -type_utilization 控制各类（如 ULVT/LVT DCAP）比例（总和 ≤ 100 ），可用 -fill_remaining 填剩余，-prefer_type_ordering 按组内从左到右优先。
- 区域：-bboxes、-voltage_area。
- 电源网违例：默认不查（省时）；metal filler 建议 -rules check_pnet。
- 防间隙：-smallest_cell_size 为 1/2/3 倍 unit site（advanced legalizer 开启时忽略）。
- **Hard placement blockage**：默认遵守；-ignore_hard_blockages 可忽略。
- 命名：xofiller!...；-prefix/-separator。
- 合法化错误：默认遇错失败；-continue_on_error 可强制完成。

示例：

```
icc2_shell> create_stdcell_fillers -lib_cells $FILLER_CELLS \
    -type_utilization { {*/DCAP16*ULVT */DCAP8*ULVT ...} 70
                      {*/DCAP16*LVT */DCAP8*LVT ...} 30 } \
    -prefer_type_ordering -fill_remaining
```

6.8.3 检查 Filler Cell DRC 违例

Checking for Filler Cell DRC Violations

默认 remove_stdcell_fillers_with_violation 检查名称含 xofiller 的实例与顶层信号布线对象之间的 DRC，并移除违例者。可控制：

- -name：识别 filler 的字符串；
- -boundary：检查区域；
- -check_between_fixed_objects true：与所有邻接对象检查；
- -shorts_only true：仅因与布线形状短路而移除；
- 双重图案化：当 route.common.color_based_dpt_flow=true 且 technology file 有相应规则时检查；
- -check_only true：只检查不移除，写 block_fillers_with_violation.rpt。

6.8.4 修复剩余 Mask 设计规则违例

Fixing Remaining Mask Design Rule Violations

部分 mask 规则因出现概率低、计算重而未在插入时处理。插入后用 `replace_fillers_by_rules` -`replacement_rule` 修复。表 6.7 列出支持的规则与关键字（原书 Table 34）。

表 6.7: `replace_fillers_by_rules` 支持的设计规则（原书 Table 34）

设计规则	-replacement_rule	说明
End orientation	end_orientation	翻转最左/最右 filler 方向
Half row adjacency	half_row_adjacency	半高 filler 邻接限制
Illegal abutment	illegal_abutment	禁止某些 filler 邻接
Maximum horizontal length	od_horiztonal_distance	连续 filler 行水平长度
Maximum horizontal edge length	horizontal_edges_distance	特定层邻接对象水平边长
Maximum stacking for small fillers	small_filler_stacking	小 filler 堆叠高度
Maximum vertical edge length	max_vertical_constraint	堆叠 filler 垂直边长
Tap cell spacing	od_tap_distance	Tap OD 与邻接 OD 距离
Random	random	随机替换为定制 filler

End Orientation 规则

End Orientation Rule

对连续水平单元序列，修改最左/最右且属于目标列表的 filler 方向。须指定 `-target_fillers`，并用 `-left_end/-right_end`（`inverse`、`R0_or_MX`、`MY_or_R180`）。`-check_only` 仅报告将改方向的单元。

Half Row Adjacency 规则

Half Row Adjacency Rule

按邻接标准单元类型（**Inbound**：顶/底边界在半高处，需 ICIP；**Regular**：全高边界，邻接侧不得有 ICIP）替换半高 filler。替换单元与原 filler 同高、同宽、同 VT；库提供带 ICIP 的 p/n 型。图 6.8；表 6.8。

（原书 Figure 92: *Half-Height Filler Cell Replacement With Half Row Adjacency Rule*）
请参阅原版 PDF 插图。

图 6.8: 半高 Filler 替换与半行邻接规则

每个列表每种尺寸只能含一个单元。

表 6.8: 半高替换 Filler Cell（原书 Table 35）

邻接 inbound 标准单元	指定替换 filler 的选项
无	-p_none、-n_none
仅左	-p_left、-n_left
仅右	-p_right、-n_right
左与右	-p_left_right、-n_left_right
顶或底（仅 2X filler）	-p_left_center_right、-n_left_center_right

Illegal Abutment 规则

Illegal Abutment Rule

```
icc2_shell> replace_fillers_by_rules -replacement_rule illegal_abutment \  
-replace_abutment { {FILL1a CFILL1a} {FILL1b CFILL1b} } \  
-illegal_abutment {FILLx FILLy}
```

-skip_fixed_cells 可忽略 physical_status 为 fixed/locked 的 filler。图 6.9。

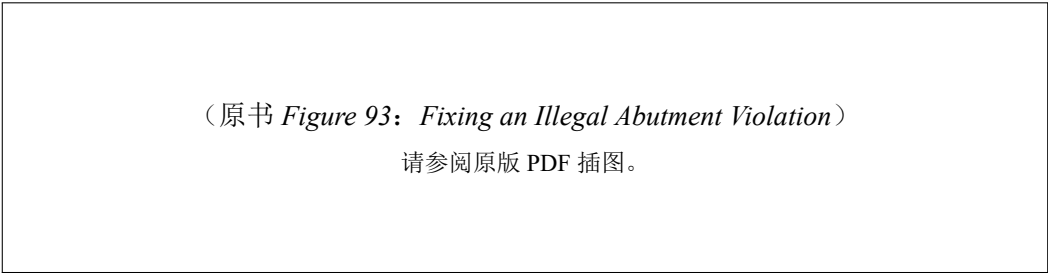


图 6.9: 修复非法邻接违例

最大水平边长规则

Maximum Horizontal Edge Length Rule

-replacement_rule horizontal_edges_distance, 须指定 -max_constraint_length、-layer, 以及 -refill_table 或 -constraint_fillers/-non_constraint_fillers。

```
icc2_shell> replace_fillers_by_rules \  
-replacement_rule horizontal_edges_distance \  
-max_constraint_length 30 -layer M2 \  
-refill_table { {{VT1_FILL1} {VT1_DECAP4 VT1_DECAP2}}  
               {{VT2_FILL1} {VT2_DECAP4 VT2_DECAP2}} }
```

最大水平长度规则

Maximum Horizontal Length Rule

-replacement_rule od_horiztonal_distance（原文关键字拼写如此）。示例限制连续 filler 水平长度 70 μm:

```
icc2_shell> replace_fillers_by_rules \
-replacement_rule od_horizontal_distance -max_constraint_length 70 \
-refill_table { {{VT1_FILL1}} {{VT1_DECAP4 VT1_DECAP2}}
               {{VT2_FILL1}} {{VT2_DECAP4 VT2_DECAP2}} }
```

图 6.10（原书 Figure 94）。

（原书 *Figure 94*：相关最大长度违例修复示意）
请参阅原版 PDF 插图。

图 6.10: 修复最大水平长度相关违例

小 Filler 最大堆叠规则

Maximum Stacking for Small Filler Cells Rule

-replacement_rule small_filler_stacking,指定 -max_constraint_length、-small_fillers、-replacement_fillers。超出高度时用大 filler 替换相邻 2–3 个小 filler；无法替换则重排。保持违例处 VT，非违例处 VT 可能改变且不检查。

```
icc2_shell> replace_fillers_by_rules \
-replacement_rule small_filler_stacking -max_constraint_length 10 \
-small_fillers {FILL1 FILL2} -replacement_fillers {FILL3 FILL4}
```

图 6.11、6.12。

（原书 *Figure 95: Fixing Maximum Stacking Violation by Replacement*）
请参阅原版 PDF 插图。

图 6.11: 通过替换修复最大堆叠违例

最大垂直边长规则

Maximum Vertical Edge Length Rule

-replacement_rule max_vertical_constraint。指定 -max_constraint_length、-constraint_fillers（默认所有标准单元受约束，-exception_cells 排除；-no_violation_along_exception_cells）、以及 -non_constraint_fillers/-non_constraint_left_fillers/-non_constraint_right_fillers。

(原书 *Figure 96: Fixing Maximum Stacking Violation by Reordering*)

请参阅原版 PDF 插图。

图 6.12: 通过重排修复最大堆叠违例

```
icc2_shell> replace_fillers_by_rules \
  -replacement_rule max_vertical_constraint -max_constraint_length 180 \
  -constraint_fillers {DECAP8 DECAP4} -non_constraint_fillers {FILL1} \
  -exception_cells {BUF1 TAP1}
```

图 6.13。

(原书 *Figure 97: Fixing a Maximum Vertical Edge Length Violation*)

请参阅原版 PDF 插图。

图 6.13: 修复最大垂直边长违例

Tap Cell Spacing 规则

Tap Cell Spacing Rule

-replacement_rule od_tap_distance。指定 -tap_distance_range (site 数)、-tap_cells，以及 -left_violation_tap/-right_violation_tap/-both_violation_tap。目标与替换 tap 须同尺寸。对无 OD 邻接单元用 -adjacent_non_od_cells 跳过。

```
icc2_shell> replace_fillers_by_rules -replacement_rule od_tap_distance \
  -tap_cells {TAP} -tap_distance_range {3 3} \
  -left_violation_tap {TAP1} -right_violation_tap {TAPr} \
  -both_violation_tap {TAPb}
```

图 6.14。

6.8.5 随机 Filler Cell 替换

Random Filler Cell Replacement

```
icc2_shell> replace_fillers_by_rules -replacement_rule random \
  -random_replace { {FILL1 {FILL1a FILL1b FILL1c}}
                  {FILL2 {FILL2a FILL2b}} }
```

(原书 *Figure 98: Fixing a Tap Cell Spacing Violation*)

请参阅原版 PDF 插图。

图 6.14: 修复 Tap Cell 间距违例

可用 `-skip_fixed_cells`。

6.8.6 用特定 Filler 邻接标准单元

Abutting Standard Cells With Specific Filler Cells

用 `create_left_right_filler_cells` 在特定标准单元左/右侧插入必选 filler，然后再做标准 filler 插入填剩余空隙。

`-lib_cells` 规则格式：

```
{standard_cells} {left_filler_cells} {right_filler_cells}
```

要求：至少一侧有 filler；filler 的 `design_type` 为 `lib_cell` 或 `filler`；与标准单元同 site；标准单元高度为 filler 高度整数倍；宜从大到小。通配符顺序与 `get_lib_cells` 一致。

```
icc2_shell> create_left_right_filler_cells \
-lib_cells {
  { {mylib/SC1} {mylib/LF4X mylib/LF2X} {mylib/RF2X mylib/RF1X} }
  { {mylib/SC2} {mylib/LF2X mylib/LF1X} {} } }
```

插入时检查合法位置，但不检查最小 VT、OD、TPO 等先进规则；紧贴标准单元插入，不考虑单元间距规则。

6.8.7 控制基于单元的 Filler 插入

Controlling Cell-Based Filler Cell Insertion

`create_left_right_filler_cells` 还可控制：

- 区域： `-boundaries`、`-voltage_areas`；
- 方向：默认按行允许方向； `-follow_stdcell_orientation` 跟随标准单元（翻转时左右互换）；
- 一单位 tile 间隙： `-rules no_1x`（此时勿在列表中包含宽度为 1 tile 的 filler）；
- 命名约定：同标准 filler（`xofiller!...`）。

6.8.8 基于阈值电压的 Filler 插入

Threshold-Voltage-Based Filler Cell Insertion

步骤：

1. 确保 VT 类型信息可用；
2. 用 `set_vt_filler_rule` 定义规则；
3. (可选) 为多种邻接 VT 组合分别设规则；
4. 用 `create_vtcell_fillers` 插入。

```
icc2_shell> set_vt_filler_rule -vt_type {default default} \
    -filler_cells {myLib/filler2x myLib/filler1x}
icc2_shell> set_vt_filler_rule -vt_type {vtA vtA} \
    -filler_cells {myLib/fillerA2x myLib/fillerA1x}
icc2_shell> create_vtcell_fillers
```

6.8.9 控制基于阈值电压的 Filler 插入

Controlling Threshold-Voltage-Based Filler Cell Insertion

默认填充整个 block 水平标准单元行空隙。可控制 `-region`、`-voltage_area`，以及命名（默认 `xofiller!...`；`-prefix/-separator`）。

6.8.10 移除阈值电压 Filler 信息

Removing the Threshold-Voltage Filler Cell Information

```
icc2_shell> create_vtcell_fillers -clear_vt_information
```

移除 **Filler Cell**

Removing Filler Cells

```
icc2_shell> remove_cells [get_cells xofiller*]
```

仅移除有 DRC 违例的标准流程 filler：用 `remove_stdcell_fillers_with_violation`。

6.9 插入 Metal Fill

Inserting Metal Fill

布线后，可用金属导线填充 block 空隙以满足金属密度规则。插入前 block 应接近满足时序，且 DRC 很少或没有。

运行 `signoff_create_metal_fill`，详见第 七 章「使用 IC Validator In-Design 插入 Metal Fill」。

注意

运行 `signoff_create_metal_fill` 需要 IC Validator license。

Chapter 7

IC Validator In-Design

IC Validator In-Design

翻译占位

本章对应原书第 7 章：*IC Validator In-Design*。

正文待续译。可将原版 PDF 对应页提取后按第 1 章体例写入本文件。

原书起始页约 p.545。

Chapter 8

使用 Custom Router 布线

Routing Using Custom Router

翻译占位

本章对应原书第 8 章：*Routing Using Custom Router*。

正文待续译。可将原版 PDF 对应页提取后按第 1 章体例写入本文件。

原书起始页约 p.631。

Chapter 9

相对布局物理数据通路

Physical Datapath With Relative Placement

翻译占位

本章对应原书第 9 章：*Physical Datapath With Relative Placement*。
正文待续译。可将原版 PDF 对应页提取后按第 1 章体例写入本文件。

原书起始页约 p.666。

Chapter 10

层次化实现

Hierarchical Implementation

翻译占位

本章对应原书第 10 章：*Hierarchical Implementation*。

正文待续译。可将原版 PDF 对应页提取后按第 1 章体例写入本文件。

原书起始页约 p.702。

Chapter 11

RedHawk 与 RedHawk-SC Fusion

RedHawk and RedHawk-SC Fusion

翻译占位

本章对应原书第 11 章：*RedHawk and RedHawk-SC Fusion*。

正文待续译。可将原版 PDF 对应页提取后按第 1 章体例写入本文件。

原书起始页约 p.740。

Chapter 12

ECO 流程

ECO Flow

翻译占位

本章对应原书第 12 章：*ECO Flow*。

正文待续译。可将原版 PDF 对应页提取后按第 1 章体例写入本文件。

原书起始页约 p.821。